

Данная книга описывает разработку и реализацию программного обеспечения распределенных систем реального времени с использованием подхода «снизу вверх». Освещаются проблемы, встречающиеся в программных проектах систем реального времени; представлен новый метод простого и эффективного выполнения всех этапов разработки программного обеспечения.

Рассматриваемые темы:

- основы систем реального времени, включая архитектуру реального времени и распределенные системы реального времени;
- операционная система реального времени, охватывающая концепции задач, памяти и управления вводом/выводом;
- подробное пошаговое построение ядра операционной системы реального времени, которое затем используется для тестирования различных реализаций более высокого уровня;
- периодическое и аperiodическое планирование, управление ресурсами и распределенное планирование;
- процесс разработки приложений – от методов проектирования высокого уровня до деталей проектирования и реализации низкого уровня;
- языки программирования в реальном времени и методы отказоустойчивости.

Интернет-магазин:
www.dmkpress.com

Оптовая продажа:
КТК «Галактика»
books@alians-kniga.ru

 Springer


ИЗДАТЕЛЬСТВО
www.dmk.pf



Распределенные системы реального времени

Кайхан Эрджиес

Распределенные системы реального времени

Теория и практика

 Springer


ИЗДАТЕЛЬСТВО

Кайхан Эрджиес

Распределенные системы реального времени

Теория и практика

Distributed Real-Time Systems

Theory and Practice

Kayhan Erciyes

Распределенные системы реального времени

Теория и практика

Кайхан Эрджиес



Москва, 2020

УДК 004.45
ББК 32.973
Э75

К. Эрджиес

Э75 Распределенные системы реального времени. Теория и практика / пер. с англ. В. А. Яроцкий. – М.: ДМК Пресс, 2020. – 382 с.: ил.

ISBN 978-5-97060-852-4

В книге описываются разработка и реализация программного обеспечения распределенных систем реального времени с использованием подхода «снизу вверх». В начале каждой главы обсуждаются основные концепции, представлен обзор соответствующих методов и доступного ПО. Затем рассматривается реализация концепций в образце ядра, сопровождаемая исполняемым кодом. В завершение главы приводится обширный код на языке C, многочисленные примеры, реализующие описанные методы.

Издание предназначено студентам, инженерам, разработчикам ПО, имеющим базовый опыт работы в области компьютерной архитектуры и операционных систем

УДК 004.45
ББК 32.973

First published in English under the title Distributed Real-Time Systems;

Theory and Practice by K Erciyes, edition: 1

Copyright © Springer Nature Switzerland AG, 2019 *

This edition has been translated and published under licence from Springer Nature Switzerland AG.

Springer Nature Switzerland AG takes no responsibility and shall not be made liable for the accuracy of the transl

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN (анг.) 978-3-030-22569-8
ISBN (рус.) 978-5-97060-852-4

© Springer Nature Switzerland AG 2019
© Оформление, издание, перевод, ДМК Пресс, 2020

*Посвящается памяти профессора Синана Йылмазу,
который был прекрасным педагогом и глубоким исследователем
систем реального времени, а также профессору Невзет Тархану,
приверженному науке и научному поиску*

Оглавление

Предисловие от автора	15
Предисловие от издательства	18
ЧАСТЬ I. ВВЕДЕНИЕ	19
Глава 1. Введение в системы реального времени.....	21
1.1. Введение	21
1.2. Что такое система реального времени	22
1.3. Базовая архитектура	23
1.4. Характеристики систем реального времени.....	25
1.5. Классификация систем реального времени.....	25
1.6. Пример системы: конвейер бутылок.....	27
1.7. Обзор книги	29
1.8. Контрольные вопросы	29
1.9. Примечания к главе	30
Справочные материалы	30
Глава 2. Аппаратура.....	31
2.1. Введение	31
2.2. Архитектура процессора.....	32
2.2.1. Шина данных с одним циклом	33
2.2.2. Канал передачи данных многими циклами	38
2.2.3. Конвейер	40
2.2.3.1. Риски	43
2.2.4. Микроконтроллеры	46
2.3. Память.....	46
2.3.1. Интерфейс процессора.....	47
2.3.2. Кеш.....	48
2.4. Доступ к вводу/выводу.....	50
2.4.1. Интерфейс устройства ввода	51
2.4.2. Интерфейс устройства вывода	52
2.4.3. Отображение в память и изолированный ввод/вывод	53
2.4.4. Программный интерфейс ввода/вывода	54
2.4.4.1. Опрос	54
2.4.4.2. Ввод/вывод с прерыванием	55
2.4.4.3. Прямой доступ к памяти.....	56
2.4.4.4. Исключения	57
2.4.4.5. Таймеры	57
2.5. Многоядерные процессоры.....	58
2.6. Мультипроцессоры	59
2.7. Контрольные вопросы.....	60
2.8. Примечания к главе	61
2.9. Упражнения	62
Справочные материалы	62

Глава 3. Распределенные системы реального времени	63
3.1. Введение	63
3.2. Модели	64
3.2.1. Распределение по времени и событию	64
3.2.2. Конечные автоматы.....	65
3.3. Распределенные операционные системы реального времени и промежуточное программное обеспечение	68
3.3.1. Промежуточное программное обеспечение.....	69
3.3.2. Распределенное планирование	69
3.3.3. Динамическая балансировка нагрузки	71
3.4. Связь в реальном времени	72
3.4.1. Трафик в реальном времени.....	72
3.4.2. Модель взаимосвязи открытых систем.....	73
3.4.3. Топология	74
3.4.4. Уровень канала передачи данных	76
3.4.4.1. Протоколы доступа к среде.....	77
3.4.5. Протокол контроллерной сети.....	77
3.4.6. Протокол запуска по времени	79
3.4.7. Сеть реального времени Ethernet	80
3.4.8. Стандарт реального времени IEEE 802.11	80
3.5. Проблемы в распределенных системах реального времени со встроенными элементами.....	81
3.6. Примеры распределенных систем реального времени	82
3.6.1. Современный автомобиль	82
3.6.2. Беспроводная мобильная сенсорная сеть	83
3.7. Контрольные вопросы.....	84
3.8. Примечания к главе	85
3.9. Упражнения	86
Справочные материалы	87

Часть II. СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

Глава 4. Операционные системы реального времени	91
4.1. Введение	91
4.2. Общие операционные системы и операционные системы реального времени.....	92
4.3. Управление задачами.....	93
4.3.1. Задача управления в UNIX	95
4.3.2. Синхронизация задач.....	97
4.3.3. Межзадачные коммуникации.....	98
4.3.4. Межпроцессорное взаимодействие в UNIX	101
4.4. Потоки.....	102
4.4.1. Управление потоками	102
4.4.2. Потоки POSIX	103
4.4.2.1. Взаимное исключение	104
4.4.2.2. Синхронизация.....	105
4.4.2.3. Связь.....	106

4.5. Управление памятью	108
4.5.1. Статическое распределение памяти	108
4.5.2. Динамическое распределение памяти	108
4.5.3. Виртуальная память	108
4.5.4. Управление памятью в реальном времени.....	109
4.6. Управление вводом/выводом.....	110
4.6.1. Управляемый прерываниями ввод/вывод.....	110
4.6.2. Драйверы устройств	111
4.7. Обзор операционных систем реального времени	112
4.7.1. Операционная система с открытым кодом RTOS.....	113
4.7.2. Операционная система VxWorks.....	113
4.7.3. Система реального времени Linux	113
4.8. Контрольные вопросы	114
4.9. Примечания к главе	115
4.10. Упражнения по программированию	115
Справочные материалы	116

Глава 5. Проектирование экспериментального

распределенного ядра реального времени	117
5.1. Введение	117
5.2. Стратегия дизайна	118
5.3. Функции ядра нижнего уровня	119
5.3.1. Структуры данных и операции с очередями	119
5.3.1.1. Тип блока данных	120
5.3.1.2. Тип данных блока управления задачами	121
5.3.2. Планировщик с несколькими очередями	123
5.3.3. Обработка прерываний и управление временем.....	126
5.3.3.1. Дельта-очередь	127
5.3.4. Управление состоянием задачи.....	128
5.3.4.1. Программа обработки прерываний по времени.....	129
5.3.5. Управление вводом/выводом	130
5.4. Функции ядра верхнего уровня	132
5.4.1. Синхронизация задач.....	132
5.4.2. Коммуникация задач.....	136
5.4.3. Управление верхней памятью с использованием пулов	139
5.4.4. Управление задачами.....	140
5.5. Инициализация.....	142
5.6. Тестирование DRTK.....	144
5.7. Контрольные вопросы.....	145
5.8. Примечания к главе	145
5.9. Проекты программирования	146
Справочные материалы	147

Глава 6. Операционные системы реального времени

и промежуточное программное обеспечение.....	148
6.1. Введение	148

6.2. Распределенные операционные системы реального времени	149
6.2.1. Интерфейс транспортного уровня	150
6.2.2. Интерфейс уровня канала передачи данных.....	151
6.3. Промежуточное программное обеспечение реального времени.....	153
6.3.1. Группы задач в реальном времени	154
6.3.2. Синхронизация часов.....	155
6.3.2.1. Логические часы	156
6.3.2.2. Векторные часы.....	157
6.3.2.3. Протокол сетевого времени.....	158
6.3.2.4. Алгоритм Беркли.....	160
6.3.2.5. Синхронизация часов в беспроводных сенсорных сетях	160
6.3.3. Алгоритмы выбора	162
6.3.3.1. Выбор в однонаправленном кольце.....	162
6.3.3.2. Выборы в беспроводных сенсорных и мобильных специальных сетях.....	163
6.4. Реализация DRTK	165
6.4.1. Инициализация сети	165
6.4.2. Интерфейс транспортного уровня	167
6.4.3. Задачи интерфейса канального уровня передачи данных.....	171
6.4.4. Групповое управление	173
6.4.5. Алгоритм синхронизации часов.....	175
6.4.6. Выбор лидера в кольце.....	176
6.5. Контрольные вопросы	178
6.6. Примечания к главе	178
6.7. Проекты программирования.....	179
Справочные материалы	179

ЧАСТЬ III. ПЛАНИРОВАНИЕ И РАСПРЕДЕЛЕНИЕ РЕСУРСОВ

181

Глава 7. Задача планирования однопроцессорной независимой задачи.....

183

7.1. Введение	183
7.2. Предпосылки	184
7.2.1. Тест планируемости.....	185
7.2.2. Применение.....	186
7.3. Политики планирования	186
7.3.1. Приоритетное или неприоритетное планирование	186
7.3.2. Статичное или динамичное планирование	188
7.3.3. Независимые или зависимые задачи	189
7.4. Таксономия алгоритмов планирования в реальном времени.....	190
7.5. Потактовое планирование.....	191
7.5.1. Планирование на основе таблиц	191
7.5.2. Циклическое выполнение	193
7.6. Приоритетное планирование.....	195

7.6.1. Монотонное планирование.....	195
7.6.2. Планирование с первым самым крайним сроком	197
7.6.2.1. Аperiodический алгоритм EDF	198
7.6.2.2. Периодический алгоритм EDF	199
7.6.3. Алгоритм наименьшего времени незанятости	201
7.6.4. Анализ времени отклика.....	202
7.7. Аperiodическое планирование задач	203
7.7.1. Основные методы.....	204
7.7.2. Периодические серверы	206
7.7.2.1. Сервер опроса.....	206
7.7.2.2. Отложенный сервер	207
7.7.2.3. Спорадический сервер.....	207
7.7.2.4. Серверы с динамическим приоритетом	207
7.8. Планирование спорадических задач	208
7.9. Реализация в DTRK.....	210
7.9.1. Планировщик монотонного рейтинга	210
7.9.2. Планировщик самого раннего первого срока	212
7.9.3. Планировщик первой наименее занятой задачи	212
7.9.4. Сервер опроса.....	214
7.10. Контрольные вопросы.....	214
7.11. Примечания к главе.....	215
7.12. Упражнения	217
Справочные материалы	218

Глава 8. Планирование однопроцессорной

зависимой задачи	219
8.1. Введение	219
8.2. Планирование зависимых задач.....	220
8.2.1. Алгоритм первым последний конечный срок.....	220
8.2.2. Модифицированный алгоритм первым ранний конечный срок.....	221
8.3. Планирование задач при совместном использовании ресурсов.....	223
8.3.1. Случай марсианского зонда.....	226
8.3.2. Основной протокол наследования приоритетов	227
8.3.3. Протокол приоритетного потолка.....	231
8.4. Реализация DRTK	233
8.4.1. Зависимое планирование задач LDF.....	234
8.4.2. Протокол приоритетного наследования.....	235
8.5. Контрольные вопросы	237
8.6. Примечания к главе	237
8.7. Упражнения	238
Справочные материалы	239

Глава 9. Планирование многопроцессорных

распределенных задач реального времени	240
9.1. Введение	240
9.2. Многопроцессорное планирование.....	241

9.2.1. Раздельное планирование	242
9.2.1.1. Распределение задач	242
9.2.1.2. Алгоритм балансировки использования с EDF	243
9.2.1.3. Алгоритмы	244
9.2.1.4. Алгоритм EDF с упаковкой первое соответствие	246
9.2.1.5. Монотонный алгоритм первое соответствие (RM-FF).....	246
9.2.2. Глобальное планирование	247
9.2.2.1. Глобальный монотонный алгоритм	248
9.2.2.2. Аномалии	249
9.2.2.3. Пропорциональный алгоритм справедливого планирования.....	249
9.3. Распределенное планирование	249
9.3.1. Балансировка нагрузки	250
9.3.1.1. Центральная балансировка нагрузки	250
9.3.1.2. Распределенная балансировка нагрузки	251
9.3.2. Метод целенаправленной адресации и назначения ставок	252
9.3.3. Алгоритм Buddy	253
9.3.4. Планирование сообщений	254
9.4. Реализация DRTK	255
9.4.1. Центральные задачи балансировки нагрузки	255
9.4.2. Задача балансировки распределенной нагрузки	257
9.5. Контрольные вопросы	259
9.6. Примечания к главе	259
9.7. Упражнения	260
Справочные материалы	260
ЧАСТЬ IV. ПРИКЛАДНОЕ ПРОЕКТИРОВАНИЕ	263

Глава 10. Разработка программного обеспечения систем реального времени	265
10.1. Введение	265
10.2. Жизненный цикл разработки программного обеспечения.....	266
10.2.1. Пошаговая модель Waterfall.....	267
10.2.2. V-модель	268
10.2.3. Спиральная модель Spiral Model.....	268
10.3. Разработка программного обеспечения систем реального времени.....	269
10.4. Анализ требований и спецификация	270
10.5. Временной анализ.....	271
10.6. Структурное проектирование с диаграммами потоков данных	272
10.7. Объектно-ориентированное проектирование	274
10.8. Методы реализации в реальном времени.....	275
10.8.1. Еще о конечных автоматах	275
10.8.1.1. Параллельные иерархические конечные автоматы	277
10.8.2. Временные автоматы	278
10.8.3. Сети Петри	279
10.8.3.1. Временные сети Петри.....	282
10.9. Унифицированный язык моделирования в реальном времени.....	283
10.9.1. UML-диаграммы	283
10.9.2. Функции реального времени	285

10.10. Метод практического проектирования и реализации	286
10.11. Контрольные вопросы	287
10.12. Примечания к главе	288
10.13. Программирование проектов	289
Справочные материалы	290

Глава 11. Языки программирования в реальном времени

11.1. Введение	291
11.2. Требования	292
11.3. Приложение в реальном времени.....	293
11.4. Операционная система C/POSIX в реальном времени.....	293
11.4.1. Инкапсуляция данных и управление модулями	294
11.4.2. Управление потоком POSIX	295
11.4.2.1. Управление временем	296
11.4.2.2. Синхронизация потоков и связь	297
11.4.2.3. Сигналы.....	297
11.4.2.4. Взаимное исключение.....	298
11.4.2.5. Условная синхронизация	299
11.4.2.6. Семафоры.....	299
11.4.3. Обработка исключений и низкоуровневое программирование	300
11.4.4. Реализация управления процессом C/POSIX в реальном времени.....	300
11.5. Ада.....	302
11.5.1. Параллелизм	303
11.5.1.1. Управление временем	304
11.5.1.2. Periodic Tasks.....	304
11.5.1.3. Приоритеты задач	305
11.5.1.4. Синхронизация задач и связь.....	305
11.5.2. Обработка исключений	306
11.5.3. Реализация управления процессами на языке ADA.....	308
11.6. Язык программирования Java	309
11.6.1. Потоки Java.....	310
11.6.2. Синхронизация потоков	311
11.6.2.1. Управление временем и расписание	311
11.6.3. Обработка исключений	312
11.7. Контрольные вопросы.....	312
11.8. Примечания к главе	313
11.9. Упражнения по программированию	314
Справочные материалы	314

Глава 12. Отказоустойчивость

12.1. Введение	315
12.2. Понятия и терминология.....	316
12.3. Классификация неисправностей	317
12.4. Резервирование.....	318
12.4.1. Аппаратное резервирование	318
12.4.2. Избыточность информации.....	320
12.4.2.1. Кодирование	320

12.4.3. Резервирование времени	322
12.4.4. Резервирование программного обеспечения.....	323
12.4.4.1. Методы с одной версией.....	323
12.4.4.2. Многократное резервирование.....	324
12.5. Отказоустойчивые системы реального времени.....	325
12.5.1. Статичное планирование	326
12.5.2. Динамическое планирование	326
12.6. Отказоустойчивость в распределенных системах реального времени.....	327
12.6.1. Классификация отказов	327
12.6.2. Пересмотр состава целевых групп	328
12.6.2.1. Надежная многоадресная связь	328
12.7. Реализация DRTK.....	331
12.8. Контрольные вопросы	334
12.9. Примечания к главе	335
12.10. Упражнения	335
Справочные материалы	336

Глава 13. Тематический пример: мониторинг окружающей среды по беспроводной сенсорной сети

13.1. Введение	338
13.2. Вопросы проектирования.....	339
13.3. Требования к спецификации	339
13.4. Временной анализ и функциональные характеристики	340
13.5. Связующее дерево и кластеризация.....	341
13.6. Вопросы проектирования.....	345
13.7. Листовой узел	346
13.7.1. Дизайн высокого уровня	347
13.7.2. Детальная разработка и реализация	349
13.8. Промежуточный узел.....	355
13.8.1. Дизайн верхнего уровня.....	355
13.8.2. Детальное проектирование и реализация	356
13.9. Узел управления кластером.....	360
13.9.1. Дизайн высокого уровня	361
13.9.1. Дизайн верхнего уровня.....	361
13.10. Приемник.....	364
13.10.1. Дизайн высокого уровня	364
13.11. Тестирование	365
13.12. Альтернативная реализация с потоками POSIX	367
13.13. Примечания к главе	367
13.14. Упражнения по программированию	368
Справочные материалы	368

Приложение А. Соглашение о псевдокоде

Приложение В. Функции нижнего ядра

Предметный указатель

Предисловие

Распределенные системы реального времени со встроенными элементами присутствуют повсюду – от производственных площадок заводов до автомобилей и авионики. Распределенные системы реального времени характеризуются числом вычислительных элементов, объединенных в сеть и выполняющих задачи в реальном времени. Задачи реального времени имеют конечные сроки, и многие применения требуют получения решения до их истечения. Современные технологические достижения привели к значительному увеличению числа элементов в распределенных системах реального времени, что, в свою очередь, вызвало необходимость разработки соответствующего программного обеспечения. Элементы распределенных систем реального времени имеют вычислительные мощности и обычно связаны с внешним миром при помощи датчиков и исполнительных механизмов. Не все системы со встроенными элементами работают в режиме реального времени, и мы будем использовать термин *распределенная система реального времени* только для таких распределенных систем, которые характеризуются работой в реальном времени.

Эта книга — о разработке и внедрении программного обеспечения для распределенных систем реального времени с использованием метода восходящего проектирования. На протяжении нескольких десятилетий я читал соответствующие курсы бакалаврам и магистрам и принимал участие в крупных программных проектах систем реального времени, что дало мне возможность наблюдать основные узкие места, которые встречаются при разработке и проектировании таких систем. Прежде всего разработчик или проектировщик часто сталкивается с проблемой сопряжения приложения с некоторой коммерческой операционной системой или промежуточным программным обеспечением реального времени и иногда вынужден писать патчи к ним. Это требует глубокого понимания концепций как аппаратного обеспечения, так и операционной системы, обеспечивающих обработку в реальном времени, и поэтому часть книги посвящена системному программному обеспечению. Первая часть книги состоит из трех глав. В первой главе мы рассматриваем основные концепции операционных систем реального времени. Затем, во второй главе, мы строим с нуля экспериментальное ядро распределенной операционной системы реального времени (*experimental distributed real-time operating system kernel*, DRTK), раскрывая все необходимые детали.

Далее, в последней главе этой части книги, рассматриваются концепции связи распределенной операционной системы реального времени и промежуточного программного обеспечения и рассказывается, как спроектировать

сетевые коммуникации, чтобы в режиме реального времени ядра могли взаимодействовать и иметь распределенный системный программный фрейм. Постепенно мы превращаем экспериментальное ядро в ядро распределенной операционной системы реального времени со связанным промежуточным программным обеспечением, показывая в последующих разделах книги все детали реализации.

Вторая проблема – это, конечно же, планирование задач, которое обеспечивает выполнение всех конечных сроков. Задачи в системе реального времени можно в широком смысле классифицировать как жесткие, мягкие и твердые задачи, которые могут быть также периодическими и аperiodическими и требующими различных стратегий планирования. Более того, они могут быть независимыми или зависеть от каждого из необходимых средств синхронизации задач. Вместе с тем нам нужно обеспечить такое сквозное планирование задач, чтобы сроки были соблюдены, а нагрузка распределялась между элементами распределенной системы реального времени равномерно. Еще одной связанной с этим проблемой является управление ресурсами в сети. Все эти проблемы исследуются во второй части, и всегда имеются в виду методы их реализации.

Наконец, перед разработчиком стоит задача выполнения всех этапов разработки программного обеспечения, начиная с определения требований и разработки общего плана и до последующей детальной разработки и кодирования. Именно здесь встречается большинство затруднений. Мы предоставляем новый метод для простого и эффективного выполнения всех этапов. В книге есть глава о проектировании верхнего уровня и детальной разработке с использованием конечных автоматов, которые могут быть реализованы с применением потоков в операционной системе. В системе реального времени для предотвращения катастрофических событий обязательна отказоустойчивость, и в книге есть глава на эту тему. Также рассматриваются различные языки программирования в реальном времени, включая C/POSIX, Ada и Java. Наконец, в последней главе представлено тематическое исследование режима реального времени, включающее все методы, которые были рассмотрены и разработаны, начиная с проектирования верхнего уровня с последующей подробной разработкой и кодированием.

В каждой главе мы сначала рассматриваем концепции, а затем описываем методы разработки и реализации необходимого программного обеспечения, давая краткое описание коммерчески доступного программного обеспечения. В главах второй части книги, где это целесообразно, мы показываем, как реализовать описанные концепции в экспериментальном образце ядра, отображая исполняемый код. Обычно эти части в каждой главе называются «Реализация DRTK», и их можно пропустить вместе с главой 5, которая описывает DRTK, если книга используется для курса с ограниченным охватом систем реального времени. Наконец, мы включили обзорные вопросы, а затем и краткое примечание к главе, подчеркивая основные моменты, давая современный обзор соответствующей литературы и возможные открытые для исследований области.

О ядре РАСПРЕДЕЛЕННОЙ ОПЕРАЦИОННОЙ СИСТЕМЫ РЕАЛЬНОГО ВРЕМЕНИ

Основные модули DRTK, рассмотренные в деталях в главе 5, тестированы при обучении студентов на различных курсах по обработке в реальном времени. Однако материал глав этой части книги, следующих за этой главой и относящихся к распределенной обработке, не полностью тестирован и с некоторой вероятностью может иметь синтаксические или другие ошибки при реализации. Я буду сохранять домашнюю страницу книги по адресу http://akademik.ube.ege.edu.tr/*erciyes/DRTS для кода DRTK, учебных слайдов и ошибок. Приветствуются любые изменения в коде DRTK в надежде сделать его практическим экспериментально распределенным ядром реального времени, которое может затем быть исследовано на соответствующих учебных курсах.

Целевой аудиторией книги являются студенты и выпускники старших курсов, проектировщики и исследователи электротехники и вычислительной техники, информатики и машиностроения в целом, все, имеющие базовый опыт работы в области компьютерной архитектуры и операционных систем. Текст содержит значительное количество кода Си в примерах реализации DRTK и других различных примерах. Я хотел бы поблагодарить студентов и аспирантов различных университетов, включая (в хронологическом порядке) Университет Эге, Университет штата Орегон, Калифорнийский государственный университет Дэвис, Калифорнийский государственный университет Сан-Маркос, Измирский Технологический институт, Измирский университет и Ускюдарский университет, в которых я преподавал курсы «Системы реального времени», «Встроенные системы», «Современные операционные системы» или другие аналогичные курсы, за их ценную обратную связь, когда различные части включенных в книгу материалов излагались во время лекций, а образец ядра тестировался в лабораторных работах. Я также хотел бы поблагодарить главного редактора издательства Springer Уэйна Уилера (Wayne Wheeler) и помощника редактора Саймона Рис (Simon Rees) за их постоянную поддержку в процессе написания книги.

Стамбул, Турция К. Эрджиес (K. Erciyes)

Предисловие от издательства

ОТЗЫВЫ И ПОЖЕЛАНИЯ

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв прямо на нашем сайте **www.dmkpress.com**, зайдя на страницу книги, и оставить комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу **dmkpress@gmail.com**, при этом напишите название книги в теме письма.

Если есть тема, в которой вы квалифицированы, и вы заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу **http://dmkpress.com/authors/publish_book/** или напишите в издательство по адресу **dmkpress@gmail.com**.

СПИСОК ОПЕЧАТОК

Хотя мы приняли все возможные меры для того, чтобы удостовериться в качестве наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в тексте или в коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от расстройств и поможете нам улучшить последующие версии этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу **dmkpress@gmail.com**, и мы исправим это в следующих тиражах.

НАРУШЕНИЕ АВТОРСКИХ ПРАВ

Пиратство в интернете по-прежнему остается насущной проблемой. Издательства «ДМК Пресс» и Packt очень серьезно относятся к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконно выполненной копией любой нашей книги, пожалуйста, сообщите нам адрес копии или веб-сайта, чтобы мы могли применить санкции.

Пожалуйста, свяжитесь с нами по адресу электронной почты **dmkpress@gmail.com** со ссылкой на подозрительные материалы.

Мы высоко ценим любую помощь по защите наших авторов, помогающую нам предоставлять вам качественные материалы.

ЧАСТЬ

I

ВВЕДЕНИЕ

Глава 1

Введение в системы реального времени

1.1. ВВЕДЕНИЕ

Системы реального времени присутствуют повсюду, от автомобилей и мобильных телефонов до авиационного оборудования и систем управления ядерными установками. Они характеризуются своевременностью реагирования на некоторый вход. Ошибка в реагировании в течение определенного времени может иногда привести к катастрофе. Правильность работы системы реального времени зависит как от правильности результатов, так и от времени получения этих результатов, которые соответственно называются *логической корректностью* (logical correctness) и *временной корректностью* (temporal correctness). В *системе реального времени со встроенными элементами* (embedded real-time system) вычислительный элемент реального времени встроен в контролируемую систему реального времени, и в эту категорию попадает большая часть элементов такой системы.

Существует много типов систем реального времени; система управления процессом в реальном времени получает входные данные от датчиков, выполняет некоторые операции с этими данными и создает выход для управления различными функциями системы, такими как включение и выключение, при необходимости активирует аварийные сигналы и отображает системные данные. Самолет в режиме реального времени является системой со строгими временными ограничениями, которые должны быть соблюдены, в то время как несоблюдение временных ограничений мультимедийной системы, как правило, приведет лишь к некачественному воспроизведению. Другие примеры систем реального времени включают роботизированные системы, атомные электростанции и мобильные телефоны. В этой главе мы рассмотрим основные понятия, связанные с системами реального времени, и, начиная с определения, разберем типы систем реального времени и свойства этих систем.

Затем опишем работу систем реального времени на примерах и в заключение дадим краткий обзор этой книги.

1.2. ЧТО ТАКОЕ СИСТЕМА РЕАЛЬНОГО ВРЕМЕНИ

Система реального времени работает с временным ограничением, то есть значимым временем вывода результата. Другими словами, получать результат в другое время, чем требуемое, не имеет смысла. Альтернативный вариант системы реального времени – система обработки данных, которая реагирует на внешние входы в течение заданного времени, называемого крайним сроком. Неспособность дать реакцию на ввод за определенный промежуток времени может нанести существенный вред жизни или имуществу. Система реального времени разрабатывается в соответствии с динамикой физических процессов, которые она должна контролировать. Вычислительная система реального времени состоит из аппаратного и программного обеспечения, работающих с временным ограничением. Программное обеспечение реального времени в целом включает в себя следующие компоненты:

- *операционная система реального времени*: операционная система имеет две основные функции: предоставление удобного доступа к аппаратному обеспечению и эффективное управление ресурсами. Для этого она обеспечивает управление процессом (задачей), памятью, а также управление вводом/выводом в целом. Операционная система реального времени должна выполнять эти задачи со строгими временными ограничениями. Операционная система реального времени, как правило, небольшого размера и легко размещается в системах со встроенными элементами, а также должна быть быстрой при минимальных потерях;
- *язык программирования реального времени*: язык программирования реального времени обеспечивает основные схемы, такие как межзадачное общение и синхронизация, обработка ошибок и планирование задач в реальном времени. Заметим, что эти функции обеспечиваются также и операционной системой реального времени. Следовательно, язык программирования реального времени может быть использован для обработки в реальном времени альтернативно операционной системе. На самом низком уровне для доступа к регистрам и другому аппаратному обеспечению процессора может использоваться ассемблер. Однако язык ассемблера подвержен ошибкам и не переносим на другой процессор. Язык Си широко используется в режиме реального времени системы, поскольку обеспечивает доступ к оборудованию и имеет простой интерфейс ввода/вывода. Вместе с тем для систем реального времени, с целью упрощения программирования, разработаны другие языки реального времени, такие как Ada и Java;
- *сеть реального времени*: компьютерная сеть обеспечивает передачу данных между различными вычислительными устройствами. Сеть реального времени должна обеспечивать надежную доставку сообщений в заданные

сроки. Ключ к работе сети в реальном времени – это надежная и своевременная передача сообщений. Протокол общения в реальном времени обеспечивает своевременную и гарантированную доставку сообщений по сети.

Следует отличать систему реального времени от системы, которая выглядит похожей на систему реального времени, но таковой на самом деле не является. Например, *онлайновая система* взаимодействия с пользователем не считается системой реального времени, поскольку решение ею задач не имеет определенных сроков. Кроме того, если система должна просто реагировать быстро, это не обязательно означает, что она является системой реального времени.

Система со встроенными элементами характеризуется тем, что в состав системы включен вычислительный элемент. Такие системы не являются общими вычислительными системами, в которых компьютер может быть динамически запрограммирован для решения различных задач. Такие системы, например, как микроволновая печь, имеют специальное назначение. Система со встроенными элементами тесно связана с внешним физическим миром. Примерами систем со встроенными элементами могут служить домашние приборы бытового назначения, используемые для улучшения качества жизни. Системы со встроенными элементами фактически составляют большинство компьютерных систем. Система со встроенными элементами может быть или не быть системой с режимом реального времени в зависимости от назначения приложения. Система со встроенными элементами не является системой реального времени, если не имеет временных границ. Такой системой, например, является MP3-плеер. Вместе с тем значительный процент приложений с реальным временем имеет встроенные элементы. Система реального времени со встроенными элементами разрабатывается для реализации определенной функции и работает с ограничениями по реальному времени. Наша терминология в этой книге относится к системам реального времени в целом, включая ее встроенные элементы.

Системы реального времени можно классифицировать, используя различные подходы; базовым условием является наличие синхронизации и взаимодействия между различными компонентами системы. Соответственно, система реального времени может быть управляема временем, в которой каждому действию определено специфическое время; внешние входы определяют поведение системы и ее взаимодействие, и пользователь может изменять режимы операций. Во многих случаях система реального времени работает как комбинация всех режимов подобного рода.

1.3. БАЗОВАЯ АРХИТЕКТУРА

Система реального времени состоит из компонентов аппаратного обеспечения, показанных на рис. 1.1. Их можно детализировать следующим образом.

- *Датчики*: датчик преобразует физический параметр в обработанный электрический сигнал. Например, термопара – это датчик температуры, который преобразует тепло в электрический сигнал.

- *Элемент обработки (Processing Element, PE)*: элемент обработки в реальном времени или компьютер реального времени, который принимает цифровые данные, предоставляемые интерфейсным узлом ввода, обрабатывает их и выбирает некоторую функцию вывода для выполнения.
- *Приводы*: привод вводит электрический сигнал выходного интерфейса обрабатываемого элемента и активирует физическое действие. Примером привода может являться реле, которое при активации сигналом замыкает/размыкает контакты, осуществляя переключение.
- *Интерфейс ввода*: интерфейс ввода в компьютер реального времени преобразовывает и обрабатывает электрический сигнал в цифровую форму для обработки. Датчики могут иметь необходимые встроенные в них преобразователи.
- *Интерфейс вывода*: двоичный вывод из компьютера реального времени, обработанный и преобразованный в форму сигнала, необходимую для привода или других устройств вывода в выходной интерфейс.

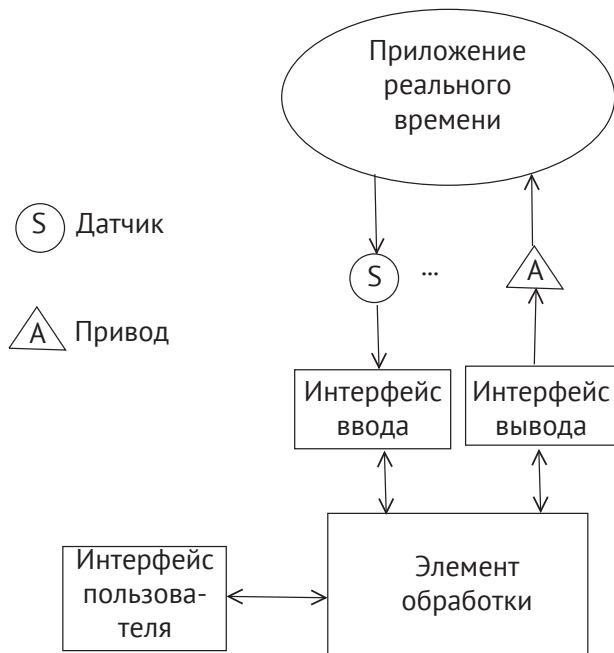


Рис. 1.1. Архитектура типичной системы реального времени

1.4. ХАРАКТЕРИСТИКИ СИСТЕМ РЕАЛЬНОГО ВРЕМЕНИ

Общие характеристики систем реального времени:

- *соблюдение сроков*: для предотвращения гибели людей и имущества должны быть выдержаны в реальном времени сроки всех важных задач;
- *большой размер*: системы реального времени обычно большие и сложные в части аппаратного обеспечения. Однако особенно это относится к величине используемого программного обеспечения. Даже небольшая система реального времени может потребовать для реализации сотни тысяч строк кода;
- *предсказуемость*: мы должны быть в состоянии предсказать время ответа для наихудшего варианта реализации и знать, будут ли соблюдены системой реального времени заданные сроки выполнения всех предписанных задач. Предсказуемость предполагает обычно теоретическое доказательство того, что все сроки в реализуемой системе реального времени будут соблюдаться;
- *безопасность и надежность*: системы реального времени работают и управляют процессами в таких средах, как атомные станции, где безопасности и надежности уделяется предельное внимание. Небезопасные и ненадежные системы подвержены ошибкам, которые могут привести к ущербу или к потере здоровья людей и потере имущества;
- *отказоустойчивость*: считается, что *сбой* происходит, когда компонент аппаратного или программного обеспечения компьютерной системы выходит из строя, и система перестает вести себя в соответствии с ее спецификацией. *Отказ* системы реального времени является результатом сбоя, что может привести к потере жизни людей и собственности. Сбой может быть постоянным, вызывая либо временный выход системы из строя, который через некоторое время может исчезнуть, либо постоянный. Отказоустойчивость – это способность системы противостоять сбоям и продолжать правильно функционировать при наличии сбоев;
- *параллелизм*: физическая среда, управляемая компьютером реального времени, в которой обычно реализуются параллельные события. Компьютер реального времени должен быть в состоянии справиться с параллельной операцией, используя возможности параллельного системного программного обеспечения или распределенного оборудования.

1.5. КЛАССИФИКАЦИЯ СИСТЕМ РЕАЛЬНОГО ВРЕМЕНИ

Мы можем указать типы систем реального времени по временным ограничениям и типу процессов в системе в зависимости от полученного ввода. Можно дать следующую классификацию при рассмотрении конечных сроков:

- *жесткое реальное время*: несоблюдение сроков может привести к катастрофическим последствиям – потере имущества или жизни. Авиационные

системы маневрирования, химические станции и атомные электростанции являются примерами таких систем. Все сроки выполнения задачи в жесткой системе реального времени устанавливаются однозначно. Полезность системы, если заданный срок пропущен, равна нулю, как показано на рис. 1.2а;

- *мягкое реальное время*: пропуск мягких сроков не критичен, однако когда это происходит, качество сервиса системы ухудшается. Мягкая система реального времени выполняет задачи с мягким, не жестко установленным сроком. Примерами мягких систем реального времени являются мультимедийные системы и системы бронирования авиабилетов. Полезность системы, когда крайний срок пропущен, ухудшается и обнуляется, однако, не сразу, как показано на рис. 1.2б;
- *твердое реальное время*: эти системы в основном представляют собой мягкий тип системы реального времени, полезность которых при пропуске срока отсутствует, но он допускается. В этих системах полезность сразу сводится к нулю, как показано на рис. 1.2а, но система допускает такие пропуски сроков.

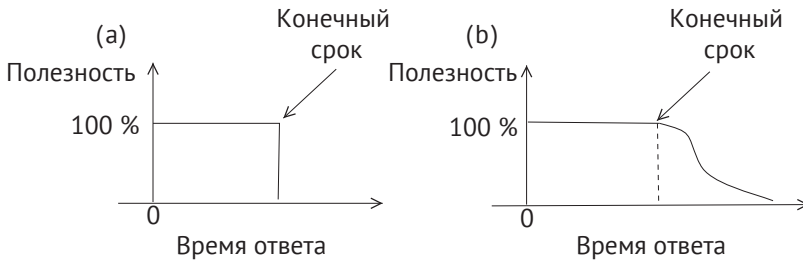


Рис. 1.2. Полезность (а) в жестких, твердых и (б) мягких системах реального времени

Система реального времени может сочетать задачи, выполняемые жестко, мягко или твердо в реальном времени. Процессы в системе могут быть *статическими* с заранее известными детерминированными характеристиками, тогда как характеристики в динамической системе реального времени меняются. Другое отличие заключается в том, как выполняется обработка:

- *системы реального времени, управляемые событиями*: когда и как выполняется обработка в реальном времени в этих системах, определяют внешние события. Система, управляемая событиями, обрабатывает асинхронные входы, активируемые событиями;
- *системы реального времени, управляемые временем*: обработка в этих системах выполняется в четко определенной временной базе, а действия, которые необходимо предпринять, выполняются в строго определенные моменты времени. Управляемая временем система реального времени имеет синхронные входы.

Например, мониторинг влажности воздуха с помощью датчика, измеряющего влажность каждые t единиц времени, является системой, управляемой временем, тогда как система, которая активируется асинхронным внешним событием, – это система, управляемая событиями. Обычно в типичной системе реального времени мы имеем сочетание этих режимов. Например, контроль температуры печи может быть выполнен путем измерения температуры через равные промежутки времени (управление событиями) и выключением обогревателя при открытии дверцы печи (по событию).

Распределенная система состоит из автономных компьютеров, каждый из которых способен функционировать в одиночку. Эти компьютеры связаны сетью и взаимодействуют, выполняя глобальную задачу и делясь ресурсами. Распределенная система реального времени является распределенной вычислительной системой, которая работает с ограничениями по времени.

Распределенные системы обеспечивают эффективное совместное использование ресурсов и избыточность. Предоставляет удобство также выполнение вычислений на нескольких компонентах, поскольку при этом возможно проведение быстрой локальной обработки. Еще одной причиной, по которой использование распределенной системы реального времени является целесообразным, является то, что различные подсистемы разных производителей могут быть связаны между собой с использованием стандартных протоколов связи. Автомобиль – это распределенная система реального времени, так как данные считываются из многих датчиков и передаются по сети на различные узлы для обработки в реальном времени. Еще одним преимуществом использования распределенных систем реального времени является повышенная надежность, поскольку сбой узла не может значительно повлиять на работу всей системы, если заранее приняты меры предосторожности.

1.6. ПРИМЕР СИСТЕМЫ: КОНВЕЙЕР БУТЫЛОК

Давайте рассмотрим очень простую систему управления процессом реального времени, наполняющую молоком бутылки. Молочные бутылки движутся на конвейерной ленте, и есть три машины: моечная машина, моющая бутылки, наполнитель, который наполняет бутылку молоком, и упаковщик, который устанавливает крышку на заполненную молоком бутылку, как показано на рис. 1.3.



Рис. 1.3. Архитектура типичной системы реального времени

Каждая машина подключена к компьютерной системе реального времени с датчиками и исполнительными механизмами (приводами). Компьютерная система должна выполнять три задачи в следующей последовательности:

- 1) *мойка*: о появлении грязной бутылки на конвейерной ленте сигнализирует датчик. Лента останавливается, и мойщик активируется на время t_w секунд, затем активируется лента. Мы видим здесь операцию, основанную на событии (поступление бутылки);
- 2) *наполнение*: прибытие бутылки фиксируется, и активированный наполнитель осуществляет наполнение. Уровень молока в бутылке контролируется датчиком, и наполнитель останавливается, когда желаемый уровень достигнут;
- 3) *упаковка*: бутылка прибывает к этому механизму, крышка помещается на горлышко бутылки, и она упаковывается.

Здесь мы видим обработку, преимущественно управляемую событиями. Тем не менее при определенном времени весь процесс может быть выполнен за счет периодического выполнения процессов мойки, розлива и укупорки, при условии что всегда присутствуют пустые бутылки. Обратите внимание, что весь процесс идет медленно, и простой компьютер оставляет значительное время для выполнения другой обработки.

1.7. Обзор книги

Мы имеем в книге следующие четыре части.

1. *Введение*: эта часть книги служит введением в системы реального времени с главами по архитектуре реального времени и распределенным системам реального времени.
2. *Системное программное обеспечение*: операционная система реального времени находится в центре внимания данной части, где мы опишем основные понятия, такие как задачи, память и управление вводом/выводом. Мы также предоставляем подробное пошаговое построение ядра операционной системы в режиме реального времени, которое используется для тестирования различных реализаций более высокого уровня в последующих главах.
3. *Планирование и управление ресурсами*: планирование задач таким образом, чтобы были выполнены конечные сроки, является фундаментальной функцией любой системы реального времени. Сначала в этой части книги мы опишем независимое периодическое и аperiodическое планирование, а затем зависимое планирование с управлением ресурсами и распределенным планированием.
4. *Проектирование приложения*: эта часть является фундаментальной с точки зрения помощи разработчикам программного обеспечения систем реального времени. Мы описываем процедуру проектирования от методов проектирования высокого уровня до низкоуровневого проектирования и реализации. Мы также представляем обзор языков программирования и методов отказоустойчивости в режиме реального времени. Эта часть завершается подробным примером, в котором мы рассматриваем реализацию методов на основе реального приложения.

1.8. Контрольные вопросы

1. Какова связь между системой реального времени и системой со встроенными элементами? Все ли системы со встроенными элементами работают в режиме реального времени? Обсудите, приводя примеры.
2. Каковы основные компоненты системы реального времени?
3. Что такое датчик и исполнительный механизм в системе реального времени? Приведите пример каждого из них.
4. Какой тип оборудования вы ожидаете встретить во входном интерфейсе компьютерной системы реального времени?
5. Что такое жесткий режим реального времени, мягкий и твердый режимы реального времени в системах реального времени?
6. Приведите пример каждой из управляемых событиями и управляемых временем систем реального времени.
7. Каковы основные характеристики распределенной системы реального времени?

1.9. ПРИМЕЧАНИЯ К ГЛАВЕ

Мы рассмотрели основные компоненты компьютерной системы реального времени, ее основную архитектуру и различные классификации. Системы реального времени могут иметь жесткие, мягкие и твердые сроки выполнения заданий. С другой стороны, обработка может быть выполнена как управляемая событиями, так и управляемая временем. В режиме реального времени компьютер может быть как встроен в среду, которой он должен управлять в системе со встроенными элементами, так и быть распределен по сети реального времени. Как правило, в режиме реального времени система имеет комбинацию этих свойств. Общие понятия о реальном времени системы описаны в [2], а языки программирования в реальном времени содержатся в [1].

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. *Burns A., Wellings A. (2001) Real time systems and programming languages: Ada 95, Real-time Java and real-time C/POSIX, 3rd edn. Addison-Wesley.*
2. *Laplante P. A., Ovaska S. J. (2011) Real-time systems design and analysis: tools for the practitioner, 4th edn. Wiley.*

Глава 2

Аппаратура

2.1. ВВЕДЕНИЕ

В большинстве вычислительных систем реального времени используются аппаратные компоненты общего назначения, такие как процессор, память и устройства ввода/вывода. Система реального времени со встроенными элементами может иметь специальные интерфейсные блоки ввода/вывода, соответствующие назначению приложения.

Основное различие между системой нереального времени и системой реального времени на уровне аппаратного программного обеспечения заключается в структуре программного обеспечения нижнего уровня, такого как обработка прерываний и управление временем. Для того чтобы понять работу такого программного обеспечения и интерфейса операционной системы, необходимо рассмотреть характеристики аппаратуры.

В этой главе мы рассмотрим аппаратное обеспечение узла сети системы реального времени на уровне компонентов, откладывая обзор архитектуры и базовой структуры программного обеспечения распределенной системы реального времени до следующей главы. Начнем с основных интерфейсных модулей: датчиков и исполнительных механизмов – и дадим обзор характеристик распространенных типов этих компонентов. Далее рассматривается процессор, выполняющий фактическую обработку, а затем обсуждаются каналы передачи данных с одним циклом, многими циклами и конвейерные. Как мы увидим, блоки памяти и узлы ввода/вывода также являются основными частями системы реального времени. Для описания аппаратных концепций мы будем использовать MIPS-процессор [1], так как этот процессор широко используется в системах со встроенными элементами и при этом достаточно прост для отображения операции.

2.2. АРХИТЕКТУРА ПРОЦЕССОРА

Модель вычисления фон Неймана основана на концепции *компьютера с установленной программой*, в которой команды и данные хранятся в одной и той же памяти. Эта архитектура является основой большинства современных процессоров. В этой модели вычислений процессор состоит из блока управления, арифметико-логического блока (АЛУ) и регистров, как показано на рис. 2.1.

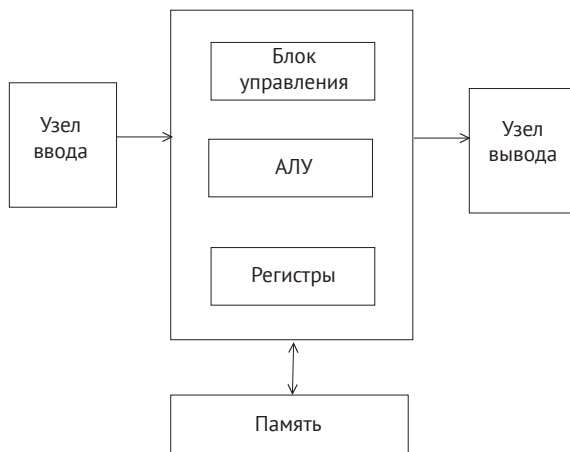


Рис. 2.1. Архитектура процессора

Блок АЛУ, блок регистров, который содержит все регистры и структуру интерфейса, обычно называют *внутренней шиной данных (datapath)*.

Процессор сначала выбирает команду из памяти и декодирует ее, чтобы принять решение, какое действие выполнить. Затем он находит все необходимые данные, выполняет команды и при необходимости записывает полученный результат в память. Стандартная промышленная архитектура шины ISA (*instruction set architecture*, ISA) формирует интерфейс между программным и аппаратным обеспечением. Шина ISA построена в основном на языке ассемблера процессора и обеспечивает команды для выполнения операций процессором, такие как *add*, *sub* и *and*. Другими словами, шина ISA – это взгляд программиста на аппаратное обеспечение. Она определяет организацию памяти, расположение временной памяти в центральном процессоре (CPU), называемой регистрами, и наборы команд.

Обычно используются два типа вычислений на основе ISA: сложный набор команд вычисления (*complex instruction set computing*, CISC) и вычисления с сокращенным набором команд (*reduced instruction set computing*, RISC).

Процессор CISC использует как можно меньшее количество строк ассемблера и потому ближе к языку более высокого уровня, чем процессор с RISC, который обычно выполняет простые команды за один такт. Например, умножение двух чисел может быть выполнено простой командой `MULT M1, M2, M3` в вычислениях CISC, которая умножает две величины в ячейках памяти M1 и M2 и сохраняет результат в ячейке памяти M3. Такую же операцию умножения в RISC необходимо разбить на несколько команд, таких как загрузка значений из памяти в два регистра, добавление значений в регистры внутри цикла и сохранение результата в памяти. На первый взгляд процессор RISC должен занимать больше строк программы, чем процессор CISC. Кроме того, в вычислениях CISC компилятор выполняет меньше работы по преобразованию языка высокого уровня в язык ассемблера и требует меньше оперативной памяти для хранения программы. Однако процессору CISC может потребоваться несколько временных циклов для выполнения команды, тогда как RISC-процессор обычно завершает выполнение строки кода ассемблера за один цикл, что делает проектирование процессора RISC проще по сравнению с процессором CISC. Режим работы канала передачи данных может быть с одним циклом, со многими циклами или конвейерным. Это рассматривается в следующих разделах.

2.2.1. Шина данных с одним циклом

В качестве примера мы кратко рассмотрим микропроцессор без взаимосвязанных этапов конвейера (*microprocessor without interlocked pipeline stages*, MIPS) – 32-битный процессор обработки данных и его блок управления. MIPS – это RISC, обычно используемый в системах со встроенными элементами и вместе с тем достаточно простой для демонстрации основных функций процессора. MIPS имеет 32 регистра каждый 32-битной длины; следовательно, для выбора регистра необходим 5-битный адрес. Сначала мы рассмотрим три режима ISA MIPS: R-type, I-type и J-type. R-type (тип регистра) считывает командное слово, состоящее из полей, показанных на рис. 2.2а.

Поля в команде следующие:

- *opcode*: это 6-битовое поле используется для декодирования команды операции. Это основной вход в блок управления, который генерирует необходимые управляющие сигналы для команды;
- *rs*: 5-битный адрес первого регистра источника;
- *rt*: 5-битный адрес второго регистра источника;
- *rd*: 5-битный адрес регистра назначения;
- *shamt*: 5-битная величина сдвига;
- *func*: 6-битный код функции. Это поле используется для выполнения подфункции, такой как добавление команды R-типа. Он может использоваться как вход в АЛУ.

Для примера команда: `add $, $14, $10`

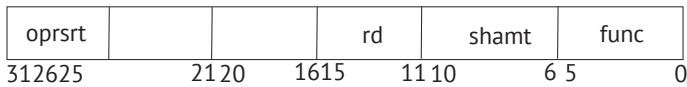
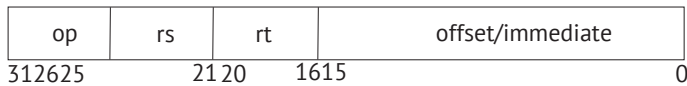
(a)
R-type(b)
I-type(c)
J-type

Рис. 2.2. Режимы команд MIPS

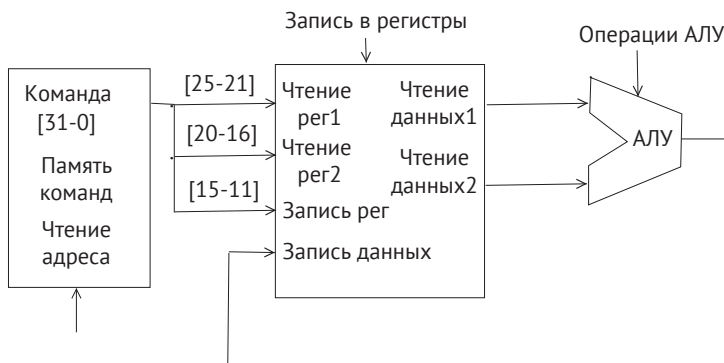


Рис. 2.3. Канал передачи данных MIPS для команды R-type

В двоичной и шестнадцатеричной форме счисления:

op	rs	rt	rd	shamt	func
-----	-----	-----	-----	-----	-----
00 0000	0 1110	0 1010	0 0111	0 0000	10 0000

32-битовая команда в шестнадцатеричной форме счисления будет тогда 0x01CA:3820. Давайте подробнее посмотрим, как эта команда выполняется в простом канале передачи данных, на рис. 2.3.

1. Адрес чтения передается в память команд, и выбрано 32-битное слово команды (0x01CA:3820).

2. Слово команды находится на выходе из памяти команд, и ее биты 11–25 поступают в файл регистров, чтобы выбрать регистры чтения и *регистр записи*.
3. Содержимое *регистров чтения* 14 и 10 появляется на *выходах данных чтения* файла регистров и на входах АЛУ.
4. Операция *add* выбирается в АЛУ посредством выполнения *ALUOp*.
5. Сумма содержимого регистров 14 и 10 появляется на выходе АЛУ и на входе записи данных файла регистра.
6. Активация сигнала *RegWrite* обеспечивает запись суммы в содержимое регистра 7.

Задача блока управления состоит в предоставлении сигналов управления *ALUOp* и *RegWrite* значения слова команды, присутствующего в поле *opcode*. Предоставление этих сигналов в описанной выше последовательности сложное, но в этом и нет необходимости. Мы можем иметь в наличии управляющие сигналы на протяжении всего цикла, и если цикл достаточно длинный, полученный результат будет правильно вычислен при реализации *одного цикла передачи данных*. Определение длины цикла имеет решающее значение в этом методе проектирования, и оно должно быть больше самой длинной команды. Обычно нам нужно записать сгенерированный вывод вместо регистра назначения во внешнюю память данных. Для этой цели используется I-тип адресации в формате, показанном на рис. 2.2, и MIPS имеет две команды для хранения и удаления данных из памяти данных:

```
lw$2, 12($1)
sw$2, 12($1)
```

В этом примере команда загрузки слова (*load word*, *lw*) загружает 32-битное слово из ячейки памяти данных по адресу, указанному содержимым регистра $\$I+12$, в регистр $\$2$, и команда сохранения слова (*storeword*, *sw*) делает обратное, сохраняя содержимое регистра $\$2$ по адресу $\$I+12$ памяти данных. Теперь мы расширим канал данных, чтобы включить I-тип команды, как показано на рис. 2.4. Есть три дополнительных 2:1 мультиплексора, чтобы иметь возможность различать режимы команд. В команде I-типа второе поле регистра теперь содержит адрес записи, поэтому нам нужно выбрать биты (20–16) в качестве этого значения для данного режима путем активации управления *RegDest* первого мультиплексора. Схема *Sign-extend* расширяет 16-битное значение в нижней половине командного слова для добавления значения первого регистра в содержимое адреса, содержащегося в регистре, плюс значение смещения. Второй мультиплексор выбирает операцию R-типа или I-типа путем его входа *ALUSrc*. Наконец, третий мультиплексор выбирает, является содержимое, расположенное в памяти данных (I-тип), или результаты операции АЛУ (R-тип) записанными в файл регистров управляющим сигналом *MemToReg* или нет. Загрузка или запись в память данных выбирается сигналами *MemWrite* или *MemRead*.

Давайте перечислим управляющие сигналы, необходимые для команды *lw* 2, 12(\$1). Нам нужно только ввести адрес команды в память команд без необходимости чтения сигнала, поскольку все операции записи контролируются блоком управления. Код операции для этой команды, 100011, подается на блок управления, который выводит следующие значения сигнала:

- *RegDest* = 0, поскольку мы хотим интерпретировать биты (20–16) слова команды в качестве адреса *writereg*, так как это команда I-типа;
- *RegWrite* = 1, так как нам нужно записать в файл регистра содержимое памяти данных;
- *ALUSrc* = 1, поскольку мы хотим добавить к значению в регистре 1 знак расширенных битов (15–0) слова команды;
- *ALUOp* = 010 в двоичном формате для операции добавления;
- *MemRead* = 1 и *MemWrite* = 0, чтобы разрешить чтение значения, содержащегося в месте, вычисленном ALU;
- *MemToReg* = 1 обеспечивает запись выходного значения из памяти данных в регистр 2 в файле регистра.

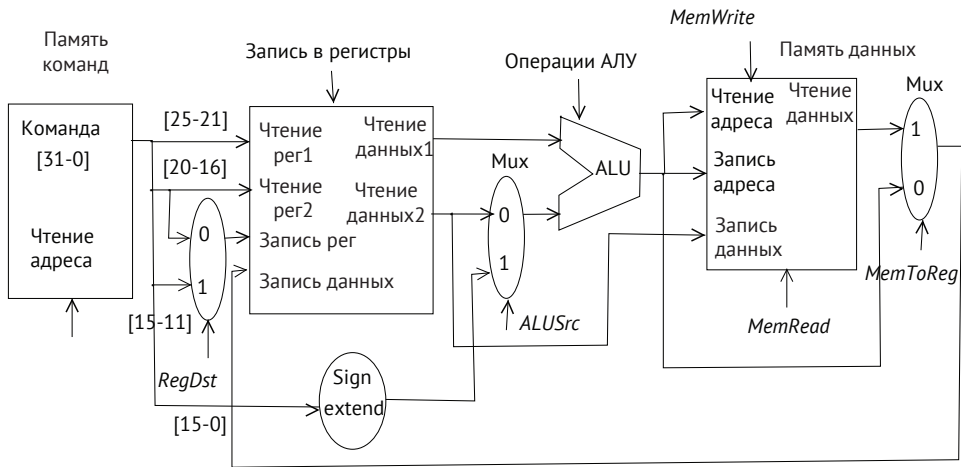


Рис. 2.4. Канал передачи данных MIPS с памятью данных

Поддержание этих значений на протяжении всего цикла блоком управления обеспечивает правильная операция для этой команды. J-тип команды используется для переадресации обычного потока программы на новый адрес команды. Команда *перехода* (branch instruction) в основном используется для перевода оператора *if* на язык высокого уровня и имеет следующий формат:

beq \$1, \$2, offset

Значения в регистрах 1 и 2 сравниваются путем вычитания их значений в ALU, и адрес, рассчитанный по значению указателя команд (instruction pointer, IP) $+ 4 + (4 \times \text{offset})$, загружается в IP, если результат равен 0. Если эти значения не равны или команда не является *ветвью*, значение IP увеличивается на 4, чтобы прочитать следующее местоположение из памяти команд. Теперь у нас есть два сумматора, чтобы можно было работать параллельно со сравнением ALU двух входных регистров 1 и 2; один для расчета $IP + 4$ и еще один для расчета адреса ветви, как показано на рис. 2.5. Обратите внимание, что адрес рассчитывается в любом случае, но мы просто не выбираем его, не активируя сигнал *PCSrc*, если сравнение ветвей не удастся или у нас есть команда R-типа.

Блок управления

Блок управления имеет в качестве его входов 6-битный код операции, 6-битное поле функции задач и 1-битный нулевой выход ALU, и он должен генерировать все необходимые сигналы для каждой команды. Это может быть реализовано с помощью простой комбинационной схемы, имеющей 13-битный вход, который обеспечивает необходимые сигналы управления. Управляющие сигналы для некоторых примеров команд MIPS приведены в табл. 2.1.

Производительность

Команда *загрузки слова* – это самая длительная инструкция в MIPS, поскольку она читает из памяти команд, затем читает из файла регистров, выполняет операцию *add* в ALU, читает из памяти данных, а потом записывает эти данные в файл регистров. Каждая из них, кроме операции ALU, является операцией чтения и записи, приводящей к задержкам в памяти. Работа ALU также занимает время, и сумма этих задержек дает представление о величине необходимого времени цикла. Например, предполагая, что каждый из пяти шагов команды загрузки слова занимает 2 наносекунды и мультиплексоры имеют нулевую задержку, мы получаем время цикла 10 наносекунд и частоту 100 МГц. Вместе с тем многие команды MIPS требуют меньше времени, например команды типа R не используют память данных и, следовательно, выполняются в четыре этапа, требуя времени цикла 8 наносекунд. На самом деле доступ к внешней памяти данных намного медленнее, и потому канал передачи данных с одним циклом имеет низкую производительность.

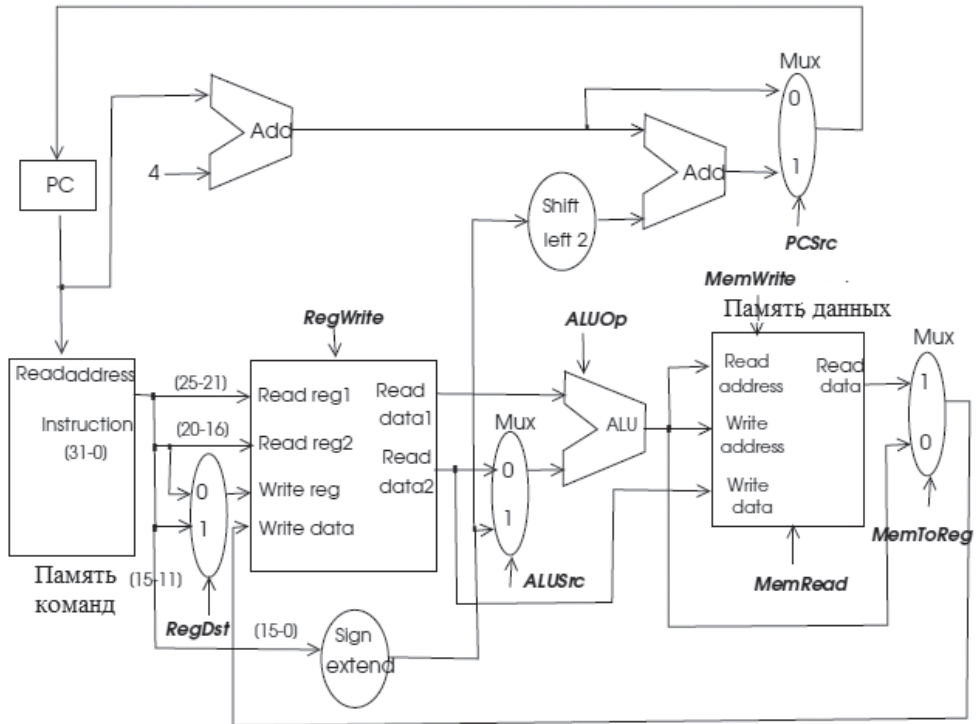


Рис. 2.5. Канал передачи данных MIPS с одним циклом

Таблица 2.1. Управляющие сигналы MIPS для команд

Операция	RegDest	RegWrite	ALUSrc	ALUOp	MemWrite	MemRead	MemToReg
add	1	1	0	010	0	0	0
and	1	1	0	000	0	0	0
lw	0	1	1	010	0	1	1
sw	X	0	1	010	1	0	X
beq	X	0	0	010	0	0	X

2.2.2. Канал передачи данных многими циклами

Путь к данным с одним циклом не эффективен, так как он использует одинаковую продолжительность цикла для всех команд. Наша цель состоит в улучшении реализации одним циклом путем применения переменной продолжительности времени выполнения каждой команды. Начнем с деления команд в MIPS на этапы, приведенные в табл. 2.2, в которых выполнение каждой команды занимает один цикл.

Таблица 2.2. Этапы конвейера MIPS

Этап	Аббревиатура	Описание
Выборка команды	IF	Команда вызывается из памяти
Декодирование команды	ID	Операция декодирования команды
Выполнение	EX	Команда выполняется в АЛУ
Память	MEM	Обеспечение доступа к чтению или записи в память
Чтение	WB	Чтение из памяти данных, записанных в файле регистров

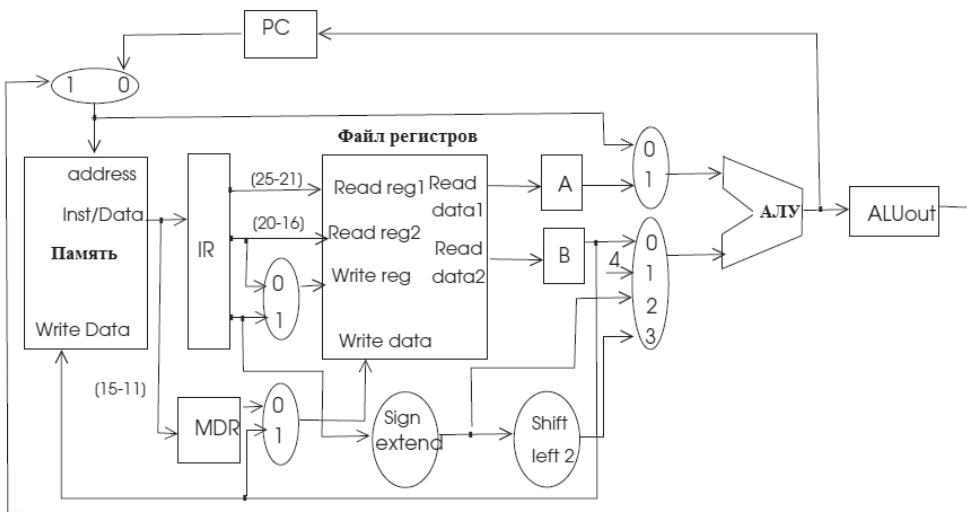


Рис. 2.6. Канал передачи данных MIPS со многими циклами без сигналов управления

Команды теперь занимают несколько циклов, переходы занимают три цикла, команды регистра занимают четыре цикла, и только команда загрузки слова использует все пять указанных выше шагов. Некоторые компоненты канала передачи данных необходимо использовать повторно, что требует изменений в аппаратуре. Теперь нужны дополнительные регистры, чтобы сохранять промежуточные результаты между циклами. Команды и память данных могут быть объединены, и мы можем иметь один ALU, который будет обрабатывать также команды перехода просто потому, что процессор не может использовать эти аппаратные компоненты одновременно, так как они заняты в разных циклах. Новый канал передачи данных многими циклами без управляющих сигналов изображен на рис. 2.6. Вновь введенные регистры: регистр команд (IR), регистр памяти данных (MDR), два регистра A и B на выходе из файла регистров и регистр ALUout для хранения значения, полученного из ALU для обратной записи в память данных.

Блок управления многими циклами может быть реализован с использованием конечного автомата (FSM), который состоит из конечного числа состояний и переходов между состояниями. Неформально говоря, FSM получает

входные данные в некотором состоянии и на основе его текущего состояния и входов производит выходы и может изменять состояние. Каждый этап многих циклов можно представить состоянием автомата, и необходимые на этом этапе сигналы управления будут выходом автомата [1]. Хотя канал передачи данных многими циклами приводит к лучшей производительности, чем одним циклом, описанная далее конвейерная обработка повышает производительность в большей степени и является основой современных процессоров.

2.2.3. Конвейер

Конвейеры в реальной жизни обычно используются для повышения производительности. Рассмотрим систему, предназначенную для стирки и имеющую в составе три машины: стиральную (W), сушилку (D) и укладчик (F). Будем предполагать, что человек должен использовать эти машины последовательно: стиральная машина, сушилка, затем укладчик. Далее давайте предположим, что операции стирки, сушки и укладки занимают по 20 минут каждая. Распределение всех этих машин для человека X, как показано на рис. 2.7а, не имеет смысла, поскольку стиральная машина, например, может использоваться другим человеком Y на стадии сушки человеком X. Конвейер, показанный на рис. 2.7б, позволяет использовать освободившиеся машины другим человеком и уменьшает время процесса стирки для трех человек.

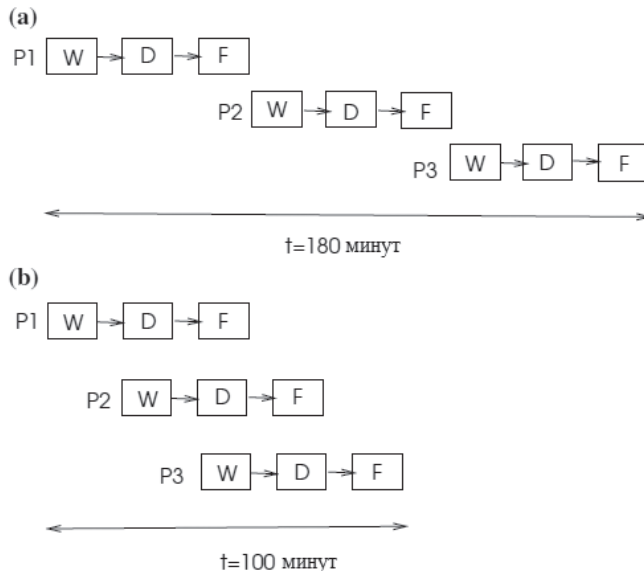


Рис. 2.7. Пример конвейера. Три машины в прачечной используются без конвейера (а), и в результате общее время завершения работы составляет 180 минут. Как показано, конвейер уменьшает общее время завершения работы до 100 минут. Обратите внимание, что конвейерная обработка не уменьшает время для каждого человека, у каждого оно все еще составляет 60 минут, однако общее время выполнения задач уменьшается

Из приведенного примера можно сделать важный вывод: для использования конвейерной обработки необходимо четко указать этапы задачи, и она должна выполняться последовательно. Этими этапами стирки были мойка, сушка и укладка для каждого человека. В терминах процесса вычислений, чтобы команда выполнялась в конвейерном процессоре, нам нужно разделить выполнение на несколько отдельных этапов. Теоретически ускорение, получаемое конвейерным процессором, можно определить следующим образом.

- Пусть k – общее количество этапов конвейера, t_s – время обработки на каждом этапе, а n – общее количество задач для конвейера.
- Первая команда занимает для завершения время kt_s .
- У нас есть команда, выходящая из конвейера в каждом цикле для оставшихся $(n - 1)$ задач, что приводит к общему времени выполнения остальных задач $(n - 1)t_s$.
- Общее время для $(n - 1)$ задач:

$$kt_s + (n - 1)t_s = (k + n - 1)t_s.$$

- Полученное ускорение S представляет собой отношение последовательного времени выполнения n команд без конвейера ко времени выполнения n команд конвейером и составляет:

$$S = \frac{nkt_s}{(k + n - 1)t_s}.$$

При $n \rightarrow \infty$ S приближается к следующему значению:

$$S = \frac{kt_s}{t_s} = k.$$

Можно сделать относительно большее количество этапов, как это делается в некоторых современных процессорах, путем аккуратного разделения промежуточных результатов. Мы будем использовать архитектуру MIPS, чтобы продемонстрировать, как конвейерная обработка может быть реализована в процессоре. У нас есть пять этапов: извлечение команд (Instruction Fetch, IF), декодирование инструкций (Instruction Decode, ID), выполнение (EX), память (Memory, MEM) и обратная запись (Write Back, WB). Команда *lw* выполняет все пять шагов, показанных в табл. 2.2, и это максимальное количество этапов, которое мы можем выполнить. Как короткий сегмент программы сборки в MIPS может работать с конвейерной обработкой, показано на рис. 2.8. Мы просто загружаем слова из двух последовательных ячеек памяти, складываем их и сохраняем результат в третьей последовательной ячейке.

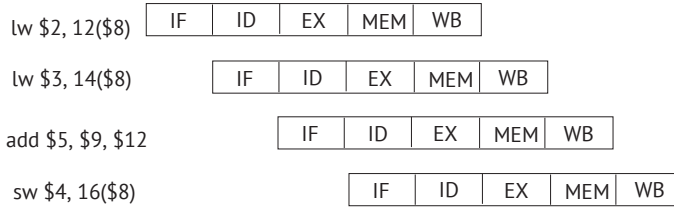


Рис. 2.8. Программа конвейера в MIPS на языке ассемблера

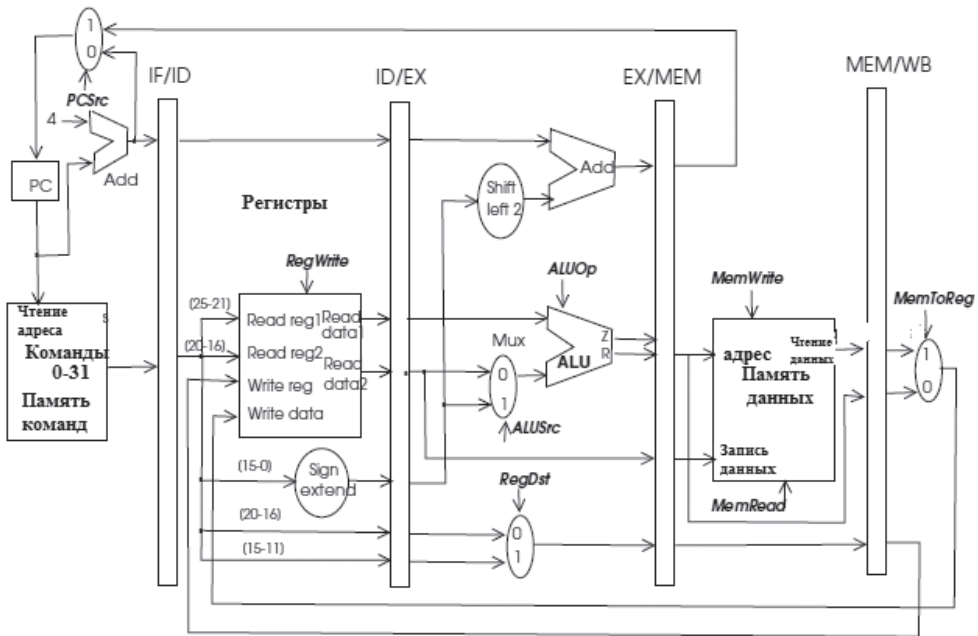


Рис. 2.9. Конвейер в MIPS

В первой простой реализации мы будем использовать временные регистры, которые будут служить буфером для результатов, полученных с помощью предыдущей команды. На рис. 2.9 эти регистры названы IF/ID, ID/EX, EX/MEM и MEM/WB, чтобы показать, между какими этапами они находятся. Эта диаграмма очень похожа на MIPS одного цикла с незначительными модификациями. Например, вычисление команды перехода (beq) может быть выполнено параллельно с вычислением ALU, а вычисление следующего адреса выполняется при получении команды, поскольку у нас есть адресная информация на первом этапе. Значение данных, требуемое на более позднем этапе, должно быть передано между регистрами конвейера. Например, адрес регистра записи передается через все этапы, так как это необходимо на последнем этапе в случае коман-

ды сохранения слова (sw). Обратите внимание, что инструкция проходит одну стадию в каждом цикле, требующем пяти циклов для всей команды.

Сигналы управления

Необходимые для команды сигналы управления должны передаваться вместе с командой, используя регистры конвейера. На каждом этапе нужны разные управляющие сигналы, и как только команда проходит этап, от управляющих сигналов, необходимых для этого этапа, можно отказаться, что приведет к уменьшению количества сигналов, передаваемых через этапы по мере выполнения команды, как показано на рис. 2.10.

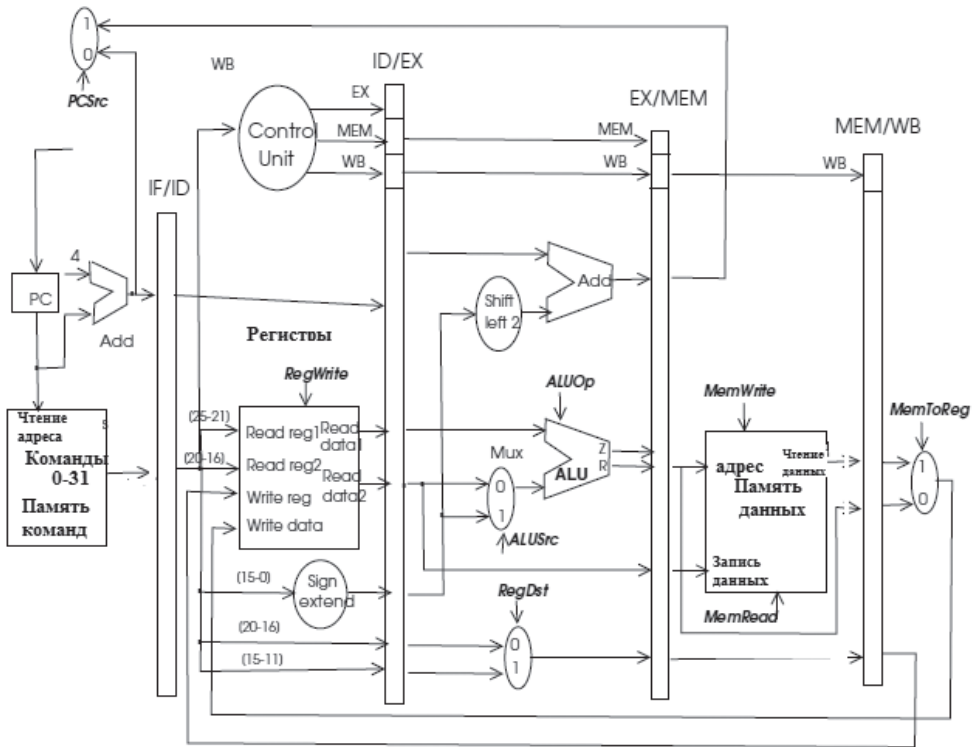


Рис. 2.10. Конвейер в MIPS с сигналами управления

2.2.3.1. Риски

Конвейерная структура процессора повышает эффективность; однако мы предполагали, что команды, которые одна за другой подаются на этапах конвейера, независимы. Команды, которые следуют друг за другом, могут быть зависимыми, поскольку значения созданного регистра используются в одной из следующих команд, что создает опасности.

Опасности для данных

Рассмотрим следующие строки кода MIPS:

```
lw $3, 10($2) Первый пример опасности для данных
\
and $4, $3, $5

add $3, $1, $2 Второй пример опасности для данных
|
sw $3, 12 ($4)
```

Первое слово загрузки команд (*lw*) загружает значение в ячейке данных (\$2)+10 в регистр 3, который выполняется на заключительном и пятом этапах обратной записи (WB). Однако последующая команда требует значения в регистре 3 при выполнении в третьем цикле (EX). Эта ситуация в конвейерном процессоре называется *опасностью для данных (datahazard)*. Возможным решением может быть пересылка значения \$3, которое выбирается из памяти в памяти (MEM) четвертого этапа загрузки слова команды *and* и которое нуждается в этом значении на втором этапе декодирования (ID) команды. Этот тип пересылки приводит к передаче на два этапа назад результатов, поступающих на регистр. Чтобы выполнить операцию правильно, нам нужно остановить процессор на один цикл между этими двумя командами. Остановка процессора означает остановку его работы на один цикл путем перестановки значений в требуемых регистрах конвейера.

Во втором примере, приведенном выше, инструкция *storeword (sw)* нуждается в правильном значении регистра 3 на четвертом этапе MEM, и, следовательно, это значение может быть передано с помощью инструкции *add* на втором этапе EX без остановки процессора. Когда и куда передавать значения регистра, может определять *блок пересылки*, представляющий собой структуру, которая сравнивает значение регистра назначения (*rd*) в конце этапа EX (регистр конвейера EX/MEM), а также в конце этапа MEM (регистр конвейера MEM/WB), со значениями первого (*rs*) и второго (*rt*) значений регистра в конце этапа ID (регистр конвейера IF/ID). Всякий раз, когда найдено совпадение, значение регистра передается на один или два этапа назад в зависимости от того, где найдено совпадение. Подробное описание операции пересылки в MIPS приведено в [1]. Сравнения, которые должны быть выполнены в блоке пересылки, состоят в следующем:

- EX/MEM.rd = ID/EX.rs
- EX/MEM.rd = ID/EX.rt
- MEM/WB.rd = ID/EX.rs
- MEM/WB.rd = ID/EX.rt

Канал передачи данных MIPS с блоком пересылки изображен на рис. 2.11. Этот блок в основном сравнивает входные данные идентификатора регистра

назначения (*rd*) этапа MEM с входными регистрами этапа EX (*rs* и *rt*) и регистра назначения (*rd*) идентифицируемого входа на этапе WB с входами регистров этапа EX (*rs* и *rt*). Если совпадение найдено, значение *rd* возвращается на один или два этапа назад. Эта передача назад значения *rd* достигается выбором 1 (2 этапа назад) или 2 (1 этап назад) входов мультиплексоров, присутствующих теперь на входе ALU.

Опасности для управления

Последовательность инструкций может перенаправиться в другое место после ветки (*beq*) инструкции, которая сравнивает значения в двух регистрах и переходит в указанное место в инструкции, если эти значения одинаковы. Инструкции извлекаются на первом этапе (IF), и решение филиала принимается на третьем этапе (EX). Рассмотрим следующий сегмент кода MIPS:

```
beq $1, $2, 16($6)
and $4, $3, $5
add 7, $8, $9
```

Переход может или не может быть зависимым от значений в регистрах 1 и 2, и это решается при операции в ALU на этапе EX, в котором будут вычитаться значения в этих регистрах, и флаг нулевого результата, установленный в конце операции, означает, что адрес команды должен быть изменен. Если переход сделан, то команды *and* и *add*, которые выполняются на этапах ID и IF соответственно, следует исключить. Исключение этих этапов путем переустановки регистров конвейера ID/IF и ID/EX является одним из решений данной проблемы за счет потери циклов CPU. Основные решения проблемы опасности для управления конвейерными процессами следующие:

- *остановка процессора*: остановка конвейера до тех пор, пока результат завершения операции ветвления не будет известен;
- *слоты задержки перехода*: несколько команд, которые следуют за командой перехода, всегда приняты. Компилятор или программист ставит полезные команды или отсутствие операций (NOP) в эти слоты. Для процессоров с большим количеством конвейеров это решение не слишком удобно;
- *предсказание перехода*: результат команды перехода предсказывается с использованием некоторой эвристики. *Статичное предсказание перехода* может либо всегда реализовывать переход, либо нет. Улучшение достигается на основе предположения, что скачки для цикла переход назад / переход вперед не являются более вероятными скачками цикла и будут приняты большую часть времени, а прыжки вперед выполняются редко. Динамическое предсказание переходов использует текущий статус для принятия решения, осуществлять переход или нет.

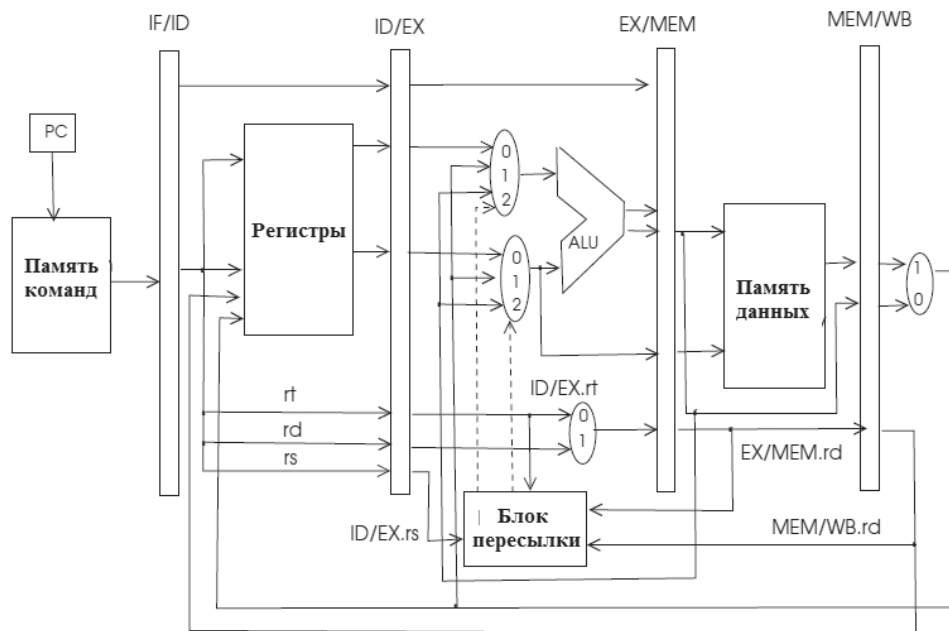


Рис. 2.11. Блок пересылки MIPS

2.2.4. Микроконтроллеры

Микроконтроллер имеет процессор, память и интерфейс ввода/вывода в одной микросхеме. Микроконтроллеры обычно используются для маломощных и малогабаритных встроенных приложений. Программируемая логическая матрица ПЛИС (field-programmable gate array, FPGA) представляет собой динамически реконфигурируемый микроконтроллер. Для создания системы реального времени разработчик приложения может использовать ПЛИС. Эти процессоры обычно используются в системах связи и системах обработки изображений. Процессоры цифровых сигналов (digital signal processor, DSP) – микропроцессоры специального назначения, используемые для обработки сигналов, которые требуют частых арифметических операций с большими объемами данных.

2.3. Память

Память наряду с вводом/выводом является основным узким местом, ограничивающим скорость передачи данных в компьютер. Всегда существует необходимость в большем, а также быстром доступе к данным. В современных компьютерах требуется большое хранилище, потому что данные доступны в беспрецедентно больших количествах, и размер доступных данных увеличивается ежедневно. С другой стороны, если доступ к памяти не достаточно быст-

рый, мы не можем использовать все возможности высокоскоростного процессора. Статическая оперативная память (SRAM) является быстрой и дорогой, тогда как динамическая RAM (DRAM) того же размера более чем на порядок дешевле SRAM, но намного медленнее. Однобитная ячейка SRAM состоит из двух вентилях, тогда как однобитное значение хранится в DRAM в виде заряда на конденсаторе. Стираемая программируемая память (EPROM) сохраняет программный код и данные, когда питание отключено. Жесткий диск может хранить данные порядка терабайтов, и хотя это самая дешевая память в иерархии устройств памяти, доступ к нему очень медленный. Нам нужна память с быстрым и эффективным способом доступа.

2.3.1. Интерфейс процессора

Среда связи между процессором и памятью состоит из трех шин: адресной шины для определения местоположения адреса данных, информационной шины для передачи данных и шины управления для операций управления. Типичными управляющими сигналами для оперативной памяти (RAM) являются программные строки чтения (RD) и записи (WR), которые позволяют выполнять операции чтения и записи из памяти и в память соответственно, как показано на рис. 2.12. Памяти EPROM нужен только сигнал RD, так как запись не может быть сделана во время нормальной работы. Для n -битных адресов и m -битных данных строк программы процессор может адресовать 2^n областей памяти, каждая из которых имеет длину m бит. Старшие биты адресной шины обычно используются для выбора памяти и устройства, которое подключено к шине. Например, 10-битная адресная шина (A_9 – A_0) и 8-битная шина данных позволяют адресовать 1024 ячейки памяти. При делении пространства в 512 байт EPROM и 512 байт RAM мы можем использовать A_9 в качестве входа в память микросхемы. В этом случае данные в диапазоне адресов 0–511 находятся в EPROM, а данные в адресах 512–1023 находятся в RAM. Обратите внимание, что выбор входа микросхемы EPROM является активным низким ($\overline{CS}$), чтобы активировать его, когда $A_9 = 0$. Современные процессоры обычно связаны шинами с блоками памяти и интерфейсами ввода/вывода в иерархические структуры.

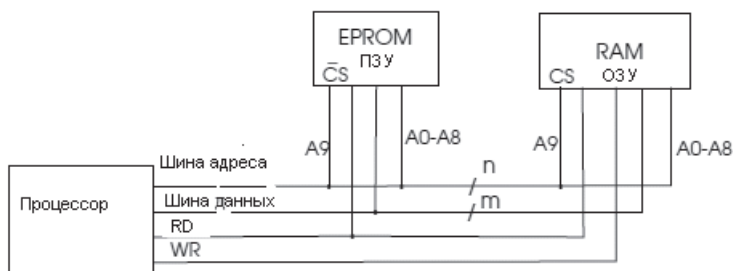


Рис. 2.12. Интерфейс памяти в процессоре

2.3.2. Кеш

Кеш – это статичная память RAM небольшого размера, размещенная между процессором и динамической оперативной памятью для обеспечения быстрого доступа к командам или данным. Идея памяти кеша заключается в том, чтобы сохранять в ней наиболее часто используемые данные и возвращать их из более близкого места, что быстрее, чем возвращать их из памяти DRAM. Принцип *временной локализации* (*temporal locality*) данных предполагает, что доступ к адресу будет с высокой вероятностью снова обрабатываться процессором, как в случае с переходами. С другой стороны, принцип *пространственной локальности* (*spatiallocality*) предполагает, что соседние адреса уже посещенных мест имеют высокий шанс доступа, что является серьезным аргументом, так как программа большую часть времени выполняется последовательно. Обратите внимание, что цикл также имеет пространственную локальность, поскольку последовательность строк цикла выполняется несколько раз, пока цикл не завершится. *Попадание в память кеша* (*cachehit*) – это ситуация, когда кеш содержит данные, которые мы ищем, в противном случае происходит *кеш-промах* (*cachemiss*) – отсутствие затребованных данных в кеше. Производительность кеша измеряется *частотой обращений* (*hit-rate*), которая является процентом обращений к кешу от общего доступа к памяти. Время, необходимое для отправки данных из кеша в процессор (*hittime*), время обращения и время, необходимое для передачи данных из основной памяти в кеш, когда происходит промах, является *штрафом за промах* (*misspenalty*). Процент промахов – это *частота промахов* (*missrate*), и основанное на этих параметрах *среднее время доступа к памяти* (*average memory access time*) для компьютера с кеша равно времени попадания + (частота промаха × штраф за промах).

Принцип пространственной локальности может быть использован путем копирования блока данных, который содержит соседнее расположение, а также требуемое расположение для кеша. В соответствии с этим принципом данные считываются и записываются в кеш в блоках, каждый из которых имеет *индекс* (*index*). Предполагая, что кеш имеет 2^k блоков, нам нужен индекс длиной k бит для адресации блока. Всякий раз при обращении (*addr*) к ячейке памяти младшие k бит *addr* могут быть использованы для поиска соответствующего блока в памяти кеша. Кроме того, нам нужно проверить, точно ли искомый адрес соответствует адресу памяти кеша. Блок содержит значение *тега* для сопоставления с оставшимися битами адреса; и, следовательно, если значение тега блока равно оставшимся битам старшего порядка адреса, у нас есть попадание. У нас также есть *бит достоверности* (*validbit*) для каждого блока кеша для индикации того, есть в блоке достоверные данные или он пуст. Для примера давайте рассмотрим основную память с 12-битными адресов, которая делится на 4-байтовые блоки, что дает 4 КБ памяти. Каждый блок может быть адресован 6 битами; поэтому необходим кеш с 6-битным индексным значением и 4-битным значением тега, как показано на рис. 2.13. Поскольку каждый блок имеет 4 байта, младшие 2 бита адреса памяти используются как

плексора 4:1, как показано на рис. 2.13. Этот кеш называется *кеш с прямым отображением* (*direct-mapped cache*), в котором каждое расположение основной памяти точно отображает одно расположение блока кеша, которое может быть рассчитано из физического адреса памяти. Данные, переданные из памяти, могут быть записаны в любое неиспользуемое место в *полностью ассоциативном кеше* (*fully associative cache*). Этот тип кеша требует проверки всех мест кеширования параллельно с использованием многих компараторов, и, следовательно, это дорого. Наборно-ассоциативный кеш (*set-associative cache*) является промежуточным решением между кешем с прямым отображением и полностью ассоциативным кешем. Кеш теперь состоит из групп блоков, называемых *наборами*, и каждая ячейка памяти отображается на один такой набор, но расположение данных внутри набора является переменным. Размещение данных в блоке похоже на полностью ассоциативный кеш, позволяющий искать данные в меньшей области кеша. Ассоциативный набор называется *k-ассоциативным кешем* (*k-way associative cache*), если он имеет *k* блоков в наборе.

Еще одна проблема, вызывающая беспокойство, – это выбор данных для удаления из кеша, когда извлекаются новые данные из памяти, а кеш заполнен. Данные, которые долго остаются в кеше неиспользованными, выбираются *методом замещения блока данных с наиболее длительным отсутствием обращений* (*least recently used, LRU*). Дважды связанный список может быть использован для сохранения отслеживания использования данных в кеше. Новые справочные данные помещаются в начале списка, а удаляемые данные берутся из конца списка. Различные другие политики, такие как «*первым пришел – первым обслужен*» (*first-in-first-out, FIFO*) и «*пришел последним – первым обслужен*» (*last-in-first-out, LIFO*), также могут быть использованы для принятия решения о данных, которые должны быть удалены как первый и последний доступные блоки соответственно.

Когда процессор записывает данные в кеш, он также должен записывать данные в основную память для согласованности. *Политика записи* (*write policy*), которая определяет, как выполняется эта процедура, следующая:

- *сквозная запись* (*Write Through*): одновременная запись в кеш и основную память;
- *обратная запись* (*Write Back*): запись в кеш с последующей записью модифицируемых блоков в основную память.

2.4. Доступ к вводу/выводу

Процессор обменивается данными с внешними устройствами, используя порты ввода и вывода, которые в основном являются буферами данных для взаимодействия с шиной данных процессора. Устройства ввода/вывода (I/O) бывают разных форм, от клавиатуры до принтера. Устройства ввода/вывода на много порядков медленнее по сравнению с процессором и памятью. Диски и сети – это две основные системы ввода/вывода, которые часто взаимодей-

ствуют с процессором. Для увеличения скорости передачи данных с/на устройства ввода/вывода в аппаратном обеспечении может быть использован параллелизм. Например, избыточный массив недорогих дисков (redundant arrays inexpensive/independent disks, RAID) системы обеспечивает параллельный доступ к нескольким жестким дискам одновременно, и память можно разделить на банки, к которым осуществляется параллельный доступ. Мы рассмотрим в этом разделе сначала общий интерфейс с устройствами ввода/вывода в системах реального времени, а затем опишем методы интерфейса ввода/вывода в аппаратном и программном обеспечении.



Рис. 2.14. Вход интерфейса системы реального времени

2.4.1. Интерфейс устройства ввода

В системах реального времени преобладают два типа устройств ввода/вывода: датчики и исполнительные механизмы. Датчик представляет собой устройство ввода, способное воспринимать внешний физический параметр и создавать электрический сигнал, представляющий этот параметр. Есть много типов датчиков, таких как датчики температуры, влажности, ускорения и давления. Типичные параметры для датчика – его рабочая температура, технические характеристики ошибки и диапазон чувствительности.

Входной интерфейс процессора содержит электрические схемы для преобразования сигнала от датчика до формы, пригодной для обработки компьютером реального времени. Выходной сигнал от датчиков обычно небольшой по величине в диапазоне милливольт и нуждается в усилении аналогового сигнала, а затем использует аналого-цифровой преобразователь, который преобразует электрический сигнал в цифровые данные. Датчики могут иметь необходимые встроенные преобразователи. Структура типичного интерфейса ввода, использующего аналого-цифровой преобразователь (АЦП), показана на рис. 2.14. Усилитель (amplifier, AMP) увеличивает напряжение, получаемое от датчика, которое обычно имеет порядок милливольт, а схема *выборки и удержания* (sample and hold, S&H) обеспечивает дискретизацию сигнала, подаваемого на аналого-цифровой преобразователь, и стабильность преобразования. Количество бит, выводимых аналого-цифровым преобразователем, отражает точность вводимого сигнала. Для типичного диапазона аналогового входа 0–5 В 32-разрядный аналого-цифровой преобразователь выдает цифровые данные в диапазоне 0–65 535.

Детальный типовой интерфейс аналого-цифрового преобразователя к процессору показан на рис. 2.15. Процессор инициирует преобразование по им-

пульсу *запуска*, когда ему нужно ввести данные датчика. В зависимости от характеристик аналого-цифрового преобразователя этот импульс может быть с активным высоким или активным низким уровнем, а иногда просто переходным уровнем между высоким и низким уровнями. Порождающий импульс, генерируемый процессором, просто выполняется записью 0, а затем 1, и снова 0, чтобы начать бит с выходного порта. Задержка после каждой записи необходима для формирования импульса. Ширина импульса должна быть больше минимальной ширины импульса, необходимой для аналого-цифрового преобразователя, и этот параметр можно регулировать по длине задержки, которая обычно достигается путем загрузки регистра величиной и уменьшением ее до 0. Выход из преобразователя доступен, когда бит конца преобразования (*end-of-conversion*, EOC) установлен. Этот сигнал может быть подключен к входному порту процессора, который проверяется непрерывным опросом этой строки или может использоваться для прерывания процессора.

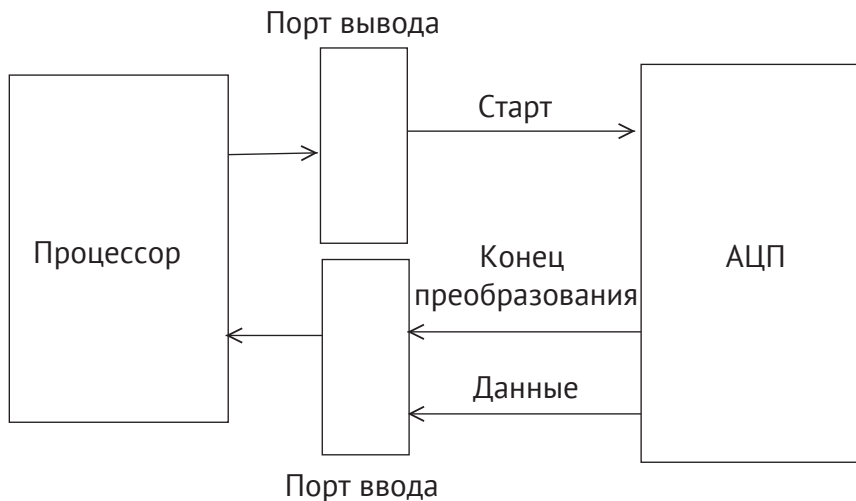


Рис. 2.15. Аналого-цифровой интерфейс

2.4.2. Интерфейс устройства вывода

Привод работает противоположно датчику, преобразуя электрический сигнал к некоторой форме физического параметра, такого как звук, тепло или движение. Например, соленоид – это исполнительный механизм, который создает магнитное поле при прохождении через него тока. Чтобы запустить привод, двоичные выходные данные сначала преобразуются в аналоговый

сигнал с помощью цифроаналогового преобразователя (ЦАП). Этот сигнал может быть преобразован в удобную форму с помощью схемы формирования сигнала для активации привода, как показано на рис. 2.16. Формирование сигнала может включать в себя усилитель и схему для ограничения значения аналогового сигнала. Его рабочая температура и диапазон уровней выходов могут представлять основные характеристики привода.

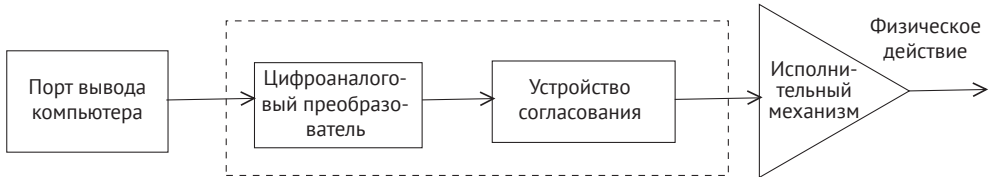


Рис. 2.16. Интерфейс вывода системы реального времени

2.4.3. Отображение в память и изолированный ввод/вывод

Связь между процессором и периферийными устройствами ввода/вывода может быть выполнена с использованием двух основных методов: ввод/вывод с отображением в память (*memory-mapped I/O*) и *изолированный ввод/вывод (isolated I/O)*. В первом методе адресное пространство делится на области памяти и периферийные устройства ввода/вывода. Доступ к модулю ввода/вывода осуществляется так же, как и к физическому адресу памяти с помощью блоков ввода/вывода, подключенных к общей шине. Как показано на рис. 2.17, процессор отправляет адрес блока ввода/вывода и данные по общей шине к периферийным устройствам ввода/вывода, подключенным к шине, которые при обращении к ним реагируют. Метод с изолированным вводом/выводом обеспечивает четкий ввод инструкций, которые могут использовать то же адресное пространство, что и память. В этом случае процессор должен подавать отдельный управляющий сигнал, который показывает, является данная операция памятью или операцией ввода/вывода. Изолированный ввод/вывод может быть удобен, когда существует много периферийных устройств ввода/вывода и, соответственно, их адресов, поскольку для операций ввода/вывода может использоваться одно и то же адресное пространство. С другой стороны, отображаемый в память ввод/вывод является более гибким, поскольку все команды, используемые для операций с памятью, могут быть использованы для операций ввода/вывода. Типичным управляющим сигналом, используемым в изолированном вводе/выводе, является сигнал ввода/вывода *IO/M* в линейке процессоров компании Intel, который при активации показывает команду ввода/вывода, и память в противном случае, как показано на рис. 2.17.

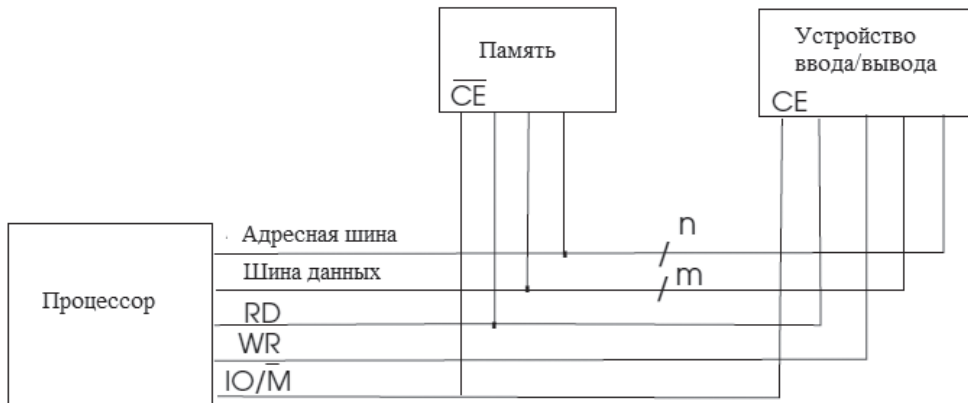


Рис. 2.17. Интерфейс изолированного ввода/вывода

2.4.4. Программный интерфейс ввода/вывода

Обсудив, как периферийные аппаратные устройства ввода/вывода могут быть подключены к процессору, мы можем рассмотреть основные методы программного обеспечения для доступа к модулям ввода/вывода. Процессор считывает некоторые данные с медленных устройств ввода и записывает на медленные устройства вывода. Два основных метода программного интерфейса к устройствам ввода/вывода – это методы опроса и управляемого прерывания.

2.4.4.1. Опрос

Опрос – это основной метод программного обеспечения интерфейса для устройства ввода/вывода, в котором процессор отправляет запрос устройству и постоянно проверяет, выполнен ли запрос. Типичная запись файла процессором на жесткий диск с использованием этого метода показана в алгоритме 2.2. Процессор отправляет запрос на запись на диск, и диск генерирует управляющий сигнал, когда он готов с головкой диска в правом секторе и т. д. Сигнал постоянно проверяется процессором, и когда он активен, передача файла инициируется. Очевидным недостатком этого метода является потеря циклов процессора в ожидании медленного устройства.

Алгоритм 2.2. Запись на диск

- 1: **отправить** требование *записи* на диск
 - 2: **пока** диск не готов записывать
 - 3: *ждать*
 - 4: **end, while**
 - 5: **while NOT end файл do**
 - 6: *передать* блок файла из памяти на диск
 - 7: **end while**
-

2.4.4.2. Ввод/вывод с прерыванием

Прерывание инициируется внешним источником, и когда это происходит, процессор останавливает текущую обработку. Он обслуживает прерывание, а затем продолжает остановленное задание. Этот тип операции аналогичен примеру в повседневности, когда человек читает книгу и раздается телефонный звонок. Человек помещает в книгу закладку (сохраняет текущую среду), отвечает посетителю (обслуживает прерывание), а затем продолжает чтение. Обработка прерывания процессором аналогична действиям в этом примере; текущая среда, состоящая из значений регистров, данных, файловых указателей и т. д., сохраняется, запрошенное прерыванием действие выполняется, и затем среда восстанавливается, и обработка продолжается с той позиции (адреса), в которой она была остановлена. Вся эта обработка обычно осуществляется операционной системой.

Возвратимся к примеру с диском. Процессор делает запрос на запись, как и раньше, но продолжает другую обработку до тех пор, пока прерывание диска не покажет, что он готов к получению информации. Прерывание затем обслуживается записью на диск, и когда этот процесс заканчивается, остановленная задача продолжается. Программа обработки прерываний может быть или не быть прерванной. Все эти процессы находятся под управлением операционной системы реального времени и могут иметь такие вариации, как наличие *программы многослойной обработки прерывания (multi-layered interrupt service routine, ISR)* с разделами кода с высоким и низким приоритетом, в которых часть с высоким приоритетом подается в первую очередь.

Процессор обычно имеет контакт прерывания (часто обозначаемый как *INT* или *INT*), который может быть активирован внешним устройством. Эта линия проверяется во время каждой команды процессора, и если она найдена активированной в предопределенном месте, активируется служебная программа. Как правило, процессор имеет более одного контакта активации прерывания вывода, один из которых обозначен как *немаскируемое прерывание (non-maskable interrupt)*, а другой, который может быть отключен программным обеспечением, – *маскируемое прерывание (maskable interrupt)*. Обслуживание прерывания может осуществляться с использованием разных подходов. В методе *опроса прерываний (interrupt polling)* все запросы устройства прерывания передаются на вход процессора *INT*, и когда эта линия активирована, процессор прочитывает регистр состояния каждого устройства через общую шину или чаще проверяет входной порт, чтобы найти источник прерывания, как показано на рис. 2.18.

У нас есть три устройства, подключенных к процессору, и устройство, запрашивающее сервис, активирует свою линию запроса *REQ* и помещает 1 в линию связи с входным портом процессора. Процессор обнаруживает активную линию порта, когда линия *INT* становится активной, как показано в алгоритме 2.3, и переходит в область памяти, где хранится устройство с программой *ISR*. Если одновременно происходит более одного прерывания, может использоваться метод приоритета.

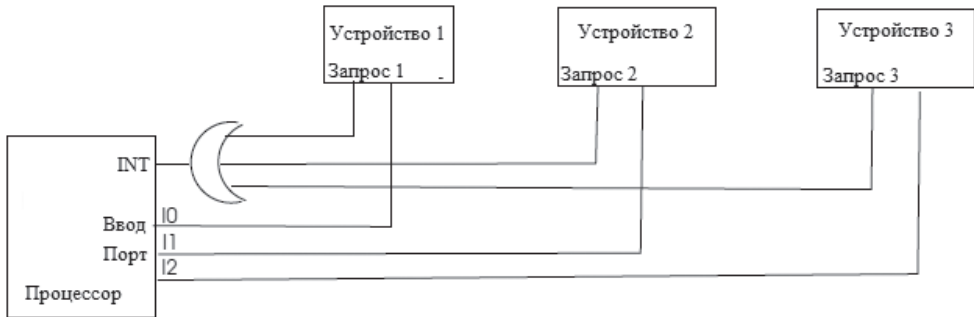


Рис. 2.18. Прерывание опросом

Алгоритм 2.3. Опрос ISR

```

1:
2: активация INT
3: вход в порт 1 по запросу 1
4: счет ← 0
5: загрузка 1 в регистр 2
6: while счет < 3 do
7: сложить регистр 1 с регистром 2 и запомнить в регистре 3
8: if регистр 3 ≠ 0, then
9: перейти к местоположению ISR база + счет
10: exit
11: end if
12: регистр сдвига 2 делает один
13: подсчет ← счет + 1
14: end while

```

Другой и более быстрый подход – метод векторного прерывания, в котором источник прерывания активирует вход INT и помещает адрес своего ISR на входной порт или шину данных процессора. Заметим, что задача обслуживания прерывания в этом случае проще. Процессор просто загружает новый адрес команды из содержимого шины. В более практическом подходе устройство, запрашивающее обслуживание, устанавливает свое смещение из общей таблицы прерываний, что приводит к упрощению интерфейса с процессором. Например, пяти входных битов будет достаточно для 32 устройств.

2.4.4.3. Прямой доступ к памяти

Такие устройства, как диски и сети, работают быстрее, чем такие устройства, как клавиатуры, и они требуют передачи больших блоков данных. Прямой доступ к памяти (direct memory access, DMA) – метод, позволяющий передавать большие блоки данных на такие устройства со скоростью, с которой они работают. Контроллер DMA является интерфейсным процессором между процессором и устройством и связывается с процессором для передачи данных. Как

правило, процессор предоставляет начальный адрес памяти, количество байтов для передачи и направление передачи. Процессор должен будет воздерживаться от любых внешних операций, когда шины заняты передачей. В режиме *захвата циклов* (*cycle stealing*) методом DMA контроллер DMA занимает шины для передачи всех байтов, когда шины простаивают. В другом случае шины контролируются исключительно контроллером DMA до тех пор, пока передача не завершена в монопольном режиме (*burst mode*).

2.4.4.4. Исключения

Исключением являются внутренние прерывания, которые обнаруживаются процессором. Незаконные команды, попытка деления на 0 и арифметическое переполнение являются частыми причинами исключения. В самой простой форме программа, которая вызвала исключение, будет остановлена и удалена из памяти операционной системой. Однако есть случаи, когда восстановление возможно. Обычно, когда обнаружено исключение, процессор уведомляет операционную систему о причине исключения и команду, которая вызвала его, и она решает проблему. Процессор MIPS имеет *счетчик программ исключений* (*exception program counter, EPC*), который содержит адрес команды, выполнявшейся при возникновении исключения. Регистр причины (*cause register*) имеет биты, помогающие идентифицировать причину исключения. Когда прерывание или исключение происходит в MIPS, выполняется следующее:

- 1) текущий программный счетчик передается в EPC;
- 2) причина исключения запоминается в регистре причины;
- 3) исключения и прерывания отключаются путем модификации регистра;
- 4) осуществляется передача адреса обработчику исключений/прерываний.

При возвращении от обработчика содержимое регистра EPC перемещается в счетчик программ, и прерывания и исключения включаются путем изменения статуса регистра.

2.4.4.5. Таймеры

Таймер обычно используется в системе реального времени для измерения временного интервала. Он загружается на величину, соответствующую интервалу, и с течением времени через определенные промежутки эта величина уменьшается. Когда установленная на таймере величина уменьшается до нуля, генерируется прерывание процессора. Таймер также может быть настроен для отсчета от нулевого значения до некоторой величины, и когда время интервала заканчивается, таймер генерирует бит переполнения, который может использоваться для активации линии прерывания процессора.

Этот тип прерывания по таймеру удобен в системе реального времени, в которой основной целью является соответствие времени выполнения задач заданным срокам. Таймеры в системах со встроенными элементами обычно называются сторожевыми таймерами (*watchdog timer*), которые используются для сброса или запуска процесса восстановления при возникновении сбоя

в системе. Процессор регулярно сбрасывает счетчик сторожевого таймера с более короткими интервалами, чем время, необходимое сторожевому таймеру для переполнения. Следовательно, если этот таймер переполняется, то существует аппаратная или программная ошибка, и бит переполнения может использоваться для прерывания процессора, чтобы начать процесс восстановления. Таймер прерывания находит также много применений в системах не в реальном времени.

2.5. Многоядерные процессоры

Многоядерный процессор имеет два или более процессора, встроенных в один чип для повышения производительности. Многие современные процессоры являются многоядерными. Многоядерный процессор может иметь однородные/симметричные процессоры ядра, как в случае общего центрального процессора, или может иметь гетерогенные/асимметричные ядра, как в графическом процессоре. Ядра используют иерархию кешей для повышения производительности. Кеши L1 и L2 обычно используются ядром, а кеш L3 распределяется между ядрами. На рис. 2.19 изображен четырехъядерный процессор с кешами L1, предназначенными для команд и данных.

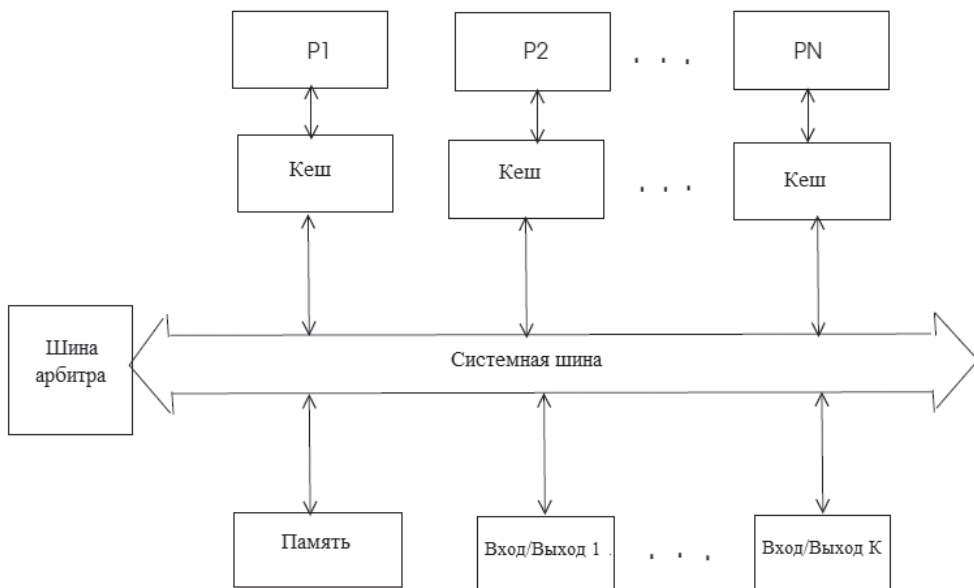


Рис. 2.19. Четырехъядерный процессор

Многоядерный процессор повышает производительность в основном за счет параллельного выполнения задач в ядрах. Используемая мощность снижена по сравнению с двумя или более процессорами в отдельных чипах, что продлевает

срок службы батареи. Однако совместное использование ресурсов, таких как кеш и шины, требует защиты от одновременного доступа. Кроме того, задачи, обычно называемые потоками или тредами (treads), работающие на ядрах, должны быть синхронизированы, подобно задаче синхронизации в общей операционной системе, работающей на одноядерном процессоре.

2.6. МУЛЬТИПРОЦЕССОРЫ

Мультипроцессор имеет два или более процессоров, которые совместно используют память и устройства ввода/вывода. *Симметричная многопроцессорная система* (*symmetric multiprocessor, SMP*) имеет однородные процессоры, которые разделяют глобальную память. SMP-систему также называют *сильно связанной многопроцессорной системой* (*tightly coupled multiprocessor system*), так как часто обмен данными между процессорами осуществляется с использованием общей памяти, в отличие от распределенной системы, где основной метод передачи через обмен сообщениями. Система SMP с n процессорами и k устройствами ввода/вывода, подключенная к общей шине с *арбитром шины* (*bus arbiter*), который управляет использованием шины, показана на рис. 2.20. Многопроцессорные вычислительные системы можно разделить на две основные ветви следующим образом:

- *системы с одной командой и несколькими данными* (*Single-Instruction-Multiple-Data, SIMD*): в этой модели один блок управления управляет несколькими каналами передачи данных. Для достижения высокой производительности в компьютерах SIMD одна и та же команда выполняется параллельно со многими элементами данных. SIMD-компьютеры имеют специальное оборудование и используются для научных расчетов, требующих больших объемов данных, особенно при выполнении матричных операций;
- *системы с несколькими командами и несколькими данными* (*Multiple-Instruction-Multiple-Data, MIMD*): эти мультипроцессоры обмениваются данными в централизованных системах памяти и общаются с помощью сообщений в многопроцессорной модели с *распределенной памятью* (*distributed memory*).

Когда коммуникационная среда в распределенной памяти многопроцессорной системы является компьютерной сетью, мы имеем распределенную систему и обычно в этом случае название «многопроцессорный» не используется. Многопроцессорная система с централизованной памятью чаще всего имеет одну операционную систему, которая решает основные задачи управления системой. Этими основными функциями являются функции планирования и управления задачами в многопроцессорных системах, межзадачная синхронизация, а также связь и защита общих ресурсов, таких как глобальная память при одновременных доступах.

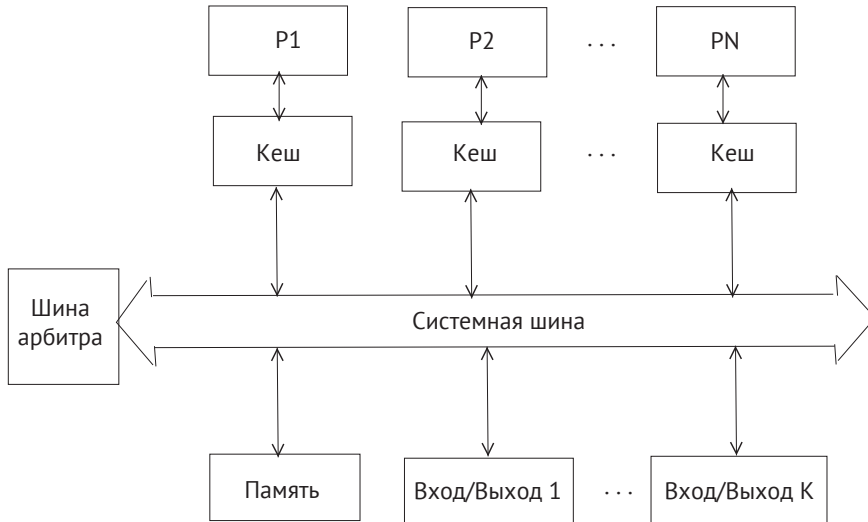


Рис. 2.20. SMP с n процессорами $P1, \dots, PN$ и k I/O-устройствами $1, \dots, I/O K$

2.7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Назовите основные компоненты процессора.
2. Что такое стандартная промышленная архитектура (ISA), и каково ее значение?
3. Если бы вы разрабатывали процессор, какой была бы ваша отправная точка?
4. Какова основная идея одиночного цикла передачи данных?
5. Как бы вы вычислили продолжительность одного цикла в канале передачи данных?
6. Каково обоснование разработки канала передачи данных со многими циклами?
7. Назовите основные этапы конвейерного канала передачи данных MIPS.
8. Что такое опасности данных и опасности контроля в конвейерном канале передачи данных? Дайте пример каждого на языке ассемблера в MIPS.
9. Сравните ввод/вывод с отображением в памяти и изолированный ввод/вывод с точки зрения возможности программирования на языке ассемблера и количества устройств ввода/вывода, используемых в приложении.
10. Что такое кеш?
11. Зачем нужен DMA?
12. В чем разница между портом ввода/вывода, интерфейсом ввода/вывода и устройством ввода/вывода? Приведите пример каждого.
13. В чем разница между прерыванием и исключением?
14. Сравните основные проблемы программного обеспечения, встречающиеся в многоядерном процессоре с многопроцессорной системой.

2.8. ПРИМЕЧАНИЯ К ГЛАВЕ

Мы рассмотрели основные аппаратные компоненты вычислительного узла в реальном времени. Базовыми типами канала передачи данных являются каналы с одним циклом, многими циклами и конвейерные. Канал передачи данных с одним циклом имеет одинаковую продолжительность цикла для всех команд и, следовательно, неэффективен.

Канал передачи данных с несколькими циклами повышает производительность благодаря тому, что каждая команда имеет свою продолжительность цикла. В конвейерном процессоре команда выполняется в несколько этапов. В процессоре MIPS это этапы: выбор команды (IF), декодирование команды (ID), выполнение (EX), память (MEM) и обратная запись (WB). Мы основали наш обзор на анализе аппаратного обеспечения процессора на базе процессора MIPS, который обычно используется в разработке систем со встроенными элементами.

Память является еще одним фундаментальным компонентом аппаратного обеспечения узла реального времени. Кэши на разных уровнях близости к процессору используются для повышения производительности. Часто используемые данные могут храниться в кеше для более дешевого доступа к ним в будущих преобразованиях. Интерфейс ввода/вывода для процессора обрабатывается входными и выходными портами, которые используются в качестве буфера данных при передаче с этих устройств и на них. Общими устройствами ввода/вывода, подключенными к узлу компьютера, являются блоки отображения, клавиатура, диски и устройства сетевого интерфейса. Применяется два основных метода программного интерфейса для ввода/вывода: опрос, при котором устройство опрашивается процессором, пока оно не готово к передаче данных, и использование прерываний. Прерывание создается внешним устройством путем активации сигнальной линии процессора, который останавливается, выполняет требуемое действие по прерыванию подпрограммы обслуживания и возвращается туда, где она перестала выполняться. Эффективное обслуживание прерываний является важным условием для соблюдения сроков при наличии прерываний в системах реального времени, так как прерывания происходят в этих системах часто. Многоядерный процессор имеет два или более процессорных блока, называемых ядрами в одном чипе, которые делятся данными через кэши. Современные процессоры обычно используют многоядерные технологии для повышения производительности, поскольку при этом достигаются необходимые пределы тактовой частоты. Мультипроцессор – это увеличенное отображение многоядерного процессора, в котором кэш-память третьего уровня заменена глобальной памятью, а ядра теперь становятся процессорами. Синхронизация задач и защита глобальной памяти являются основными проблемами обработки в многопроцессорной операционной системе.

2.9. УПРАЖНЕНИЯ

1. Разработать комбинационную схему, которая будет выполнять функции блока управления с одним циклом передачи данных MIPS для команд загрузить слово (load word), сохранить слово (store word) и добавить (add).
2. Опишите опасность в следующем коде MIPS и предложите решение для устранения этой опасности в MIPS.

```
sw $1, 16($6)
and $1, $2, $3
add $5, $7, $6
```

3. Опишите опасность в следующем коде MIPS и предложите решение для устранения этой опасности в MIPS.

```
beq $3, $4, 8($12)
add $1, $2, $7
sub $5, $6, $8
```

4. Предположим, у вас есть процессор с 20-битной адресной шиной и 16-битной шиной данных. Нарисуйте схему интерфейса этого процессора, чтобы получить 256 килобайт EPROM в [0–256) килобайтах ячеек, 256 килобайт SRAM в [256–512) килобайтах ячеек и 512 килобайт DRAM в [512–1024) килобайтах ячеек.
5. Нарисуйте схему интерфейса к процессору для векторного прерывания трех устройств. Напишите подпрограмму обработки прерываний для процессора в псевдокоде.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. *Hennessy J. L., Patterson D. A. (2011) Computer architecture: a quantitative approach, 5th edn. Morgan Kaufmann.*

Глава 3

Распределенные системы реального времени

3.1. ВВЕДЕНИЕ

Распределенная система реального времени (DRTS) состоит из автономных вычислительных узлов, соединенных сетью реального времени. Узлы в такой системе взаимодействуют для достижения общей цели в установленные сроки. Распределенные системы реального времени необходимы по ряду причин. Прежде всего вычисления в приложениях, требующих работы в реальном времени, естественным образом распределяются по узлам, обрабатывающим части приложения, которые могут быть отделены друг от друга. Во-вторых, одной из наиболее важных причин использования распределенных систем является получение отказоустойчивости, являющейся фундаментальным требованием для любой системы реального времени. Кроме того, балансировка нагрузки на узлы DRTS повышает производительность.

Вычисления, выполняемые на каждом узле, должны соответствовать временным ограничениям задачи. Кроме того, сеть должна обеспечивать обработку в реальном времени с ограничениями по задержке сообщений. В реальности распространены многие приложения реального времени, где задачи, выполняемые в одном узле системы, влияют на задачи, выполняемые в других узлах сети этой системы. Задачи должны взаимодействовать и синхронизироваться в сетях реального времени DRTS. Современный автомобиль оснащен DRTS, где узлы – датчики температуры, скорости, уровня охладителя, уровня масла и т. д. связаны сетью реального времени.

Мы начнем эту главу с обзора моделей DRTS. Затем проведем краткий обзор функций распределенной операционной системы реального времени и основных требований промежуточного программного обеспечения DRTS. Коммуникации в реальном времени формируют основу DRTS. Мы опишем модели коммуникаций в реальном времени с акцентом на топологии и протоколы канального уровня. Мы дадим краткий обзор часто используемых протоколов реального времени и за-

вершим главу двумя примерами архитектуры DRTS. Эта глава в основном служит введением в различные концепции DRTS, представленные в данной части книги.

3.2. Модели

УТОЧНИТЬ

Общая архитектура распределенной системы реального времени изображена на рис. 3.1. Каждый узел сети отвечает за определенные выделенные функции и для выполнения своих функций должен общаться с другими узлами через сеть реального времени, получив требуемый результат. Принципиальное отличие в дизайне DRTS – стремится к системе, запускаемой по времени или событию.

3.2.1. Распределение по времени и событию

Мы опишем системы, запускаемые по времени и событиям, в контексте DRTS. *Trigger (trigger)* – это событие, которое инициирует ответ в системе реального времени. Как было сказано ранее, в системах, *запускаемых по времени*, система должна ответить на внешние события в предопределенные моменты времени. Например, измерение температуры жидкости в системе, запускаемой по времени, выполняется каждые 5 секунд. Планирование событий может быть выполнено в автономном (offline) режиме, поскольку время и продолжительность исполнения известны заранее с незначительными издержками по времени выполнения. Тестирование и отказоустойчивость могут быть выполнены более просто в DRTS, запускаемых по времени, поскольку характеристики неисправного аппаратного или программного модуля известны и их заранее можно заменить точной копией. Системы с запуском по времени подходят для периодических жестких задач в реальном времени; однако они имеют очень небольшую гибкость, когда происходит изменение в наборе задач и нагрузке, испытываемой системой.

Архитектура запускаемых по времени систем требует синхронной связи и, соответственно, использования общих часов. Такую аппаратуру сложно реализовать в распределенной системе реального времени. Можно, однако, синхронизировать в DRTS свободно работающие через равные промежутки времени часы, используя подходящие алгоритмы. Описанная в [8] запускаемая по времени архитектура (time-triggered architecture, ТТА) может использоваться в качестве шаблона для реализации распределенной системы реального времени. Каждому узлу в этой сети выделяется фиксированный временной интервал для трансляции любого сообщения, которое он имеет, следовательно, обеспечивается гарантия доставки сообщения. Архитектура ТТА реализуется во многих приложениях реального времени, таких как автомобильные приложения.

В запущенной событием системе, как мы кратко рассмотрели ранее, система, напротив, должна реагировать на внешние события, которые время от времени спорадически наступают. В такой системе необходимо онлайновое,

приоритетное планирование задачи. В общем, система, управляемая событиями, более гибкая и более адаптивна к изменению характеристик системы. Однако такие системы при работе страдают от значительных временных издержек из-за более сложных алгоритмов планирования, чем в случае запускаемых по времени. Связь в распределенной системе реального времени также может быть классифицирована как инициируемая событием, начинающаяся, когда поступает команда *send* (*отправить*) или наступает время запуска, при котором сообщения отправляются периодически. В целом триггерный подход по событиям удобен в тех системах DRTS, в которых часто встречаются непредсказуемые события из внешнего мира. С другой стороны, метод с временным срабатыванием подходит для детерминированных DRTS, которые имеют известные наборы задач с заранее известными характеристиками задач. В общем случае DRTS будет иметь компоненты, запускаемые и по времени, и по событиям.

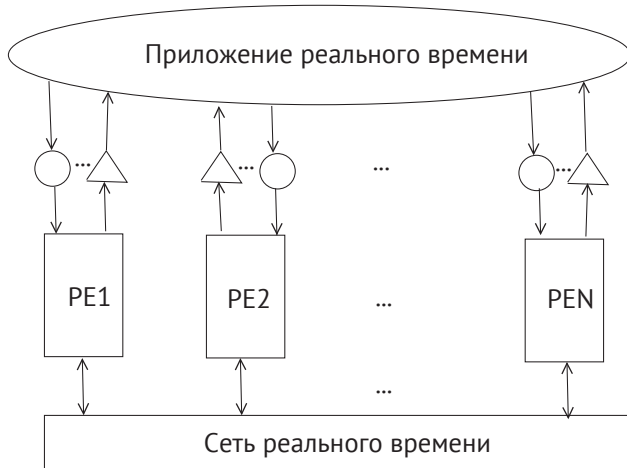


Рис. 3.1. Распределенная структура системы реального времени

3.2.2. Конечные автоматы

Конечный автомат (*finite-state machine, FSM*) состоит из дискретных состояний и переходов между этими состояниями. FSM, представленная шестью кортежами, выглядит следующим образом:

- I – множество конечных входов;
- O – множество конечных выходов;
- S – множество конечных состояний;
- $S_0 \subset S$ – начальное состояние;
- δ – следующая функция состояния: $I \times S \rightarrow S$;
- λ – выходная функция.

Есть два основных типа автоматов: FSM Мура (Moore-type FSM) и FSM Мили (Mealy FSM). В первом выход зависит только от текущего состояния ($\lambda: S \rightarrow O$), во втором выход является функцией как входного, так и текущего состояния ($\lambda: I \times S \rightarrow O$). FSM представляется в форме ориентированного графа, где состояния – вершины графа, а направленные ребра представляют переходы между состояниями. Ребра помечаются как f (вход)/выход, что означает, что этот переход (представленный дугой, arc) происходит, когда указанная функция ввода принимает значение «истина» (true) или просто когда вход получен с указанным генерированным выходом.

Давайте смоделируем простой торговый автомат, который выдает фрукты по 20 центов каждый и принимает только монеты 5 и 10 центов. Он имеет два выхода – R, чтобы освободить плод, и C, чтобы предоставить возможность изменений. Человек может внести четыре монеты по 5 центов подряд, и, следовательно, нам нужно как минимум четыре транзакции по 5 центов, и, конечно, могут быть транзакции по 10 центов. Мы будем представлять 5- или 10-центовые входы как 2-битное двоичное число, чтобы индентифицировать соответствующие входы. Например, 01 означает 5 центов, а 10 – 10 центов. Выход – аналогичное 2-битное двоичное число с первым битом, представляющим вывод, и вторым, представляющим кнопки изменения. Автомат использует эти спецификации с четырьмя состояниями, как показано на рис. 3.2.

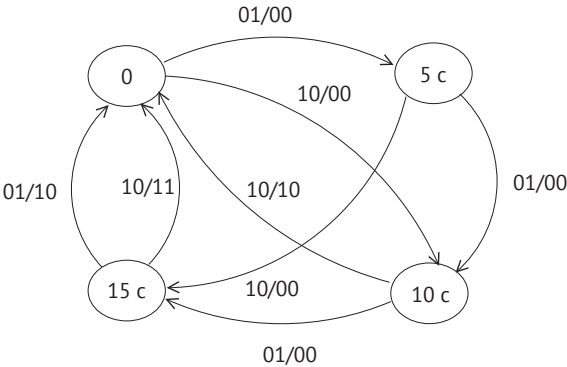


Рис. 3.2. Диаграмма FSM для автомата продаж

Таблица 3.1. Таблица FSM для автомата продаж

Вход		01	10
Состояния	0	act_00 (действие 00)	act_00 (действие 01)
	1	act_10 (действие 10)	act_00 (действие 11)
	2	act_20 (действие 20)	act_00 (действие 21)
	3	act_30 (действие 30)	act_00 (действие 31)

Чтобы написать программу на языке Си для реализации этого FSM, мы можем использовать простой и эффективный подход, сначала формируя *таблицу* FSM, которая имеет состояния в виде строк и входные данные в виде столбцов. Обратите внимание, что мы не можем иметь вход 11, поскольку одновременная вставка монет 5 и 10 центов невозможна. Мы будем представлять поступления монет 0, 5, 10 и 15 центов состояниями 0, 1, 2 и 3 соответственно. Таблица FSM для торгового автомата показана как табл. 3.1. Обратите внимание, что нам не нужно иметь состояние 20 центов, поскольку получение 20 центов означает, что нам нужно выдать фрукты, дать сдачу или нет и вернуться в состояние 0.

Для каждой записи в таблице мы можем определить действие, которое необходимо выполнить, и поступить соответствующим образом: просто получить код программы, перейдя к записи входа в таблице, определенной строкой текущего состояния, и текущий ввод будет столбцом в таблице FSM, как показано в приведенном ниже коде Си:

```

/*****
Реализация торгового автомата FSM
*****/
typedef void (*func_ptr_t)();
func_ptr_t fsm_tab[4][2];
int current_state=0;

act_00{current_state=1; printf("Total is 5 cents");}
act_01{current_state=2; printf("Total is 10 cents");}
act_10{current_state=2; printf("Total is 10 cents");}
act_11{current_state=3; printf("Total is 15 cents");}
act_20{current_state=3; printf("Total is 15 cents");}
act_21{current_state=0; printf("Total 20, release,no change");}
act_30{current_state=0; printf("Total 20, release, no change");}
act_31{current_state=0; printf("Total 20, release, change");}

void main(){
    int input;
    // initialize
    fsm_tab[0][0]=act_00; fsm_tab[0][1]=act_01;
    fsm_tab[1][0]=act_10; fsm_tab[1][1]=act_11;
    fsm_tab[2][0]=act_20; fsm_tab[2][1]=act_21;
    fsm_tab[3][0]=act_30; fsm_tab[3][1]=act_31;

    current_state=0;
    while(true)
    { printf("Input 0 for 5 cent and 1 for 10 cent");
      scanf("%d", &input);
      (*fsm_tab[current_state][input])();
    }
}

```

Автоматы просты в использовании, и алгоритмы для проверки и синтеза распределенных систем реального времени могут быть эффективно разработаны. Количество состояний автомата FSM может быть очень большим, что затруднит управление. Система DRTS может моделироваться сетью FSM. Обычно используемые типы сложных автоматов FSM в DRTS следующие:

- иерархические автоматы: состояние FSM может быть другим FSM, что упрощает проектирование. Следовательно, активация состояния в HFSM приводит к активации FSM, связанной с этим состоянием;
- параллельные иерархические автоматы: этот тип автомата формируется произвольной комбинацией иерархии и параллелизма. Например, мы можем иметь FSM, который может быть представлен как состояние, встроенное в другой FSM, но он и сам может состоять из параллельно работающих FSM. Обычно системы реального времени могут быть определены как параллельные иерархические конечные автоматы (CHFSM), которые имеют свойства и параллельных, и иерархических автоматов.

3.3. РАСПРЕДЕЛЕННЫЕ ОПЕРАЦИОННЫЕ СИСТЕМЫ РЕАЛЬНОГО ВРЕМЕНИ И ПРОМЕЖУТОЧНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

Операционная система реального времени должна управлять ресурсами узла, а также, где это применимо, обеспечить удобный пользовательский интерфейс (см. главу 1). Планирование задач, управление задачами, межадачное общение и синхронизация, управление вводом/выводом, обработка прерываний, управление памятью являются основными задачами, выполняемыми такой операционной системой. Распределенная операционная система реального времени (distributed real-time operating system, DRTOS) должна выполнять все вышеперечисленные функции локально на узле, а также обеспечивать синхронизацию и связь между задачами, решаемыми разными узлами системы.

Своевременность является основным требованием DRTOS, которая состоит из небольшого ядра реального времени, обычно называемого *микроядром* (*microkernel*), повторенным на каждом узле распределенной системы и выполняющим перечисленные выше функции. Более того, распределенное ядро реального времени должно теперь координировать ретрансляцию в реальном времени сообщений через сеть. Сообщения должны быть доставлены надежно и своевременно и требуют поддержки протоколом и быстрой доставки сетевым оборудованием.

Подмножество задач более высокого уровня, решаемых операционной системой реального времени, может быть реализовано на отдельном узле, то есть необходимые функции могут быть распределены между узлами. Более высокие уровни операционной системы, работающей с микроядром, близким к аппаратным и недублируемым компонентам, выполняющим низкоуровневые функции, вовлекаются в выполнение функций более высокого уровня.

3.3.1. Промежуточное программное обеспечение

Уровень промежуточного программного обеспечения в общем смысле находится между операционной системой и приложением. Оно предоставляет функции, которые обычно являются расширениями функций, выполняемых операционной системой, а также управлением сетью. Необходимость применения этого уровня вместо встраивания некоторых функций в операционную систему возникает в связи с тем, что разные приложения могут нуждаться в различных действиях промежуточного программного обеспечения, но все они нуждаются в выполнении основных рабочих функций. Вместе с тем эти процедуры являются общими, могут быть востребованы многими приложениями и должны иметь общую структуру поверх операционной системы. Три основные функции промежуточного программного обеспечения, необходимые в распределенной системе реального времени, – это синхронизация часов, сквозное планирование задач в реальном времени и сеть управления.

В качестве часов процессора обычно используют кварцевый генератор тактовых импульсов. Из-за неточностей в тактовых частотах с течением времени часы узлов в распределенной системе реального времени будут расходиться, и тогда сохранить общий временной интервал будет невозможно. Узлы с разными значениями часов приведут к ошибочным операциям в системе. Для сохранения общего времени обычно используются различные алгоритмы синхронизации часов. Как мы увидим в главе 6, существуют центральные алгоритмы, в которых мастер-узел периодически задает свое значение часов всем узлам или распределяет алгоритмы, где узлы обмениваются сообщениями для достижения согласования по времени.

Еще одна функция промежуточного программного обеспечения, необходимая в распределенных системах реального времени, заключается в обеспечении сквозного планирования задач. Эта операция специфична для DRTS и необходима при планировании подзадач, когда должен быть выполнен общий срок задачи.

3.3.2. Распределенное планирование

Важной проблемой, которая должна решаться распределенной операционной системой реального времени, является такое планирование задачи узлам распределенной системы, чтобы каждая задача соответствовала сроку и нагрузка распределялась надлежащим образом. Как мы увидим в главе 9, эти задачи, называемые *распределенное планирование (distributed scheduling)* и балансировка загрузки (*load balancing*), являются нетривиальными задачами. *Статическое распределенное планирование (static distributed scheduling)* относится к планированию задач, выполняемых до истечения заданного срока. Одним из способов достижения этого является построение *графа задач (task graph)* с использованием некоторой эвристики. Давайте рассмотрим набор задач реального времени, имеющих жесткие сроки выполнения, приведенный в табл. 3.2, и задачи $\tau_1, \dots, \tau_6$, вычислений C_i с конечными сроками D_i .

Таблица 3.2. Пример набора задач

τ_i	C_i	D_i
1	4	12
2	5	20
3	3	15
4	10	60
5	6	30
5	15	60

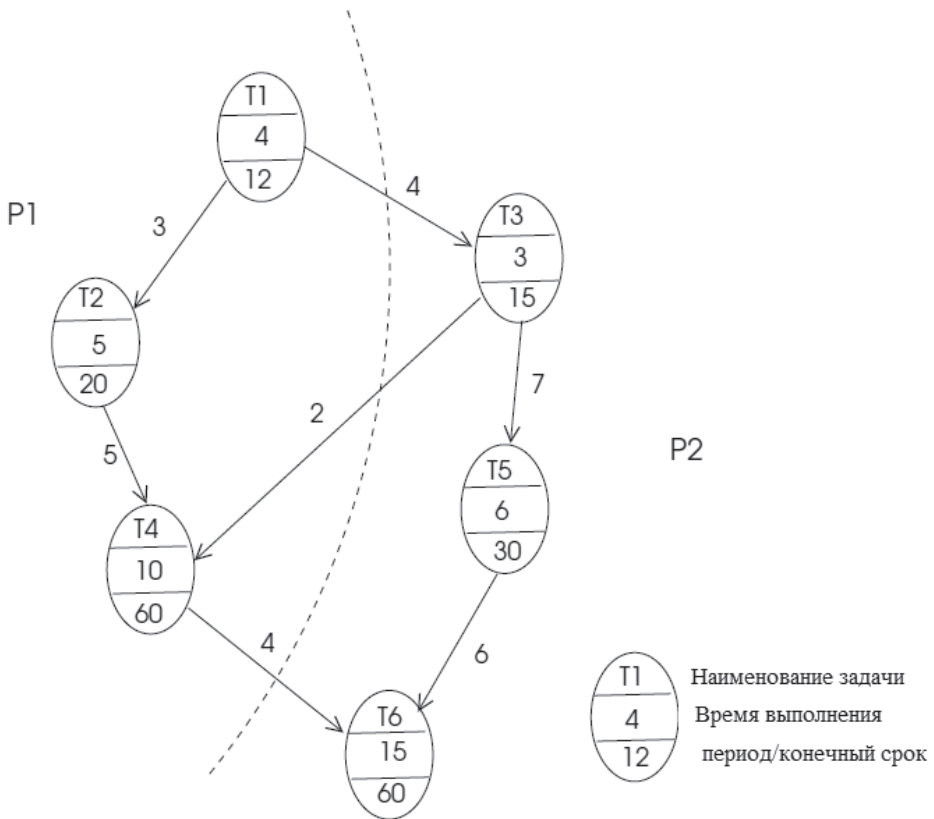


Рис. 3.3. Пример графа задач

Давайте далее предположим связь между этими задачами, как показано в графе задач на рис. 3.3, где стрелка, указывающая на задачи $\tau_i, \dots, \tau_j$, показывает, что τ_i должна завершиться, прежде чем начнет работать τ_j . Планирование этих шести периодических задач для двух процессоров P_1 и P_2 может быть достигнуто путем разделения графа задач, как показано пунктирной линией.

При планировании этих задач связь между двумя задачами, запущенными на одном процессоре, рассматривается как имеющая нулевую стоимость, хотя связь между процессорами является существенным фактором, и это должно быть принято во внимание.

Возможное планирование данного набора для двух процессоров с использованием диаграммы Гантта (Gantt) изображено на рис. 3.4 и показывает выполнение задач с течением времени. Подобно этому τ_4 ждет сообщения τ_3 и τ_6 для τ_4 . Заметьте также, что это планирование может повторяться каждые 60 единиц времени. Задачи распределенного планирования являются жесткими (NP-hard), и обычно при поиске подходящего планирования используется эвристика.

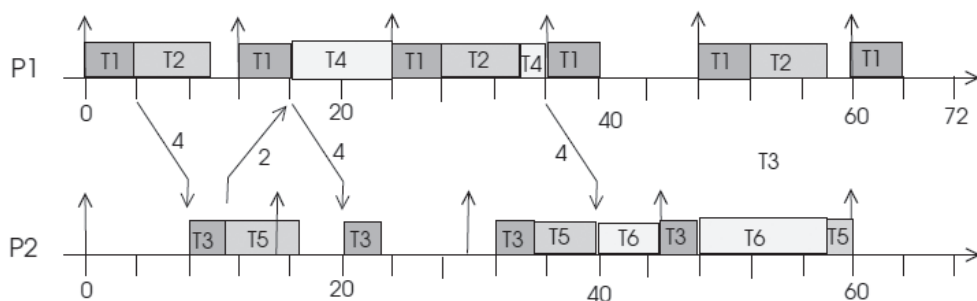


Рис. 3.4. Планирование задач рис. 3.3 для двух процессоров

3.3.3. Динамическая балансировка нагрузки

Иные из узлов распределенной системы реального времени могут быть сильно загружены непредсказуемыми аperiodическими и спорадическими задачами, в то время как некоторые другие узлы могут иметь для запуска немного задач. В таких случаях можно рассмотреть вопрос переноса задач с сильно загруженных узлов на менее загруженные узлы. *Динамические методы балансировки нагрузки (dynamic load balancing)* направлены на уравнивание нагрузок процессором нагрузок на узлы. Эти подходы предоставляют возможность динамически добавлять в систему задачи и разрешают планирование различных типов задач, таких как асинхронные и спорадические задачи.

Эти методы могут быть централизованными, или полностью распределенными, или с промежуточным подходом [2]. Централизованные методы позволяют принимать решения по планированию в центральном узле, который может стать узким местом при большом количестве узлов. Кроме того, информация о состоянии, которая должна храниться в центральном узле, также может быть слишком большой. Более того, центральный узел является той точкой отказа, дефект в которой может быть причиной остановки всей системы. Однако для систем с небольшим количеством узлов такой подход может быть практичным и легко применимым.

При распределенной балансировке нагрузки решения принимаются для каждого узла с использованием методов *инициирующего отправителя* (*sender-initiated*) или *инициирующего приемника* (*receiver-initiated*). В методе иницирующего отправителя узел, который находит свою нагрузку тяжелой, ищет для получения задач узел с легкой нагрузкой, в то время как в методе иницирующего приемника узел с легкой нагрузкой ищет узел с тяжелой нагрузкой.

3.4. СВЯЗЬ В РЕАЛЬНОМ ВРЕМЕНИ

Общие сетевые коммуникации обеспечивают наилучшее обслуживание, однако сеть связи для приложений в реальном времени должна обеспечивать также и определенный уровень качества обслуживания (*quality of service, QoS*). Основные физические характеристики такой сети следующие:

- *пропускная способность (Throughput)*: количество пакетов, которые сеть может доставить за единицу времени;
- *длина битов (Bit Length)*: количество битов, проходящих через носитель одновременно;
- *полоса пропускания (Bandwidth)*: это число битов, которые могут пройти канал в единицу времени. Пропускная способность канала связи должна быть достаточно высокой, чтобы обеспечить требуемую пропускную способность для приложения;
- *задержка распространения (Propagation Delay)*: время, необходимое для прохождения одного бита от одного конца до другого конца канала связи;
- *задержка (Latency)*: время, необходимое для доставки пакета;
- *джиттер (Jitter)*: это отклонение периодического сигнала от его периода.

Основные требования к QoS для сети реального времени связаны с задержкой в сети, ограничением частоты потерь в сети и небольшой вероятностью блокировки. В общих сетевых коммуникациях надежность является главной проблемой, хотя скорость передачи данных явно предпочтительнее. С другой стороны, общение в реальном времени требует доставки сообщений надежно и своевременно. Сообщения могут иметь конечные сроки доставки – сроки, в течение которых они должны быть доставлены по назначению по сети в реальном времени. Обычно предусматривается приоритетная передача сообщений, при которой в сети реального времени высокоприоритетные сообщения доставляются до низкоприоритетных сообщений.

3.4.1. Трафик в реальном времени

Существуют три основных типа трафика в *сетях реального времени (real-time net, RTN)* [7]:

- *трафик с постоянной скоростью передачи битов (constant bit rate, CBR)*: сеть используется с постоянной скоростью, например когда датчики генерируют данные периодически. Обычно сложные задачи в реальном

времени осуществляют CBR-трафик, используя сообщения фиксированной длины;

- *трафик с переменной скоростью передачи битов (variable bit rate, VBR)*: передача данных по сети время от времени варьируется;
- *спорадический трафик (sporadic traffic)*: передача данных по сети происходит пакетами, за которыми обычно следуют длинные периоды без передачи. Этот тип трафика является частным случаем трафика VBR. Основное отличие состоит в том, что существует определенный временной интервал между передачей двух спорадических данных. Типичным примером является возникающий в сети реального времени сигнал тревоги.

3.4.2. Модель взаимосвязи открытых систем

Модель взаимодействия открытых систем (open systems interconnection, OSI) предполагает семь уровней передачи данных и функционирует посредством телекоммуникации или компьютерной сети. Уровень в этой модели получает команды и отправляет ответы на более высокий уровень. Это разделение сложного процесса общения упрощает дизайн и реализацию основных функций сети. Уровни в этой модели следующие:

- *физический уровень (Physical Layer)*: основная функция этого уровня – передача данных между физической средой и устройствами;
- *уровень канала передачи данных (Data Link Layer)*: этот уровень обеспечивает двухточечную передачу данных между двумя подключенными устройствами. При этом он обеспечивает механизмы контроля потока, обнаружение и исправление ошибок. Он состоит из двух подуровней: *контроль доступа к среде (medium access control, MAC)* и *управление логическим каналом (logical link control, LLC)*. Уровень MAC обеспечивает управление доступом к сети, а уровень LLC отвечает за обнаружение ошибок и контроль потока;
- *сетевой уровень (Network Layer)*: основная задача этого уровня – включить маршрутизацию пакетов данных от исходного узла к целевому узлу;
- *транспортный уровень (Transport Layer)*: этот уровень отвечает за правильную сквозную передачу данных между двумя приложениями. Два основных протокола на этом уровне: *протокол управления передачей (transmission control protocol, TCP)* – ориентирован на соединения между приложениями – и *протокол дейтаграмм пользователя (user datagram protocol, UDP)* – протокол для бесконтактной связи;
- *уровень сессии (Session Layer)*: основная задача на этом уровне – управление диалогом между двумя взаимодействующими объектами;
- *уровень представления (Presentation Layer)*: на этом уровне осуществляется преобразование синтаксиса и контекста между двумя взаимодействующими приложениями.

Модель связи в интернете TCP-IP обычно использует четыре уровня: прикладной уровень, транспортный уровень (*транспортный и сеансовый уровни модели стандарта взаимодействия открытых систем OSI (open system interconnection)*), уровень межсетевого взаимодействия (подмножество сетевого уровня OSI) и уровень канала передачи данных (канал передачи данных и частично физические уровни модели OSI). Семиуровневая модель OSI требует значительных затрат. Более того, многие сети реального времени предназначены для конкретного применения и использования локальной сети. Следовательно, в этих сетях презентация и сетевые уровни модели OSI могут быть исключены. Кроме того, для передачи сообщений в сети в реальном времени обычно необходимы короткие фрагментированные сообщения и средства трансляции с языка ассемблера не требуются. В небольшой сети RTN обычно используется сжатая модель OSI с тремя уровнями, как показано на рис. 3.5, в которой приложение может напрямую обращаться к канальному уровню. Однако часто присутствует и другой уровень, обычно транспортный, чтобы обеспечить независимый от приложения интерфейс сети.

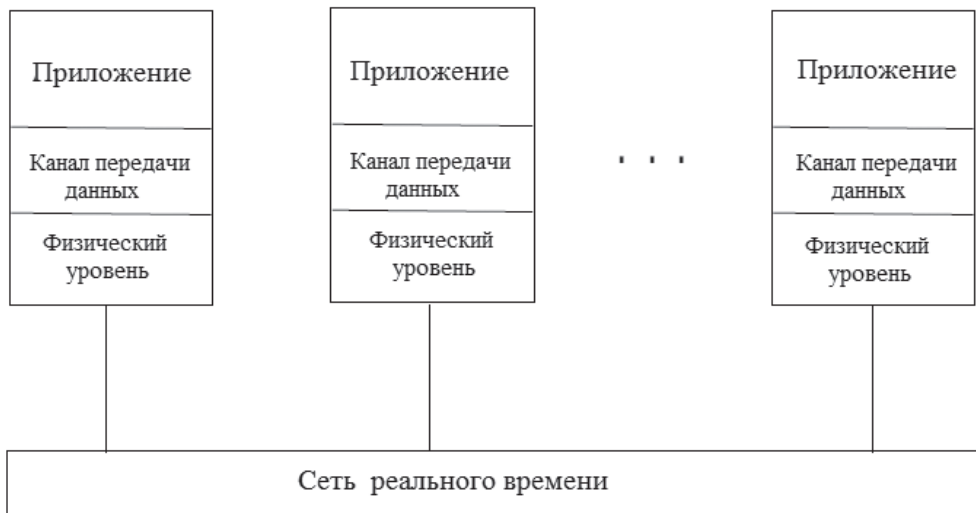


Рис. 3.5. Сжатая модель OSI, используемая в сети реального времени

3.4.3. Топология

Приложение реального времени обычно состоит из сайтов обработки, которые находятся поблизости друг от друга, а сеть реального времени чаще представляет собой локальную сеть, состоящую из шины, кольца, дерева или смешанной структуры, как показано на рис. 3.6. Широко распространенная

сеть в виде шины также применяется и в системах реального времени. Все узлы привязаны к общей шине, что уменьшает количество соединений, однако нужен механизм разделения шины. В наиболее часто применяемом методе Ethernet станции осуществляют передачу в любое необходимое им время и одновременно прослушивают шину. Если происходит конфликт, все станции, участвующие в нем, ждут случайное количество времени и пытаются повторить передачу. Шину можно разделить по времени, выделив ее каждой станции для связи на определенное время. В архитектуре связи, называемой *маркерной шиной (token bus)*, среди станций в заранее определенном порядке может распространяться маркер (токен), и любая станция, которая получила маркер, будет иметь право передачи.

Кольцевая структура позволяет передавать сообщение от одной станции к другой, пока оно не достигнет своего места назначения. В этой структуре обычно используется маркер для определения права на передачу. *Древовидная структура* также может быть использована для коммуникации. Худшее время для передачи сообщения от одного узла дерева другому – через корень дерева – оно вдвое больше глубины дерева. Обратите внимание, что в древовидной структуре возможна параллельная связь через непересекающиеся пути. *Смешанная структура*, где каждая станция связана со всеми другими станциями, имеет наилучшую производительность, так как каждый узел находится на расстоянии одного шага от любого другого за счет необходимого большого количества соединений. Требуемое количество соединений для сети из n узлов будет равно $n(n - 1)/2$.

Сеть звездой обеспечивает двухзвенную связь между любыми двумя станциями в сети, используя всего n соединений для сети с n узлами. Однако, как и в случае со всеми системами связи, имеющими центральный узел, этот узел представляет собой единую точку сбоя, и при сбое прекращаются все коммуникации. Кроме того, он также является узким местом, поскольку все сообщения доставляются через него. *Беспроводные сети связи* используют для передачи сообщений радиоволны и многопереходную связь, следовательно, зона должна быть перекрыта так, чтобы связь была между любыми двумя станциями. Беспроводные сенсорные сети обычно используют древовидную структуру, которая сосредоточена для связи вокруг более мощной корневой станции.

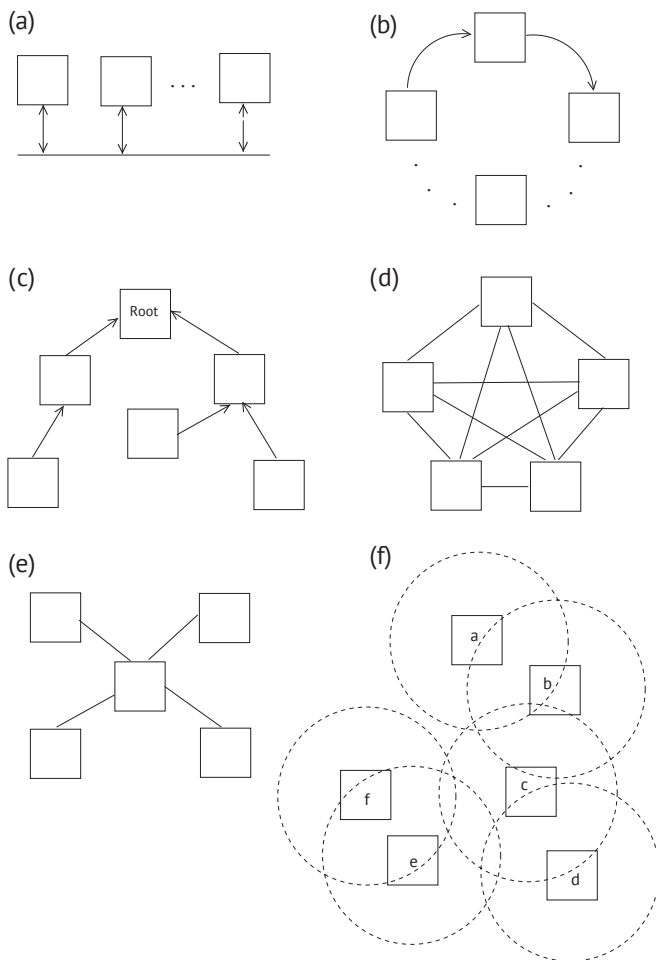


Рис. 3.6. Топология сетей реального времени: а – шина, б – кольцо, с – дерево, д – смешанная и ф – беспроводная архитектуры. Диапазон передачи беспроводной сети показан пунктирной линией. Обратите внимание, что узел а в беспроводной сети соединен с узлом f посредством узлов b и c либо последовательно, либо просто через узел c использованием многопереходной связи

3.4.4. Уровень канала передачи данных

Уровень канала передачи данных управляет потоком данных между двумя подключенными устройствами и контролирует ошибки. Обратите внимание, что кадры, обрабатываемые в канале передачи данных, могут принадлежать различным приложениям, находящимся в этом устройстве. *Транспортируемый пакет* является единицей информации, переносимой между двумя приложениями на транспортном уровне хостов. Ссылка на данные состоит из уровней MAC и LL, описание которых приводится ниже.

3.4.4.1. Протоколы доступа к среде

Уровень управления доступом к среде (*medium acceses control, MAC*) главным образом имеет дело с арбитражем совместно используемой коммуникационной среды. Три основных метода арбитража в реальном времени на уровне MAC следующие:

- определение несущей с многостанционным доступом/обнаружением конфликтов (*carrier-sense multiple access and collision detection, CSMA/CD*): это группа протоколов, основанных на измерении шины перед попыткой передачи. Механизм обнаружения конфликтов используется для обнаружения, если два узла пытаются передавать сообщения в одно и то же время. Когда это происходит, станции ждут случайную продолжительность времени и пробуют возобновить передачу. Этот протокол используется в сетях Ethernet и Wi-Fi. Из-за его недетерминированного поведения он не подходит для использования в RTN. Разновидностью протокола CSMA/CD, который может использоваться в RTN, является *CSMA с арбитражем (предотвращением) конфликтов (CSMA/collision avoidance, CA)*, где устанавливаются приоритеты сообщений. Всякий раз, когда происходит конфликт, передается сообщение с наивысшим приоритетом, что обеспечивает необходимую определенность;
- *многостанционный доступ с временным разделением (time-division multiple access, TDMA)*: как мы уже отмечали, каждой станции в сети выделяется заданная продолжительность использования сети, называемая *кадром (frame)*, которая обычно является фиксированным временным интервалом. Этот тип протокола подходит для распределенных систем реального времени из-за его детерминированной характеристики. Каждая станция осведомлена о времени права на доступ к сети. TDMA обычно используется в общей шине распределенных систем реального времени;
- *коммуникация на основе токенов (token, маркер)*: владение токеном дает право на использование сети для передачи сообщений. Стандарт Token Bus (IEEE 802.4) использует общую шину, стандарт Token Ring (IEEE 802.5) применяет кольцевую архитектуру для передачи токена. Станция, у которой нет данных для отправки, передает токен следующему узлу, после последнего из которых токен постоянно вращается в сети. Токен также распространяется в сети с архитектурой Token Bus, но последовательность станций, которые он может посещать, определяется программным обеспечением, то есть каждая станция имеет предшественника и преемник приписан к нему. Распределенная передача данных с частотным разделением (*frequency-devision data link, FDDI, ANSI X3T9.5*) имеет двойной счетчик вращающихся токенов.

3.4.5. Протокол контроллерной сети

Контроллерная сеть (controller area network, CAN) – это небольшая локальная сеть, обычно используемая для соединения компонентов встроенных контроллеров [1]. Длина сети CAN – обычно не более нескольких десятков метров, и скорость связи чаще 1 Мбит/с. Первоначально она была создана фирмой BOSCH как сеть для автомобильной промышленности для подключения автомобиль-

ных компонентов, таких как тормоза, впрыск топлива, кондиционер и т. д. В настоящее время широко используется в системах автоматизации, на кораблях, в бортовом электронном оборудовании и в медицинском оборудовании.

Современные автомобили могут иметь несколько десятков электронных блоков управления (ЭБУ) для управления тормозами, навигацией, двигателем и т. д. Сеть CAN использует два соединения для мультимедиа к коммуникационной шине, что приводит к значительному уменьшению количества соединений. Сеть CAN основана на методе MAC CSMA/CA, в котором станция, которая хочет передавать, контролируется шиной и начинает отправлять свое сообщение, когда шина свободна. Специальный протокол арбитража шины CAN обеспечивает эффективную обработку конфликтов. CAN использует четыре типа сообщений:

- *фрейм данных (Data frame)*: это наиболее распространенный тип сообщений для передачи данных;
- *удаленный фрейм (Remote frame)*: этот фрейм используется для запроса данных передачи с удаленного узла;
- *фрейм ошибки (Error frame)*: узел, обнаруживший ошибку в сообщении, отправляет этот фрейм, вызывая другие узлы для отправки фрейма ошибки. Отправитель сообщения затем повторно передает сообщение;
- *фрейм перегрузки (Overload frame)*: этот фрейм используется занятым узлом для задержки передачи сообщения.

Сеть CAN охватывает только физический и канальный уровни модели OSI. Физические функции уровня, такие как сигнализация и интерфейс, зависящие от среды, обрабатываются приемопередатчиком сети CAN, а функции канального уровня MAC и LLC обрабатываются контроллером CAN, как показано на рис. 3.7.

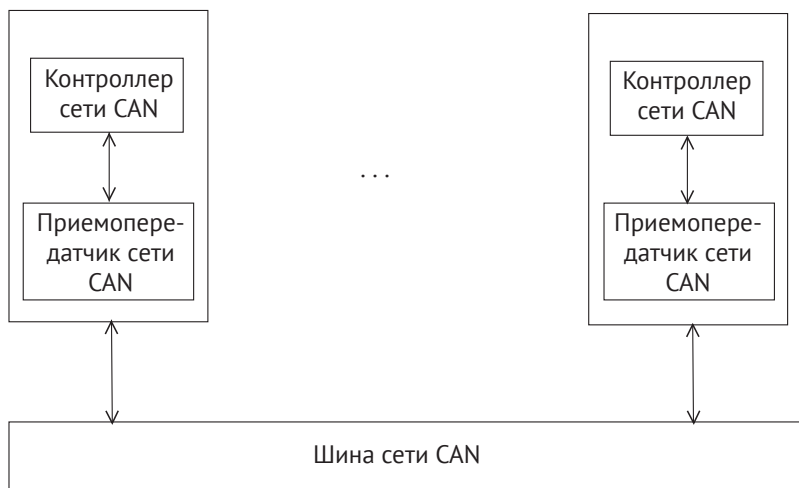


Рис. 3.7. Архитектура сети CAN

3.4.6. Протокол запуска по времени

Протоколы временного срабатывания (time-triggered protocol, TTP) – это семейство отказоустойчивых протоколов, спроектированных Копецом (Kopetz) и Грунштейдлом (Grunsteidl) [5]. TTP имеет две версии: TTP/A для автомобилей класса А и для недорогих программных приложений реального времени и TTP/C для автомобилей класса С и для сложных приложений реального времени. TTP/A использует основную рабочую шину арбитража, в которой главный узел контролирует доступ к шине. TTP использует TDMA для доступа к сети и основан на распределенной архитектуре реального времени, в которой каждое событие запускается по времени, определяемому общей временной базой. Все часы узлов синхронизируются с высокой точностью, а сообщения, которые в основном являются периодическими, передаются в заранее определенные моменты времени.

Протокол TTP/C

Протокол TTP/C, используемый для жестких систем реального времени, имеет в сетевом узле *сетевой интерфейс связи* (*communication network interface*, CNI) между контроллером связи и хост-компьютером. Контроллер связи (*communication controller*, CC) TTP/C является фактическим интерфейсом между сетью и станцией TTP/C. Каждому узлу назначается фиксированный временной интервал для доступа в сеть. Сеть TTP/C соединяет узлы двумя реплицированными каналами с именами канал 1 и канал 2, как показано на рис. 3.8. TTP/C может работать с общей топологией шины или комбинацией звезда или звезда/шина. Синхронизация часов между станциями предоставляется в каждом из доступов TDMA.

Дальнейшим развитием протокола TTP/C является протокол Flexray, разработанный европейскими автопроизводителями. Он объединяет подходы, основанные на событиях и времени, где управляемая событиями связь возможна в некоторых слотах TDMA.

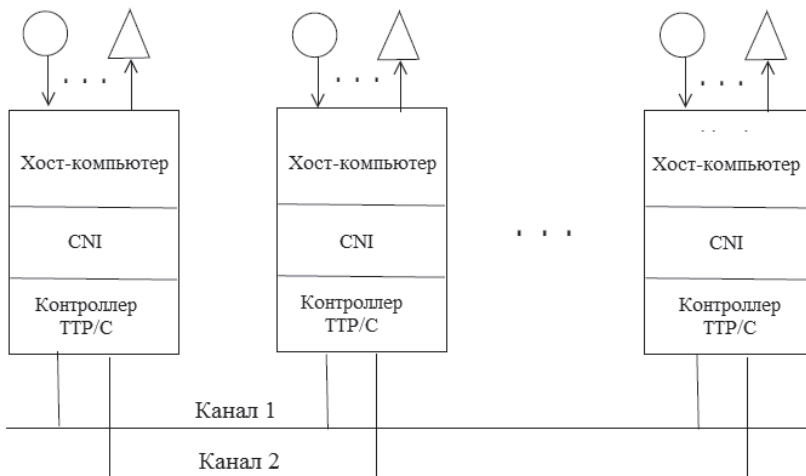


Рис. 3.8. Сеть TTP/C

3.4.7. Сеть реального времени Ethernet

Ethernet, стандарта IEEE 802.3, не подходит для приложений реального времени из-за его недетерминированности в доступе к сети. Запрос на передачу в Ethernet может не предоставляться после ограниченного числа случайных ожиданий в случае конфликта. Предотвращение конфликта является первым шагом к созданию детерминированного Ethernet.

Было несколько попыток преобразовать Ethernet в протокол Ethernet реального времени. Одним из таких усилий был протокол Ethernet реального времени (RETHER). Он был разработан в государственном университете Нью-Йорка в Стони Брук (Stony Brook) [11]. В многосегментной версии данного протокола обеспечивается гибридная операция, в которой режим CSMA/CD активируется в случае трафика не в реальном времени. В режиме реального времени станции переходят в сети в режим трафика RETHER. Этот режим реализуется путем распространения токена в общей шине и резервирования полосы пропускания станции, прежде чем можно будет осуществлять передачу в цикле. Этот протокол не делает различий между жесткой и мягкой коммуникациями в реальном времени и требует установки сетевых драйверов на каждой станции. Протокол Ethernet в реальном времени (RT-EP) – еще один проект, направленный на обеспечение функциональности Ethernet в реальном времени с использованием приоритетов прохождения токена [6].

Протокол RTnet, разработанный в университете Твенте (Twente University), является еще одним подходом для реализации связи в реальном времени с использованием Ethernet без каких-либо модификаций [3]. Этот протокол также использует распространение токенов по шине и алгоритм с приоритетным прерыванием связи самым ранним крайним сроком (early deadline first, EDF) для выделения заранее временного слота, в котором станция может хранить токен. Потеря токена предотвращается способностью узла, который передает токен, быть монитором. При обнаружении потери токена, например из-за сбоя в узле, новый токен генерируется узлом монитора. Протокол RTnet обеспечивает динамическое дополнение в сеть или удаление узлов из сети. Когда нет трафика в реальном времени и присутствует трафик не в реальном времени, узлы используют алгоритм циклического перебора EDF для передачи токена. Основным недостатком этого протокола является то, что он также не различает жесткий и мягкий трафик в реальном времени (CBR, VBR или спорадический).

3.4.8. Стандарт реального времени IEEE 802.11

IEEE 802.11 представляет собой набор спецификаций MAC и спецификаций физического уровня для беспроводной связи по локальной сети [12]. Сеть 802.11 состоит из базовых наборов услуг (basic service sets, BSS), которые связаны с системой распределения (distribution system, DS). Протокол 802.11 MAC обеспечивает два режима связи для беспроводных узлов: распределенную функцию координации (distributed coordination function, DCF), которая использует многостанционный доступ с контролем несущей и исключением конфликтов

(CSMA/CA), и функцию координации точек (point coordination function, PCF), которая используется для разделения времени между периодом без конфликтов (contention-free period, CFP) и периодом конфликтов (contention period, CP). Используя PCF, узел может передавать данные во время бесконфликтного периода опроса.

Протокол IEEE 802.11e является усовершенствованием протокола IEEE 802.11, предлагая приоритет передачи данных, голоса и видео. Механизм связи с расширенным распределенным доступом (enhanced distributed channel access, EDCA), определенный в стандарте IEEE 802.11e, используется для поддержки трафика в реальном времени по протоколу IEEE 802.11. Высокоприоритетный трафик имеет более высокую вероятность доступа к сети с использованием EDCA, чем трафик с более низким приоритетом. Протокол IEEE 802.11e предоставляет услуги для чувствительных ко времени программных приложений реального времени, таких как потоковая передача видео и голоса по интернет-протоколу (video and voice over Internet protocol, VoIP). Работа ведется на радиочастотах в двух диапазонах. По стандарту IEEE 802.11e возможны частоты в диапазонах 2,400–2,4835 ГГц и 5,725–5,850 ГГц, однако он не предназначен для жесткого трафика в реальном времени.

Существует несколько попыток применить стандарт IEEE 802.11 для жесткого трафика в режиме реального времени. Беспроводной протокол многократного перехода (realtime wireless multi-hop protocol, RT-WMP) является одним из таких протоколов на основе RT-EP, использующим распространение токенов с приоритетными сообщениями в стандарте 802.11 [10]. Он поддерживает жесткий трафик в режиме реального времени путем предоставления ограниченной сквозной задержки сообщения с заданной продолжительностью. Сообщения располагаются по приоритетам, и для увеличения перекрытия сетью используется многозвенная связь.

3.5. ПРОБЛЕМЫ В РАСПРЕДЕЛЕННЫХ СИСТЕМАХ РЕАЛЬНОГО ВРЕМЕНИ СО ВСТРОЕННЫМИ ЭЛЕМЕНТАМИ

Есть несколько проблем при проектировании DRTS. Прежде всего общие задачи, выполняемые операционной системой, должны быть расширены в DRTS в двух направлениях: операционная система теперь распределена и является системой реального времени. Распределенная операционная система рассматривается пользователем как единая операционная система. Реализация этой функции вносит значительную сложность в разработку операционной системы как таковой. Важной задачей является необходимость в заранее определенной манере обеспечить основные функции распределенной операционной системы, такие как межзадачная связь и синхронизация сети в режиме реального времени. Промежуточное программное обеспечение в DRTS находится между DRTOS и распределенным приложением реального времени и обеспечивает общий набор сервисов для приложения, поэтому может использовать

ся многими разнообразными приложениями реального времени. Типичным промежуточным сервисом в DRTS является синхронизация часов.

Отказоустойчивость является одной из основных задач при реализации распределенной системы, будь то в реальном времени или не в реальном времени. В случае реального времени стоимость ошибки может быть намного выше с точки зрения затрат, не говоря уже о сохранении человеческой жизни. Поэтому обнаружение отказов, таких как сбои в сети, сбои программного обеспечения и восстановление системы после этих неисправностей, являются основными функциями, которые должны выполняться в DRTS. Они обычно реализуются с помощью DRTOS и/или промежуточного программного обеспечения. Общими источниками неисправностей в DRTS являются аппаратные сбои узлов, сбои сети, задержки передачи, превышающие заданные, и проблемы распределенной координации.

Как и в системе реального времени с одним узлом, нам нужно, чтобы задачи соответствовали их срокам в качестве основной и фундаментальной задачи. В распределенном случае реализация этой функции сложнее и потому является проблемой в DRTS. Мы рассмотрим различные подходы, которые решают эту проблему, в главе 9. Планирование задач с известным временем выполнения и конечными сроками может быть выполнено путем разделения графа задачи на доступные вычислительные узлы. Эта функция является трудно реализуемой даже в случае задач не в реальном времени. Реализация разбиения графа вносит дополнительное требование по соответствию задач их срокам. Другой проблемой, которая встречается в DRTS при сравнении с распределенными системами не в реальном времени, является тестирование, которое становится самостоятельной проблемой при решении таких задач, как моделирование среды и отказоустойчивость.

Таким образом, основными проблемами при разработке и реализации DRTS являются разработка и внедрение DRTOS, промежуточного программного обеспечения реального времени, отказоустойчивых методов и распределенного планирования.

3.6. ПРИМЕРЫ РАСПРЕДЕЛЕННЫХ СИСТЕМ РЕАЛЬНОГО ВРЕМЕНИ

В качестве примеров распределенных систем реального времени мы кратко опишем структуру двух часто используемых распределенных систем реального времени: современного автомобиля и мобильной беспроводной сенсорной сети.

3.6.1. Современный автомобиль

Современный автомобиль, использующий протокол CAN, является типичным примером системы распределенного реального времени, в которой основные модули, подключенные к сети реального времени, имеют ограничения по времени. Пример системы управления автомобилем изображен на рис. 3.9, на котором две шины CAN соединены шлюзом. Высокоскоростная шина CAN

подключает такие модули, как управление двигателем, подвеской и коробкой передач, тогда как низкоскоростная сеть используется для связи между фарами, сиденьем и блоком управления дверями.

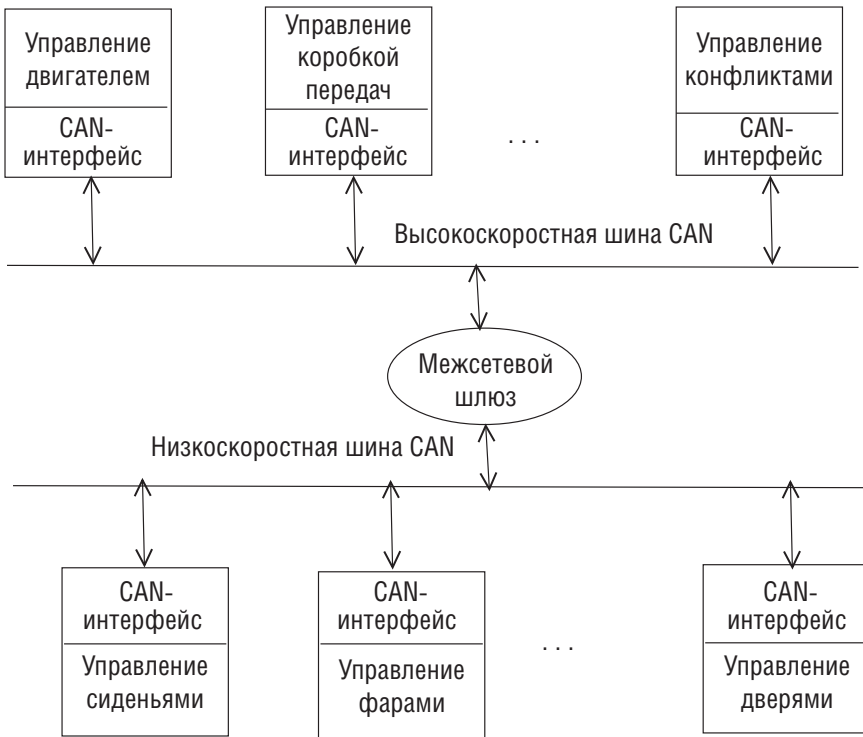


Рис. 3.9. Коммуникационная сеть в современном автомобиле

3.6.2. Беспроводная мобильная сенсорная сеть

Беспроводная сенсорная сеть (wireless sensor network, WSN) состоит из небольших автономных узлов с датчиками, беспроводной связи с использованием радиоволн и имеет ограниченные вычислительные возможности. **Мобильная беспроводная сенсорная сеть** WSN (mobile wireless sensor network, MWSN) – это сенсорная сеть с мобильными узлами. Сети MWSN имеют многочисленные приложения, такие как мониторинг окружающей среды, спасательные операции, военные системы наблюдения и здравоохранение [4]. Эти сети могут иметь различные топологии – дерево, сетка или кластер, но обычно предпочтительными являются гибридные топологии. Основными проблемами в приложении MWSN выступают ограниченный срок службы батареи, даже когда узлы небольшие, а также эффективное использование общей среды при изменении топологии. Маршрутизация в этих сетях использует многопереходный обмен сообщениями и должна иметь дело с правильной доставкой сообще-

ний при динамической топологии. Сеть MWSN изображена на рис. 3.10 с узлами $a, \dots, i$. Пунктирный круг вокруг узла показывает дальность передачи, и мы видим, что каждый узел находится в пределах дальности передачи, по крайней мере одного другого узла. Следовательно, сеть соединена, и каждый узел может общаться с любым другим узлом в сети. Узлы перемещаются в направлениях, указанных стрелками, и могут появиться узлы, в какой-то момент времени отключенные от сети. Таким кандидатом, например, является узел b . Как видно из этого примера, обеспечение подключений в сети всегда является фундаментальной проблемой в MWSN.

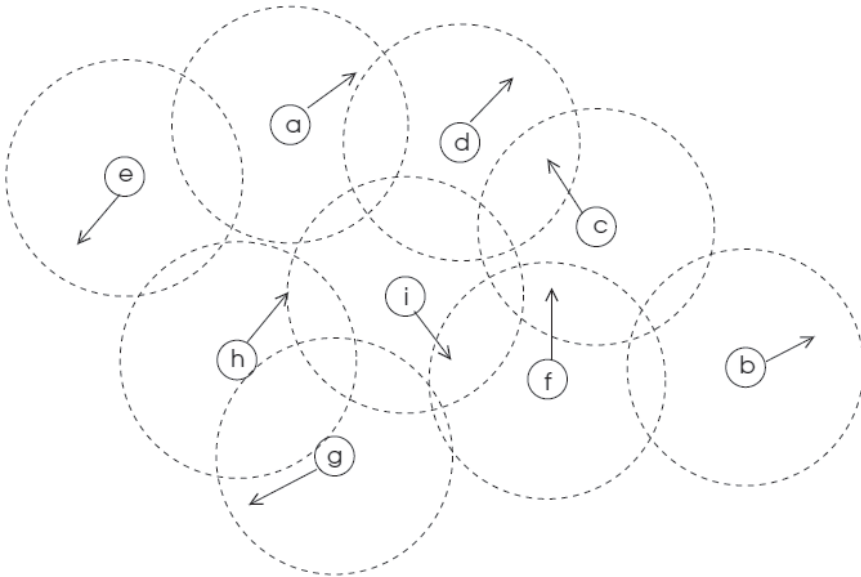


Рис. 3.10. MWSN

С помощью узлов MWSN можно осуществлять мониторинг людей, животных и транспортных средств. Мониторинг можно проводить периодически с использованием режимов запуска по времени, или генерации данных при наступлении некоторого события. Можно иметь совмещение этих режимов, как в общем случае распределенного реального времени.

3.7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чем разница между распределенными системами реального времени, запускаемыми по времени и запускаемыми по событию?
2. Что такое конечный автомат FSM и параллельный, иерархический FSM?
3. Каковы различия между операционной системой реального времени и распределенной операционной системой реального времени?

4. Какова основная функция промежуточного программного обеспечения в распределенной системе?
5. Сравните процессы планирования и баланса нагрузки в распределенной системе реального времени.
6. Каковы общие средства передачи в основных сетях?
7. Какова обычная сетевая архитектура распределенной системы реального времени?
8. Какие основные требования предъявляются к сетевому протоколу реального времени?
9. Каковы основные типы трафика в системах реального времени?
10. Назовите три основных метода, реализуемых в протоколах MAC реального времени.
11. Сравните протоколы CAN и TTP в терминах сетевого доступа.
12. Каковы основные методы реализации сети Ethernet реального времени?
13. Сравните системы реального времени с одним узлом с распределенными системами реального времени по требованиям, предъявляемым к аппаратному обеспечению, программному обеспечению и реализации.

3.8. ПРИМЕЧАНИЯ К ГЛАВЕ

В этой главе мы рассмотрели основные понятия распределенной системы реального времени. Сеть DRTS состоит из узлов обработки в реальном времени, соединенных сетью реального времени. Основное требование от сети реального времени – своевременная доставка сообщений. Что касается программного обеспечения, операционная система должна обеспечивать сетевой интерфейс с предоставлением всех функций реального времени в системе реального времени с одним узлом, а также обеспечить механизмы для распределенного планирования задач. Распределенное статичное планирование должно предусматривать назначение задач узлам распределенной системы и такое время их выполнения, чтобы каждая задача выполнялась к заданному сроку. Для аperiodических и sporadicических задач, активирующихся во время выполнения, обычно используются различные политики динамического планирования. Эти методы могут быть в широком смысле классифицированы как центральные с центральным узлом управления распределения нагрузки и распределения, при котором каждый узел свободно решает задачу баланса своей нагрузки. При инициировании передачи нагрузки существуют инициированные отправителем и инициированные получателем подходы к балансу нагрузки. Основным требованием, предъявляемым к распределенному промежуточному программному обеспечению системы реального времени, является обеспечение синхронизации часов, так как все задачи должны синхронизироваться в определенные моменты времени в соответствии с глобальной временной базой.

Основными проблемами при проектировании и внедрении рассмотренных распределенных систем реального времени является выбор подходящей сети

реального времени для приложения, выбор распределенной операционной системы реального времени, промежуточного программного обеспечения и, по мере необходимости, написание патчей к этим программным модулям. Кроме того, все приложение может рассматриваться как единый граф задачи, и необходим алгоритм автономного назначения задач узлам. Для непредсказуемых активаций задач, таких как получение аperiodической задачи, может быть применен механизм динамического преобразования задачи миграции.

3.9. УПРАЖНЕНИЯ

1. *Бит четности* двоичного числа добавляется в конец, чтобы сделать общее число битов, включая четный бит, четным или нечетным. Например, к данной 8-битной двоичной строке 10011010 нам нужно добавить 0 для четной четности и 1 для нечетности четности. Средство проверки четности вводит один за другим биты двоичной строки и показывает текущую четность хранимых битов, 0 для нечетной четности и 1 для четной четности. Создайте средство проверки четности с четными и нечетными состояниями, используя автомат FSM, и реализуйте этот FSM на языке Си.
2. Набор задач $T = \{\tau_1, \dots, \tau_5\}$ имеет следующую приоритетную связь. Задача τ_1 не имеет предшественников, она предшествует τ_2 и τ_3 ; задача τ_2 предшествует τ_4 , τ_3 предшествует τ_4 и τ_5 , как показано в табл. 3.3. Задача τ_4 является конечной задачей и не имеет преемников, как показано в табл. 3.3. Сначала нарисуйте граф для этого набора задач. Учитывая характеристики задач в этом наборе, покажите, выполнимо ли планирование с набором из двух процессоров. Покажите планирование с помощью диаграммы Ганта.

Таблица 3.3. Пример набора задач

τ_i	C_i	D_i	Предшественники	Связь
1	2	12	–	(1,2)=3; (1,3)=5
2	4	15	1	(2,4)=4
3	5	20	1	(3,4)=2; (3,5)=6
4	9	30	2,3	
5	12	60	3	

3. Предложите метод приоритетной доставки сообщений в сети реального времени.
4. Протокол Token Bus должен быть реализован в DRTS. Напишите в псевдокоде подпрограмму уровня MAC с циркулирующим основным токеном, которая может использоваться для получения токена, проверки, можно ли его использовать, и при невозможности пересылки его на следующую станцию.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. CAN bus. <http://www.can-cia.org/can/protocol/>.
2. *Erciyes K., Ozkasap O., Aktas N.* (1989) A semi-distributed load balancing model for parallel real-time systems. *Informatica* 19 (1): 97–109 (Special Issue: Parallel and Distributed Real-Time Systems).
3. *Hanssen F., Jansen P., Scholten H., Hattink T.* (2004) RTnet: a real-time protocol for broadcastcapabl networks, University of Twente, Enschede, 2004. <http://www.ub.utwente.nl/webdocs/ctit/1/000000e5.pdf>.
4. *Hayes T., Ali F. H.* (2016) Mobile wireless sensor networks: applications and routing protocols. *Handbook of research on next generation mobile communications systems*. IGI Global. ISBN 9781466687325, pp. 256–292.
5. *Kopetz H., Grunsteidl G.* (1993) TTP – a time-triggered protocol for fault-tolerant real-time systems. In: *IEEE CS 23rd international symposium on fault-tolerant computing, FTCS-23*, Aug. 1993, pp. 524–533.
6. *Martinez J. M., Harbour M. G.* (2005) RT-EP: a fixed-priority real time communication protocol over standard ethernet. In: *Proceedings of Ada-Europe*, pp. 180–195.
7. National Programme on Technology Enhanced Learning. Real-time systems course. Govt. of India.

ЧАСТЬ II

.....

**СИСТЕМНОЕ
ПРОГРАММНОЕ
ОБЕСПЕЧЕНИЕ**

Глава 4

Операционные системы реального времени

4.1. ВВЕДЕНИЕ

Операционная система располагается между аппаратным обеспечением компьютерной системы и приложением, принимая вызовы от приложения и реализуя их в аппаратном обеспечении. Аппаратное обеспечение в типичной вычислительной системе состоит из процессоров, памяти и устройства ввода/вывода. Основной функцией операционной системы является эффективное управление различными аппаратными и программными ресурсами, скрывая от приложения и пользователя детали различных аппаратных и программных компонентов. Самый низкий уровень операционной системы, управляющий непосредственно оборудованием и выполняющий различные низкоуровневые функции, называется *ядром*.

Современные операционные системы основаны на концепции *процесса* (или *выполнении задания*), которая осуществляется этой программой. Одной из основных функций операционной системы является *управление задачами*, которое включает процедуры для создания, удаления и планирования задач. Средства для передачи задач и синхронизации также обеспечиваются операционной системой. Другими ресурсами, которые находятся под управлением операционной системы, являются память, ввод/вывод (I/O) и такое программное обеспечение, как базы данных. Операционная система в режиме реального времени также должна выполнять все эти функции, но всегда с учетом времени. Выполнение должно быть предсказуемо, и все сервисы, которые требуются от операционной системы, должны быть доставлены в установленные сроки. Например, должны быть представлены по расписанию отправка, получение сообщений и их синхронизация. Приложение также должно обеспечиваться возможностью доступа к определенным аппаратным функциям управления. Желаемым свойством операционной системы реального времени является также возможность выбора различных задач при выборе политики планирования.

Мы начинаем эту главу с обзора концепций операционных систем и сравнения общих операционных систем и операционных систем реального времени, отмечая основные отличия. Затем продолжаем рассматривать задачи, память и функции управления вводом/выводом.

4.2. ОБЩИЕ ОПЕРАЦИОННЫЕ СИСТЕМЫ И ОПЕРАЦИОННЫЕ СИСТЕМЫ РЕАЛЬНОГО ВРЕМЕНИ

Операционная система состоит из программных модулей, которые обеспечивают удобный доступ к аппаратным и программным ресурсам. Это программное обеспечение также служит в качестве менеджера эффективного использования ресурсов компьютерной системы. Хотя доступ пользователя непосредственно к аппаратуре в различных компьютерных системах обеспечивается, это условие обычно не используется из-за сложности написания программ, которые напрямую обращаются к аппаратуре. Кроме того, код доступа к аппаратуре в общем случае не будет передаваться на другой компьютер. Основными ресурсами, которые управляются операционной системой, являются процессоры, память, устройства ввода/вывода и такое программное обеспечение, как базы данных.

Операционная система реального времени обладает большинством функциональных возможностей обычной операционной системы, но она должна обеспечить выполнение операций предсказуемо и своевременно. Основные характеристики операционной системы реального времени таковы:

- *обработка прерываний*: прерывания генерируются внешними устройствами, и процедуры, которые обслуживают их, называются *подпрограммами обработки прерываний* (*interrupt service routines, ISR*). ISR обладает высоким приоритетом в общей операционной системе, прерывая выполнение задачи. В системе реального времени прерывание задачи с жестким крайним сроком может привести к потере этого крайнего срока. Поэтому ISR должна быть короткой и быстрой, так как в системе реального времени запрещение дальнейших прерываний на длительное время нежелательно. Обычный подход заключается в том, чтобы иметь ISR в качестве еще одной задачи с высоким приоритетом, которая должна быть планируемой операционной системой реального времени;
- *синхронизация и связь*: функции создания и уничтожения задач, связи и синхронизации должны быть обеспечены своевременно;
- *планирование*: планирование – это процесс назначения процессору готового задания. Основная проблема при составлении расписания в реальном времени заключается в том, что должны быть удовлетворены конечные сроки выполнения задач. Контекстное время переключения должно быть минимальным, что требует эффективных готовых процедур обработки очереди. Должны быть обеспечены средства для планирования периодических, аperiodических и спорадических задач и обеспечено

управление ими. Различные политики планирования в реальном времени, каждая из которых имеет свои преимущества и недостатки, будут подробно рассмотрены в части III;

- *управление памятью*: пространство памяти приложения реального времени должно эффективно управляться операционной системой реального времени. Чтобы быть предсказуемым, в системе реального времени динамическое распределение памяти не является предпочтительным, поскольку необходимо предотвратить нежелательные ожидания и неопределенности. Вся необходимая память обычно выделяется статично во время компиляции, так что ожидание памяти в процессе работы существенно уменьшается. Для управления памятью, однако, все еще необходима возможность повторно использовать пространство памяти, когда оно освобождается. Например, сетевой драйвер, забрав данные из сети, заполнит уже выделенный буфер и доставит его в протокол более высокого уровня. В конце концов, сообщение будет доставлено приложению, которое будет использовать данные в буфере и должно будет вернуть буфер в свободное пространство памяти для дальнейшего использования. Если такая циркуляция свободного пространства не предусмотрена, в течение короткого времени система встретится с недостатком памяти;
- *масштабируемость*: приложение в реальном времени может варьироваться от простого до очень сложного. Поэтому операционная система реального времени должна быть масштабируемой, чтобы обеспечивать диапазон приложений. В некоторых случаях создается две или более версий операционной системы в реальном времени, чтобы система могла удовлетворять как малые, так и большие приложения.

4.3. УПРАВЛЕНИЕ ЗАДАЧЕЙ

Современные операционные системы разработаны с использованием концепции задачи, которая является экземпляром программы, выполняющейся процессором. Наличие ряда задач обеспечивает параллелизм и, следовательно, улучшает производительность. Например, вместо одной программы, ожидающей медленного устройства ввода/вывода или ресурса, который недоступен, задача, которая ждет устройство или ресурс, может быть остановлена, и процессор может быть переключен на другую задачу. Когда ресурс или устройство ввода/вывода становится доступным, ожидающая задача может быть возобновлена. Задача имеет код и некоторые данные, которыми она может использоваться совместно с другими процессами в системе. Как правило, задача реального времени имеет следующие атрибуты:

- *идентификатор задачи*: обычно это уникальное целое число или иногда имя, которое определяет задачу в системе;
- *состояние*: состояние задачи показывает, выполняется ли она, готова к выполнению или находится в ожидании события;

- *приоритет*: в системе реального времени задачи будут иметь разные приоритеты, отражающие порядок их исполнения;
- *счетчик программы*: это регистр, показывающий адрес следующей команды, которая должна быть выполнена задачей;
- *регистры*: текущие значения регистров в процессоре. Эти значения должны храниться и восстанавливаться при переключении задач;
- *указатель стека*: это специальный регистр, который в основном используется для вызова процедур. Он указывает на стек, который является очередью типа «первым пришел – первым вышел» для хранения адресов возврата вызова процедур;
- *период*: для периодических задач это продолжительность периода;
- *абсолютный крайний срок*: крайний срок выполнения задачи в реальном времени в абсолютном масштабе времени;
- *относительный крайний срок*: крайний срок задачи в реальном времени относительно времени ее поступления;
- *указатели памяти*: это указатели на текущие открытые файлы;
- *информация ввода/вывода*: должна быть запомнена текущая информация об устройствах ввода/вывода, независимо от того, размещены они или нет, время и продолжительность их использования и т. д.;
- *статистика*: любая статистическая информация о задаче, например время, потраченное на ожидание, ресурсы и пропущенные крайние сроки.

Вся эта информация хранится в структуре данных, называемой *блоком управления задачами* (*task control block*, TCB). Запущенный процесс может блокировать ожидание события, относительно которого его текущие данные запомнены операционной системой в ее TCB, с тем чтобы его можно было возобновить с последней команды, как если бы он не был заблокирован, когда это событие произошло. При выполнении задача проходит через некоторое число состояний, которые в сжатой форме могут быть описаны следующим образом:

- *готова*: готовая задача имеет все ресурсы, необходимые для выполнения, кроме процессора, который в настоящее время назначен на другую задачу;
- *выполняется*: задача, выполняемая на процессоре, находится в состоянии выполнения;
- *блокирована*: задача, ожидающая ресурса или события, находится в заблокированном состоянии;
- *задержана*: задача задерживается на определенный интервал времени, после которого ее состояние изменяется на состояние *готовности*.

Переходы между этими состояниями изображены на рис. 4.1. Задержка состояния важна в системе реального времени для реализации периодических задач, которые вызываются в регулярные отрезки времени.

Планировщик является центральным компонентом операционной системы, которая выбирает задачу из очереди (или очередей) готовых задач для выполнения процессором. Существуют различные методы планирования, чтобы

определить, какую готовую задачу назначить процессору. Самый низкий уровень планировщика, называемый *диспетчером*, вызывается для сохранения среды текущей задачи в своем ТСВ и восстанавливает среду следующей задачи для исполнения из своего ТСВ. Общие политики планирования в порядке очереди «*первым пришел – первым обслужен*» (*first-come-first-served, FCFS*), *планирование наименьшего задания* (*shortest-job-first, SJF*) и *планирование на основе приоритетов*. Системы реального времени требуют определенных алгоритмов планирования, которые мы подробно рассмотрим в части III.

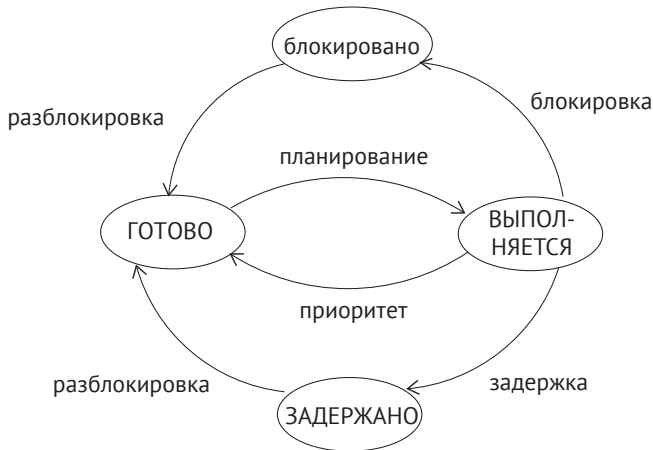


Рис. 4.1. Основные состояния процесса

4.3.1. Задача управления в UNIX

Операционная система UNIX предоставляет различные системные вызовы для управления процессами. Системный вызов *fork* создает процесс с тем же кодом и адресным пространством, что и вызывающий процесс. Созданный процесс становится дочерним элементом (ребенком, *child*) вызывающего процесса, а вызывающий является *родителем* вызываемого (*parent*). Родительский процесс может узнать идентификатор дочернего по возвращенному целочисленному значению из вызова *fork*.

UNIX предоставляет различные схемы взаимодействия процессов; *магистраль* (*pipe*) – это простой способ передавать данные между процессами. Магистраль – это специальный файл FIFO с ограниченной памятью. Две основные операции магистрали – чтение и запись; процесс, который хочет читать из пустой магистрали, блокируется. Для операций чтения и записи необходимо создать магистраль, включить доступ к ней для чтения/записи. Магистрали могут быть объединены с системным вызовом *fork*, чтобы провести обмен данными между родительским и дочерним процессами, как показано ниже в коде Си. Мы имеем родительский процесс, разветвляющий дочерний, для чего используются два массива двух целых чисел *p1* и *p2* сохранения иден-

тификаторов магистрали двунаправленной связи между этими процессами. Родитель записывает содержимое второй половины массива A в магистраль, который имеет идентификатор, сохраненный в p_1 , который один за другим читает ребенок, и вычисляется общая сумма. Эта сумма возвращается родителю через идентификатор магистрали, хранящийся в p_2 , который добавляет к нему вычисленную сумму первой половины A , а затем отображает ее. Обратите внимание, что первый идентификатор магистрали используется для чтения, второй – для записи, p_1 используется для связи между родителем и ребенком, а p_2 применяется для другого направления. Конец магистрали можно безопасно закрыть, как это сделано в этом примере. Если вызов по магистрали возвращает общую ошибку, которой является -1 , нам нужно выйти из программы. Массив A расположен в родителе и ребенке, но инициализация не реализуется в ребенке, так как его начальные значения передаются ребенку родителем.

```
#include <stdio.h>
#define n 8
int i, c, child_sum, p1[2], p2[2], A[n], total_sum, my_sum=0;

main()

{ if (pipe(p1) == -1) {
    exit(2);
  }
  if (pipe(p2) == -1) {
    exit(2);
  }
  c=fork(); // p2 от ребенка к родителю
  if(c!=0) { // это родитель */
    close(p1[0]); // закрыть чтение конец p1
    close(p2[1]); // закрыть чтение конец p2
    for(i=0;i<n;i++) // инициализация массива
      A[i]=i+1;
    write(p[1],&A[n/2],n/2*sizeof(int)); // отправить половину массива
    for(i=0;i<n/2;i++)
      my_sum=my_sum+A[i];
    read(p2[0], &child_sum, sizeof(int)); // прочитать сумму ребенка
    total_sum=my_sum + child_sum;
    printf("Total sum is = %d", total_sum);
  }
  else { /* это ребенок */
    close(p2[0]); // закрыть чтение конец p2
    close(p1[1]); // закрыть чтение конец p1
    read(p1[0], &A[n/2], n/2*sizeof(int)); // прочитать половину массива
    for(i=n/2;i<n;i++)
      my_sum=my_sum+A[i];
    write(p2[1],&my_sum,sizeof(int)); // отправить
  }
}
```

Многозадачная операционная система допускает приоритетность задач и позволяет назначить одно из готовых заданий процессору. Разделение приложения на несколько задач предотвращает трату процессорного времени, но операционная система должна для этих задач обеспечивать средства синхронизации и связи.

4.3.2. Синхронизация задач

Синхронизация задач необходима в двух распространенных случаях: для защиты общих данных от одновременного доступа и когда задача ожидает какого-либо действия от другой задачи или прерывания, чтобы иметь возможность продолжить. *Критическим разделом* задачи является кодовый сегмент, в котором она получает доступ к общим данным. Выполнение задачи в критическом разделе должно взаимно исключать другие задачи, совместно использующие одни и те же данные. Давайте рассмотрим пример, где две задачи $T1$ и $T2$ обращаются к общей переменной t . Они обе увеличивают значение t в выражении языка высокого уровня, таком как $t \leftarrow t + 1$, которое обычно преобразовывается в три команды ассемблера следующим образом:

$T1$:	$T2$:
1. LOAD R1, @t	3. LOAD R2, @t
2. INC R1	4. INC R2
6. STORE R1, @t	5. STORE R2, @t

где R1 и R2 – регистры процессора. Строки перед командами показывают возможный порядок выполнения, где $T1$ останавливает выполнение после увеличения прерыванием, а процессору назначается $T2$, которая завершает последовательность команд. Предположим, что переменная t перед выполнением $T1$ и $T2$ загружена значением 5. Регистр R1 загружается значением 5 и затем увеличивает его до 6, и поскольку $T1$ останавливается, его окружение, включая регистр R1, загружается в его TCB. Теперь запускается задача $T2$, загружает регистр R2 значением 5, увеличивает его и запоминает 6 в t . В этой точке процессору назначается $T1$, и его окружение с R1=6 восстанавливается из его TCB. Наконец, $T1$ запоминает значение 6 переменной t , которая уже имеет это значение. Следовательно, мы увеличили значение t один раз вместо запланированных двух. Этот тип ситуации известен как *состояние соперничества* (*race condition*) и должно быть предотвращено операционной системой обеспечением наличия только одной задачи в ее критическом разделе. Если задачам в приведенном выше примере было бы разрешено завершить все три команды без прерывания, мы бы имели правильную операцию. Один из возможных способов исправить состояние соперничества в приведенном выше примере – отключить прерывания в начале критического раздела и включить их, когда задача завершит выполнение. Однако такое управление приложением имеет недостатки, так как невключение прерываний может нарушить работу всей системы. Поэтому удобнее, чтобы задача синхронизации управлялась операционной системой.

Синхронизация задач (или просто задачи) может быть достигнута на аппаратном уровне, уровне операционной системы или уровне приложения с использованием подходящего алгоритма. Мы будем конкретизировать примитивы операционной системы, которые обеспечивают синхронизацию. Семафор является структурой данных, используемой для синхронизации задач, которая состоит из целого числа и очереди процессов, как показано на рис. 4.2.

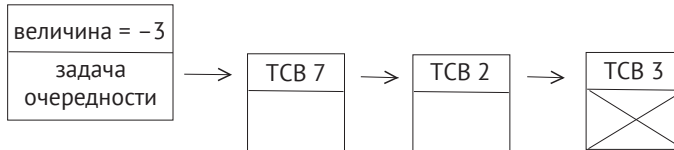


Рис. 4.2. Простой семафор с тремя задачами в его очереди

Два вызова операционной системы (или *примитивов*) на семафорах – это *ожидание* и *сигнал* операции. Простой способ реализовать *ожидание* – уменьшить значение семафора, проверить, не становится ли он меньше нуля, и поставить вызванную задачу в очередь семафора, если значение отрицательное, как показано в алгоритме 4.1. *Сигнал* вызова делает обратное, увеличивая значение семафора, и если это значение равно или меньше нуля, должна присутствовать хотя бы одна ожидающая задача в очереди семафора, которую нужно убрать оттуда. Как показано в алгоритме 4.1, его состояние должно быть изменено на *готовое* и помещено в очередь готовности. Должен быть также запущен планировщик, чтобы эта задача была запланирована или немедленно, или в будущем в зависимости от ее приоритета. Обратите внимание, что сама операция «сделать готовым» (*make ready*) может включать вызов планировщика, что мы и будем реализовывать в образце ядра. *Ожидание* и *сигнальные* операции должны быть атомарными, то есть они должны выполняться без прерывания. Простой способ для достижения этой неделимости – отключить прерывания в начале системного вызова и разрешить прерывания в конце, как и во многих других системных вызовах ядра. Заметим, что данный подход отключения и включения прерываний может быть практически реализован на уровне ядра, поскольку этот код написан и протестирован, но, как говорилось ранее, на уровне пользователя может вызвать проблемы.

Системное приложение реального времени может нуждаться во временном ожидании системного вызова, который проверяет значение семафора, и если это значение равно 0 или меньше, оно само задерживается. После пробуждения он снова проверяет значение семафора и возвращает ошибку, если значение равно 0 или отрицательно, как показано в алгоритме 4.2.

4.3.3. Межзадачные коммуникации

В ходе выполнения задачи должны отправлять данные друг другу. Эта упорядоченная доставка сообщений между задачами обеспечивается операцион-

ной системой. Обычно данные хранятся в буферах, называемых почтовыми ящиками, или портами. Почтовый ящик может быть реализованным семафором отправителя, семафором получателя и очередью буфера, как показано на рис. 4.3. Почтовый ящик имеет ограниченное пространство для хранения указателей на сообщения, и отправитель сообщения блокируется на семафоре отправителя, когда в почтовом ящике нет места. Семафор получателя ставит в очередь задачи, которые пытаются прочитать сообщение из почтового ящика, когда почтовый ящик пуст.

Алгоритм 4.1. *Вызовы системного семафора*

```

1:
2: procedure wait(semaphore s)
3: s.value ← s.value - 1
4: if s.value < 0 then
5:   enqueue the caller in s.queue
6: end if
7: end procedure
8:
9: procedure signal(semaphore s)
10: s.value ← s.value + 1
11: if s.value ≤ 0 then
12:   dequeue the first task from s.queue
13:   make the task ready
14:   call the Scheduler
15: end if
16: end procedure
17:

```

Алгоритм 4.2. *Ожидание вызова семафора с блокировкой по времени*

```

1:
2: procedure wait_timed(semaphore s)
3: if s.value ≤ 0 then
4:   delay myself n_wait_sem time
5: if s.value ≤ 0 then
6:   return NOT_AVAILABLE
7: end if
8: end if
9: s.value ← s.value - 1
10: return DONE
11: end procedure
12:

```

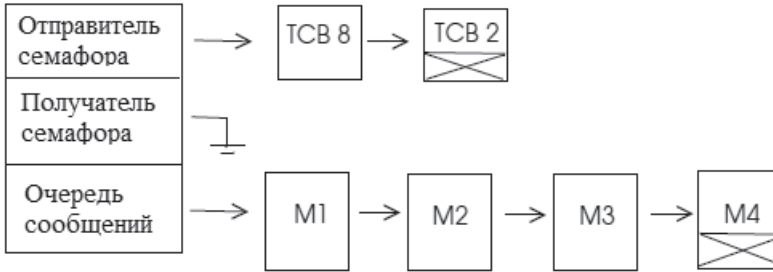


Рис. 4.3. Структура примера почтового ящика. Две задачи T8 и T2 ждут размещения, и 4 сообщения M1, M2, M3 и M4 находятся в очереди для поступления в почтовый ящик

Мы видим, что в примере почтового ящика есть четыре сообщения, что означает, что будет четыре операции получения задач из этого почтового ящика без блокировки на семафоре получателя. Две задачи, T8 и T2, ожидают свободного места в почтовом ящике и будут хранить свои сообщения, когда две задачи получают сообщения M1 и M2 из почтового ящика. Два основных системных вызова в почтовом ящике – операции отправки и получения, как показано в алгоритме 4.3.

Алгоритм 4.3. Системные вызовы почтового ящика

```

1:
2: procedure SEND(mailbox mb, message msg)
3:   wait (mb.send_sem)
4:   enqueue message in mb.message_queue
5:   signal(mb.receive_sem)
6: end procedure
7:
8: procedure RECEIVE (mailbox mb, message msg)
9:   wait (mb.receive_sem)
10:  dequeue message msg from mb.message_queue
11:  signal(mb.send_sem)
12: end procedure
13:

```

В среде реального времени задача не должна блокироваться на неопределенное время на семафоре почтового ящика. Поэтому можно обеспечить отправку и получение процедур с тайм-аутом. Получение с тайм-аутом будет тогда ждать сообщения, а отправка с тайм-аутом будет ожидать свободного места для сообщений в течение указанных интервалов времени, обычно порядка миллисекунд. Если требуемое пространство или сообщение отсутствует после истечения времени ожидания, эти процедуры вернутся с ошибкой, как показано в алгоритме 4.4. В противном случае процедуры отправки и получения работают как блокирующие версии.

4.3.4. Межпроцессорное взаимодействие в UNIX

UNIX предоставляет различные методы межпроцессорного взаимодействия, включая магистрали, которые мы уже рассмотрели. Очереди сообщений и общая память – это две другие базовые схемы связи, предоставляемые UNIX. Общая память защищена семафорами с очередями сообщений, являющимися более общим коммуникационным подходом среди процессов, которым не нужно иметь общую память. Кратко рассмотрим очереди сообщений, поскольку этот метод является общим, а также поскольку мы будем использовать эти процедуры для моделирования сетевых коммуникаций при реализации экспериментального ядра. Очередь сообщений создается системным вызовом *msgget* в следующем формате:

```
int msgget (key_t key, .int msgflg);
```

Он возвращает неотрицательное целое число, которое идентифицирует очередь, и младшие биты из *msgflg* используются для разрешения доступа к очереди.

Алгоритм 4.4. Системный вызов почтового ящика с блокировкой по времени

```
1:
2: procedure send_tout(mailbox mb, message msg)
3: if mb.send_sem.value <= 0 then
4:   delay myself n_wai t time
5: if mb.send_sem.value <= 0 then
6:   return NO_SPACE
7: end if
8: end if
9: wai t (mb.send_sem)
10: enqueue message in mb.message_queue
11: signal(mb.receive_sem)
12: return DONE
13: end procedure
14:
15: procedure receive_tout(mailbox mb, message msg)
16: if mb.receive_sem.value <= 0 then
17:   delay myself n_wai t time
18: if mb.receive_sem.value <= 0 then
19:   return NO_MSG
20: end if
21: end if
22: wai t (mb.receive_sem)
23: dequeue message from mb.message_queue
24: signal(mb.send_sem)
25: return DONE
26: end procedure
27:
```

Ключевая переменная используется в качестве начального числа для создания определенной очереди сообщений. Отправка сообщения в очередь сообщений выполняется процедурой *msgsend*, описанной ниже:

```
int msgsnd (int msqid, const void * msgp, size_t msgsz, int msgflg);
```

где *msqid* – целое число, возвращаемое из системного вызова *msgget*, *msgp* – адрес сообщения для отправки, а *msgsz* – размер сообщения. Поле *msgflg* указывает действие, когда очередь заполнена; *IPC_NOWAIT* означает, что вызов возвращается без ожидания и с установленной ошибкой, и установка этого флага в 0 означает, что отправитель блокируется до тех пор, пока сообщение не будет отправлено, или процесс не найдет сигнал, или очередь не будет удалена из системы. Получение сообщения имеет аналогичный формат, но дополнительно указывает тип сообщения, чтобы разрешить выборочный прием:

```
int msgrcv (int msqid, void * msgp, size_t msgsz, long msgtyp, int msgflg);
```

Он возвращает количество байтов, полученных в случае успеха. Набор полей *msgflg* *IPC_NOWAIT* означает, что процесс получения не должен быть заблокирован, если нет сообщений, и *MSG_NOERROR* указывает, что сообщения должны быть усечены, если выделенное пространство недостаточно велико. Очереди сообщений UNIX требуют копирования данных из пространства пользовательской памяти в пространство ядра при отправке и получении, что может привести к значительному увеличению затрат при больших объемах данных. Кроме того, передача затруднена, поскольку чтение из очереди сообщений удаляет данные из очереди.

4.4. Потоки

Процесс или задача может состоять из нескольких небольших подпроцессов, называемых *потоками* (*threads*). Поток – это самая маленькая единица выполнения процессором. Потоки используются для повышения производительности в связи с низким временем переключения контекста, так как только небольшой объем окружения, обычно состоящий из набора регистров, индивидуальных данных памяти и индивидуальной памяти, должен быть сохранен и восстановлен. Остальная часть памяти является глобальной для всех цепочек одной и той же задачи. Потоки также обеспечивают эффективное совместное использование ресурса и улучшение отклика, например поток, ответственный за обслуживание запроса, с точки зрения затрат будет отвечать на этот запрос быстрее, чем задача активации. Многопоточное приложение также можно удобно запланировать в многоядерной/многопроцессорной системе.

4.4.1. Управление потоками

Используются два типа потоков: *потоки ядра* и *пользовательские потоки*. Поток ядра запланирован ядром и известен ядру, тогда как пользовательский поток действителен только в пользовательском пространстве. Планирование потока ядра требует переключения контекста как задач планирования, однако требует на порядок меньше затрат, чем задача переключения контекста из-за размера данных, хранящихся в его *блоке управления потоками*. Пользовательский по-

ток ядру неизвестен, поэтому поток должен управляться библиотекой потоков в пространстве пользователя, которая предоставляет все утилиты управления потоками, такие как создание и удаление потоков, межпотоквая синхронизация и связь. Пользовательский контекст потока переключения требует гораздо меньше затрат при переключении контекста, чем поток ядра. Блокировка потока ядра ядром не остановит другие потоки задач ядра, но блокировка пользовательского потока на ресурсе заблокирует всю задачу. Поэтому удобно использовать потоки ядра, когда они участвуют в частых операциях ввода/вывода, которые могут заблокировать их. Напротив, пользовательские потоки обычно используются в приложениях, которые для простоты проектирования требуют интенсивных вычислений с редкими командами ввода/вывода и модульности. Потоки обычно используются для параллельной обработки приложений в качестве веб-интерфейса, где многие пользователи могут пытаться подключиться к серверу, или в многопользовательской операционной системе.

4.4.2. Потоки POSIX

Интерфейс переносимой операционной системы (Portable Operating System Interface, POSIX), стандарт IEEE 1003.1c [4], является попыткой стандартизировать UNIX для различных приложений. Многие современные операционные системы соответствуют POSIX и потому обеспечивают переносимость кодов приложений. Этот стандарт определяет *интерфейс прикладного программирования* (*application programming interface*, API) и другие утилиты для обеспечения общего интерфейса разных версий UNIX и иных операционных систем. Потоки в POSIX называются *Pthreads*. Мы остановимся более подробно на утилитах управления потоками POSIX. Подпрограммы, которые включают Pthreads API, могут быть сгруппированы следующим образом [5]:

- *управление потоками* (*Thread Management*): подпрограммы, которые определены непосредственно в потоках, такие как создание, удаление;
- *мьютексы* (*Mutexes*): мьютекс используется для взаимного исключения. Связанные процедуры используются для взаимного исключения при доступе к общему ресурсу;
- *условные переменные* (*Condition Variables*): процедуры, используемые для синхронизации между потоками, которые совместно используют мьютекс;
- *синхронизация* (*Synchronization*): процедуры, используемые для синхронизации через блокировки чтения/записи, барьеры и семафоры.

Следующая функция в интерфейсе POSIX применяется для создания потока и его вызова:

```
pthread_create (& t, NULL, my_thread, (void *));
```

Эта функция создает поток с атрибутами по умолчанию, сохраняет его идентификатор в переменной *t* для дальнейшего доступа и вызывает функцию потока по адресу *my_thread* без передачи ему каких-либо параметров. Как пра-

вило, увеличение счетчика, начиная с 1, используется для отправки значения счетчика каждому созданному потоку, чтобы поток мог использовать это значение в качестве собственного идентификатора вместо идентификатора, назначенного операционной системой.

4.4.2.1. Взаимное исключение

Все потоки задачи разделяют адресное пространство задачи, которое должно быть защищено против гонки условий, как в синхронизации с общей памятью задач. Эта проблема может быть преодолена при наличии критичного раздела потока для взаимного выполнения, исключая другие потоки, которые могут использовать ту же переменную общего доступа. Взаимно исключающие переменные в POSIX объявляются как переменные *mutex_t*, блокируются системным вызовом *mutex_lock* и разблокируются вызовом *mutex_unlock*, как показано в следующем примере, в котором два потока *T1* и *T2* совместно используют переменную *shared* и выполняют безопасные операции с ней, блокируя и разблокируя переменную мьютекса *m*. Эта блокировка будет установлена первым потоком, который обращается к переменной, а другой поток будет ждать, пока этот поток не завершит выполнение критичного раздела и не сбросит блокировку, чтобы подключить ожидающий поток. Обратите внимание, что вывод данной программы может быть 6 или 4 в зависимости от того, какой поток выполняется первым.

```
#include <stdio.h>
#include <pthread.h>

int shared=1;
pthread_mutex_t m;
void *T1(){
    pthread_mutex_lock(&m);
    shared=shared+1;
    pthread_mutex_unlock(&m);
    pthread_exit(NULL);
}
void *T2(){
    pthread_mutex_lock(&m);
    shared=3*shared;
    pthread_mutex_unlock(&m);
    pthread_exit(NULL);
}
main(){
    pthread_mutex_init(&m, NULL); // инициализация мьютекса
    pthread_t t1,t2;
    pthread_create(&t1, NULL, T1, NULL);
    pthread_create(&t2, NULL, T2, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf(" shared = %d", shared);
}
```

Обратите внимание, что основная программа создает потоки и ожидает их завершения вызовом функции *pthread_join*, которая синхронизируется вызовом *pthread_exit*. Основная программа в приведенном выше коде является *main_thread*, которая управляет созданием и прекращением потоков и контролирует их.

4.4.2.2. Синхронизация

Встречаются случаи, когда поток должен ждать сигнала от другого потока. Эта ситуация отличается от ожидания блокировки для взаимного исключения, и семафоры обычно используются для этой цели. Интерфейс POSIX предоставляет типы семафоров и операции *ожидания* (*sem_wait*) и *сигнала* (*sem_post*) для семафоров, как показано в следующем примере с двумя потоками *T1* и *T2*, где *T1* записывает содержимое первого целочисленного массива *data1* в общую целочисленную переменную, совместно используемые одна за другой, и активирует *T2* после каждой записи. *T2* копирует содержимое переменной во второй массив *data2* и сигналы *T1* для продолжения. Нужны два семафора *s1* и *s2*, и без их использования могут быть перезаписаны данные в *общую* переменную. Создание потока и код выхода не отображены. Для правильной операции семафор *s1* инициализируется в 1 и *s2* – в 0.

```
#include <pthread.h>
#include <semaphore.h>

int shared;
int data1[10]={...},data2[10];
sem_t s1,s2;

T1(){
    int i;
    ...
    for(i=0; i< 10; i++)
    { sem_wait(&s1);
      shared=data1[i];
      sem_post(&s2);
    }
    ...
}
T2(){
    int i;
    ...
    for(i=0; i< 10; i++)
    { sem_wait(&s2);
      data2[i]=shared;
      sem_post(&s2);
    }
    ...
}
```

Интерфейс POSIX определяет условные переменные типа *pthread_cond_t* и две процедуры *pthread_cond_wait* и *pthread_cond_signal* в ожидании условной переменной и сигнализирует это. Синхронизация между потоками с применением условных переменных возможна, однако семафоры обеспечивают общую структуру, которая может быть использована для синхронизации и связи.

4.4.2.3. Связь

Связь между потоками возможна посредством общих переменных, защищенных взаимно исключенными переменными, и синхронизации посредством семафоров или условных переменных. Однако обычно для передачи сообщений между потоками в связи с простотой программирования и интерфейса подпрограмм передачи данных используется механизм более высокого уровня. Эта функциональность не обеспечивается интерфейсом POSIX, и поэтому мы опишем простой, новый интерфейс передачи сообщений на языке Си, который использует статично распределенные очереди сообщений, защищенные семафорами, как в [2]. *Буфер* является базовым блоком данных, передаваемых между потоками, а *очередь сообщений* является смежным блоком указателей буфера, который использует *индекс чтения* и *индекс записи*. Два семафора (*fullsem* и *emptysem*) обеспечивают синхронизацию отправки и получения сообщений из очереди, и переменная взаимного исключения (*msgquemu*) используется для защиты индексов от одновременного доступа. Структура очереди сообщений показана в коде Си ниже.

```
/******
```

```
Структура сообщений очереди данных
```

```
*****/
```

```
#define MSGQUEUE_SIZE 10
```

```
#define N_MSGQUES 10
```

```
#define ALLOCATED 1
```

```
#define ERR_MSGQUEUEEMPTY -1
```

```
#define ERR_MSGQUEFULL -2
```

```
typedef struct msgqueue *msgqueueptr_t;
```

```
typedef struct msgqueue{
```

```
    int state ;
```

```
    int msgqueue_size;
```

```
    int read_idx;
```

```
    int write_idx;
```

```
    sem_t fullsem;
```

```
    sem_t emptysem;
```

```
    pthread_mutex_t msgquemu;
```

```
    bufptr bufs[MSGQUEUE_SIZE]; } msgqueue_t;
```

Структура инициализирована, чтобы сохранить число сообщений с семафорами, как показано ниже.

```

/*****
Инициализация очереди сообщений
*****/
int init_msgque(msgqueptr mp) {
    int msgqueid;
    mp->state=ALLOCATED;
    mp->msgque_size=MSGQUE_SIZE;
    sem_init(&mp->fullsem,0,0);
    sem_init(&mp->emptysem,mp->msgque_size,0);
    pthread_mutex_init(&fp->msgquemut,0);
    mp->read_idx=0;
    mp->write_idx=0;
    return(msgqueid); }

```

Отправка к очереди сообщений реализована ожиданием на семафоре отправителя и затем запоминанием сообщения.

```

/*****
Получение буфера из очереди сообщений
*****/
bufptr recv_msgque(msgqueptr mp){
    bufptr bp;
    sem_wait(&fp->fullsem);
    pthread_mutex_lock(&fp->msgquemut);
    bp=fp->bufs[fp->read_idx++];
    fp->read_idx MOD=fp->msgque_size;
    pthread_mutex_unlock(&fp->msgquemut);
    sem_post(&fp->emptysem);
    return(bp); }

```

Получение сообщения вызывающей программой выполняется путем ожидания на получающем семафоре и затем удаления сообщения, как показано ниже. Заметим, что эта структура подобна связи между задачами на основе почтового ящика.

```

/*****
Отправка буфера в очередь сообщений
*****/
bufptr send_msgque(msgqueptr mp, bufptr bp){
    bufptr bp;
    sem_wait(&fp->emptysem);
    pthread_mutex_lock(&fp->msgquemut);
    fp->bufs[fp->write_idx++]=bp;
    fp->write_idx MOD=fp->msgque_size;
    pthread_mutex_unlock(&fp->msgquemut);
    sem_post(&fp->fullsem);
    return(bp); }

```

4.5. УПРАВЛЕНИЕ ПАМЯТЬЮ

Управление памятью является фундаментальной функцией, необходимой в режиме реального времени или общей операционной системе. Распределение памяти можно широко классифицировать как *статическое* и *динамическое распределение*.

4.5.1. Статическое распределение памяти

Статическое распределение памяти – это выделение памяти, необходимой приложению во время компиляции. В этом способе выделение и освобождение памяти во время работы не требуется. В простейшем виде программист выделяет пространство в виде структур, таких как переменные и массивы в коде. Метод управления статичной памятью подходит для систем реального времени, поскольку он является детерминированным, и поэтому этот метод часто используется в операционных системах реального времени. Основная проблема при статичном выделении памяти заключается в том, что мы можем зарезервировать пространство больше или меньше, чем требуется.

4.5.2. Динамическое распределение памяти

Динамическое распределение памяти включает выделение пространства памяти, необходимого во время выполнения задач. Этот метод использует пространство памяти более эффективно, чем статичный, однако он может привести к значительным затратам во время выполнения, и, что еще хуже, пространство может быть недоступно во время выполнения задач. Динамическое распределение памяти может быть выполнено вручную или автоматически. Программист контролирует, когда память выделяется или освобождается в ручном управлении динамической памятью, используя вызовы, обычно предоставляемые языком программирования. Например, язык программирования Си предоставляет функцию *malloc* для резервирования пространства во время выполнения. Выделенное хранилище должно быть освобождено после использования программой посредством вызова (*free* на языке C). При автоматическом распределении памяти язык программирования или его расширение предоставляет автоматический менеджер памяти, обычно называемый сборщиком мусора (*garbage collector*), который собирает неиспользуемую память и возвращает в оборот.

4.5.3. Виртуальная память

Пользовательское приложение не имеет прямого доступа к физической памяти, вместо этого для него предусматривается *виртуальная память*, которая в значительной степени резервируется им самим. При виртуальной памяти нам не требуется, чтобы все адресное пространство задачи находилось в памяти. Нужные код/данные могут быть перенесены при необходимости с диска.

Виртуальная память делится на *страницы*, а физическая память – на *фреймы* фиксированной длины. Виртуальная страница отображается в физическом фрейме, и по существу виртуальную память можно рассматривать как кеш для страниц, хранящихся на диске. Виртуальные адреса преобразуются в адреса физической памяти с помощью *блока управления памятью* (*memory management unit*, MMU), который обычно для ускорения реализуется аппаратно. Для выполнения этого преобразования MMU поддерживает *таблицу страниц*. Страница ссылок не может быть в памяти, поскольку это вызвало бы *сбой страницы*, поэтому эта страница переносится в память с диска. Место для этой страницы должно быть найдено путем удаления существующей страницы. Проблемы, подобные тем, которые мы видели в кешах, проявляются и в управлении виртуальной памятью. Мы должны решить, какую страницу удалить, используя алгоритм замены страниц, например *наименее недавно использованную* (*least recently used*, LRU) или *«первым пришел – первым вышел»* (*first in – first out*, FIFO). Виртуальная память обычно не используется в системах реального времени из-за больших затрат при доступе к страницам.

4.5.4. Управление памятью в реальном времени

Распределение статичной памяти в системах реального времени, как говорилось ранее, удобно ввиду ее детерминированности. Однако приложение реального времени может динамически изменять размеры данных для обработки, что затрудняет оценку размера необходимого пространства при проектировании или при компиляции. В связи с этим обычно используемым подходом в операционных системах реального времени является выделение памяти разделами, называемыми *пулами памяти*, или *буферными пулами* (*memory pools* или *buffer pools*), с фиксированным размером блоков в каждом разделе и динамическое выделение буферов из этих пулов во время выполнения программ. Это промежуточная стратегия между статичным и динамическим распределением использования неиспользуемой памяти для задач, требующих обработки больших массивов данных. На рис. 4.4 показан пул из четырех буферов.



Рис. 4.4. Буферный пул

Два основных системных вызова подается на пул – *получить* (*get*), который возвращает адрес свободного буфера из пула, и *поместить* (*put*), который возвращает адрес используемого буфера в пул, как показано в алгоритме 4.5. Семафор в каждом пуле применяется для блокировки вызывающей задачи, когда

в пуле нет свободных буферов. Задача, которая возвращает буфер в пул, должна подать сигнал семафору, чтобы разблокировать любую задачу, которая может ожидать свободного буфера.

Алгоритм 4.5. Системный вызов буферного пула

```
1:
2: procedure get(pool p, buffer_ptr bp)
3: wait (p.semaphore)
4: dequeue the first free buffer address from the pool to bp
5: return(bp)
6: end procedure
7:
8: procedure put(pool p, buffer_ptr bp)
9: enqueue bp to the pool p
10: signal(p.semaphore)
11: end procedure
12
```

4.6. УПРАВЛЕНИЕ ВВОДОМ/ВЫВОДОМ

Система ввода/вывода состоит из устройств ввода/вывода, контроллеров устройств и программного обеспечения, связанного с этими устройствами. Системы сбора данных и преобразователь аналог–цифра (A/D) являются примерами устройств ввода/вывода в системах реального времени. Контроллер устройства представляет собой электронную схему, которая используется в качестве интерфейса между устройством и процессором. Например, программа записывает некоторую величину в регистр контроллера устройства для выполнения операции записи на это устройство.

4.6.1. Управляемый прерываниями ввод/вывод

Прерывание – это событие, которое останавливает процессор при выполнении текущей задачи. Прерывание может быть получено от внешнего или внутреннего источника, например от таймера, когда он переполняется. Затем вызывается подпрограмма обработки прерывания (ISR), которая обрабатывает запрос, а потом прерванная задача возобновляется. *Время ожидания (interrupt latency)* – это временной интервал прерывания между генерацией прерывания и активацией подпрограммой ISR. Процессор со многими этапами конвейера должен сбросить эти этапы, прежде чем он сможет запустить подпрограмму ISR, и, следовательно, время ожидания прерывания может быть значительным. При обработке *векторного прерывания* устройство, которое вызывает прерывание, снабжает процессор через системную шину адресом ISR для активации. Процессор обычно возвращается к тому же адресу памяти в любом и *невекторном прерывании*. Внешние каналы управления могут пользоваться методом опроса, чтобы обнаружить источник в последнем, как мы видели в главе 2. В системе реаль-

ного времени со многими устройствами ввода/вывода векторное прерывание предпочтительнее, так как при каждом прерывании опрос многих устройств занимает много времени. Некоторые процессоры имеют два или более каналов ввода прерываний на аппаратном уровне с различными приоритетами.

Система реального времени должна обеспечивать уровни приоритета для прерываний, чтобы ISR с низким приоритетом могла быть вытеснена ISR с более высоким приоритетом. ISR может вызывать другие функции и, как правило, снимает блокировку заблокированной задачи, чтобы активировать ее. ISR не должна ждать семафор или мьютекс, так как она может быть заблокирована, что нарушит обработку любых дальнейших прерываний из того же источника. Типы обработчиков прерываний в общих чертах следующие:

- *непривилегированный обработчик прерываний (Non-preemptive Interrupt Handler)*: прерывания отключены до тех пор, пока текущая подпрограмма ISR не завершает выполнение, то есть работающая ISR не может быть прервана. Этот метод не подходит для систем реального времени, которые имеют прерывания с различными приоритетами;
- *обработчик прерываний с вложениями (Preemptive Interrupt Handler)*: прерывания активируются во время работы ISR, делая возможным вложение прерываний. Однако вложение обычно выполняется после некоторых жизненно важных действий, таких как сохранение важных данных. Таким образом, ISR можно считать состоящим из критического раздела, после выполнения которого включаются прерывания некритического раздела. Среди ISR может не быть приоритетов, и при использовании этого метода следует внимательно относиться к размеру стека, так как он увеличивается с количеством вложенных прерываний;
- *привилегированный обработчик прерываний (Prioritized Interrupt Handler)*: ISR имеют различные приоритеты, и вложение ISR допустимо только с более высоким приоритетом. Этот подход является наиболее удобным для системы реального времени, поскольку он отражает внешнюю обработку в подобном окружении.

В заключение отметим: подпрограммы ISR системы реального времени должны быть как можно короче, задержка прерывания должна быть небольшой, и ISR должны быть приоритетными.

4.6.2. Драйверы устройств

Драйвер устройства – это, как правило, программный модуль, который скрывает детали устройства от операционной системы. Драйвер устройства имеет код, специфичный для устройства, и он действует на *контроллере устройства*, который является аппаратным компонентом, управляющим этим устройством. Драйвер диска, например, будет выдавать команды на контроллер диска для управления движением головки диска и передачей данных. *Блок управления устройством (device control block, DCB)* представляет собой структуру данных, которая содержит информацию об устройстве. Как правило, адрес драйвера

устройства хранится в его блоке DCB, а что касается операционной системы, то вызов драйвера устройства из DCB является существенным действием для выполнения требуемой операции.

Драйвер устройства инициализирует устройство, с которым он связан, интерпретирует команды процессора, такие как чтение или запись на устройство, обрабатывает прерывания и управляет передачей данных. Эта многоуровневая программная архитектура для обработки ввода/вывода изображена на рис. 4.5. Аппаратные компоненты – это само устройство и контроллер, который находится внутри него. Когда прикладная программа выдает команду на устройство, такую как считывание количества байтов, операционная система вызывает соответствующую подпрограмму из блока DCB. Подобная абстракция обеспечивает различным устройствам взаимодействие с операционной системой, например в операционной системе UNIX.

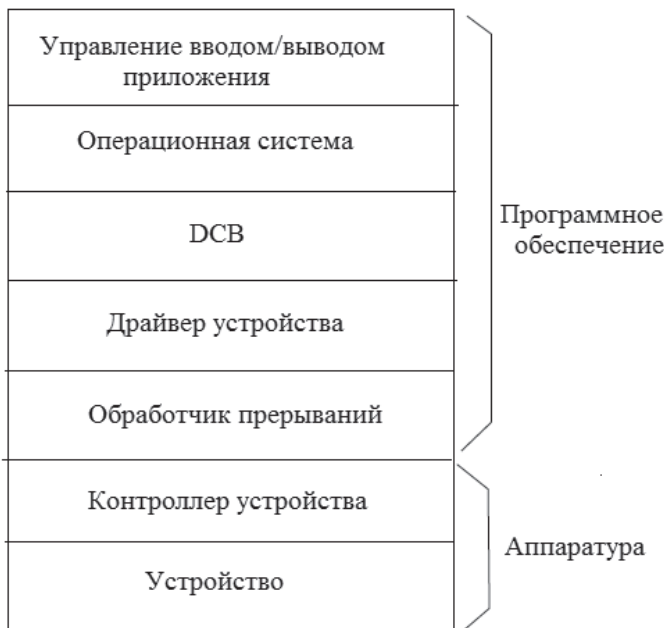


Рис. 4.5. Аппаратура ввода/вывода устройства и структура программного обеспечения

4.7. ОБЗОР ОПЕРАЦИОННЫХ СИСТЕМ РЕАЛЬНОГО ВРЕМЕНИ

В этом разделе мы кратко рассмотрим часто используемые операционные системы реального времени с упором на их структуры, методы для межзадачной синхронизации и связи и обработку прерываний. Эти выбранные нами операционные системы имеют широкий спектр применений от авионики до промышленного контроля.

4.7.1. Операционная система с открытым кодом RTOS

Операционная система с открытым кодом (FreeRTOS) – это простое ядро операционной системы реального времени со встроенными элементами. Она распространяется по лицензии Массачусетского технологического института (Massachusetts Institute of Technology, MIT) [7]. FreeRTOS написана по большей части на языке программирования Си и лишь с небольшой частью целевой архитектуры написана на ассемблере. Система поддерживается множеством процессорных архитектур, включая Intel и ARM. Как и многие другие ядра реального времени, она обеспечивает процедуры управления потоками и задачами, включая семафоры для синхронизации. Обычно ядро занимает небольшое место в памяти от 6 до 12 КБ и работает быстро. Оно широко использовалось в небольших приложениях реального времени, применяющих микроконтроллеры и микропроцессоры.

4.7.2. Операционная система VxWorks

VxWorks – это многозадачная операционная система реального времени, предназначенная для приложений со встраиваемыми элементами, разработанная компанией Винд Ривер Системс (Wind River Systems) [6]. Она использовалась во многих приложениях, включая различные проекты NASA, в том числе Mars Rover и Mars Pathfinder. Поддерживает MIPS, Intel, Power и ARM-архитектуры. VxWorks основан на процессах, которые могут состоять из ряда задач. VxWorks обеспечивает системный уровень, обслуживание задач, управление задачами, управление сетью и функции ввода/вывода. Приоритетное планирование и циклический перебор-планирование – два основных метода назначения задач процессору. Приоритет определяется по потребностям в ресурсах, таких как время и память. Межзадачное общение и синхронизация обрабатывается семафорами, очередями и магистралями. Основные сигнальные процедуры также предоставляются в виде программных прерываний процессов. VxWorks предоставляет различные POSIX-совместимые интерфейсы API. Управление сетью обеспечивается интерфейсом структуры данных или удаленным вызовом процедуры для связи с одинаковым по рангу приложением.

4.7.3. Система реального времени Linux

Linux – бесплатная операционная система, которая имеет большинство основных функций операционной системы Unix. Linux реального времени (RTLinux) был разработан как расширение операционной системы Linux с добавленными возможностями в реальном времени, делающими ее предсказуемой [1]. Основным источником непредсказуемости в операционной системе Linux является планировщик, который оптимизирован для лучшей пропускной способности, а не предсказуемости, обработки прерывания и управления виртуальной памятью.

RTLinux построен как небольшое ядро реального времени, которое работает под Linux и имеет более высокий приоритет, чем ядро Linux, как показано на рис. 4.6. Прерывания передаются ядру RTLinux, а ядро Linux замещается, когда

становятся доступными задачи реального времени. Когда происходит прерывание, оно сначала обрабатывается подпрограммой ISR RTLinux, которая активирует задачу реального времени, в результате чего вызывается планировщик. Передача прерываний между RTLinux и Linux обрабатывается разными версиями RTLinux по-разному.

Операционная система Linux обрабатывает инициализацию устройства, любую динамику блокировки распределения ресурсов и устанавливает компоненты RTLinux [11]. Планировщик и FIFO в режиме реального времени – это два основных модуля RTLinux, которые обеспечивают монотонный рейтинг обработки и политику планирования в RTLinux самых ранних сроков первыми. Интерфейс приложения включает системные вызовы для управления прерываниями и задачами.

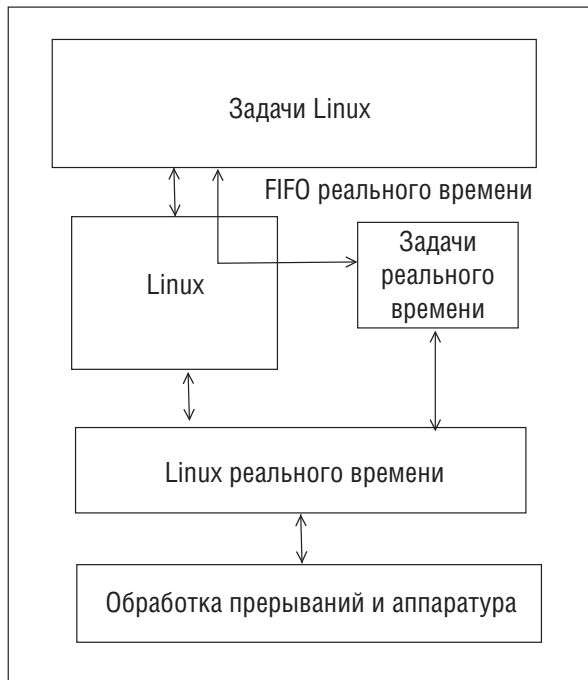


Рис. 4.6. Структура RTLinux. Адаптировано из [11]

4.8. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы общие состояния задачи?
2. Когда задача входит в операционную систему, каким должно быть ее начальное состояние и почему?
3. Какова основная процедура создания новой задачи в UNIX?

4. Можно ли иметь положительные целочисленные значения в обоих семафорах структуры почтового ящика рис. 4.3? Покажите причину.
5. Как поток улучшает скорость отклика приложения в реальном времени? Дайте пример.
6. Сравните преимущества и недостатки статичного и динамического управления памятью. Какой тип управления памятью следует предпочесть в системе реального времени?
7. Что такое буферный пул?
8. Сравните управляемый прерыванием ввод/вывод с опрашиваемым вводом/выводом.
9. Что такое драйвер устройства и контроллер устройства? Кратко опишите интерфейс между ними.
10. Что делает RTLinux операционной системой реального времени?

4.9. ПРИМЕЧАНИЯ К ГЛАВЕ

В этой главе мы подробно рассмотрели общие концепции операционных систем и систем реального времени. Операционная система – это менеджер ресурсов, который также предоставляет приложению удобный интерфейс для выполнения различных задач в компьютерной системе. Современные операционные системы сосредоточены на концепции задачи (процесса), обеспечивающей возможность эффективно выполнять вышеуказанные функции. Операционная система управляет задачами, памятью и системой ввода/вывода. Существуют различные книги по операционным системам, которые описывают эти функции гораздо более подробно, в том числе [8–10].

Операционная система реального времени должна обеспечивать функции общей операционной системы с существенными отличиями. Во-первых, обработка прерываний в операционной системе реального времени требует приоритетности прерывания и, возможно, многоуровневого управления планированием прерываниями. Планирование должно выполняться с учетом сроков выполнения задач, что служит наиболее важным критерием. Распределение памяти обычно статично, что устраняет неопределенность, которая встречается при динамическом распределении памяти. Операционные системы реального времени должны быть масштабируемыми для удовлетворения потребностей очень разнообразного набора приложений реального времени.

4.10. УПРАЖНЕНИЯ ПО ПРОГРАММИРОВАНИЮ

1. Напишите псевдокод часов для обработчика прерываний подпрограммы ISR, который активируется при каждом такте часов, уменьшает величину такта часов в начале *дельта-очереди* (*delta-queue*) и, если начальная задача имеет нулевую величину такта, пробуждает ее и вызывает планировщик.

2. Укажите процедуры *блокировки*, *разблокировки* и *задержки* в псевдокоде.
3. Предоставьте структуру данных для счетного семафора на языке С.
4. Предоставьте процедуры *ожидания* и *сигнала* в псевдокоде для двоичного семафора.
5. Обеспечьте модификацию операции ожидания на двоичном семафоре так, чтобы ожидание в очереди было возможно, когда ресурс недоступен.
6. Предоставьте статичный массив блоков управления задачами (TCB) так, чтобы каждый TCB изначально указывал следующему в массиве. Структура головы массива содержит начало и заднюю часть свободных TCB. Напишите процедуру на языке Си, чтобы выделить свободный TCB из этого массива.
7. Покажите, как можно реализовать структуру буферного пула, используя статичный массив и заголовок.
8. Напишите программу на языке Си, в которой сформированы два потока $T1$ и $T2$. Поток $T1$ считывает с клавиатуры строки символов до конца строки пять раз и отправляет прочитанную строку в $T2$, используя структуру очереди сообщений в разделе 4.4.2.3.
9. Три потока POSIX $T1$, $T2$ и $T3$ работают одновременно. $T1$ периодически получает данные от датчика температуры, и $T2$ периодически получает входные данные от датчика давления. И $T1$, и $T2$ записывают в общее место 2 байта, 1 байт для типа данных (тепло или давление) и 1 байт для величины. Затем активируется $T3$, читает тип и величину и отображает их на экране. Напишите программу на языке Си с короткими комментариями, используя потоки POSIX, обеспечив правильную работу. Не показывайте обработку ошибок, когда система делает вызовы.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Barabanov M., Yodaiken V. (1997) Real-time Linux. Linux J.
2. Erciyes K. (2013) Distributed graph algorithms for computer networks. Springer, App. B.
3. Erciyes K. (1989) Design and realization of a real-time multitasking kernel for a distributed operating system. PhD thesis, Ege University.
4. <http://pubs.opengroup.org/onlinepubs/9699919799/>.
5. <https://computing.lln.gov/tutorials/pthreads/>.
6. https://www.windriver.com/products/product-notes/PN_VE_6_9_Platform_0311.pdf.
7. FreeRTOS open source licensing. www.freertos.org/a00114.html.
8. Silberschatz S., Galvin P. B. (2012) Operating system concepts. Wiley. ISBN 10: 1118063333.
9. Stallings W. (2017) Operating systems, internals and design principles, 9th edn. Pearson. ISBN 10: 0-13-380591-3.
10. Tanenbaum A. (2016) Modern operating systems. Pearson. ISBN 10: 93325.
11. Yodaiken V. (1999) The RTLinux manifesto. In: Proceedings of 5th Linux conference.

Глава 5

Проектирование экспериментального распределенного ядра реального времени

5.1. ВВЕДЕНИЕ

Мы опишем дизайн и реализацию модели *распределенного ядра реального времени* (*distributed real-time kernel*, DRTK) операционной системы UNIX, которую можно использовать для тестирования стратегии дизайна программного обеспечения реального времени. Мы будем расширять функции DRTK на протяжении всей книги и в конце спроектируем и реализуем простое приложение распределенной системы реального времени с использованием DRTK: систему мониторинга окружающей среды с применением беспроводной сенсорной сети. Это ядро, как мы увидим, будет структурировано как ядро операционной системы UNIX и во многих отношениях аналогично [1], но намного меньше и проще, чем UNIX, а также оно будет иметь внутреннюю структуру с аналогичными характеристиками в реальном времени, подобно структуре, описанной в [2]. Построение модели поверх операционной системы не в реальном времени может привести к потере некоторых характеристик ДРТК в реальном времени. Однако наша цель – показать внутреннюю архитектуру и низкоуровневые детали распределенного ядра в реальном времени, а не реализацию ядра реального времени с приемлемой производительностью.

Ядро состоит из нижнего и верхнего модулей. Нижний модуль имеет в основе планировщика, обработку времени и прерываний, управление состоянием задач и управление вводом/выводом в качестве более высоких уровней. Верхний модуль состоит из задачи синхронизации и связи, управления верхней

памятью и управления уровнями задач. Мы расширим это ядро до распределенного, добавив в главе 6 уровень сетевого взаимодействия. Процедуры DRTK должны быть атомарными, возможно реализованными путем отключения и включения прерываний, что для простоты в коде опущено.

5.2. СТРАТЕГИЯ ДИЗАЙНА

Мы будем реализовывать прикладные задачи в реальном времени как потоки POSIX и использовать очередь сообщений UNIX межпроцессорного взаимодействия для имитации сетевого взаимодействия между узлами системы реального времени, как показано на рис. 5.1, где каждый узел в режиме реального времени – это процесс UNIX.

Следующие атрибуты DRTK делают его пригодным для использования в качестве модели ядра реального времени.

- У нас есть политики планирования в режиме реального времени для соблюдения сроков выполнения задач. Это свойство – самый важный аспект системы реального времени.
- Все структуры данных статично объявляются во время компиляции, и поэтому мы не имеем непредсказуемых задержек выделения памяти.
- Синхронизация задач и системные вызовы связи имеют версии *ожидания с блокировкой по времени (waiting-with-time-out)*, которые могут использоваться в системе реального времени.
- Код DRTK сделан максимально коротким и простым.

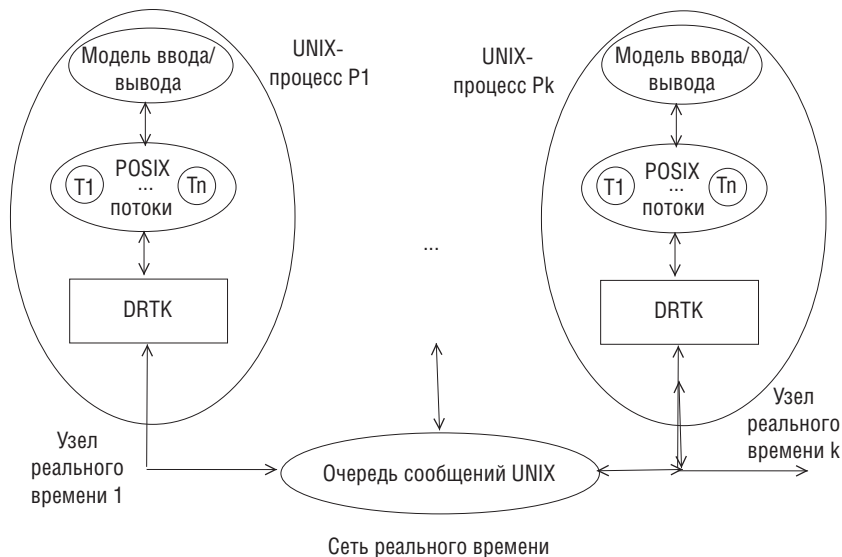


Рис. 5.1. Структура распределенной системы реального времени с использованием DRTK

5.3. Функции ядра нижнего уровня

DRTK нижнего уровня состоит из планировщика, управления часами и обработки прерывания, управления состоянием задачи и управления вводом/выводом, как показано на рис. 5.2. Причина выбора такой иерархии заключается в том, что всякий раз, когда появляется изменение состояния текущей выполняемой задачи, может потребоваться, например, отправить новую задачу, когда текущая задача блокируется в ожидании события. Мы сначала опишем структуры данных и процедуры очереди, которые будут использоваться в DRTK, а затем эти уровни при подходе снизу вверх.

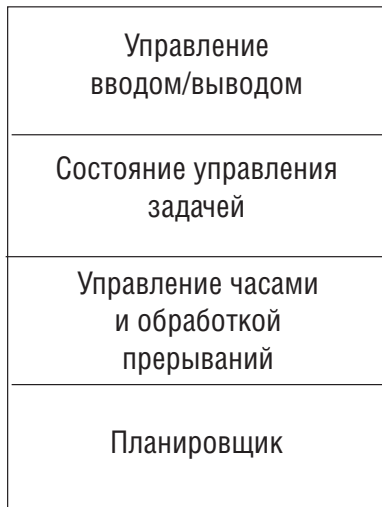


Рис. 5.2. Нижний уровень DTRK

5.3.1. Структуры данных и операции с очередями

Ключевой структурой данных является системная таблица (System_Tab), которая содержит различные системные параметры, такие как максимально допустимое количество задач, максимальное количество пулов и т. д. Это глобальная структура данных в файле «system_data.h», которая может быть доступна всем задачам, находящимся в узле реального времени. Она также содержит идентификаторы системных задач. Некоторые из этих задач будут реализованы в главе 6, но для полноты изложения мы включаем их здесь. Локальные данные, связанные с модулем, обычно включены в его файле-заголовке.


```

/*****
Системная таблица
*****/
typedef unsigned short ushort;
typedef int TASK;

typedef struct {
    ushort this_node; // идентификатор узлов
    ushort N_TASK=120; // структура данных ограниченных величин
    ushort N_SEM=300;
    ushort N_MBOX=100;
    ushort N_MBOX_MSG=100;
    ushort N_GROUP=50;
    ushort N_GROUP_MEM=30;
    ushort N_POOL=20;
    ushort N_POOL_MSG=400;
    ushort N_DEVICE=120;
    ushort N_DATA=1024;
    ushort N_INTERRUPT=32;
    ushort DELAY_TIME=10;
    ushort preempted_tid; // идентификация приоритетной задачи
    ushort DUNIT_LEN;
    ushort DL_Out_id; // идентификатор системной задачи
    ushort DL_In_id; // используется для запроса передачи (RTS)
    ushort DL_Out_mbox;
    ushort DL_In_mbox;
    ...
    sem_t sched_sem; // POSIX семафор для планировщика
    task_queue_ptr_t system_que_pt;
    task_queue_ptr_t realtime_que_pt;
    task_queue_ptr_t user_que_pt;
    task_queue_ptr_t delay_que_pt;
}system_tab_t;

```

5.3.1.1. Тип блока данных

У нас есть блок данных в качестве базовой структуры связи, который нужен для обмена между задачами локально или по сети. Он содержит данные, а также транспортный уровень и заголовки канального уровня для использования в сетевых коммуникациях, которые будут расширены, когда будет рассматриваться сетевая связь. Эта структура объявлена в файле-заголовке «data_unit.h», как показано ниже. Мы также объявляем тип очереди блока данных, которые будут использоваться в качестве очереди блоков данных вместе с типами указателей.

```

// data_unit.h
/*****
Тип блока данных
*****/
typedef struct {
    ushort sender_id;

```

```

    ushort receiver_id;
    ushort type;
    ushort seq_num;
}TL_header_t;

typedef struct {
    ushort type;
    ushort length;
    ushort sender_id;
    ushort receiver_id;
}MAC_header_t;

typedef struct *data_ptr_t{
    TL_header_t TL_header;
    MAC_header_t MAC_header;
    int type;
    char data[N_DATA];
    ushort MAC_trailer;
    data_ptr_t next;
}data_unit_t;
typedef data_unit_t *data_unit_ptr_t;

typedef struct {
    int state;
    data_unit_ptr_t front;
    data_unit_ptr_t rear;
}data_unit_que_t;

typedef data_unit_que_t *data_que_ptr_t;

```

У нас есть следующие функции для получения и сохранения блока данных в очереди блоков данных, содержащихся в файле «data_unit_que.c», а также в функции для проверки наличия данных: очередь пуста или нет. Коды для этих функций содержатся в приложении В.

- enqueue_data_unit (data_que_ptr_t dataque_pt, data_unit_ptr_t data_pt): включает блок данных, показанный, как data_pt, в очереди блоков данных по адресу dataque_pt.
- data_unit_ptr_t data_pt dequeue_data_unit (data_que_ptr_t dataque_pt): выполняет вытеснение первого блока данных из очереди блоков данных по адресу dataque_pt и возвращает свой адрес в data_pt.

5.3.1.2. Тип данных блока управления задачами

Ключевой структурой задачи данных является блок управления задачами (TCB), который содержит всю жизненно важную информацию о ней. Он находится в файле-заголовке «task.h» и имеет содержание, показанное ниже. Мы покажем параметры, необходимые только на данный момент, но расширим их, когда будем расширять структуру DRTK.

```
// task.h
/*****
Состояния задач
*****/
#define RUNNING 1
#define READY 2
#define BLOCKED 3
#define DELAYED 4
#define SUSPENDED 5
#define RECEIVING 6
/*****
Типы задач
*****/
#define SYSTEM 1
#define REALTIME 2
#define PERIODIC 3
#define APERIODIC 4
#define SPORADIC 5
#define USER 6
/*****
Данные, связанные с задачей
*****/
#define VOIDTASK 0
#define TIMEQUANT 10
#define N_REGS 32
```

Тип `task_tab` также объявлен, так как у нас все блоки управления задачами будут храниться в этой таблице, которая статично размещается во время компиляции. Нам также нужно определить структуру очереди задач, которая будет использоваться при планировании и синхронизации задач.

```
/*****
Блок управления задачами
*****/
typedef struct *task_ptr{
  ushort init_address; // адрес начальной задачи
  ushort ISR_address; // адрес прерывания сервисной программы
  int REGS[N_REGS]; // значения регистра
  ushort tid; // определитель задачи
  ushort type; // тип задачи
  ushort priority; // тип приоритета
  ushort active_prio; // действующий приоритет
  ushort state; // состояние задачи
  sem_t sched_sem; // ожидание семафора
  ushort allocation; // расположение блока управления задачей
  ushort mailbox_id; // почтовый ящик задачи
  ushort group_id; // группа, которой задача принадлежит
  ushort group mailbox; // определитель почтового ящика группы
  ushort n_children; // число детей дерева
  ushort mailbox_access[N_MBOX]; // доступ к другому почтовому ящику
  ushort abs_deadline; // абсолютный срок выполнения задачи
```

```

ushort rel_deadline; // относительный срок выполнения задачи
ushort delay_time; // время задержки задачи в тактах
ushort executed; // временная задача выполнена
ushort wset; // худший случай времени выполнения
ushort n_predecessors; // число предшественников
ushort n_successors; // число последователей
ushort predecessors[N_PREDS]; определение задач предшественников
ushort successors[N_SUCCS]; определение задач последователей
task_ptr next; // указатель следующей задачи в очереди
}task_control_block_t;
typedef task_control_block_t *task_control_block_ptr_t,
*task_ptr_t;
typedef task_control_block_t task_tab_t[System_Tab.N_TASK];
typedef struct {
int state;
int n_task;
task_control_block_ptr_t front;
task_control_block_ptr_t rear;
}task_queue_t;

```

У нас есть следующие функции для управления очередями задач, содержащимися в файле «task_que.c». Коды для этих функций приведены в приложении В.

- *enqueue_task (task_que_ptr_t taskque_pt, task_ptr_t task_pt)*: ставит задачу с указателем *task_pt* на очередь задач по адресу *taskque_pt*.
- *task_ptr_t task_pt dequeue_task (task_que_ptr_t taskque_pt)*: исключает первую задачу из очереди задач по адресу *taskque_pt* и возвращает адрес задачи в *task_pt*.
- *insert_task (task_que_ptr_t taskque_pt, task_ptr_t task_pt)*: вставляет задачу с указателем *task_pt* на позицию в очереди задач по адресу *taskque_pt* согласно ее приоритету.

5.3.2. Планировщик с несколькими очередями

Планировщик операционной системы является одним из наиболее часто исполняемых кодов сегментов, которые должны быть короткими и эффективными, поскольку это влияет на всю производительность системы. Мы разрабатываем алгоритмы планирования в реальном времени для DRTK в следующей части книги. Сейчас мы рассмотрим общий планировщик, который работает с тремя очередями: системная очередь без приоритетов «первым пришел – первым обслужен» (FCFS) для системных задач, очередь реального времени с упреждающим приоритетом для жестких задач реального времени и упреждающая очередь пользовательских задач FCFS, как показано на рис. 5.3. Системные задачи, такие как сетевые задачи входа и выхода, служат задачам приложения в реальном времени, поэтому они рассматриваются как задачи с более высоким приоритетом, чем задачи реального времени. Работа планировщика в этой версии DRTK – выбрать задачу с наивысшим приоритетом в соответствии со следующей логикой.

1. Если вызывающая программа (текущая задача) запущена и является системной задачей, следующая задача, которую нужно выполнить, – это вызывающая программа без необходимости в вытеснении (прерывании обслуживания).
2. Если вызывающая программа работает и является задачей реального времени, проверить очередь системных задач. Если она не пустая, удалите первую задачу из этой очереди и выберите ее для запуска в качестве следующей. Если очередь системных задач пуста, проверьте очередь задач реального времени. Если она не пустая, сравните приоритет первой задачи с вызывающей программой. Если вызывающая задача имеет более высокий приоритет, следующая задача – вызывающая программа, в противном случае удалите первую задачу из очереди реального времени и выберите ее для запуска, прервав (вытеснив) текущую задачу.
3. Если вызывающая программа заблокирована, выберите первую задачу в одной из очередей путем поочередной их проверки, начиная с очереди системных задач с наивысшим приоритетом до очереди самых низких пользовательских задач.



Рис. 5.3. Пример операций планировщика

На этом этапе нам нужен интерфейс UNIX, чтобы можно было остановить (прервать) запущенный поток. В нашей реализации *планировщик (Scheduler)* представляет собой системную задачу, представленную потоком POSIX, то есть активируемым системным вызовом расписания (*Schedule*) и вызываемым программой управления состоянием задачи при изменении состояния задачи. Эта программа просто сохраняет идентификатор вытесненной задачи и сигнализирует семафору планировщика, что данный поток ожидает, как это показано ниже

в файле «schedule.c». Обратите внимание, что если планировщик был реализован как функция, а не как задача, или если он не изменял идентификатор текущей задачи, нам не нужно было сохранять идентификатор замещаемой задачи в системной таблице идентификаторов. Эта подпрограмма также останавливает выполнение вызывающей программы своего планирующего семафора. Обратите внимание, что это необходимо даже в случае продолжения текущей задачи.

Поток планировщика всегда работает и активируется только своим семафором, как показано ниже. Обратите внимание, что нам нужно хранить регистры и другие переменные окружения текущей задачи при каждой записи в расписании и восстанавливать регистры и другие переменные окружения выбранной задачи перед выходом этой задачи в реальное приложение. Глобальное значение *current_tid* устанавливается идентификатору следующей задачи, запускаемой в соответствии с планировщиком. Мы не указали, как пользовательская задача может быть прервана, мы будем предполагать, что она переходит в состояние с *задержкой*, когда ее кванты времени истекают.

```
//schedule.c
/*****
Расписание системного вызова
*****/
void Schedule(){

    if (current_pt->state==RUNNING)
        System_Tab.preempted_tid=current_tid;
    sem_post(&(System_Tab.schedule_sem));
    sem_wait(&(task_tab[current_tid].schedule_sem));
}

//schedule.c
/*****
Задача планировщика
*****/
TASK Scheduler()

    task_ptr_t task_pt;
    task_queue_ptr_t task_qpt;
    ushort next_tid;

    while(TRUE) {
        sem_wait(&(System_Tab.sched_sem));
        //сохранить (current_tid); сохранить регистры и окружение

        switch(task_tab[current_tid].state) {
            case BLOCKED:
            case DELAYED:
                taskq_pt=System_Tab.system_que_pt;
                if (taskq_pt->front!=NULL) {
                    task_pt=dequeue_task(System_Tab.system_que_pt);
```

```

}
else {
    taskq_pt=System_Tab.realtime_que_pt;
    if (taskq_pt->front!=NULL) {
        task_pt=dequeue_task(System_Tab.realtime_que_pt);
        next_tid=task_pt->tid;
    }
}
else {
    taskq_pt=System_Tab.user_que_pt;
    task_pt=dequeue_task(System_Tab.user_que_pt);
}
next_tid=task_pt->tid;
break;
case RUNNING:
    if(task_tab[current_tid].type==SYSTEM)
        next_tid=current_tid;
    else if (task_tab[current_tid].type==REALTIME){
        taskq_pt=System_Tab.realtime_que_pt;
        if (taskq_pt->front != NULL)
            task_pt=taskq_pt->front;
        if(task_tab[current_tid].priority>task_pt->priority)
            next_tid=current_tid;
        else {
            task_pt=dequeue_task(System_Tab.realtime_que_pt);
            next_tid=task_pt->tid;
        }
    }
    else next_tid=current_tid;
    break;
}
current_tid=next_tid;
current_pt=&(task_tab[current_tid])
//restore(current_tid); restore registers and environment
sem_post(&(task_tab[current_tid].sched_sem));
}

```

5.3.3. Обработка прерываний и управление временем

Прерывания являются ключом работы системы реального времени, независимо от того, запущена она по времени или вызвана событием. Когда происходит прерывание, вызывается процедура обработки прерывания (*interrupt service routine*, ISR), которая обычно разблокирует заблокированную задачу или обновляет переменную и т. п. Для быстрого ответа ISR в системе реального времени может состоять из низкого и высокого уровней. Низкий уровень выполняется после прерывания, а более высокий уровень обычно задерживается, оставляя его выполняться по расписанию, когда процессор будет не занят. У нас в DRTK для простоты будет одноуровневая ISR-структура. Таблица ISR (*ISR_tab*) содержит адреса ISR, как в методе векторного прерывания, что показано ниже.

```
//isr.h
/*****
Таблица ISR
*****/
typedef struct {
    int (*func_ptr)() [System_Tab.N_INTERRUPT];
}ISR_tab_t;
```

Общий ISR (*ISR_Gen*) затем может быть закодирован, как показано ниже. Он получает номер прерывания и активирует необходимый ISR из *ISR_tab*, как определил этот номер.

```
//isr.c
/*****
Обработчик общего прерывания
*****/
int ISR_Gen(ushort int_num){
    (*ISR_tab[int_num])();
    return(DONE);
}
```

5.3.3.1. Дельта-очередь

Важной функцией, требуемой от ядра реального времени, является задержка задачи, как того требует необходимая периодическая активация. Дельта-очередь – это очередь отложенных задач, которые отсортированы по времени задержки. Любая задача в этой очереди задерживается на суммарный период времени задержки всех задач, предшествующих ей в очереди. Таким образом, только уменьшая время задержки первой задачи в очереди, обеспечивается уменьшение времени задержки всех задач, как показано на рис. 5.4.

Структура данных очереди *Delta_Queue* реализуется в виде очереди задач. Нам нужно предоставить операцию вставки в эту очередь с помощью *insert_delta_queue (task id, n ticks)*, функциональные подробности которой приведены в приложении В. Время задержки вызова может быть меньше, чем время задержки вызова первой задачи в этой очереди, и в этом случае вызов программы помещается в начале очереди, а задержка начала предыдущей задачи устанавливается на ее предыдущее значение минус значение задержки вызывающего абонента. Значение задержки вызова может быть больше, чем первая задача в очереди. В этом случае нам необходимо продвигаться в очереди до первой задачи, в которой общая задержка больше, чем задержка вызова.

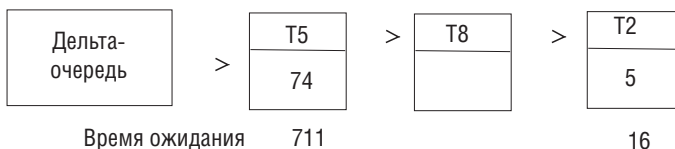


Рис. 5.4. Реализация дельта-очереди

5.3.4. Управление состоянием задачи

Основными функциями этого модуля являются управление переходами задач между состояниями. Мы предполагаем многоуровневый планировщик, но дадим описание различных планировщиков, подходящих для систем реального времени, в следующей части книги. Все эти системные вызовы представлены в файле «*task_state.c*».

В случае если задача с более низким приоритетом удержания ресурса, мы можем захотеть изменить приоритет задачи, в результате чего задача с более высоким приоритетом ожидает и пропускает крайний срок. Эта проблема известна как инверсия приоритетов, которую мы рассмотрим далее.

```
// task_state.c
/*****
Изменение приоритета задачи
*****/
int change_prio(ushort task_id, ushort prio){
    task_ptr_t task_pt;
    if (task_id < 0 || task_id >= System_Tab.N_TASK)
        return(ERR_RANGE);
    task_pt=&(task_tab[task_id]);
    task_pt->priority=prio;
    return(DONE);
}
```

Задача, пытающаяся получить ресурс, может быть заблокирована системным вызовом *block_task*. Мы обеспечили возможность разрешения блокировки задачи другой задачей, как показано ниже. В связи с этим планировщик вводится только в том случае, если вызывающая программа выполняет текущую задачу, то есть она заблокирована из-за отсутствия какого-то ресурса. Остановка текущей задачи осуществляется, как сказано ранее, в функции *Schedule* (расписание). Эта функция сигнализирует семафору, что *Scheduler* (планировщик) ждет, чтобы сделать ее выполняемой.

```
/*****
Блокировка задачи
*****/
int block_task(ushort task_id){
    task_ptr_t task_pt;
    if (task_id < 0 || task_id >= System_Tab.N_TASK)
        return(ERR_RANGE);
    task_pt=&(task_tab[task_id]);
    task_pt->state=BLOCKED;
    if (task_id==current_tid){
        Schedule();
    }
}
```

Разблокировка задачи достигается путем изменения ее состояния в состояние *готовности* и постановки в очередь / вставки ее блока управления в одну из готовых очередей в зависимости от ее типа. Мы проверяем параметр *sched*,

чтобы определить, должен планировщик быть активирован или нет. Это желательно, так как вызывающая эту функцию программа может захотеть продолжить выполнение без необходимости изменения расписания, как в случае выполнения жесткой задачи в реальном времени, которая должна соответствовать крайнему сроку и, следовательно, не должна быть замещена.

```

/*****
Разблокировка задачи
*****/
int unblock_task(ushort task_id, ushort sched){

    task_ptr_t task_pt;

    if (task_id < 0 || task_id >= System_Tab.N_TASK)
        return(ERR_RANGE);
    task_pt=&(task_tab[task_id]);
    task_pt->state=READY;
    switch task_pt->type {
case SYSTEM : enqueue_task(&systask_que, task_pt);
        break;
case REALTIME : insert_task(&realtime_que, task_pt);
        break;
case USER : enqueue_task(&user_que, task_pt);
    }
    if (sched==YES)
        Schedule();
}

```

У задачи может возникнуть потребность в задержке на определенное количество тактов. Системный вызов для этой операции – *delay_task*, который обычно вызывается периодической жесткой задачей реального времени, позволяя отложить выполнение на свой период.

```

/*****
Задержка задачи
*****/
int delay_task(ushort task_id, ushort n_ticks){

    if (task_id < 0 || task_id >= System_Tab.N_TASK)
        return(ERR_RANGE);
    insert_delta_queue(task_id, n_ticks);
    task_tab[task_id].state=DELAYED;
    if (task_id==current_tid)
        Schedule();
}

```

5.3.4.1. Программа обработки прерываний по времени

Программа обработки прерываний по времени (Time_ISR) вызывается при каждом такте, который мы установим 100 мкс. Время задержки задач указы-

вается в виде тактов, и, следовательно, для этого ISR достаточно обеспечить уменьшение величины всех тактов ожидающих задач в дельта-очереди путем уменьшения значения такта часов для первой задачи в этой очереди. Затем проверяется, имеет ли первая задача нулевое значение задержки, и если это так, программа удаляет эту задачу из очереди и делает ее готовой к выполнению, как показано ниже, вызывая процедуру *разблокировки*, которая будет описана в следующем разделе.

```
// isr.c
/*****
Время ISR
*****/
int Time_ISR(){

    task_ptr_t task_pt;

    while(TRUE){
        sleep(System_Tab.N_CLOCK);
        task_pt->delay_time--;
        if (task_pt->delay_time==0){
            task_pt=dequeue_task(delta_queue);
            unblock_task(task_pt->tid,YES);
        }
    }
}
```

5.3.5. Управление вводом/выводом

Каждое устройство ввода/вывода представлено структурой данных блока управления устройством, как в UNIX. Эта структура данных в основном имеет указатели на драйверы устройств, связанные с блоком ввода/вывода. В нашей реализации мы имеем указатели функций для чтения/записи блока байтов от/к устройствам в структуре данных *dev_cont_block* в файле «device.h». Таблица устройства, *dev_tab*, имеет в качестве записей структуру данных *dev_cont_block*.

```
//device.h
/*****
Структура устройства данных
*****/
typedef struct dev_block{
    ushort state;
    func_ptr_t (*read_pt)(char* read_addr, ushort n_byte);
    func_ptr_t (*write_pt)(char* write_addr, ushort n_byte);
    dev_block *dev_pt;
} dev_cont_block_t, *dev_ptr_t;

typedef dev_cont_block_t *dev_cont_block_ptr_t;
typedef dev_cont_block_t dev_tab_t[System_Tab.N_DEVICE];
```

Мы формируем структуру данных *dev_cont_block* с помощью системного вызова *make_dev*, как показано ниже. Он ищет в таблице устройств выделенный блок управления устройством записи, начиная с первой записи:

```
// device.c
/*****
Создание устройства
*****/
int make_device(char* read_addr, char* write_addr){

    ushort i;
    for(i=0; i<System_Tab.N_DEVICE; i++)
        if (dev_tab[i].state!=ALLOCATED){
            dev_tab[i].state=ALLOCATED;
            dev_tab[i].read_pt=read_addr;
            dev_tab[i].write_pt=write_addr;
            return(i);
        }
    return(ERR_NO_SPACE);
}
```

Чтение блока байтов с устройства выполняется системным вызовом *read_dev*, который в основном активирует драйвер чтения устройства посредством блока управления устройством, как показано ниже.

```
/*****
Чтение с устройства
*****/
int read_dev(ushort dev_id, char *data_pt, ushort n_bytes) {

    dev_cont_block_ptr_t dev_pt;
    if (dev_id < 0 || dev_id >= System_Tab.N_DEVICE)
        return(ERR_RANGE);
    dev_pt=&(dev_tab[dev_id]);
    (*dev_pt->read_pt)(data_pt,n_bytes);
    return(DONE);
}
```

Запись блока байтов на устройство выполняется аналогично системному вызову *write_dev*, как показано ниже:

```
/*****
Запись на устройство
*****/
int write_dev(ushort dev_id, char *data_pt, ushort n_bytes) {

    dev_cont_block_ptr_t dev_pt;
```

```

if (dev_id < 0 || dev_id >= System_Tab.N_DEVICE)
return(ERR_RANGE);
dev_pt=&(dev_tab[dev_id]);
(*dev_pt->write_pt)(data_pt,n_bytes);
return(DONE

```

Удаление устройства из системы выполняется системным вызовом *delete_dev*, который отменяет выделение блока управления устройством, выделенного устройству, как показано кодом ниже.

```

/*****
Удаление устройства
*****/
int delete_dev(ushort dev_id){

if (dev_id < 0 || dev_id >= System_Tab.N_DEVICE)
return(ERR_RANGE);
dev_tab[dev_id].state=NOT_ALLOC;
return(DONE);

```

5.4. ФУНКЦИИ ЯДРА ВЕРХНЕГО УРОВНЯ

Функции ядра верхнего уровня используют нижние уровни, которые являются процедурами управления данными, управления состоянием задачи, обработки прерываний и времени, а также устройства управления. Вызовы верхнего уровня могут быть сгруппированы в задачу синхронизации, задачу связи и управления верхней памятью, как показано на рис. 5.5.

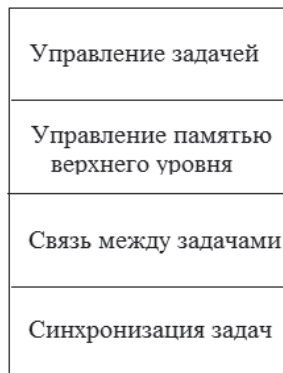


Рис. 5.5. Верхние уровни образца ядра

5.4.1. Синхронизация задач

Семафор – это основной объект синхронизации в DRTK. Таблица семафоров *semaphore_tab* – это массив семафоров, созданных при инициализации си-

стемы. Для выделения элемента этой таблицы используется вызов *make_sema*, инициализация семафора осуществляется с помощью *init_sem*, и сброс для освобождения от всех ожидающих его задач достигается системными вызовами *reset_sem*, как показано ниже.

```
// semaphore.h
/*****
Структура данных семафора
*****/
typedef struct{
    int state;
    int value;
    task_queue_t task_queue;
} semaphore_t;

typedef semaphore_t *semaphore_ptr_t;
typedef semaphore_t semaphore_tab_t[N_SEM];
```

Первая функция, которая нам нужна, – это инициализация семафора *init_sem*, которая устанавливает его значение для данного входного параметра и сбрасывает указатели очереди начальной и конечной задач семафора.

```
// sempahore.c
/*****
Инициализация семафора
*****/
int init_sem(ushort sem_id, int val) {
    if (sem_id < 0 || sem_id >= System_Tab.N_SEM)
        return(ERR_RANGE);
    sem_tab[sem_id].value=val;
    sem_tab[sem_id].task_queue.front=NULL;
    sem_tab[sem_id].task_queue.rear=NULL;
    return(DONE);
}
```

Выделение семафора в таблице семафоров выполняется системным вызовом *make_sem*, который также инициализирует его вызовом *init_sem*. Обратите внимание, что функция *init_sem* может еще использоваться приложением.

```
/*****
Создание семафора
*****/
int make_sema(int value) {

    ushort i;
    for(i=0; i<System_Tab.N_SEM; i++){
        if (sem_tab[i].state!=ALLOCATED){
            sem_tab[i].state=ALLOCATED;
            init_sem(sem_tab[i],value);
        }
    }
}
```

```

    return(i);
}
return(ERR_NO_SPACE);
}

```

Ожидание семафора выполняется системным вызовом *wait_sema*, показанным ниже. Значение семафора уменьшается, и если оно меньше или равно нулю, вызывающий объект заблокирован.

```

/*****
Ожидание семафора
*****/
int wait_sema(ushort sem_id){

    semaphore_ptr_t sem_pt;

    if (sem_id < 0 || sem_id >= System_Tab.N_SEM)
        return(ERR_RANGE)
    sem_pt=&(semaphore_tab[sem_id]);
    sem_pt->value--;
    if (sem_pt->value < 0){
        enqueue_task(sem_pt->task_queue);
        block(current_tid);
    }
    return(DONE);
}

```

Задаче в реальном времени может потребоваться проверить, доступен ли ресурс или, возможно, ей необходимо продолжить без ресурса, чтобы уложиться в срок, когда ресурс отсутствует. Для этой цели существует системный вызов *notwait_sema*, который не блокируется ни при каких обстоятельствах. Вызывающая программа должна проверить возвращаемое значение, чтобы определить порядок действий.

```

/*****
Проверка семафора без ожидания
*****/
int notwait_sema(ushort sem_id){

    semaphore_ptr_t sem_pt;

    if (sem_id < 0 || sem_id >= System_Tab.N_SEM)
        return(ERR_RANGE)
    sem_pt=&(semaphore_tab[sem_id]);
    if (sem_pt->value <= 0)
        return(ERR_RES_NOTAV);
    else
        sem_pt->value--;
    return(DONE);
}

```

Сигнализация семафора выполняется системным вызовом *signal_sema*, в котором значение семафора увеличивается, и если оно меньше или равно нулю, первая задача ожидания в очереди семафора при этом освобождается, а затем снимается блокировка, чтобы она могла быть активирована планировщиком. Наряду с обычной сигнализацией семафора у нас есть параметр *sched* этой функции для определения, должен ли планировщик вызываться с этой операцией. Этот параметр может быть полезен в случае, когда жесткая задача в реальном времени имеет сжатые сроки. Обратите внимание, что такой подход имеет недостаток, так как предоставление приложению данного средства может повлиять на производительность системы, создавая зависание ожидающих на семафоре задач.

```

/*****
Сигнал семафора
*****/
int signal_sema(ushort sem_id, int sched){

    semaphore_ptr_t sem_pt;
    task_ptr_t task_pt;

    if (sem_id < 0 || sem_id >= System_Tab.N_SEM)
        return(ERR_RANGE)
    sem_pt=&(semaphore_tab[sem_id]);
    sem_pt->value++;
    if (sem_pt->value <= 0){
        task_pt=dequeue_task(sem_pt->task_queue);
        unblock_task(task_pt->task_id, sched);
    }
}

```

Нашим последним вызовом в этом модуле является сброс семафора с помощью системного вызова *reset_sema*, который необходим, когда семафор должен быть удален из системы. Удаление семафора осуществляется просто изъятием из таблицы семафора структуры данных и его переустановкой.

```

/*****
Переустановка семафора
*****/
int reset_sema(ushort sem_id, sched){

    semaphore_ptr_t sem_pt;
    task_ptr_t task_pt;

    if (sem_id < 0 || sem_id >= System_Tab.N_SEM)
        return(ERR_RANGE)
    sem_pt=&(semaphore_tab[sem_id]);
    if (sem_pt->value < 0) {
        for(i=sem_pt->value; i<0; i--) {
            task_pt=dequeue_task(sem_pt->task_queue);
            unblock_task(task_pt->task_id,NO);
        }
    }
    sem_pt->state=NOT_ALLOC;
}

```


5.4.2. Коммуникация задач

Передача задач в основном обрабатывается косвенно с помощью объектов *почтовый ящик (mailbox)*. Структура данных почтового ящика имеет очередь блоков данных, состояние, семафор отправителя для блокировки вызывающей программы, когда почтовый ящик заполнен, и семафор получателя, чтобы заблокировать вызывающую программу, когда в почтовом ящике нет сообщений, как показано ниже.

```
// mailbox.h
/*****
Структура почтового ящика данных
*****/
typedef struct{
    int state;
    ushort next;
    semaphore_t send_sem;
    semaphore_t recv_sem;
    data_unit_que_ptr_t queue;
} mailbox_t;

typedef mailbox_t *mailbox_ptr_t;
typedef mailbox_t mbox_tab_t[System_Tab.N_MBOX];
```

Выделение почтового ящика выполняется системным вызовом *make_mbox* обычным образом, получением свободной записи из таблицы почтовых ящиков (*mailbox_tab*). Почтовый ящик-отправитель инициализируется максимальным количеством сообщений, которые могут быть сохранены в почтовом ящике, и семафоры получателя сбрасываются в отсутствие доступных сообщений.

```
// mailbox.c
/*****
Создание почтовых ящиков
*****/
int make_mailbox() {

    ushort i;
    for(i=0; i<system_tab.N_MBOX; i++){
        if (mailbox_tab[i].state!=ALLOCATED){
            mailbox_tab[i].state=ALLOCATED;
            sem_id=make_sema(System_Tab.N_MBOX_MSG);
            mailbox_tab[i].send_sem=sem_id;
            sem_id=make_sema(0);
            mailbox_tab[i].recv_sem=sem_id;
            mailbox_tab[i].queue->front=NULL;
            mailbox_tab[i].queue->rear=NULL;
            return(i);
        }
    }
    return(ERR_NO_SPACE);
}
```

Отправка сообщения в почтовый ящик осуществляется с помощью *send_mbox_notwait*, когда отправитель не хочет ждать ответа, чтобы убедиться, что сообщение получено приемником. Однако вызывающая программа должна установить *ожидание* почтового ящика на семафоре отправителя, чтобы обеспечить место для размещения сообщения.

```

/*****
Отправка сообщения в почтовый ящик
*****/
int send_mailbox_notwait(ushort mbox_id, data_unit_ptr_t data_pt){

    mailbox_ptr_t mbox_pt;

    if (mbox_id < 0 || mbox_id >= System_Tab.N_MBOX)
        return(ERR_RANGE)
    else mbox_pt=&(mailbox_tab[mbox_id]);
    wait_sema(mbox_pt->send_sema);
    enqueue_data(mbox_pt->queue, data_pt);
    signal_sema(mbox_pt->recv_sema);
    return(DONE);
}

```

Получение из почтового ящика осуществляется путем ожидания на принимающем семафоре и затем извлечением сообщения, как показано ниже.

```

/*****
Получение сообщения из почтового ящика при ожидании
*****/
data_unit_ptr_t recv_mailbox_wait( ushort mbox_id){

    mailboxptr_t mbox_pt;
    data_unit_ptr_t data_pt;

    if (mbox_id < 0 || mbox_id >= System_Tab.N_MBOX)
        return(ERR_RANGE)
    else mbox_pt=&(mailbox_tab[mbox_id]);
    wait_sema(mbox_pt->recv_sema);
    data_pt=deque_data_unit(mbox_pt->queue);
    signal_sema(mbox_pt->send_sema);
    return(DONE);
}

```

Приложению реального времени может потребоваться проверить почтовый ящик на наличие сообщения, а затем продолжить без блокировки, если нет сообщений. DRTK обеспечивает системный вызов *recv_mbox_notwait* для этой цели, как показано ниже. Если нет доступных сообщений, возвращается указатель NULL.

```

/*****
Получение сообщения из почтового ящика без ожидания
*****/
data_unit_ptr_t recv_mailbox_notwait( ushort mbox_id){

    mailbox_ptr_t mbox_pt;
    data_unit_ptr_t data_pt;

    if (mbox_id < 0 || mbox_id >= System_Tab.N_MBOX)
        return(ERR_RANGE)
    else mbox_pt=&(mailbox_tab[mbox_id]);
    if(mbox_pt->recv_sem.value > 0) {
        wait_sema(mbox_pt->recv_sem);
        data_pt=deque_data_unit(mbox_pt->queue);
        signal_sema(mbox_pt->send_sem);
        return(data_pt);
    }
    return(NULL);
}

```

В системе реального времени необходима также процедура приема с перерывом. Для этой цели используется системный вызов *recv_mbox_timeout*, который проверяет почтовый ящик вызывающего абонента, и если нет никаких сообщений, он задерживается на определенное количество тактов, и затем почтовый ящик снова проверяется.

```

/*****
Получение сообщения из почтового ящика с перерывом
*****/
data_unit_ptr_t recv_mailbox_timeout( ushort mbox_id){

    mailbox_ptr_t mbox_pt;
    data_unit_ptr_t data_pt;

    if (mbox_id < 0 || mbox_id >= System_Tab.N_MBOX)
        return(ERR_RANGE)
    else mbox_pt=&(mailbox_tab[mbox_id]);
    if(mbox_pt->recv_sem.value >0){
        wait_sema(mbox_pt->recv_sem);
        data_pt=deque_data_unit(mbox_pt->queue);
        return(data_pt);
    }
    else delay_task(current_tid, System_Tab.N_DELAY);
    if(mbox_pt->recv_sem.value >0){
        wait_sema(mbox_pt->recv_sem);
        data_pt=dequeue_data_unit(mbox_pt->queue);
        return(data_pt);
    }
    return(ERR_NO_MSG);
}

```

5.4.3. Управление верхней памятью с использованием пулов

Высокий уровень управления памятью в DRTK достигается за счет использования *блока пулов данных*. Задача, которая должна передавать данные другой задаче, получает блок данных из пула, вставляет свои данные в блок данных и передает блок данных в почтовый ящик получателя задачи. Получатель использует данные в сообщении и обычно возвращает блок данных обратно в пул. Таким образом, повторное использование пространства памяти осуществляется без истощения ее объема. Сначала мы определим структуру пула *pool_t* в файле «pool.h», который имеет семафор и который нужно проверить на наличие доступного блока данных и массива блоков данных. Тип таблицы пула (*pool_tab_t*) – это массив пулов, выделенных во время компиляции, в соответствии с нашей стратегией статического распределения памяти для детерминированного времени выполнения.

```

/*****
Структура пула блоков данных
*****/
// pool.h

typedef struct{
    int state;
    semaphore_t sem;
    unsigned int next;
    data_unit_t queue[System_Tab.N_POOL_MSG];
} pool_t;

typedef pool_t *pool_ptr_t;
typedef pool_t pool_tab_t[System_Tab.N_POOL];

```

Пул выделяется путем поиска неиспользуемого в таблице пулов *pool_tab* в файле «pool.c», как показано ниже. Когда такая запись найдена, она выделяется, и ее семафор записи инициализируется значением, указанным в системной таблице, и считывание для семафора указывается равным 0.

```

// pool.c
/*****
Выделение пула
*****/
int make_pool(){

    ushort i, sem_id;

    for(i=0; i<System_Tab.N_POOL; i++){
        if (pool_tab[i].state!=ALLOCATED){
            pool_tab[i].state=ALLOCATED;
            if(sem_id=make_sem(System_Tab.N_POOL_MSG)<0)
                return(ERR_NO_SPACE);

```

```

    pool_tab[i].sem=sem_id;
    return(i);
}
return(ERR_NO_SPACE);
}

```

Функция *get_data_unit* используется для получения свободного блока данных из указанного пула. Если нет доступных блоков данных, вызывающая программа блокируется на семафоре отправки пула.

```

/*****
Получение data_unit из пула
*****/
data_unit_ptr_t get_data_unit_(ushort pool_id) {

    pool_ptr_t pool_pt;
    data_unit_ptr_t data_pt;

    if (pool_id < 0 || pool_id >= System_Tab.N_POOL)
        return(ERR_RANGE)
    else pool_pt=&(pool_tab[pool_id]);
    wait_sema(pool_pt->sem);
    data_pt=dequeue_data_unit(pool_pt->queue);
    return(data_pt);
}

```

Функция *put_data_unit* используется для размещения используемого блока данных в указанный пул.

```

/*****
Размещение data_unit в пуле
*****/
int put_data_unit( ushort pool_id, data_unit_ptr_t data_unit_pt){
    pool_ptr_t pool_pt;
    data_unit_ptr_t data_pt;

    if (pool_id < 0 || pool_id >= System_Tab.N_POOL)
        return(ERR_RANGE)
    else pool_pt=&(pool_tab[pool_id]);
    enqueue_data_unit(pool_pt->queue, data_pt);
    signal_sema(pool_pt->sem);
    return(DONE);
}

```

5.4.4. Управление задачами

Первое, что нужно сделать, – это создать задачу, инициализировать ее переменные, назначить ей состояние *готовности* и вставить ее в соответствующую очередь с помощью системного вызова *make_task*. Эта функция имеет входной

параметр *sched*, который используется для определения того, должен ли быть введен планировщик после создания задачи.

```
// task.c
/*****
Создание задачи
*****/
int make_task(task_addr_t task_addr, ushort type, int priority,
ushort sched) {

    task_ptr_t task_pt, ushort i;
    for(i=0; i<System_Tab.N_TASK; i++)
    if (task_tab[i].state!=ALLOCATED){
        task_tab[i].state=ALLOCATED;
        task_pt=&(task_tab[i]);
        task_pt->address=task_addr;
        task_pt->type=type;
        task_pt->priority=priority;
        task_pt->state=READY;
        switch (task_pt->type) {
            case SYSTEM : enqueue_task(&system_queue, task_pt);
                break;
            case REALTIME : insert_task(&realtime_queue, task_pt);
                break;
            case USER : enqueue_task(&user_queue, task_pt);
        }
        pthread_create(NULL, NULL, task_addr, NULL);
        if (sched) Schedule();
        return(i);
    }
    return(ERR_NO_SPACE);
}
```

При удалении задачи она удаляется из DRTK, освобождая ее блок управления задачами. Имеющиеся у нее ресурсы также должны быть освобождены для использования другими задачами, которые для простоты мы не показываем.

```
/*****
Удаление задачи
*****/
int delete_task( ushort task_id ){

    task_ptr_t task_pt;
    if (task_id < 0 || task_id >= System_Tab.N_TASK)
        return(ERR_RANGE);
    task_pt=&(task_tab[task_id]);
    task_pt->allocation=NOT_ALLOC;
    // освобождение ее ресурсов
    return(DONE);
}
```

5.5. Инициализация

Мы используем второй способ доступа к глобальным данным в файле «global_data.h», который в основном предназначается для параметров, необходимых для задач приложения, тогда как системная таблица содержит данные для системных задач в целом. Этот файл также включает в себя все необходимые заголовки и объявление всех таблиц вместе с кодами ошибок, следовательно, только включение этого файла предоставит все необходимые структуры данных и очевидные для данной задачи приложения.

```
// global_data.h
/*****
Глобальные данные
*****/
#define DONE 1
#define YES 1
#define NO 0
#define ERR_RANGE -1 // ошибка диапазона
#define ERR_NO_SPACE -2 // в таблице отсутствует место
#define ERR_RES_NOTAV -3 // ресурсы недоступны
#define ERR_NO_MSG -4 // в почтовом ящике отсутствуют сообщения
#define ERR_INIT -5 // ошибка инициализации
#define ERR_NOTAV -6 // не доступен
#define UNICAST 0x00 // типы сообщений в сети
#define MULTICAST 0x10
#define BROADCAST 0x20
system_tab_t System_Tab; // размещение системной таблицы
task_control_block_t task_tab[system_tab.N_TASK]; // таблица задач
task_queue_t system_queue; // очереди планировщика
task_queue_t realtime_queue;
task_queue_t user_queue;
task_queue_t delta_queue; // задержка очереди задачи
device_tab_t dev_tab; // таблица устройств
semaphore_tab_t sem_tab; // таблица семафоров
mailbox_tab_t mailbox_tab; // таблица почтовых ящиков
pool_tab_t pool_tab; // таблица пулов
ushort current_tid; // идентификация текущей задачи
task_ptr_t current_pt; // указатель текущей задачи
```

Процедура инициализации *init_system* в файле «init.c» распределяет основные пулы блоков данных и создает системные и прикладные задачи.

```
// init.c
/*****
Инициализация системы
*****/
int init_system(){
    int pool_id;
    int task_id;
```

```

if(pool_id=make_pool() < 0) // создание пула сети
return(ERR_INIT);
System_Tab.network_pool_id=pool_id;
if(pool_id=make_pool() < 0) // создание пользовательского пула
return(ERR_INIT);
System_Tab.userpool1=pool_id; //

if(task_id=make_task(DL_Out, SYSTEM, 0, NO)<0)
return(ERR_INIT);
System_Tab.DL_Out_id=task_id;
/* make all other needed system tasks such as Timer_ISR,
Clock_Synch, Leader_Elect etc. */
if(task_id=make_task(Scheduler, SYSTEM, 0, YES)<0)
return(ERR_INIT);

```

Для простоты все файлы указателей включены в главный файл «drtk.h», как показано ниже:

```

//drtk.h
/*****
Модули DRTK
*****/
#include "system_data.h"
#include "global.h"
#include "data_unit.h"
#include "task.h"
#include "isr.h"
#include "device.h"
#include "semaphore.h"
#include "mailbox.h"
#include "pool.h"
#include "global_data.h"

```

Модули, описанные до сих пор, были включены как исходный код на языке Си в файл «drtk.c», как показано ниже. Любое приложение реального времени просто включает этот файл, чтобы запустить DRTK.

```

/*****
Модули DRTK
*****/
#include "data_unit_queue.c"
#include "task_queue.c"
#include "schedule.c"
#include "isr.c"
#include "task_state.c"
#include "device_man.c"
#include "semaphore.c"
#include "mailbox.c"
#include "task.c"
#include "pool.c"
#include "init.c"

```


5.6. ТЕСТИРОВАНИЕ DRTK

У нас есть симуляция DRTK в среде UNIX, где симулируется каждая задача по потоку POSIX. Чтобы симуляция соответствовала реальной ситуации настолько, насколько это возможно, поток POSIX, запущенный DRTK, *готов* и помещен в готовую очередь, ожидая запуска планировщиком. Сам планировщик – это POSIX-поток, который всегда работает и активируется только своим семафором. Также мы должны иметь задачи, которые являются потоками POSIX для выполнения в соответствии с решением планировщика вместо случайного выполнения потоков базовой операционной системой UNIX. Для достижения этой функции каждый поток при активации сначала ожидает своего семафора POSIX. Планировщик будет сигнализировать семафору потока, когда это запланировано. У нас есть пример производителя/потребителя, кодирования, который показан ниже:

```
// test.c
# включение <stdio.h>
# включение <pthread.h>
# включение <synch.h>
# включение "drtk.h"
# включение "drtk.c"
/*****
Создатель
*****/
char c;
semaphore_t sem_prod, sem_cons;

TASK Producer(int *me){
    sem_wait(&(task_tab[me].sched_sem));

    do {
        c=getc();
        signal_sema(sem_cons);
        wait_sema(sem_prod);
    } while (c!=EOL)
}
/*****
Потребитель
*****/
TASK Producer(void *me){
    sem_wait(&(task_tab[me].sched_sem));

    do {
        wait_sema(sem_cons);
        putc(c);
        signal_sema(sem_prod);
    } while (c!=EOL)
}
```

```

void main(){
    init_system();
    if((sem_prod=make_sema() < 0)|| (sem_cons=make_sem() < 0))
        return(ERR_SYS);
    make_task(Producer, USER, 2, NO);
    make_task(Consumer, USER, 1, YES);
}

```

Для раздельной компиляции нам необходимо компилировать DRTK, тестировать коды и затем сделать их исполнимыми показанным ниже образом. Затем мы запускаем исполнимый *тест*.

```

gcc -c drtk.c
gcc -c test.c
gcc -o test drtk.o test.o -lpthread

```

5.7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы основные характеристики DRTK, которые делают его ядром реального времени?
2. Какие другие поля могут быть необходимы в блоке управления задачами в реальном приложении?
3. Какова функция *состояния* поля в структуре данных в DRTK?
4. Почему для правильной работы планировщика нужен реальный семафор POSIX DRTK?
5. Какие системные вызовы для синхронизации задач в DRTK могут понадобиться приложению в реальном времени?
6. Какие системные вызовы для передачи задач в DRTK могут понадобиться приложениям в реальном времени?
7. Зачем нужны системные вызовы с тайм-аутами в DRTK?

5.8. ПРИМЕЧАНИЯ К ГЛАВЕ

В этой главе мы описали разработку и реализацию распределенного симулятора ядра в реальном времени. Уровни DRTK практически универсальны. Они похожи на уровни различных ядер других операционных систем, с планировщиком внизу, управлением состоянием системных вызовов задач, при необходимости активирующих планировщик, процедурами синхронизации задач на семафорах с вызовом подпрограммы управления состоянием задачи и, наконец, подпрограммы межзадачной связи для отправки/получения сообщений с использованием почтовых ящиков путем вызова семафорных процедур. Подход к проектированию структуры данных аналогичен UNIX с каждым объектом идентифицируемым целым числом, которое является его индексом в статично распределенной таблице. Доступ к устройству осуществляется через блок

управления устройством с глобальными операциями чтения и записи, которые вызывают драйверы устройств, адресованных в этой структуре данных, аналогично внутренним компонентам UNIX.

Вместе с тем системные вызовы реализованы с целью применения в приложениях реального времени. Например, планировщик предназначен для работы в режиме реального времени с очередью в реальном времени, а системные вызовы семафоров и почтовых ящиков могут возвращаться без блокировки и по истечении времени ожидания. Сетевые коммуникации и различные функции промежуточного программного обеспечения будут реализованы на верхнем уровне этого ядра в следующих главах, чтобы иметь возможность вызывать его в распределенной среде реального времени.

5.9. ПРОЕКТЫ ПРОГРАММИРОВАНИЯ

1. Операции *постановки в очередь* и удаления из блоков данных и блоков управления задачами аналогичны. Покажите, как реализовать одну очередь и одну функцию удаления очереди для обоих типов данных.

Таблица 5.1. Пример набора задач

τ_i	C_i	T_i
1	3	18
2	8	36
3	5	24

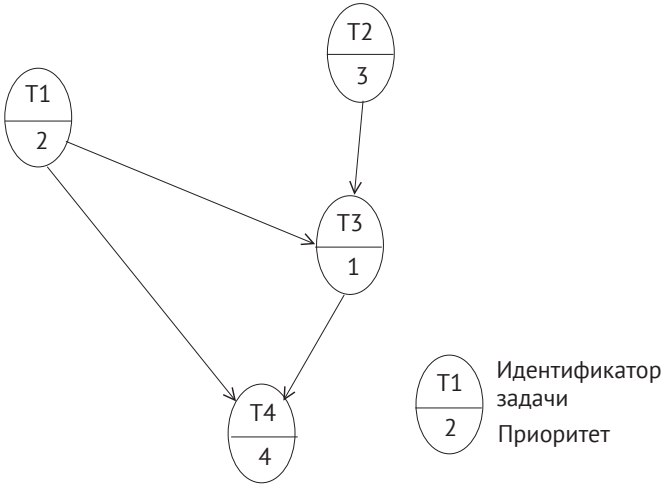


Рис. 5.6. Граф для упражнения 4

2. Напишите тестовую программу, которая является производителем, потребители которого обмениваются данными с помощью сообщений, используя почтовые ящики.
3. Три независимых периодических сложных задания реального времени с указанными в табл. 5.1 характеристиками должны быть реализованы с использованием DRTK. Напишите прикладную программу на языке Си с помощью всех необходимых системных вызовов DRTK.
4. Четыре зависимые аperiodические задачи в реальном времени показаны на графике задач на рис. 5.6. Задача T_1 и T_2 получает 20 символов от пользователя, T_1 отправляет первые 10 символов для задачи T_3 и вторые 10 символов для задачи T_4 . Задача T_2 отправляет все 20 символов задаче T_4 , которая печатает все полученные данные. Сформируйте все эти задачи и основные программы на языке Си, которые реализуются с помощью DRTK, где основное общение осуществляется по почтовым ящикам.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Comer D. (2015) Operating system design: theXINU approach, 2nd edn. Chapman and Hall/CRC.
2. Erciyes K. (1989) Design and realization of a real-time multitasking kernel for a distributed operating system. PhD thesis, Ege University.

Глава 6

Операционные системы реального времени и промежуточное программное обеспечение

6.1. ВВЕДЕНИЕ

Основным требованием распределенной операционной системы (*distributed operating system*, DOS) является управление ресурсами в сети, причем сама сеть является одним из таких ресурсов. Когда распределенная операционная система является системой реального времени (DRTOS), необходимо добавить *своевременное управление*. Упрощенная модель стандарта OSI (*open interconnection system*, OSI) распределенной системы реального времени (DRTS) обычно состоит только из канального уровня, физического уровня и, возможно, небольшого транспортного уровня. Система DRTOS должна обеспечить эффективный интерфейс функций канального уровня при управлении доступом к среде и логическое управление канальным уровнем. При этом DRTOS должна обеспечивать характеристики сети в режиме реального времени, отвечающие требованиям приложения. Например, когда прикладная задача реального времени отправляет сообщение с тайм-аутом задаче, находящейся в другом узле сети, эта функция должна быть переведена в функцию передачи данных, выполняющую необходимую операцию в режиме реального времени. Во многих приложениях систем DRTS некоторое подмножество задач вовлечено во взаимодействие для завершения выполнения общей задачи. Это требует многоадресной связи и реализации группирования задач (или процессов), что является эффективным методом для достижения необходимой формы взаимодействия. Дополнительно группирование задач обычно используется

для их резервирования, обеспечивая отказоустойчивость. Все копии задачи включаются в группу и выполняют один и тот же код. Если основная задача не выполняется, в качестве основной задачи для запуска выбирается копия. Кроме того, чтобы в группе сохранялось согласованное состояние, порядок сообщений, отправляемых на все копии в группе, должен быть одинаковым, и реализация такой функции требует детального анализа. Управление целевой группой обычно осуществляется DRTOS или иногда промежуточным программным обеспечением. Процессоры являются основным управляемым ресурсом в системе DRTOS. Статичное планирование задач для процессора может быть получено автономной и динамической балансировкой нагрузки и требует справедливого распределения нагрузки при выполнении задачи. Мы оставляем обзор этих методов для следующей части книги, где планирование в DRTS рассматривается детально.

Промежуточное программное обеспечение – это набор программных модулей, которые находятся между приложением и операционной системой. Это определение охватывает отдельные вычислительные узлы, а чаще распределенные системы. Приложение обычно запрашивает услугу у промежуточного программного обеспечения, а промежуточное программное обеспечение вызывает функцию операционной системы для реализации запроса. Основная причина реализации такого уровня программного обеспечения – обеспечение повторного использования. Многие разнообразные приложения требуют функций, которые не содержатся в общих задачах операционной системы, таких как обработка, память, ввод/вывод и управление процессами. Разработка и реализация этих задач для каждого приложения во многих случаях не нужны. Работа промежуточного программного обеспечения реального времени похожа на работу обычного промежуточного программного обеспечения с добавлением ряда функций, необходимых в режиме реального времени. Некоторые промежуточные модули могут быть явно необходимы в DRTS. В этой главе мы рассмотрим типичные модули промежуточного программного обеспечения, которые необходимы в большинстве систем DRTS: синхронизацию часов между узлами DRTS, группирование задач и выбор основных групп. Мы завершаем главу реализацией интерфейса сети, группированием задач, синхронизацией часов и выбором основной группы на примере распределенного ядра реального времени (DRTK).

6.2. РАСПРЕДЕЛЕННЫЕ ОПЕРАЦИОННЫЕ СИСТЕМЫ РЕАЛЬНОГО ВРЕМЕНИ

Как было сказано, упрощенная модель OSI обычно имеет канальный уровень, физический уровень и требуемый приложением интерфейс транспортного уровня. Обратите внимание, что мы можем исключить сетевой уровень, предполагая, что сеть в реальном времени локализована без необходимости значительных процедур маршрутизации. Важное требование к DRTOS заключается

в управлении функциями уровня канала передачи данных и их взаимодействии с операционной системой. Однако прикладное программное обеспечение, как правило, не знает и не должно знать о функциях канала передачи данных, что означает, что существует необходимость в более высоком уровне над данными канального уровня для обеспечения точки доступа интерфейса DRTOS. Этот уровень обычно имеет характеристики транспортного уровня модели OSI. Модифицированную модель OSI для DRTS можно рассматривать в таком случае как модель, имеющую уровни, показанные на рис. 6.1. Мы можем видеть, что на физическом уровне фактический сетевой интерфейс в реальном времени обрабатывается драйверами устройств и DRTOS рассматривает сеть как устройство. Обратите внимание, что функции канального уровня данных, реализуемые DRTOS, и протокол могут перекрываться. Основными функциями, выполняемыми DRTOS, являются управление канальным уровнем данных и интерфейсом между задачами приложения и функциями канального уровня данных, как описано в следующих разделах.

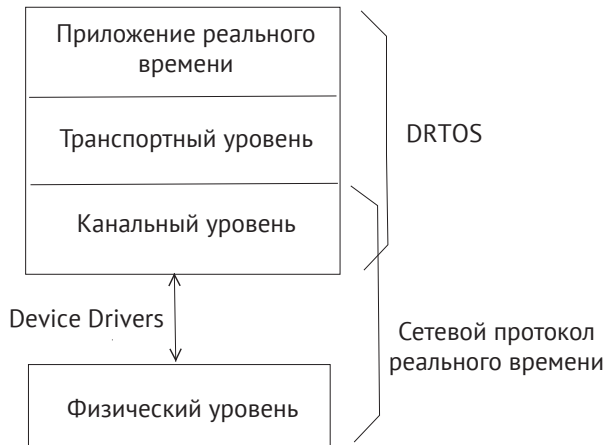


Рис. 6.1. Модифицированная модель OSI, используемая в сети реального времени

6.2.1. Интерфейс транспортного уровня

Основным сервисом, выполняемым на транспортном уровне, является прямое подключение и связь с приложением на другом хосте компьютера. Обратите внимание, что любой уровень ниже (сеть и канал передачи данных) не делает различий между сообщениями приложения. Две задачи приложения логически связаны через этот уровень, обеспечивая сквозную связь приложения и обмен. Например, веб-клиент на хосте общается непосредственно с веб-сервером на другом хосте, используя транспортный уровень. Поскольку на хосте компьютера будет запущено много приложений, важной функцией транспортного уровня является обеспечение мультиплексирования и демultipлексирования прикладных задач. Для этого сообщение прикладного уровня делится в про-

токоле на меньшие блоки данных (protocol data units, PDU), добавляется заголовок транспортного уровня, а затем сообщение передается в сетевой уровень как уровень 4 PDU. Транспортный уровень на принимающем хосте удаляет заголовки PDU, собирает сообщение и передает его приложению. Двумя основными транспортными протоколами интернета являются протокол управления передачей (TCP), который обеспечивает надежную, ориентированную на соединение передачу данных с контролем потока и протокол пользовательской дейтаграммы (user datagram protocol, UDP), являющийся услугой без установления соединения и управления потоком.

На основании общих функций, реализованных на этом уровне, мы рассмотрим возможные адаптации для случая DRTS. Мы будем предполагать, что узлы в DRTS физически расположены близко друг к другу, как в большинстве приложений DRTS. Поэтому можно предположить, что сетевой уровень почти не существует, и можно установить прямое соединение через канальный уровень. Основной задачей для DRTOS на транспортном уровне является разделение сообщения приложения на блоки PDU и повторная сборка их на принимающей стороне. Соответственно, две основные процедуры на этом уровне, которыми являются интерфейсы API для задач реального времени, следующие:

- `send_message` (сообщение, идентификаторов задачи): отправить сообщение другой задаче, которая может находиться на другом хосте;
- `receive_message` (сообщение): получить сообщение от любой задачи.

Нам нужно рассмотреть задачи в случае реального времени, а это значит, что нам нужны варианты *отправки с тайм-аутом* и, соответственно, *получение с тайм-аутом*. Процедура отправки также должна различать локальные и удаленные задачи, чтобы передать блоки сообщений локально, если получаемая задача находится на том же хосте, что и отправитель. Эта функция может быть реализована с помощью механизма *наименования*. В простейшей форме диапазон идентификаторов задач может быть зарезервирован для локальных задач, а любой идентификатор вне предела диапазона означает, что задача удалена. Еще одна важная проблема состоит в том, чтобы передать в сетевом протоколе требование приложения без срока. Мы увидим, как эти проблемы могут быть решены, в следующих разделах.

6.2.2. Интерфейс уровня канала передачи данных

Основная функция, требуемая на канальном уровне, – это управление потоком из пункта в пункт между двумя хостами или хостом и сетевым устройством в сети и обеспечение безошибочной передачи в этом уровне. Блок обмена данными на этом уровне называется *фрейм (frame)*. Типичный фрейм данных канала передачи данных показан на рис. 6.2 с полями *управление доступом к среде (medium access control, MAC)* и *управление логическими связями (logical link control, LLC)*. Заголовок MAC используется для MAC-адреса флага, а в заголовке LLC указывается порядковый номер, тип фрейма и *данные* – базовый блок дейтаграмм, связанный с сетевым уровнем. Завершающая часть MAC (трейлер)

используется для обнаружения и исправления ошибок с проверкой *циклическим избыточным кодом* (*cyclic redundancy check, CRC*), которая является одним из наиболее часто используемых методов обнаружения ошибок.

MAC- заголовок	LLC- заголовок	Данные	MAC- трейлер
-------------------	-------------------	--------	-----------------

Рис. 6.2. Фрейм уровня передачи данных

Полагая, что у нас есть сетевой протокол реального времени, такой как CAN, TTP или IEEE 802.11e, нам нужен интерфейс между драйверами канала передачи данных такого протокола и система DRTOS. Как и в нашей реализации DRTK, мы можем рассматривать сеть как любое другое устройство в операционной системе. Структура данных блока управления устройством связана с сетью реального времени (блок управления сетью) и теперь может содержать код активации для всех драйверов, необходимых для выполнения требуемого режима связи в сети. Следующим шагом для реализации интерфейса является формирование функций базового уровня канала передачи данных, наблюдаемых DRTOS. Удобный способ достичь этой цели – иметь нижеуровневые системные задачи реального времени, указанные в интерфейсе канального уровня передачи данных. Эти задачи общаются с верхним транспортным уровнем, используя свои почтовые ящики, которые являются структурами данных для асинхронного депонирования сообщений (см. раздел 4.3.3).

- *DL_Output*: эта задача непрерывно ожидает в своем почтовом ящике, и когда появляется сообщение для передачи, оно получает сообщение и активирует сетевой драйвер, используя блок управления сетью. Когда задача приложения должна ждать определенное время для ответа, эта задача «спит» в течение заранее определенного времени, а затем просыпается, чтобы посмотреть, получен ли какой-либо ответ.
- *DL_Input*: эта задача считывает фрейм из сети, используя драйвер блока управления сетью, и проверяет наличие ошибок. В случае ошибки он передает отправителю отрицательный ответ. Обратите внимание, что таким образом мы реализуем проверку наличия ошибок и контролируем поток. Когда сетевой протокол обеспечивает эту функцию, для выполнения данной задачи достаточно просто прочитать сообщение из сети и отправить его в почтовый ящик получателя. В случае *получения с тайм-аутом* эта задача читает из сети путем активации драйвера ввода, и если нет сообщений, он «спит» в течение заранее определенного времени.

На рис. 6.3 показано, как эти задачи в интерфейсе канального уровня связаны между собой, и способ, с помощью которого уровень транспортировки связан с ними.

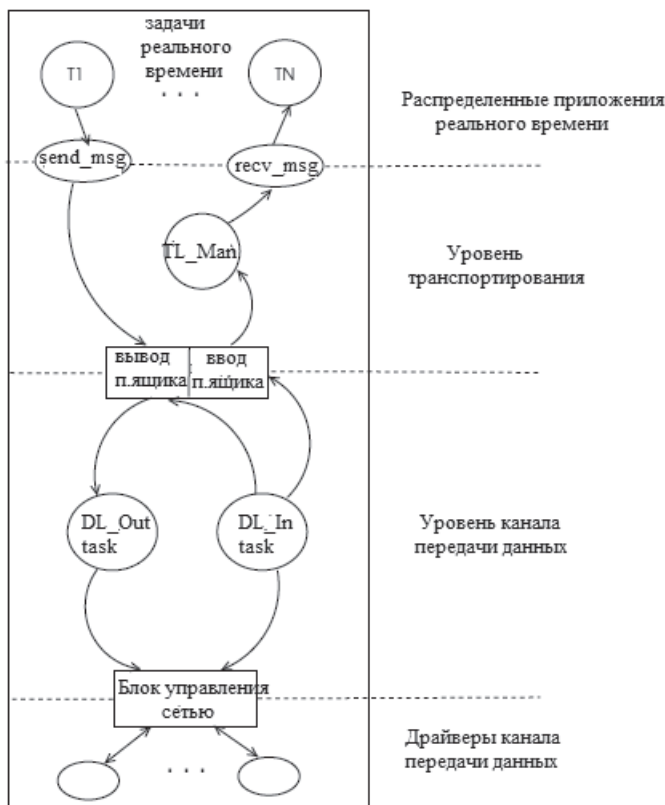


Рис. 6.3. Интерфейс DRTOS, модификация [3]

Когда задача *DL_In* обнаруживает, что это сообщение предназначено для другого узла, он просто помещает сообщение в почтовый ящик для задачи *DL_Out*, чтобы, взяв его оттуда, отправить в сеть. Использование данного почтового ящика для этой цели может быть практичным, так как в этом случае не требуется заголовка канала данных и проверки ошибок. Менеджер транспортного уровня (*TL_Man*) – это задача, которая используется для переключения входящих сообщений приложения в задачи получателя.

6.3. ПРОМЕЖУТОЧНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ РЕАЛЬНОГО ВРЕМЕНИ

Диагностика часто используемых программных модулей в DRTS и предоставление их для разнообразных приложений – основной подход к разработке промежуточного программного обеспечения. У нас есть три примера функций

промежуточного программного обеспечения, которые обычно необходимы и используются в DRTS: задача группы, синхронизация часов и выбор лидера.

6.3.1. Группы задач в реальном времени

Не все задачи могут участвовать в реализации общей функции в DRTS. *Группа задач* (или группа процессов) представляет собой подмножество задач, которые должны взаимодействовать и общаться часто. Этот набор задач требует специальных коммуникационных примитивов, чтобы эффективно сотрудничать и завершить общую работу. Есть два основных подхода к формированию структуры целевых групп: *однородные группы* и *иерархические группы*, как изображено на рис. 6.4. Однородная группа – это группа, в которой все задачи участника имеют одинаковый статус, а иерархическая группа имеет задачу лидера, которая координирует групповую деятельность. Задача может быть участником более чем одной группы.

Основными проблемами, которые необходимо решить в управлении целевой группой, являются поддержка членства в группе, групповое общение и синхронизация между членами группы. Группа задач идентифицируется уникальным идентификатором группы.

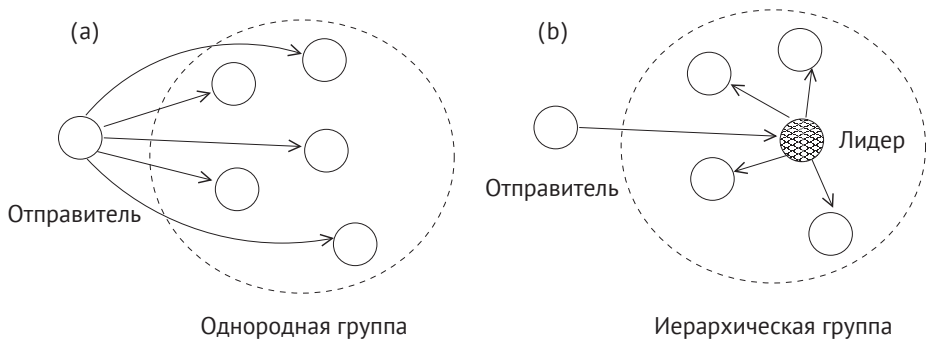


Рис. 6.4. Структуры группы для связи: а – однородная группа коммуникаций; б – иерархическая группа коммуникаций

Типичный модуль управления рабочей группой содержит следующие примитивы [1]:

- *group-id = CreateGroup* (идентификатор исходной задачи, *mun*): вызывающая задача создает группу с уникальным идентификатором и становится членом и лидером группы, если это иерархическая группа;
- *JoinGroup* (идентификатор группы, идентификатор задачи): вызывающая задача становится членом указанной группы на основе прав доступа;
- *LeaveGroup* (идентификатор группы, идентификатор задачи): вызывающая задача удаляет себя из группы. Если это лидер иерархической группы, избирается новый лидер.

Отправка сообщения в группу может быть выполнена разными способами, основанными на структуре группы. Использование избранного лидера группы облегчает реализацию групповых коммуникаций и других групповых процедур управления. Если группа однородная, сообщение должно быть доставлено всем членам группы программой транспортного уровня. Принимающие участники могут ждать в своих локальных почтовых ящиках или в групповом почтовом ящике, специально используемом для многоадресной связи. Для иерархической группы сообщение доставляется лидеру, который помещает его в почтовые ящики всех членов или в почтовый ящик группы. Эти операции изображены на рис. 6.4.

Основные процедуры групповой связи:

- *SendMcast (сообщение, идентификатор группы)*: сообщение отправляется всем членам группы. Это может быть достигнуто путем отправки сообщения руководителю группы, который, в свою очередь, отправляет сообщение всем членам группы в соответствии с указателем;
- *RecvMcast (идентификатор группы, идентификатор задачи)*: получение многоадресного сообщения либо из группового почтового ящика, либо от лидера, либо из личного почтового ящика.

Обычно целевые группы используются для резервирования в отказоустойчивых системах DRTS. Копии критической сложной задачи в реальном времени формируются в виде группы, и эти задачи выполняются параллельно с первостепенными (основными) задачами. Если основная задача не выполняется, выбирается одна из вторичных (фоновых) задач для запуска в качестве первичной задачи. Операция по отказоустойчивости требует тщательного рассмотрения того, как запустить вторичную задачу, когда происходит сбой. Обеспечение приема всех многоадресных сообщений в одном и том же порядке имеет решающее значение для правильной работы, как мы увидим в главе 12. Рассмотрение случая, когда все вторичные задачи работают с одним и тем же FSM, показывает, что получение сообщений в разном порядке приводит к различным состояниям копий, делающим ошибочным восстановление от сбоя. Средство программирования параллельной обработки (Message Passing Interface, MPI) предоставляет удобный интерфейс для группового управления и связи с приложением [11].

6.3.2. Синхронизация часов

По разным причинам соглашение об общих временных рамках имеет решающее значение в DRTS, как и в нераспределенных системах реального времени. Невыполнение этого требования может привести к ошибочным состояниям узлов, которые, в свою очередь, могут привести к коллапсу всей системы. Например, несоответствия конечных сроков выполнения задач, распределенных по сети, из-за различных значений тактовой частоты в узлах становятся неизбежными. В DRTS обычным требованием является наличие часов, синхронизированных с внешними реальными часами или опорными часами

системы в пределах допустимых значений. Всемирное координированное время (*universal time, coordinated, UTC*) хранит время на основе атомных часов с необходимыми поправками на вариации вращения Земли. К значению часов UTC можно получить доступ через спутники, которые постоянно передают его. *Глобальная система позиционирования (global positioning system, GPS)* позволяет определять географическое положение объекта [14]. Спутник GPS передает свое положение с отметкой времени, позволяющей прием получателями точного времени.

Каждый вычислительный элемент DRTS оснащен аппаратными кварцевыми часами. Хотя частота каждого кристалла стабильна, частоты кристаллов в узлах могут отличаться, что приводит к отклонению значений часов во времени. Удержание времени на узле осуществляется с помощью таймера, который уменьшает его значение с определенной скоростью, и когда это значение достигает нуля, таймер генерирует прерывание. Затем обработчик прерывания таймера увеличивает тактовое значение до значения текущего времени. Алгоритмы синхронизации часов исправляют дрейфующие значения часов в узлах DRTS. Существует два варианта достижения этой цели: с одним или с несколькими узлами, которые поддерживают свое время в соответствии со временем GPS и всех других, синхронизированных с ними узлов, или при котором ни один из узлов не имеет приемника GPS и они просто синхронизированы друг с другом в допустимых пределах.

6.3.2.1. Логические часы

Вместо синхронизации физических часов порядок событий может быть обеспечен *логическими часами*. В этом случае нашей основной заботой с точки зрения синхронизации часов является порядок событий, которые происходят в различных узлах распределенной системы. *Соотношение «событие произошло до того, как» ($\rightarrow$)* может быть использовано для описания случайного упорядочивания событий по следующим правилам:

- если a и b являются событиями в одной и той же задаче узла распределенной системы и a происходит до b , тогда $a \rightarrow b$;
- если a – событие отправки сообщения m по какой-либо задаче, а b – получение этого сообщения по какой-то другой задаче, то $a \rightarrow b$;
- если $a \rightarrow b$ и $b \rightarrow c$, то $a \rightarrow c$.

Два события a и b называются *одновременными* ($\parallel$), если соблюдается ни $a \rightarrow b$, ни $b \rightarrow a$. Мы можем реализовать эти правила для синхронизации логических часов в распределенной системе реального времени со следующими предположениями:

- задача τ_i имеет логические часы C_i ;
- часы C_i присваивают значение $C_i(a)$ событию a , которое происходит при выполнении задачи τ_i ;
- значения часов C_i монотонно возрастают.

Для обеспечения синхронизации логических часов должны удовлетворяться следующие условия:

- если два события a и b происходят в одной и той же задаче τ_i с событием a , предшествующим событию b , то $C_i(a) < C_i(b)$;
- если a является событием отправки сообщения m в задаче τ_i , а b является событием получения этого сообщения в задаче τ_j , то $C_i(a) < C_j(b)$.

Наконец, чтобы обеспечить вышеуказанные условия, следующие правила должны соблюдаться для синхронизации времени с использованием логических часов:

- 1) часы C_i увеличиваются между любыми двумя последовательными событиями в задаче τ_i ;
- 2) если a – отправка сообщения m в задаче τ_i , то m помечается как $t_m = C_i(a)$;
- 3) если b является событием получения сообщения m в задаче τ_j , то часы C_j задачи τ_j устанавливаются следующим образом:

$$C_j \leftarrow \max(C_j, t_m + 1).$$

Синхронизация логических часов между двумя задачами τ_1 и τ_2 изображена на рис. 6.5. Значения тактовых импульсов монотонно возрастают, и когда τ_2 получает сообщение $m1$, нет изменения значения его часов. Тем не менее сообщения $m2$, $m3$ и $m4$ вызывают изменения в значениях локальных часов при применении вышеприведенных правил.

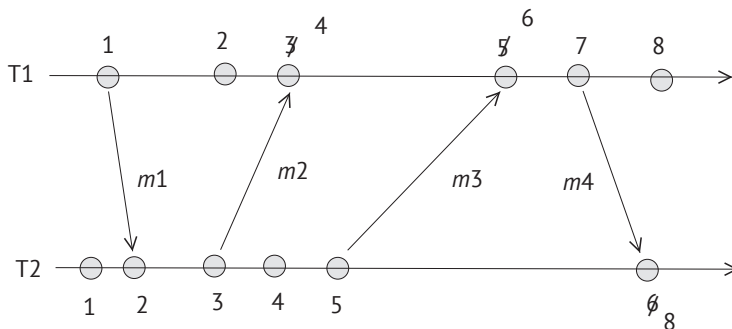


Рис. 6.5. Логические часы синхронизации между двумя задачами

6.3.2.2. Векторные часы

Основная проблема с логическими часами состоит в том, что если $C(a) < C(b)$, мы не можем определить, что $a \rightarrow b$. Векторные часы являются улучшением по сравнению с логическими часами при использовании вектора часов $C_i[1..n]$ в каждой задаче τ_i , где $C_i[i]$ – значение логической тактовой частоты задачи τ_i и $C_i[j]$ – оценка часов τ_j по τ_i . Теперь мы применим следующие правила реализации для векторных часов:

- 1) часы C_i увеличиваются между любыми двумя последовательными событиями в задаче τ_i :

$$C_i[i] \leftarrow C_i[i] + 1;$$

- 2) если a – отправка сообщения m в задаче τ_j , то m помечается меткой времени как $t_m = C_i(a)$, где t_m – векторные часы, хранящиеся в сообщении; если b является событием получения сообщения m в задаче τ_j , то часы C_j задачи τ_j устанавливаются следующим образом:

$$\forall k, C_j[k] \leftarrow \max(C_j[k], t_m[k]).$$

Две векторные метки времени $t(a)$ и $t(b)$ для двух событий a и b равны, если и только если $\forall i, t^a[i] = t^b[i]$. Соотношение меньше или равно между $t(a)$ и $t(b)$ означает, что $t^a \leq t^b$ тогда и только тогда, когда $\forall i, t^a[i] \leq t^b[i]$. С помощью приведенных выше правил можно сказать, что мы можем определить $a \rightarrow b$, если и только если $t_a < t_b$, что является решением проблемы логических часов. Три задачи на рис. 6.6 синхронизируются с использованием векторных часов применением этих правил.

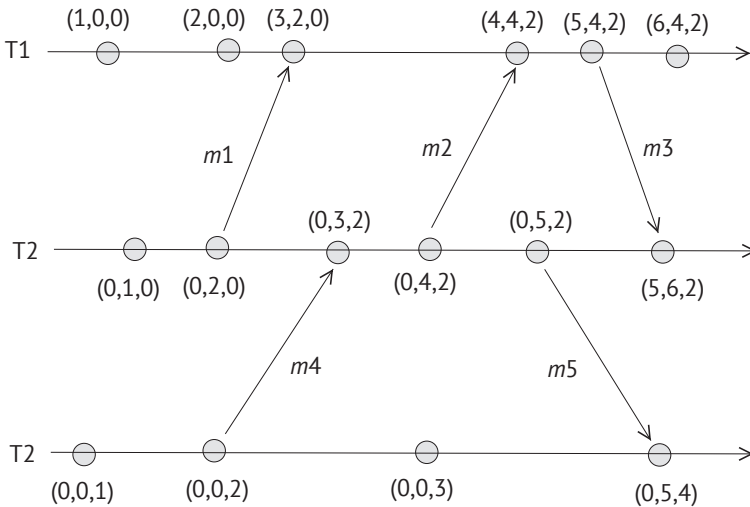


Рис. 6.6. Синхронизация векторных часов между тремя задачами

6.3.2.3. Протокол сетевого времени

Протокол сетевого времени (*network time protocol, NTP*) предполагает наличие в распределенной системе сервера, который получает точные значения часов с помощью передач UTC [9]. Синхронизация часов достигается узлами, получающими время от сервера через регулярные интервалы времени. Однако при корректировке значений часов узла должны быть приняты во внимание задержки, вносимые сетью. Временная метка, используемая NTP, состоит из

64 бит; 32 разрядов для секунд и 32 разрядов для долей секунд. Последний протокол NTPv4 удвоил длину из этих полей и дает временную метку 128 бит [10].

Для синхронизации необходимы два параметра: *сдвиг по времени* и *задержка круговой передачи*. Рассмотрим случай, когда узел пытается исправить свои часы, отправив сообщение с меткой времени в t_1 для сервера, как показано на рис. 6.7.

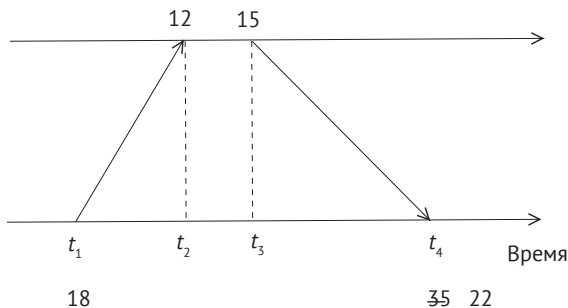


Рис. 6.7. Пример NTP. Величина времени в миллисекундах

Это сообщение, полученное сервером в момент времени t_2 и временной меткой с этим значением, отвечает в момент времени t_3 , который снова имеет временную метку, и отправляющий узел получает ответ в момент времени t_4 .

Отправляющий узел имеет все эти значения времени и может вычислить смещение времени θ , как показано ниже:

$$\theta = \frac{(t_2 - t_1) + (t_4 - t_3)}{2}. \quad (6.1)$$

Наблюдаемая отправителем двусторонняя задержка δ является разницей между общим временем, передается отправителю и обрабатывается на сервере следующим образом:

$$\delta = (t_4 - t_1) - (t_3 - t_2). \quad (6.2)$$

Предполагается, что задержка распространения в сети в обратном направлении та же самая, другими словами, сеть является симметричной. После статистического анализа из полученных значений выбирается самое низкое значение, и время локального узла корректируется, используя это значение. Мгновенные значения θ и δ , например, на рис. 6.7 равны 7 и 14 соответственно. Приемник добавил бы половину задержки туда и обратно к значению тактовой частоты сервера, исправляя его часы до 22. Когда группа серверов имеет более точные часы, чем другие, им дают более высокий приоритет, используя схему классификации. Менее сложная версия NTP называется *простой протокол сетевого времени* (*simple network time protocol*, SNTP) и используется в некоторых встроенных приложениях [13].

6.3.2.4. Алгоритм Беркли

Алгоритм синхронизации часов Кристиана [F. Cristian] предполагает наличие точного сервера времени, который получает значения времени от UTC [2]. Узел запрашивает у сервера время и в ответе к отметке времени сообщения добавляет половину времени туда-обратно (round-trip time, RTT), чтобы определить значение его часов. Предполагается, что задержки в сети равны в обоих направлениях, что разумно предполагать для коротких RTT.

Алгоритм Беркли использует аналогичную оценку RTT и разработан в основном для сетей интранет. Он работает без информации UTC, и, следовательно, нет внешнего чтения значения тактовой частоты и настройки в этом алгоритме, что облегчает реализацию. По сути, относительная синхронизация между узлами направлена не на синхронизацию в реальном времени. Она осуществляется узлом *супервизора*, который периодически опрашивает *рабочие* узлы. Узлы отвечают, сообщая о своем текущем значении времени. Супервизор рассматривает RTT и оценивает текущее время узлов. Среднее значение часов узлов затем рассчитывается путем исключения выбросов. Значения корректировок отправляются каждому узлу в виде положительных и отрицательных уточнений. Мастер может отказать, и тогда выбирается новый мастер, используя *алгоритм выбора*.

6.3.2.5. Синхронизация часов в беспроводных сенсорных сетях

Беспроводная сенсорная сеть (wireless sensor network, WSN) состоит из большого количества сенсорных узлов, которые общаются с помощью радиочастот. Центральный узел, называемый приемником, имеет в WSN большие вычислительные мощности, чем обычные узлы. Для сбора данных от узлов используется приемник (sink), который анализирует и, возможно, передает эту информацию удаленному компьютеру для более точного анализа. Этот узел в WSN имеет ограниченное время жизни, поскольку в качестве основного источника энергии у него выступает аккумулятор. Кроме того, связь по беспроводной среде рассеивает значительную энергию, и потому необходимы другие, более простые алгоритмы синхронизации часов в этих сетях. Мы рассмотрим два таких протокола: протокол эталонной широковещательной синхронизации (Reference Broadcast Synchronization Protocol, RBS) и протокол временной синхронизации (the Timing-Synch protocol).

Протокол эталонной широковещательной синхронизации

Протокол эталонной широковещательной синхронизации отличается от традиционных протоколов синхронизации тем, что синхронизируются друг с другом приемники, а не отправитель и получатель [4]. В простейшей форме сообщение m без информации о времени передается узлом WSN, скажем, двум приемникам. Каждый получатель записывает время получения сообщения, а затем они обмениваются этой информацией друг с другом, чтобы узнать разницу во времени и обеспечить необходимую коррекцию. Основным пре-

имуществом RBS является отсутствие у отправителя необходимости расчета какого-либо смещения за счет удаления. Задержка распространения и время приема вызывают смещение, и им можно пренебречь, учитывая, что широко-вещательное сообщение принимается всеми узлами WSN одновременно, когда количество переходов между двумя самыми дальними узлами, то есть диаметр сетевого графа, мало.

Протокол временной синхронизации

Протокол синхронизации по времени для сенсорных сетей (Timing-Synch Protocol for sensor networks, TPSN) учитывает при синхронизации время отправителя и время получателя [6, 12]. Он имеет две фазы: фазу обнаружения уровня и фазу синхронизации. Первый запускается один раз, когда сеть развернута. Узел приемника запускает алгоритм, отправляя соседям пакет *level_discovery*, который содержит 0 в качестве своего уровня и свой идентификатор. Любой узел, получающий это сообщение первый раз, идентифицирует свой уровень, добавляя единицу к уровню в сообщении, вставляет в сообщение это значение и свой идентификатор и затем передает. Таким образом, строится *остовое дерево* (spanning tree) с корнем в узле приемника данных. Алгоритм 6.1 отображает возможный код реализации для уровня фазы открытия.

Алгоритм 6.1. Алгоритм уровня открытия

```

1:
2: procedure level_discovery
3: int my_id, my_parent, my_level = 0, f lag = 0;
4: message msg;
5: if my_id = sink then
6:   my_parent ← my_id
7:   msg.id ← my_id
8:   msg.level ← 0
9:   send broadcast msg
10: else
11:   receive msg;
12:   if f lag = 0 then
13:     my_level ← msg.level + 1
14:     my_parent ← msg.id
15:     msg.id ← my_id
16:     msg.level ← my_level
17:     f lag ← 1
18:     send broadcast msg
19:   end if
20: end if
21: end procedure
22:

```

Фаза синхронизации периодически выполняется корнем дерева путем передачи сообщения синхронизации *time_synch*. Каждый узел уровня 1, который получает это сообщение, начинает двусторонний обмен сообщениями с кор-

нем после ожидания случайный период времени, чтобы избежать столкновений. Узел уровня 2, который прослушивает общение узлов уровня 1 с корнем, инициирует затем двустороннюю связь с узлом уровня 1 и т. д., пока все узлы на всех уровнях не будут синхронизированы. Двусторонняя связь происходит, как показано на рис. 6.7, в моменты времени t_1 , t_2 , t_3 и t_4 . Иницирующий узел A на уровне i вставляет свой уровень, метку времени и отправляет его в узел B на уровне $i - 1$ в момент времени t_1 . Узел B получает это сообщение в момент времени $t_2 = t_1 + \Delta + d$, где Δ – относительный сдвиг часов между узлами, а d – задержка распространения сообщения. Ответ от узла B , отправленного в момент времени t_3 , содержит его номер уровня и время t_1 , t_2 и t_3 , и A получает это сообщение в момент времени t_4 . Узел A теперь может вычислять Δ и d следующим образом и настраивает свои часы в соответствии с узлом B :

$$\Delta = \frac{(t_2 - t_1) - (t_4 - t_3)}{2}; \quad d = \frac{(t_2 - t_1) + (t_4 - t_3)}{2}. \quad (6.3)$$

Полагая, что задержки распространения постоянны в каждом направлении, дрейф часов между двумя узлами и задержка распространения могут быть точно определены. Утверждается, что TPSN достигает в два раза большей точности, чем RBS [6]. Роль корневого узла может быть по очереди перехвачена другими сенсорными узлами с помощью алгоритма выбора лидера, когда в WSN такого узла нет.

6.3.3. Алгоритмы выбора

Различные функции промежуточного программного обеспечения в DRTS могут быть реализованы проще и эффективнее, когда некоторые из основных функций в группе задач/узлов обрабатываются лидером. Например, сгруппировать несколько узлов в кластере и выбрать один из них в качестве лидера кластера – эффективный метод, используемый для маршрутизации в беспроводных сетях с датчиками. Мы также видели, как координатор/лидер группы может быть использован для отправки групповых сообщений. Лидер группы может отказать, и в таком случае существует необходимость в выборе нового лидера. Алгоритмы выбора лидера, рассматриваемые в следующем разделе, решают эту проблему.

6.3.3.1. Выбор в однонаправленном кольце

Мы рассмотрим алгоритм выборов с использованием автомата FSM, который будет реализован в однонаправленном кольце с узлами, каждый из которых имеет уникальный идентификатор. Общая идея этого алгоритма состоит в том, что когда обычный узел обнаруживает, что лидер не работает, он начинает выбор среди запущенных узлов сообщением о выборе. Он вставляет свой идентификатор в сообщение о выборе и меняет свое состояние на «ВЫБРАТЬ». Любой узел, который получает это сообщение $m(i)$, проверяет содержимое и выполняет одно из следующих действий, где i является идентификатором получателя:

- $i > j$: узел i заменяет j на i в сообщении и передает его следующему узлу. Он также меняет его состояние на «ВЫБОР»;
- $i < j$: узел i не изменяет содержимое сообщения, он передает сообщение следующему узлу и меняет его состояние на «ВЫБОР»;
- $i = j$: сообщение *выбора* сделало один полный обход кольца, и узел i , который имеет самый высокий идентификатор среди запущенных задач, становится лидером. Он отправляет сообщение «лидер» своему следующему за ним соседу.

Таким образом, лидером гарантированно становится задача с самым высоким идентификатором. Состояния узлов показаны на рис. 6.8. Обратите внимание, что только узел лидера находится в состоянии лидера (LDR), в то время как любой другой узел при наличии лидера находится в состоянии ожидания (IDLE). Другой алгоритм выбора лидера – это булли-алгоритм (*bully algorithm*). В этом алгоритме узел P_i , обнаружив сбой лидера, отправляет сообщение о выборе узлам со все более высоким идентификатором, чем он сам. Если нет ответа в течение заранее определенного интервала времени, P_i становится лидером и отправляет сообщение лидера всем узлам, объявляя лидером себя. Любой узел, который получает сообщение о выборе и имеет идентификатор больше идентификатора отправителя, отправляет ответное сообщение и начинает свой выбор. Узел, который получает ответ на его предвыборное сообщение, выходит из избирательного процесса и ждет сообщение лидера, чтобы определить его как лидера.

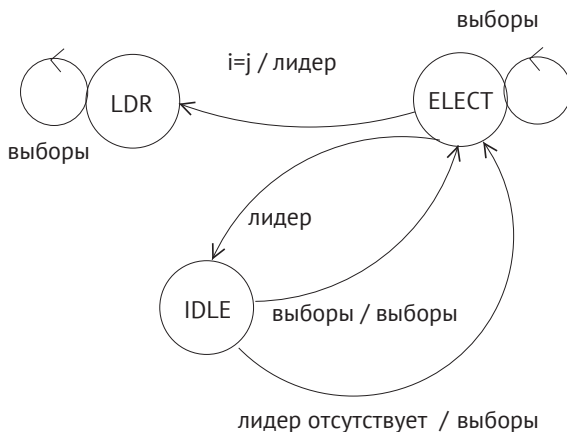


Рис. 6.8. Алгоритм FSM кольцевого выбора лидера

6.3.3.2. Выборы в беспроводных сенсорных и мобильных специальных сетях

Сеть WSN – это масштабная система DRTS, работающая со многими сложными задачами в реальном времени. Лидеры в WSN главным образом требуются для целей маршрутизации. Сеть WSN может быть разделена на несколько групп

узлов, называемых *кластерами*. При использовании кластеров управление топологией WSN становится проще. Этот метод также минимизирует размер хранимых таблиц маршрутизации на отдельных узлах. Широковещательное сообщение может быть передано лидеру узлов кластера сети WSN, и лидер передаст сообщение узлам в своем кластере. Кроме того, лидеры могут формировать опорные точки с приемниками GPS, когда другие узлы используют их, чтобы найти свое географическое положение и синхронизировать свои часы. Лидер, или голова кластера (cluster head, CH), в основном используется в этом качестве, пока уровень его энергии не падает ниже заданного значения, и тогда избирается новый лидер, управляющий узлами в кластере. Мобильная специальная сеть (mobile ad hoc network, MANET) состоит из мобильных узлов со средствами беспроводной связи. Многопользовательская связь является основным средством обмена сообщениями в этой сети. Как и в WSN, кластеризация и выбор канала для каждого кластера обеспечивают простой и эффективный метод маршрутизации. Сеть MANET, которая разделена на четыре кластера, C1, C2, C3 и C4, показана на рис. 6.9, где узел *a* отправляет сообщение узлу *b*, используя головы кластеров (CH).

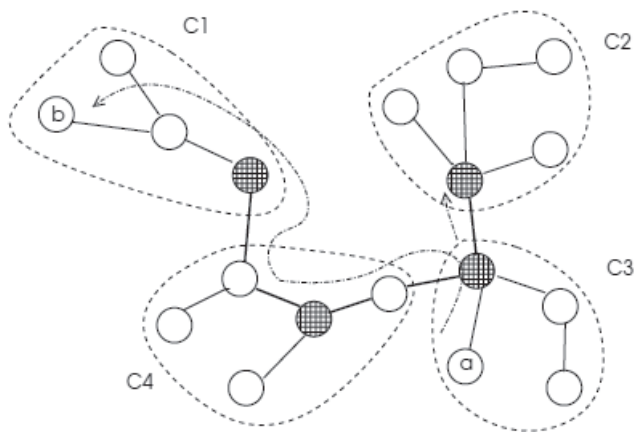


Рис. 6.9. Маршрутизация в сети MANET, основанная на кластерах. Головы – в клеточку

Узел *a* знает только свой узел CH и отправляет ему сообщение, которое транслирует его более высокому уровню среди CH. Каждый CH знает идентификаторы узлов в своем кластере, и если адрес назначения находится в его кластере, он ретранслирует сообщение на узел назначения и прекращает передачу его другим каналам. Таким образом, размер таблиц маршрутизации на каждом узле и канале поддерживается минимальным.

Голова CH в сети WSN выбирается на основе таких критериев, как уровень энергии, связность и мобильность. Изучаются обычно два типа протоколов вы-

бора СН в WSN: случайные алгоритмы и алгоритмы минимального поиска. Низкоэнергетическая адаптивная иерархичная кластеризация (low-energy adaptive clustering hierarchy, LEACH) – это алгоритм кластеризации для WSN [7]. Каждый узел в LEACH выбирает случайное число между 0 и 1, и если это число меньше, чем порог, то этот узел становится СН для текущего раунда. Избранный СН сообщает о своем выборе членам кластера. Основная роль СН в этом алгоритме состоит в том, чтобы создать множественный доступ с временным разделением (time-division multiple access, TDMA) для членов, чтобы использовать сеть, а также сообщить им, когда передавать. СН собирает данные от членов кластера и загружает эти данные в приемник. LEACH-C при выборе СН также учитывает местонахождение узлов и их энергетическое состояние [8]. Иерархическим методом кластеризации является также метод HEED [5], основанный на учете остаточной энергии на узле и затрат на внутреннюю связь в кластере.

6.4. РЕАЛИЗАЦИЯ DRTK

Мы опишем, как реализовать интерфейсы транспортного уровня и канального уровня и промежуточные модули синхронизации часов, управления группами и лидерами и выборы в ядре DRTK. Будем полагать, что станции подключены как однонаправленные кольца, и сообщения передаются путем пересылки их от станции к преемнику. Отправляющий узел удаляет сообщение, когда получает свое собственное сообщение. В нашей реализации мы будем предполагать, что существует только физический уровень, и протокола связи канала передачи данных не существует.

6.4.1. Инициализация сети

Предположим, три узла сети соединены в однонаправленной структуре кольца, как показано на рис. 6.10. Каждый узел сети является процессом UNIX и может быть закодирован как отдельная программа. Сетевая связь между узлами достигается с помощью очередей сообщений UNIX.

Мы будем называть узлы распределенной системы реального времени как *node1*, *node2* и *node3*. Например, код для *node1* будет включать в себя «drtk.h» и «drtk.c», обычную инициализацию DRTK и дополнительную процедуру инициализации сети *init_network*, которая создает подпрограмму создания очереди сообщений и показана ниже в файле «network.c». Нам нужно сформировать блок управления сетевым устройством, сформировать драйверы устройства и хранить их адреса в структуре данных, чтобы задачи передачи данных могли их вызвать. Для простоты мы опускаем программу управления ошибками.

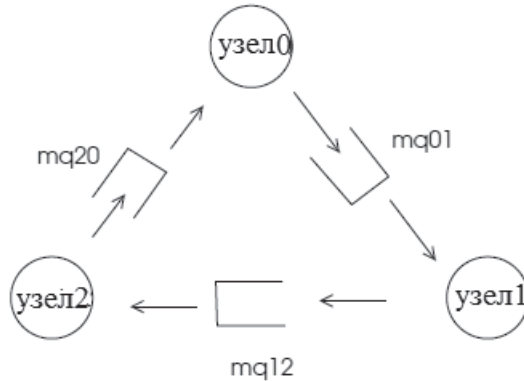


Рис. 6.10. Структура сети DRTK

```
// network.c
#include <stdio.h>
#include <pthread.h>
#include <synch.h>
#include <drtk.h>
#include <drtk.c>

long int keys[3]={1234L,2345L,3456L}, my_key_in, my_key_out;
int msgq_id;

int read_net(int *address, int n_bytes) {
    msgrecv(address, n_bytes, 0, IPC_WAIT);
}

int write_net(int *address, int n_bytes) {
    msgsnd(msgq_id_out, address, n_bytes, IPC_NOWAIT);
}

void init_network() {
    my_key_in=keys[Sys_Tab.this_node];
    msgq_id_in=msgget(my_key_in, IPC_CREAT | 0660);
    my_key_out=keys[(Sys_Tab.this_node+1) MOD 3];
    msgq_id_out=msgget(my_key_out, IPC_CREAT | 0660);
    net_dev=make_dev();
    net_dev.read_addr=read_net;
    net_dev.write_addr=write_net;
}
```

Нам будет необходимо кодировать три программы с различными идентификаторами узлов в системной таблице (*System_Tab.this_node*) и компилировать индивидуально программу каждого узла, а затем запустить эти программы, как показано ниже.



Рис. 6.11. Формат пакета транспортного узла DRTK

```

> gcc -o node1 node1.c -lpthread
> gcc -o node2 node2.c -lpthread
> gcc -o node3 node3.c -lpthread
> node1 &
> node2 &
> node3 &

```

6.4.2. Интерфейс транспортного уровня

Сначала мы определим структуру сообщения транспортного уровня (TL), как показано на рис. 6.11. Этот уровень является основным интерфейсом приложения, и нашим основным требованием является то, что приложение представлено с глобальным идентификатором задачи, не зная, где находится эта задача. Таким образом, задача этого уровня – определить, является передача данных локальной или удаленной, а иногда в случае многоадресного сообщения и то, и другое. *Поле формы* указывает, является ли сообщение одноадресным, многоадресным или широковещательным, а поле типа – более подробный тип, специфичный для приложения.

Заголовок TL указывается в файле «data_unit.h», как показано ниже. Для любого приложения / специфической информации промежуточного программного обеспечения мы проанализируем поле данных PDU, используя объявление типа данных Си *union*.

```

// data_unit.h
/*****
Заголовок транспортного уровня
*****/
typedef struct {
    ushort sender_id;
    ushort receiver_id;
    ushort form;
    ushort type;
    ushort seq_num;
}TL_header_t;

```

Основными системными вызовами на этом уровне являются процедуры отправки и получения сообщения *send not wait* и *recv wait*. Эти процедуры используются как для одноадресной, так и для многоадресной рассылки сообще-

ний. Важным вопросом, который необходимо решить, является вопрос о том, является ли получатель(и) локальным(и) и/или удаленным(и). Мы принимаем здесь простую схему наименования: каждая задача имеет глобальный 16-битный уникальный идентификатор, который формируется путем объединения уникального идентификатора хоста с локальным идентификатором задачи. Например, 0×1206 является локальной задачей с $tid = 06$ на машине 18. Мы полагаем, что отправитель сообщения знает глобальный идентификатор получателя задачи. Процедура *отправки* программы должна при необходимости разобирать сообщение на некоторое количество пакетов фиксированного размера, как показано в коде ниже в файле «*transport.c*». Он получает свободный блок данных из своего пула, заполняет его содержимым сообщения и помещает эти данные в почтовый ящик получателя, используя отправку без ожидания. Если вызов является многоадресной передачей, он ищет идентификатор локальной группы и отправляет сообщение в локальном почтовом ящике группы, а также сохраняет его в почтовом ящике задачи вывода канала передачи данных, как показано ниже. Мы полагаем, что задача может быть членом только одной группы, идентификатор которой хранится в его блоке управления. Аналогично, широковещательное сообщение доставляется всем активным задачам локально и отправляется в виде широковещательного сообщения по сети. Обратите внимание, что мы не имеем отдельной многоадресной *отправки*, но приложение указывает тип сообщения как одноадресное, многоадресное или широковещательное. Процедура *send_net* подготавливает заголовок уровня MAC для сети и помещает блок данных в почтовый ящик задачи вывода канала передачи данных.

```
/* transport.c */
/*****
Отправка сообщения без ожидания
*****/
int send_net(data_ptr_t data_pt, ushort tid, ushort f) {

    ushort mbox_id=task_tab[System_Tab.DL_Out_id].mailbox_id;
    data_pt->TL_header.form=f;
    data_pt->TL_header.sender_id=current_tid;
    data_pt->TL_header.receiver_id=tid & 0x00FF
    data_pt->MAC_header.sender_id=System_Tab.this_node;
    data_pt->MAC_header.receiver_id=(tid & 0xFF00)>>8;
    send_mailbox_notwait(mbox_id,data_pt);
}

int send_msg_notwait(ushort tid, char* msg_pt, ushort len,
    ushort type) {

    task_ptr_t task_pt;
    mailbox_ptr_t mbox_pt;
    data_unit_ptr_t data_pt1, data_pt2;
    ushort node_id, mbox_id, group_id, tid;
```

```

if (tid < 0 || tid >= System_Tab.N_TASK)
return(ERR_RANGE);
if (type == UNICAST) { // send unicast
node_id=(tid & 0xFF00)>>8;
if (node_id == System_Tab.this_node) //check remote
mbox_id==task_tab[tid & 0x00FF].mailbox_id;
while(len>0) { //do for local and remote
data_pt1=get_data_unit(System_Tab.Net_Pool);
memcpy(data_pt1, &(msg_pt.data), N_DATA_UNIT);
if(node_id == System_Tab.this_node)
send_mailbox_notwait(mbox_id, data_pt1);
else
send_net(data_pt1, tid, UNICAST);
len=len-System_Tab.N_DATA_UNIT;
msg_pt=msg_pt+len;
}
}
else { // MULTICAST or BROADCAST
while(len>0) {
data_pt1=get_data_unit(System_Tab.Net_Pool);
data_pt2=get_data_unit(System_Tab.Net_Pool);
memcpy(&(msg_pt.data), data_pt1, System_Tab.N_DATA_UNIT);
memcpy(&(msg_pt.data), data_pt2, System_Tab.N_DATA_UNIT);
if (type == MULTICAST) {
group_id=task_tab[current_tid].group_id;
mbox_id=group_tab[group_id].mailbox_id;
send_mailbox_notwait(mbox_id,data_pt1);
}
if (type == BROADCAST)
for(i=0;i<System_Tab.N_TASK;i++)
if (task_tab[i].state=ALLOCATED) {
data_pt=get_data_unit(System_Tab.Net_Pool);
memcpy(data_pt, &(msg_pt.data), N_DATA_UNIT);
send_mailbox_notwait(task_tab[i].mailbox_id,data_pt);
}
send_net(data_pt2, tid, type);
len=len-System_Tab.N_DATA_UNIT;
msgpt=msgpt+len;
}
}
return(DONE);
}

```

У нас есть задача менеджера транспортного уровня (*TL_Man*), которая получает сообщение от канального уровня в своем входном почтовом ящике. Она проверяет тип заголовка транспортного уровня, чтобы решить, является ли входящее сообщение одноадресным, многоадресным или широковещательным, и размещает или в почтовом ящике отдельных задач, или в почтовом ящике группы, или в почтовых ящиках всех задач, как показано в коде ниже. Она служит интерфейсом между данными канального уровня и приложением, а также обслуживает группу.

```

/*****
Менеджер транспортного слоя
*****/
TASK TL_Man() {

    data_unit_ptr_t data_pt;
    task_ptr_t task_pt;
    ushort tid, mbox_id3;
    ushort mbox_id1=task_tab[current_tid].mailbox_id;
    ushort mbox_id2=task_tab[System_Tab.DL_Out_id].mailbox_id;

    while(TRUE) {
        data_pt=recv_mailbox_wait(mbox_id1);
        if (data_pt->TL_header.form==UNICAST){

            tid=(data_pt->TL_header.receiver_id) & 0x00FF;
            mbox_id3=task_tab[tid].mailbox_id;
            send_mailbox_notwait(mbox_id3,data_pt);
        }
        else if (data_pt->TL_header.form==MULTICAST) {
            group_id=data_pt->TL_header.receiver_id;
            mbox_id2=group_tab[group_id].mailbox_id;
            send_mailbox_notwait(mbox_id2,data_pt);
        }
        else if (data_pt->TL_header.form==BROADCAST) {
            for(i=0;i<System_Tab.N_TASK;i++)
                if (task_tab[i].state==ALLOCATED) {
                    data_pt2=get_data_unit(System_Tab.Net_Pool);
                    memcpy(data_pt2, data_pt1, sizeof(data_unit_t));
                    send_mailbox_notwait(task_tab[i].mailbox_id,data_pt);
                }
        }
    }
}

```

Прием сообщения осуществляется путем объединения полученных блоков в сообщение. У нас есть одна процедура блокировки для одноадресного и многоадресного приемов, тип которого указан в качестве параметра, переданного этой подпрограмме, как показано в коде ниже.

```

/*****
Прием сообщения блоками
*****/
int recv_wait(char* msg_pt, ushort len, int type) {
    task_ptr_t task_pt;
    data_unit_ptr_t data_pt;
    ushort node_id, mbox_id, group_id;

    if (type == MULTICAST) { // receive multicast
        group_id=task_tab[current_tid].group_id;
        mbox_id=group_tab[group_id].mailbox_id;
    }
    else mbox_id=task_tab[current_tid].mailbox_id;
}

```

```

while(len>0) {
data_pt=recv_mailbox_wait(mbox_id);
memcpy(msg_pt, data_pt->data.data, N_DATA_UNIT);
len=len-N_DATA_UNIT;
msg_pt=msg_pt+len;
put_data_unit(Sys_Tab.Net_Pool,data_pt);
}
return(DONE);
}

```

6.4.3. Задачи интерфейса канального уровня передачи данных

У нас есть две задачи для реализации интерфейса канального уровня: *DL_Out* и *DL_In* в соответствии описанием в разделе 6.2.2. Дополнительно мы будем полагать, что основная обработка ошибок и функции управления потоком выполняются в задачах канального уровня, а сетевым протоколом реального времени определяются только сетевые коммуникации на физическом уровне. Простой протокол, который мы реализуем, называется протоколом «Остановись и жди» (Stop-and-Wait), в котором отправитель ожидает подтверждения для каждого отправляемого им фрейма. FSM-представление этого протокола показано на рис. 6.12.

Таблица 6.1. Таблица FSM для отправителя уровня канала передачи данных

	Выводы	TR_REQ	ACK	NACK	TOUT
Состояния	IDLE	act_00	NA	NA	NA
	WAIT	NA	act_11	act_12	act_13

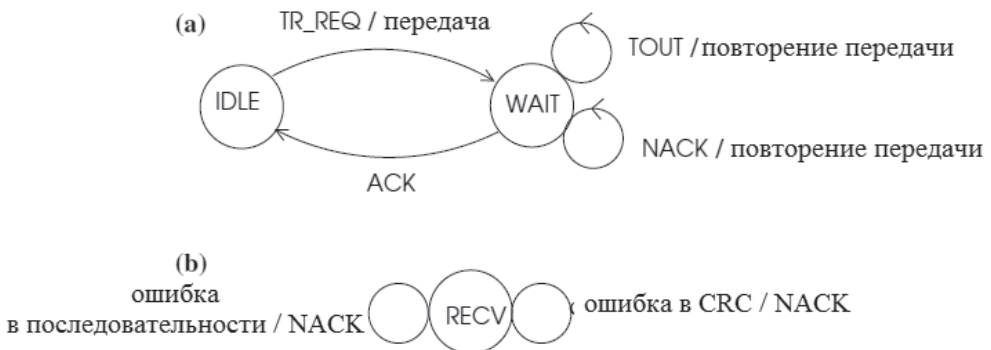


Рис. 6.12. Уровень передачи данных автоматов FSM: а – отправитель; б – приемник

Отправитель – конечный автомат (FSM) имеет два состояния: IDLE, когда данные для передачи отсутствуют, и WAIT, когда фрейм отправляется и ждет ответа. Если присутствует сообщение, что отправленный фрейм получен с ошибкой (negative acknowledge, NASK) или по истечении времени ожидания, фрейм передается повторно, и ожидается ответ. Отправка повторяется заданное количество раз, прежде чем это останавливается. Таблица FSM для отпра-

вителя изображена в табл. 6.1. Обратите внимание, что некоторые действия не применимы (not applicable, NA). Для простоты примем, что транспортный уровень не запрашивает отправку следующего фрейма командой TR_REQ до тех пор, пока текущий фрейм не будет доставлен правильно. Подпрограммы *CRC_check* и *set_timer*, которые могут быть реализованы здесь, просто не показаны. Этот FSM реализован с помощью кода задачи *DL_Out*, которая показана ниже. Все задачи передачи данных находятся в файле «datalink.c».

```

/*****
Задача вывода при передаче данных
*****/
// datalink.c
#define IDLE 0 // состояния
#define WAIT 1
#define DATA_MSG 0 // типы сообщений
#define ACK 1
#define NACK 2
#define TOUT 3
#define N_TRIES_MAX 5

fsm_table_t sender_FSM[2][4];
data_unit_ptr_t data_pt;
ushort dev_id, mbox_id, seq_no=0, crc_code, n_tries=0;

void act_00(){
    data_pt->MAC_header.type=DATA_MSG;
    data_pt->MAC_header.seq_num=seq_no;
    CRC_generate(data_pt1, &crc_code);
    MAC_trailer=crc_code;
    write_dev(dev_id,data_pt,N_DATA_LEN);}

void act_11(){
    data_pt2=get_data_unit(System_Tab.Net_Pool);
    data_pt2->type=ACK;
    seq_no=(seq_no+1) MOD 2;
    write_dev(System_Tab.Net_Dev,data_pt,N_DATA_LEN);}

void act_12(){
    if (++n_tries< N_TRIES_MAX) {
        write_dev(System_Tab.Net_Dev,data_pt,N_DATA_LEN);
        set_timer();
    } else // error log
    } }

TASK DL_Out() {

    sender_FSM[0][0]=act_00;
    sender_FSM[1][1]=act_11;
    sender_FSM[1][2]=act_12;
    sender_FSM[1][3]=act_12;
    dev_id=Sys_Tab.Net_Dev;

```

```

mbox_id=&(task_tab[current_pid])->mailbox_id;
current_state=IDLE;

while(TRUE)
{ data_pt=recv_mailbox_wait(mbox_id);
(*sender_FSM[current_state][data_pt->MAC_header.type])();
}
}

```

Прием сообщения из сети выполняется задачей DL_In, которая имеет только состояние RECV. Она всегда ждет фреймы с физического уровня и при получении проверяет их на наличие ошибки и отправляет подтверждение (acknowledgement, ACK) или отрицательное подтверждение (non- acknowledgement, NACK) соответственно.

```

/*****
Задача ввода при передаче данных
*****/
TASK DL_In() {

    data_unit_ptr_t data_pt1, data_pt2;
    ushort dev_id, mbox_id, crc_code, seq_no=0;
    dev_id=System_Tab.Net_Dev;
    mbox_id=task_tab[current_pid].mailbox_id;
    while(TRUE)
    { data_pt=read_device(dev_id,System_Tab.DUNIT_LEN);
      CRC_generate(data_pt, &crc_code);
      if ((data_pt->MAC_header.seq_num != seq_no) ||
          (crc_code != data_pt->MAC_trailer))
          data_pt-> MAC_header.type= NACK;
      else {
          data_pt1->MAC_header.type= ACK;
          mbox_id=task_tab[System_Tab.TL_Man_id].mailbox_id;
          send_mbox_notwait(mbox_id,data_pt);
          type=data_pt1->TL_header.form;
          if (type==MULTICAST||type==BROADCAST) { // forward
              data_pt2=get_data_unit(System_Tab.Net_Pool);
              memcpy(data_pt2, data_pt1, System_Tab.N_DATA_UNIT);
              mbox_id=task_tab[System_Tab.DL_Out_id].mailbox_id;
              send_mbox_notwait(mbox_id,data_pt2)
          }
      }
      write_device(dev_id,data_pt,N_DATA_UNIT);
      seqno=(seqno+1) MOD 2;
    }
}

```

6.4.4. Групповое управление

Мы начинаем с определения блока управления группой (*group control block, gcb*). Групповое управление определяется в файле «group.h» ниже. Основные процедуры управления группой предназначены для создания группы, присоединения к группе и выхода из группы. Отправка многоадресной рассылки сообщения

членам группы достигается с помощью общей процедуры отправки в интерфейс транспортного уровня. Прием сообщения из почтового ящика выполняется каждой задачей из группы, опять же с помощью общей функции *получения*.

```

/*****
Структура группы данных
*****/
/* group.h */
#определение ERR_GR_NONE -2
#определение N_MEMBERS 30

typedef struct group { ushort id;
    ushort state;
    ushort mailbox_id;
    ushort n_members;
    ushort local_members[N_MEMBERS];
}group_t;
typedef group_t* group_ptr_t;

```

Структура группы сначала выделяется системным вызовом *allocate_group* следующим образом:

```

/*****
Выделение группы
*****/
int allocate_group() {
    int i;

    for(i=0; i< System_Tab.N_GROUP; i++)
        if (group_tab[i].state != ALLOCATED) {
            group_tab[i].state= ALLOCATED;
            group_tab[i].id= i;
            group_tab[i].mailbox_id=make_mailbox();
            group_tab[i].n_members=1;
            group_tab[i].local_members[0]=current_tid;
            return(i);
        }
    return(ERR_NOT_ALLOC);
}

```

Задача может быть присоединена к уже выделенной группе следующим образом:

```

/*****
Присоединение к группе
*****/
int join_group(ushort group_id) {
    group_ptr_t group_pt=&group_tab[group_id];

```

```

if ( group_pt->state == ALLOCATED) {
group_pt->n_members++;
group_pt->local_members[n_members]=current_tid;
return(DONE);
}
else return(ERR_GR_NONE);
}

```

Выход из группы по заданию выполняется системным вызовом *leave_group*.

```

/*****
Выход группы
*****/
int leave_group(ushort group_id) {
group_ptr_t group_pt=&group_tab[group_id];

if (group_pt->state == ALLOCATED) {
for(i=0;i<n_members;i++)
if(group_pt->local_members[i]=current_pid)
group_pt->local_members[i]=NOT_ALLOC;
group_tab[i].n_members--;
return(DONE);
}
else return(ERR_GR_NONE);
}

```

6.4.5. Алгоритм синхронизации часов

Мы реализуем сетевой протокол времени (network time protocol, NTP) для промежуточного программного обеспечения распределенной синхронизации часов DRTK. Эта задача выполняется в цикле с задержкой на 10 секунд. Она генерирует сообщение, запрашивая время у сервера NTP, записывает время исходящего сообщения и время получения сообщения от сервера. Смещение времени может быть рассчитано на основе этих значений и временных меток (t_2 и t_3), хранящихся в сообщении сервера. У нас есть единый код для сервера и клиентов. Для простоты мы реализуем задачу только для одного сервера и для чтения один раз без усреднения значений, взятых несколько раз с разных серверов. Так как наша моделируемая среда UNIX, чтобы прочитать текущее время, мы используем функцию UNIX *gettimeofday*. Времена t_2 и t_3 в исходном алгоритме предполагаются равными. Поскольку мы не хотим устанавливать время UNIX, процедура *set_my_clock* в реальном приложении оставлена.

```

/*****
Задача NTP
*****/
/* clocksynch.c */
#включение <time.h>
#включение <sys/time.h>

#определение CLOCK_READ 12

```



```

TASK NTP() {

    data_unit_ptr_t data_pt;
    ushort dest_id;
    struct timeval t1, t2, t4;
    long int t_offset;
    while(TRUE) {
        if(System_Tab.this_node==System_Tab.NTP_id) {
            data_pt=recv_mbox_wait(task_tab[current_tid].mailbox_id);
            dest_id=data_pt.TL_header.sender_id;
            data_pt->TL_header.sender_id=current_tid;
            data_pt->TL_header.receiver_id=dest_id;
            gettimeofday(&t2,NULL);
            data_pt->data.timestamp=t2;
            send_mbox_notwait(Sys_Tab.DL_Out_mbox,data_pt);
        }
        else {
            delay_task(current_tid, 10000);
            data_pt=get_data_unit(Sys_Tab.Net_Pool);
            data_pt->TL_header.type=CLOCK_READ;
            gettimeofday(&t1,NULL);
            data_pt->data.timestamp=t1;
            send_mbox_notwait(Sys_Tab.DL_Out_mbox,data_pt);
            data_pt=recv_mbox_wait(task_tab[current_tid].mailbox_id);
            gettimeofday(&t4,NULL);
            t2=data_pt->data.timestamp;
            t_offset=((t2.tv_sec*1e6 + t2.tv_usec)-(t1.tv_sec*1e6
            + t1.tv_usec)+(t4.tv_sec*1e6 + t4.tv_usec)-
            (t2.tv_sec*1e6 + t2.tv_usec))/2;
            set_my_clock(t2+t_offset);
        }
    }
}

```

6.4.6. Выбор лидера в кольце

Мы реализуем алгоритм выбора лидера, описанный в разделе 6.3.3.1 в структуре кольца. Таблица FSM для изображенного алгоритма показана в табл. 6.2.

Обратите внимание, что мы не имеем никаких действий в состоянии LDR, так как это состояние терминала достигнуто только новым лидером. Код ниже может быть использован для реализации этого FSM. Задача выбора лидера (*Leader_Elect*) в каждом узле вызывается полученным сообщением из своего почтового ящика и срабатывает соответственно воздействию. Мы не показываем явно механизм тайм-аута обнаружения текущего лидера, как показано ниже, поскольку это зависит от приложения. Обратите внимание, что *act_11* имеет тот же код, что и *act_01*, так как получение сообщения о выборе из состояния IDLE или ELECT вызывает ту же самую последовательность действий. Мы предполагаем, что область данных блока данных является объединением других структур, и новое поле *winner_id* имеет самый высокий идентификатор узла, который сообщение передало по сети.

Таблица 6.1. Таблица FSM для отправителя уровня канала передачи данных

		Выводы		
		TOUT	ELECTION	LEADER
Состояния	IDLE	act_00	act_01	NA
	WAIT	NA	act_11	act_12
	LDR	NA	NA	NA

```

/*****
Задача выбора лидера
*****/
/* datalink.c */
#define IDLE 0 // states
#define ELECT 1
#define LDR 2
#define TOUT 0 // inputs
#define ELECTION 1
#define LEADER 2

fsm_table_t leader_FSM[3][2];

ushort recvd_id, my_leader, winner_id;
data_unit_ptr_t data_pt;

void act_00(){
    current_state=ELECT;
    data_pt=get_data_unit(System_Tab.Net_Pool);
    data_pt->TL_header.type=ELECTION;
    data_pt->TL_header.data.winner_id=System_Tab.this_node;
    send_net(data_pt, 0, BROADCAST);
}

void act_01(){
    current_state=ELECT;
    recvd_id=data_pt->data.winner_id;
    if (recvd_id < System_Tab.this_node)
        data_pt->data.winner_id=System_Tab.this_node;
    else if (recvd_id == System_Tab.this_node) {
        current_state=LDR;
        data_pt->data.TL_header.type=LEADER;
    }
    send_net(data_pt, 0, BROADCAST);
}

void act_12(){
    current_state=IDLE;
    my_leader=data_pt->data.winner_id
}

TASK Leader_Elect() {
    current_state=IDLE;
    sender_FSM[0][0]=act_00;
    sender_FSM[0][1]=act_01;
}

```

```

sender_FSM[1][1]=act_01;
sender_FSM[1][2]=act_12;

while(TRUE)
{ data_pt=recv_mbox_wait();
(*leader_FSM[current_state][data_pt->TL_header.type])();
}
}

```

6.5. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы основные функции DRTOS?
2. Опишите основные функции транспортного уровня DRTOS.
3. Какова цель канального уровня?
4. Что такое промежуточное программное обеспечение и каковы основные функции, реализуемые промежуточным программным обеспечением DRTS? Приведите примеры общих модулей промежуточного программного обеспечения в DRTS.
5. Какова основная область применения групп задач в реальном времени?
6. Почему необходима синхронизация часов в DRTS или не в реальном времени, или распределенной системе нереального времени?
7. В чем разница между алгоритмом Кристиана и алгоритмом Беркли?
8. Сравните методы RBS и TPSN синхронизации часов, используемые в WSN.
9. Каковы основные способы использования лидеров в WSN?
10. Каковы общие процедуры выбора лидера в WSN?

6.6. ПРИМЕЧАНИЯ К ГЛАВЕ

В этой главе мы рассмотрели основные функции DRTOS и промежуточное программное обеспечение в реальном времени. основополагающая задача для DRTOS – это интерфейс сети, который обычно выполняется на транспортном и канальном уровнях. Процедуры транспортного уровня в основном делят пользовательское сообщение на несколько блоков протокола уровня канала передачи данных на стороне отправителя, а на стороне получателя выполняется сборка сообщения. Некоторые или все функции на канальном уровне могут быть реализованы по протоколу реального времени. Мы предполагали, что протокол обеспечивает только функции физического уровня и выполняемые задачи обработки сетевых процедур канала передачи данных на этом уровне как задачи DRTOS. Эти задачи взаимодействуют и синхронизируются, как любые другие задачи операционной системы, и имеют более высокий приоритет, чем другие задачи, поскольку их задержка может привести к нарушению сроков выполнения прикладных задач в реальном времени. Мы привели реальный код реализации задач канального уровня для примера DRTK.

Промежуточное программное обеспечение реального времени состоит из программных модулей, которые обычно не являются частью DRTOS, но необходимы для многих различных приложений. Мы рассмотрели три таких модуля промежуточного программного обеспечения: задача группового управления, синхронизация часов и выборы лидера. Часы синхронизации необходимы во многих системах, как реального времени, так и нереального времени. Целевые группы задач обеспечивают простой и элегантный способ достижения отказоустойчивости в DRTS при параллельном запуске всех копий задачи в группе и активации копии, когда основная задача не выполняется. Мы увидим, как упорядочение сообщений может быть достигнуто в последующей главе, где рассматривается отказоустойчивость. Выбор лидера необходим, когда набор задач делится на несколько подмножеств. Выбор лидера в подмножестве задач облегчает выполнение функций в этом подмножестве, так же как и при формировании групп задач.

6.7. ПРОЕКТЫ ПРОГРАММИРОВАНИЯ

1. Разработайте на языке Си и реализуйте алгоритм программы *отправки* (*send*) на транспортном уровне, которая ждет ответа отправителя. Реализуйте соответствующую программу *прием* (*receive*) транспортного уровня, которая должна служить интерфейсом DRTK.
2. Напишите на языке Си код мастера для алгоритма синхронизации часов Беркли и рабочих узлов, которые должны служить интерфейсом DRTK.
3. Разработайте FSM для булли-алгоритма из раздела 6.3.3.1 и напишите его код на языке Си, который должен служить интерфейсом DRTK.
4. Разработайте на языке Си и реализуйте основанный на FSM код задачи протокола TPSN, который должен служить интерфейсом DRTK. Напишите фазу уровня обнаружения и фазу синхронизации для узла приемника данных и обычного узла сети.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Cheriton D. R., Zwaenepoel W. (1985) Distributed process groups in the V kernel. ACM Trans Comput Syst 3 (2): 77–107.
2. Cristian F. (1989) Probabilistic clock synchronization. Distrib Comput 3 (3): 146–158 (Springer).
3. Erciyes K. (1989) Design and implementation of a real-time multi-tasking kernel for a distributed operating system. PhD thesis, Computer Eng. Dept, Ege University.
4. Elson J., Estrin D. (2002) Fine-grained network time synchronization using reference broadcast. In: The fifth symposium on operating systems design and implementation (OSDI), p. 147–163.
5. Fahmy S., Younis O. (2004) Distributed clustering in ad-hoc sensor networks: a hybrid, energy efficient approach. In: Proceedings of the IEEE conference on computer communications (INFOCOM), Hong Kong.

6. *Ganeriwat S., Kumar R., Srivastava M.* (2003) Timing-Sync protocol for sensor networks. In: The first ACM conference on embedded networked sensor systems (SenSys), p. 138–149.
7. *Heinzelman W. R., Chandrakasan A., Balakrishnan H.* (2000) Energy-efficient communication protocol for wireless microsensor networks. In: Proceedings of the 33rd annual Hawaii international conference on system sciences, vol. 2, p. 10.
8. *Heinzelman W. R., Chandrakasan A., Balakrishnan H.* (2002) An application-specific protocol architecture for wireless microsensor networks. *IEEE Trans Commun* 1 (4): 660–670.
9. *Mills D. L.* (1992) Network time protocol (version 3): specification, implementation, and analysis. RFC 1305.
10. *Mills D. L.* (2010) Computer network time synchronization: the network time protocol. Taylor & Francis, p. 12. ISBN 978-0-8493-5805-0.
11. *Gropp W., Lusk E., Skjellum A.* (1999) Using MPI: portable parallel programming with the message passing interface, 2nd edn. MIT Press.
12. *Sivrikaya F., Yener B.* (2004) Time synchronization in sensor networks: a survey. *IEEE Netw* 18 (4): 45–50.
13. Simple Network Time Protocol (SNTP) Version 4 for IPv4, IPv6 and OSI. RFC 4330 14. Zogg J-M (2002) GPS basics. Technical Report GPS-X-02007, UBlox, Mar 2002.

ЧАСТЬ III



**ПЛАНИРОВАНИЕ
И РАСПРЕДЕЛЕНИЕ
РЕСУРСОВ**

Глава 7

Задача планирования однопроцессорной независимой задачи

7.1. ВВЕДЕНИЕ

Планирование при рассмотрении распределенной системы реального времени – это процесс определения задач процессору или в случае многопроцессорной системы набору процессоров или сети вычислительных элементов. Задача в реальном времени имеет отведенное для решения время, крайний срок выполнения и требования к ресурсам. Периодическая задача активируется через равные промежутки времени, аperiodическая задача может быть активирована в любое время, а спорадическая задача имеет минимальный интервал между любыми последовательными активациями. Задачи могут иметь соотношение приоритетов, что означает, что задача не может быть запущена прежде, чем завершены предшествующие ей задачи. Кроме того, задачи могут делиться ресурсами, которые влияют на планирование решений. Наша главная цель при планировании задач в реальном времени – обеспечить выполнение задач в соответствии с их конечными сроками и разумное распределение ресурсов.

Мы будем различать планирование независимых задач, зависимых задач без разделения ресурсов, зависимых задач с разделением ресурсов, многопроцессорное планирование независимых и зависимых задач и, наконец, распределенное планирование независимых и зависимых задач. Каждое из этих различных приложений может требовать несколько разных методов планирования. Наш подход состоит в том, чтобы классифицировать эти методы как однопроцессорное зависимое планирование задач и разделение ресурсов при планировании и многопроцессорного, и распределенного планирования задач. Используя эту классификацию, мы посвятили каждой из этих схем главу

и начинаем в этой главе с планирования однопроцессорной независимой задачи. Сначала мы рассмотрим основные понятия, связанные с планированием в реальном времени.

Мы опишем основные политики планирования, а затем основные методы планирования в однопроцессорной системе. Наконец, мы покажем, как различные политики планирования могут быть применены в ядре DRTK.

7.2. Предпосылки

Типы задач и их атрибуты являются основополагающими при принятии решения о том, какой метод использовать для планирования. В целом у нас могут быть следующие типы задач в реальном времени:

- *периодические задачи*. Периодическая задача τ_i активируется через равные промежутки времени, называемые ее периодом (T_i). Большинство сложных задач в реальном времени относятся к этой категории. Например, мониторинг температуры на химическом заводе может быть выполнен с помощью задачи, которая активируется каждые несколько секунд;
- *аперiodические задачи*: это задачи, которые активируются в непредсказуемое время. Обычно аперiodическая задача активируется внешним прерыванием системы реального времени, например когда пользователь нажимает кнопку на панели управления. Примером аперiodической задачи является задача активации тревоги, когда система управления процессом замечает параметр, находящийся вне диапазона;
- *спорадические задачи*: эти задачи должны запускаться нерегулярно, как аперiodические задачи, но минимальное время между каждым появлением такой задачи заранее известно.

Задачи системы реального времени могут иметь следующие атрибуты:

- *время поступления (Arrival Time) a_i* : это время, когда задача становится готовой к выполнению. Оно также называется *временем запуска* или *временем запроса* (r_i);
- *время выполнения в наихудшем случае (Worst-Case Execution Time, WCET) C_i* : это оценка худшего (самого длинного отрезка) возможного времени выполнения задачи τ_i ;
- *время начала (Start Time) s_i* : время, когда задание τ_i начинает выполняться;
- *время окончания (Finish Time) f_i* : время, когда задача τ_i завершает выполнение;
- *время отклика (Response time) R_i* : интервал времени между временем запуска задачи и временем окончания $f_i - r_i$;
- *период задачи (Task Period) T_i* : постоянный интервал между последовательными активациями периодической задачи;
- *абсолютный крайний срок (Absolute Deadline) d_i* : задание абсолютного времени τ_i , после которого выполнение должно завершиться;

- относительный крайний срок (*Relative Deadline*) D_i : временной интервал между временем поступления задачи и временем задачи T_i , когда задача должна завершиться;
- незанятое время (*Slack Time*) S_i : это время между относительным сроком выполнения задачи и ее худшее время исполнения $D_i - C_i$;
- оставшееся время (*Laxity Time*) $L_i(t)$: оставшееся время выполнения задачи в момент времени t ;
- превышение времени (*Time Overflow*): это происходит, когда задача заканчивается после ее крайнего срока.

Все эти атрибуты приведены на рис. 7.1. При попытке запланировать задачи с крайним сроком наша цель состоит в том, чтобы обеспечить назначение задачи без превышения времени, и в этом случае планирование называется *осуществимым* (*feasible*). В литературе обычно термин *работа задачи* (*job of a task*) используется, чтобы обозначить работу с момента ее активации. Мы будем ссылаться на текущий момент выполнения задачи просто как *задача*, чтобы избежать путаницы с концепцией работы операционной системы.

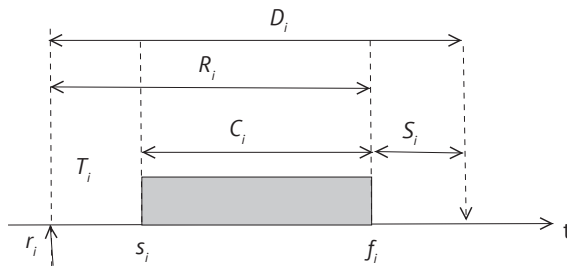


Рис. 7.1. Атрибуты задач

7.2.1. Тест планируемости

Если основные характеристики задачи известны заранее, мы можем выполнить следующие тесты планируемости набора задач:

- *необходимость*: если этот тест успешно не пройден, набор задач не может быть запланирован. Если тест выполняется успешно, у нас есть необходимое условие для планирования, но мы не можем быть уверены, что набор задач может быть запланирован. Поэтому этот тест в основном используется для обозначения незапланированного набора задач;
- *достаточность*: прохождение этого теста означает, что набор задач является планируемым. Тем не менее отрицательный результат этого теста не означает, что набор не может быть запланирован;
- *точность*: мы имеем оба вышеуказанных теста с положительными результатами.

7.2.2. Применение

Коэффициент использования μ_i (или u_i) для периодической задачи τ_i определяется следующим образом:

$$u_i = \frac{C_i}{T_i}. \quad (7.1)$$

Этот параметр показывает процент времени, в течение которого задача использует процессор во время своего периода работы. Коэффициент использования процессора U – это доля времени, затраченного на выполнение набора задач $T = \tau_1, \dots, \tau_n$:

$$U = \frac{C_i}{T_i} + \dots + \frac{C_n}{T_n}. \quad (7.2)$$

Можно видеть, что лучший коэффициент использования процессора может быть получен путем увеличения времени вычислений или уменьшения периодов задач, при которых задача остается выполнимой.

7.3. Политики планирования

Прежде всего мы должны обеспечить соблюдение конечных сроков выполнения сложных задач в реальном времени при любом методе планирования, и если мы не можем этого сделать, то не должны допускать включение задачи с невыполняемым сроком в систему. Если задачи используют ресурсы совместно, операционная система должна обеспечить эффективные механизмы защиты этих ресурсов, которые мы рассмотрели в главе 4. Необходимо также учитывать, являются задачи зависимыми или независимыми. Кроме того, нам нужно искать способы планирования комбинаций жестких, твердых и мягких задач реального времени, отражающих явления реального мира.

7.3.1. Приоритетное или неприоритетное планирование

В политике неприоритетного планирования задача выполняется до своего завершения, не будучи прерванной. Две общие политики, принятые в форме пакетной обработки в этом методе, заключаются в следующем:

- планирование «первым пришел – первым обслужен» (*First Come First Served, FCFS*): задачи помещаются в очередь отправки в соответствии с их временем прибытия и поступают на процессор по очереди, начиная с первой;
- планирование наиболее краткого задания следующим (*Shortest Job Next, SJN*): в этой политике менее продолжительная задача будет запланирована до более продолжительной задачи. Перед запуском задач в этой политике нам нужно знать время выполнения задач, и это может оказаться невозможным для ряда задач реального времени. Кроме того, задача с очень

длительным временем выполнения может быть проигнорирована процессором из-за длительного времени выполнения многих коротких задач.

С другой стороны, политика приоритетного планирования позволяет прерывать запущенную задачу, чтобы запустить задачу с более высоким приоритетом. Давайте рассмотрим в качестве примера набор из трех задач, показанный в табл. 7.1, в котором P_i является приоритетом задачи τ_i .

Таблица 7.1. Пример набора задач

τ_i	C_i	T_i	P_i
1	2	8	1
2	5	12	2
3	8	24	3

Если мы назначим статично более высокие приоритеты задачам с более короткими периодами и используем неприоритетную политику планирования, у нас получится планирование, показанное на рис. 7.2. Можно видеть, что задача τ_1 не может завершить свое выполнение до своего конечного срока с началом ее второго периода в момент времени 16.

Однако если бы мы применили приоритетную политику, то могли бы иметь планирование, показанное на рис. 7.3, где каждая задача соответствует своему конечному сроку. Задача τ_3 выгружается во время 8 до запланированного для задачи τ_1 времени, а затем в момент времени 12 начинается выполняться запланированная задача τ_2 , которая прерывается в момент времени 16 для начала задачи τ_1 , и все задачи укладываются в сроки. Обратите внимание, что мы можем повторить этот основной цикл за период времени 24. Другой путь назначения приоритетов ведет к иной политике планирования, описанной в следующем разделе.

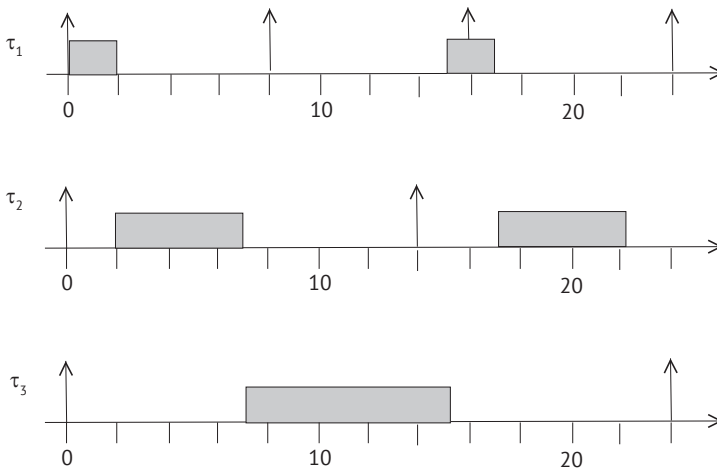


Рис. 7.2. Неприоритетное планирование для трех задач из табл. 7.1

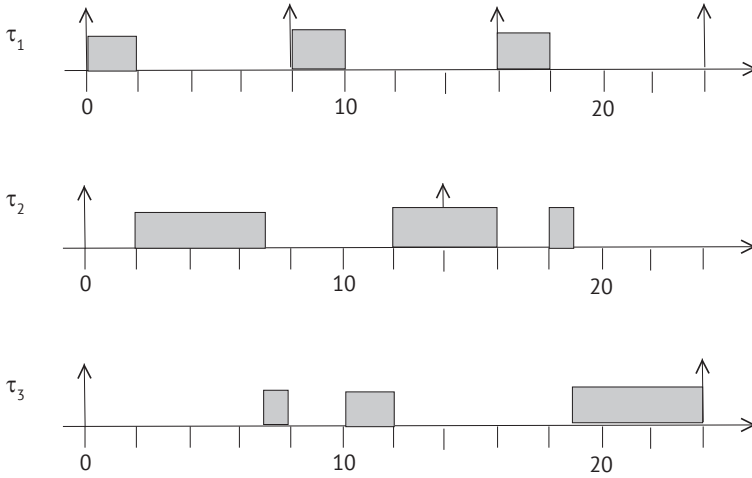


Рис. 7.3. Приоритетное планирование для трех задач из табл. 7.1

7.3.2. Статичное или динамичное планирование

Существует два типа методов планирования при принятии решений о планировании по времени:

- *статичное планирование*: метод планирования (алгоритм) использует ряд статичных параметров, и планируемые решения, когда и где активировать задачу, обычно принимаются перед выполнением задачи. Атрибуты задачи в этом методе должны быть известны заранее. Таблица распределения обычно формируется во время компиляции, и планируемые затраты времени выполнения выдерживаются минимальными просто выполнением задач, как это определено таблицей планирования;
- *динамическое планирование*: в этом случае алгоритм планирования использует некоторые динамические параметры, и решения по планированию принимаются во время выполнения задачи. Этот метод планирования адаптивен к изменению атрибутов задачи, однако для получения осуществимого графика в процессе выполнения может потребоваться значительная проработка.

Задачи с фиксированным или динамическим приоритетом

Задачи в реальном времени обычно имеют приоритеты. Приоритет задачи может быть определен в автономном режиме и может оставаться постоянным для всей системы, работающей с задачами с *фиксированным приоритетом* или *статичными приоритетными* задачами. С другой стороны, приоритет задачи с *динамическим приоритетом* может быть изменен в зависимости от его критичности. Как мы увидим, метод динамического планирования может иметь задачи со статичными конечными сроками.

Автономное или онлайн-планирование

При *автономном планировании* перед выполнением задач определяется набор задач и создается график для всего набора задач. При онлайн-планировании решения принимаются во время выполнения, когда поступают новые задачи. Обратите внимание, что онлайн-планирование может быть выполнено с использованием как статических, так и динамических параметров.

7.3.3. Независимые или зависимые задачи

Задачи системы реального времени могут быть независимыми, то есть без какого-либо взаимодействия. Чаще всего задачам реального времени необходимо общаться, чтобы передавать некоторые данные, и синхронизироваться, например чтобы сообщить о завершении события. Когда задача τ_i отправляет данные в задачу τ_j , мы говорим, что τ_i предшествует τ_j , что записывается как $\tau_i < \tau_j$, то есть τ_j не может начать выполнение до завершения τ_i . Обратите внимание, что τ_i может отправлять данные в свои первые 10 % времени выполнения, но мы полагаем, что τ_i должен завершиться до того, как τ_j начнет выполняться, поскольку обнаружение момента отправки времени данных не практично. В общем, все предшествующие задачи должны завершить выполнение, прежде чем данная задача может начаться. Это отношение предшествования изображается в *графике зависимости задач* (или просто графике задач), где дуга, направленная от τ_i к τ_j , указывает, что $\tau_i < \tau_j$, как показано на рис. 7.4. Есть семь задач, $T_1, \dots, T_7$, и T_7 – это конечная задача, которая должна ждать, пока все другие задачи не завершат выполнение. Связь между двумя задачами может иметь ненулевые затраты, когда эти две задачи находятся на двух разных узлах распределенной системы реального времени. Тем не менее затраты на связь предполагаются незначительными, когда задачи выполняются на одном процессоре, так как отправка данных будет включать передачу адреса данных и, возможно, сигнализирует об этом событии получателю сообщения.

Еще один момент, который стоит отметить, – это то, что время выполнения и отношения предшествования задач должны быть известны до того, как мы построим график задач, следовательно, нам нужно выполнить *статическое планирование* для этих задач. Поэтому важной целью алгоритма планирования любой задачи в этом случае является соблюдение отношений приоритета между задачами. Более того, когда мы стремимся распределять задачи по узлам системы распределенного реального времени, как мы увидим в главе 9, нам нужны эффективные эвристические алгоритмы для разделения графа задач.

Проблема планирования становится более сложной, когда задачи используют ресурсы совместно. Общие ресурсы защищены конструкциями операционной системы, такими как семафоры и блокировки, однако планирование конкурирующих за ресурс задач требует тщательного анализа, так как ресурс, необходимый для задачи, может быть во время ее выполнения недоступен. В этом случае выполнение задачи может быть отложено из-за несоблюдения сроков, и даже, как мы увидим в следующей главе, может возникнуть тупик.

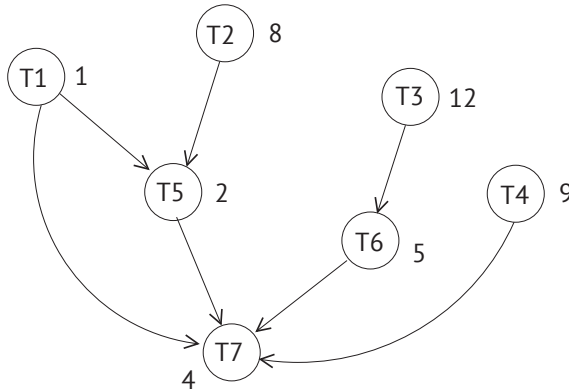


Рис. 7.4. Граф задач с разделением времени, показанным рядом с ними

7.4. ТАКСОНОМИЯ АЛГОРИТМОВ ПЛАНИРОВАНИЯ В РЕАЛЬНОМ ВРЕМЕНИ

Мы можем классифицировать алгоритмы планирования в системе реального времени, используя различные подходы. Сначала мы предположим, что задачи независимы и наша цель – найти осуществимый график выполнения задач в однопроцессорной системе в заданные сроки. На рис. 7.5 показаны предложенные в [4] (а) и в [1] (б) два разных алгоритма классификации планирования в реальном времени.

Мы видим, что для жестких задач в режиме реального времени основным различием изначально является то, осуществляется статичное или динамическое планирование или рассматриваются периодические или аperiodические задачи. Например, многие жесткие задачи реального времени являются периодическими и в то же время имеют статичные приоритеты. В следующих разделах мы рассмотрим фундаментальные алгоритмы планирования в режиме реального времени с допущениями об отсутствии взаимодействия между задачами, и наша цель будет состоять в том, чтобы получить осуществимое расписание на одном процессоре. В последующем при поиске алгоритма планирования мы будем полагать это большую часть времени. Алгоритмы планирования для зависимых задач реального времени, которые используют ресурсы совместно, будут рассмотрены в следующей главе.

- Планируемый набор задач $T = \{\tau_1, \dots, \tau_n\}$ состоит только из периодических задач.
- Конечный срок τ_i задачи D_i равен периоду T_i .
- Время вычисления C_i задачи τ_i определяется заранее и остается постоянным.

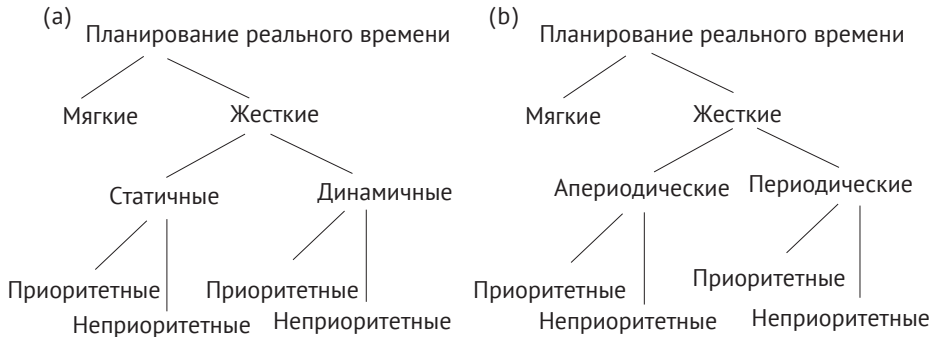


Рис. 7.5. Два типа классификации алгоритмов планирования в реальном времени

7.5. ПОТАКТОВОЕ ПЛАНИРОВАНИЕ

Сначала мы рассмотрим простую модель, в которой задачи периодические, независимые, они не делятся никакими ресурсами, и их приоритеты фиксированы. Типичное приложение с такими характеристиками состоит из набора датчиков, которые активируются через равные промежутки времени (периоды), считывают данные и отправляют эти данные в блок ввода компьютера реального времени, в котором вызывается соответствующая задача обработки входных данных в реальном времени. Эта обработка должна быть завершена до того, как с датчика будут введены следующие данные. Точки планирования известны заранее, и поэтому может быть составлена таблица, которая указывает, когда вызывать планировщик. В этом так называемом *потактовом методе* обычно используются два основных подхода: алгоритм с табличным управлением (или циклическое планирование) и циклический исполнительный алгоритм (или структурированное циклическое планирование).

7.5.1. Планирование на основе таблиц

Циклическое планирование на основе таблиц – это простой автономный метод, который в течение десятилетий используется в промышленности для сложных периодических задач в реальном времени. Он основан на предварительно рассчитанной таблице, в которой содержится время выполнения задач. Этому типу планировщика на самом деле не нужна поддержка операционной системы, достаточно аппаратного таймера, действие которого основано на точках, устанавливаемых при планировании задач. Кроме того, расчет записей в таблице может потребовать сложных алгоритмов и значительного времени, поскольку выполняется в автономном режиме. Мы полагаем, что для формирования таблицы задач в автономном режиме в этом алгоритме у нас есть n периодических задач с известными временами запуска и худшим временем выполнения. Давайте рассмотрим примерный набор из четырех задач $\tau_1, \dots, \tau_4$ с расчетными временами и периодами, показанными в табл. 7.2.

Таблица 7.2. Пример с набором задач

τ_i	C_i	T_i
1	1	4
2	2	10
3	2	10
4	4	20

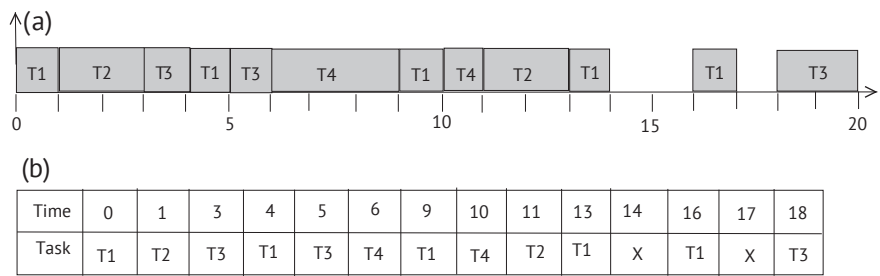


Рис. 7.6. Планирование на основе таблиц: а – выполнение четырех задач из табл. 7.1, б – содержание таблицы

Возможное планирование этих задач изображено на рис. 7.6. Обратите внимание, что после 20 временных единиц расписание повторяется, и этот период является наименьшим общим кратным из всех задач периода. Мы полагаем, что сроки выполнения задач равны их периодам, то есть каждая задача должна завершить свое выполнение и выгрузку в текущем периоде до начала следующего периода, при незначительном времени переключения контекста. Точки в таблице, отмеченные знаком X, являются временем простоя, которое можно использовать для активации любой аperiodической задачи, находящейся в режиме ожидания.

Наибольший общий делитель (greatest common divisor, GCD) периодов заданий называется *второстепенным циклом*, и *наименьшие общие кратные периоды* (lowest common multiple, LCM) являются *основными циклами* задачи.

Примечание 7.1. Нам нужно рассмотреть только переключение задач в точках GCD.

Примечание 7.2. Поскольку мы получим выполнимое расписание для основного цикла и оно будет повторяться в циклах, появилось название *циклический планировщик*.

Планирование циклического планировщика показано в алгоритме 7.1 [3].

Он ожидает прерывания только в точках переключения контекста и активирует задачу, которая указана в таблице в этой точке планирования. Обратите

внимание, что мы разрешаем выполнение аperiodических задач в этой версии циклического планировщика. Если в системе нет ожидающих аperiodических задач, строки 7–9 и 14–16 алгоритма могут быть опущены, и таблица должна быть изменена так, чтобы не было никакой активации планировщика в свободных точках времени. Это будет означать удаление активаций из таблицы в моменты времени 14 и 17.

Алгоритм 7.1. *Table_Scheduler*

```

1: Input: set  $T = \{\tau_1, \dots, \tau_n\}$  of  $n$  periodic tasks
2: aperiodic task queue  $AT\_queue$ 
3:  $i \leftarrow 0, k \leftarrow 0$ 
4: set timer to expire at  $t_k$ 
5: while true do
6:   wait for timer interrupt
7:   if  $\tau_{curr}$  is aperiodic then › если текущая задача аperiodическая, выгрузите ее
8:     preempt  $\tau_{curr}$ 
9:   end if
10:   $\tau_{curr} \leftarrow \tau(t_k)$  › выберите задачу, определенную в таблице
11:   $i \leftarrow i + 1$ 
12:   $k \leftarrow k \bmod N$ 
13:  set timer to expire at  $_i/n_H + t_k$  › установите на таймере следующую точку
    › из расписания
14:  if  $\tau_{curr}$  is  $X$  then › если текущая точка в расписании не предусмотрена, › выберите
    аperiodическую задачу
15:     $\tau_{curr} \leftarrow \text{front of } AT\_queue$ 
16:  end if
17:  schedule  $\tau_{curr}$ 
18: end while

```

7.5.2. Циклическое выполнение

Планирование на основе таблиц работает нормально, но с ростом числа задач размер таблицы растет, что может быть неудобно для систем со встроенными элементами при ограниченном объеме памяти. Мы можем улучшить работу табличного циклического планировщика с помощью включения в этот метод некоторой структуры. В этом измененном подходе к циклическому планированию время делится на фреймы с постоянными размерами, и планирование решений делается только на границах фрейма, следовательно, нет никаких предпочтений задачам внутри фрейма.

Размер фрейма f следует выбирать с осторожностью. Фрейм должен быть достаточно большим, чтобы в нем содержалась каждая задача и чтобы в приоритете внутри фрейма не было необходимости. Следовательно, мы имеем $f \geq \max(C_i), i = 1, \dots, n$, в качестве первого ограничения. Но, с другой стороны, f должна делить размер гиперпериода $H = \text{LCM}(T_1, \dots, T_n)$. Следовательно, f делит T_i по крайней мере для одной задачи τ_i :

$$\left\lceil \frac{T_i}{f} \right\rceil - \frac{T_i}{f} = 0$$

как второе ограничение. Пусть $F = H/f$, интервал H называется *основным циклом*, и интервал f – *минорным циклом*. Для обеспечения соблюдения сроков выполнения задач можно показать, что должно выполняться следующее неравенство [3]:

$$2f - CGD(T_i, f) \leq D_i, \quad (7.3)$$

что является третьим ограничением. Определим f для задач в табл. 7.2. Первое ограничение выполняется при $f \geq 4$. Гиперпериод H равен 20 временным единицам, следовательно, величина f может быть поделена на 4, 5, 10 или 20. Третье ограничение не выполняется для всех задач, и в этом случае мы можем использовать метод, называемый *разбиением задач (task splitting)*. Мы видим, что самые длительные выполняемые задачи – τ_4 , которые можно разбить на три подзадачи τ_{41} , τ_{42} и τ_{43} , и теперь мы можем набросать возможный график, как показано на рис. 7.7.



Рис. 7.7. Выполнение четырех задач из табл. 7.1 при циклическом исполнении

Алгоритм, который реализует метод структурированного циклического планирования и называется *циклическая исполнительная программа (cyclic executive)*, запускается при прерываниях по таймеру на границах фрейма и выполняет блок целиком до следующего прерывания таймера, что позволяет использовать неиспользованное время аperiodической задачи [2]. Структура его изображена в алгоритме 7.2.

Алгоритм 7.2. *Cyclic_Executive*

```

1: Input: stored schedule  $S = \{S_1, \dots, S_k\}$ ,  $F$ 
2:  $i \leftarrow 0$ ,  $t \leftarrow 0$ 
3: set timer to expire at  $t$ 
4: while true do
5: wait for timer interrupt
6: current_block  $\leftarrow S_i$ 
7:  $t \leftarrow t + 1$ 
8:  $i \leftarrow t \bmod F$ 
9: execute all tasks in current_block
10: sleep until next timer interrupt
11: end while

```

Циклическое планирование в обеих формах, в основных или структурированных методах, является простым и эффективным, однако даже незначительное изменение атрибутов задачи может потребовать значительного изменения графика, который должен быть рассчитан с нуля. Тем не менее он продолжает использоваться во многих современных приложениях реального времени благодаря своей простоте и низким затратам.

7.6. ПРИОРИТЕТНОЕ ПЛАНИРОВАНИЕ

Онлайн-планирование основано на принятии решений по планированию во время выполнения задачи. Это динамический подход к проблеме планирования. Данный метод может включать задачи со статичным или с динамическим приоритетом. Мы рассмотрим четыре основных алгоритма онлайн-планирования, основанных на приоритете: алгоритмы монотонный, монотонный с конечным сроком – как алгоритмы со статичным приоритетом – и алгоритмы с динамическим приоритетом: первый с самым ранним крайним сроком и первый с наименьшим незанятым временем.

7.6.1. Монотонное планирование

Монотонное планирование (rate monotonic, RM) – это динамический алгоритм реализации приоритетов для независимых, периодических, жестких задач реального времени со статичным приоритетом. Мы полагаем, что независимые самостоятельные периодические задачи имеют конечные сроки, равные их периодам. Основная идея этого алгоритма заключается в назначении статичных приоритетов задачам на основе их временных периодов – более короткий период означает более высокий приоритет. Приоритет выполнения задачи является обратным ее периоду, и отсюда название данного алгоритма. Задача с более высоким приоритетом предопределяет выполнение именно этой цели. Время переключений контекста считается незначительным.

Формально задача τ_i имеет более высокий приоритет, чем задача τ_j , если $T_i < T_j$, полагая, что все задачи имеют разные периоды. Когда периоды двух или более задач равны, для нарушения симметрии могут быть использованы идентификаторы задач. Рассмотрим набор задач T в табл. 7.3 с тремя задачами τ_1 , τ_2 , τ_3 с указанием времени и периодов (конечных сроков). Приоритеты, назначенные этим задачам, показаны в последнем столбце таблицы.

Отношение использования к числу задач дается следующей теоремой:

Теорема 7.1 (Liu [3]). Набор из n периодических жестких задач реального времени будет соответствовать их конечным срокам независимо от времени их запуска, если

$$\sum_i^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1). \quad (7.4)$$

Коэффициент использования U сходится к $\ln 2 = 0,69$ при $n > 10$. Поэтому, применяя это неравенство, мы можем проверить, можно ли запланировать некоторый набор задач. Обратите внимание, что эта проверка – достаточное, но не обязательное условие. Другими словами, может быть набор задач, который имеет общее использование, превышающее 0,69, но задачи в этом наборе могут все еще уложиться в сроки в расписании RM.

Рассмотрим множество задач $\tau_1(2,8)$ и $\tau_2(8,20)$ в форме $\tau_i(C_i, T_i)$ для RM-планирования. Можно видеть, что эти две задачи могут быть запланированы, так как $(2/8) + (8/20) = 0,65 < 2(2^{0.5} - 1) = 0,82$. Следовательно, этот набор задач может быть принят в систему. Планирование этих задач с использованием планирования RM показано на рис. 7.8. Поскольку наименьшее общее кратное (least common multiple, LCM) их периодов равно 40, расписание будет повторяться каждые 40 единиц времени.

Таблица 7.3. Пример набора задач

τ_i	C_i	T_i	P_i
1	3	10	1
2	6	24	3
3	7	12	2

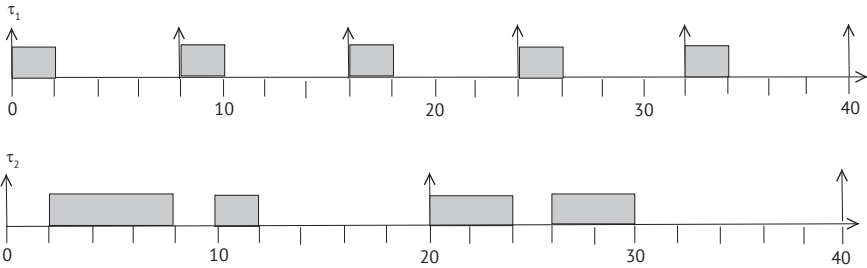


Рис. 7.8. Выполнимое RM-планирование двух независимых задач, прошедших проверку

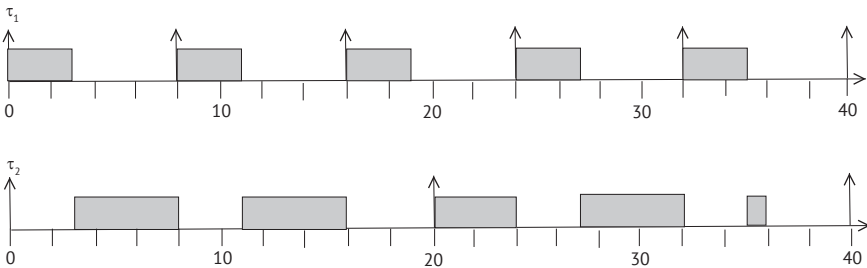


Рис. 7.9. Выполнимое RM-планирование двух независимых задач, не прошедших проверку

Теперь увеличим время вычисления задач до $\tau_1(3,8)$ и $\tau_2(10,20)$. Применение теста планируемости дает $(3/8) + (10/20) = 0,875 > 0,82$, что говорит, что алгоритм RM не гарантирует наличие подходящего графика для этого набора задач. Однако мы все еще можем запланировать выполнение этих задач, как показано на рис. 7.9.

Преимущества данного метода в том, что он прост в реализации и оптимален по сравнению со всеми алгоритмами планирования статичного приоритета. Тем не менее он не может эффективно использовать всю мощность обработки, как это показано в примере выше.

Монотонный алгоритм с конечными сроками

Могут существовать и периодические задачи реального времени, сроки которых отличаются от их периодов, то есть $\exists \tau_i: D_i < T_i$. В таких случаях может использоваться монотонный алгоритм с конечным сроком (deadline monotonic, DM), который назначает приоритеты задачам, основываясь только на их сроках. Алгоритм DM является разновидностью алгоритма RM и позволяет создавать выполнимые расписания в случаях, в которых алгоритм RM терпит неудачу.

Примером $\tau_i (C_i, D_i, T_i)$ с двумя задачами может служить задача, задаваемая следующим образом: $\tau_1 (2, 5, 5)$ и $\tau_2 (3, 4, 10)$. Запуск алгоритма планирования RM для этих двух задач приводит к графику на рис. 7.10а, где τ_2 пропускает свой крайний срок, и, поскольку τ_1 имеет более короткий период, она запланирована первой. Эти две задачи приведены, чтобы показать на графике на рис. 7.10б выполняемость завершения задачи с использованием DM-планирования в установленные сроки.

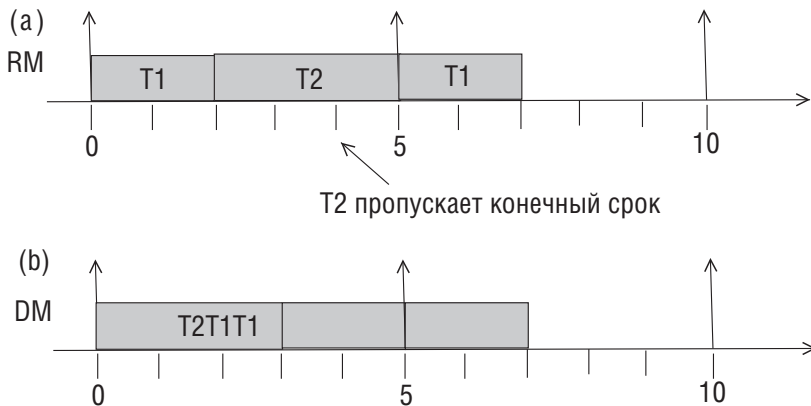


Рис. 7.10. Сравнение методов планирования RM и DM

7.6.2. Планирование с первым самым крайним сроком

Алгоритм самого крайнего срока (earliest deadline first, EDF) – это алгоритм динамического планирования реализации приоритетов, который назначает

процессору задачи с динамическими приоритетами. В этом методе приоритет задачи увеличивается по мере приближения ее срока, и задачи с самым ранним сроком всегда выполняются первыми. При каждом прерывании таймера вычисляется время, оставшееся до выполнения каждой задачи, и отправляется задание с наименьшим значением. Этот частый подсчет может при большом наборе задач привести к значительным затратам.

Примечание 7.3. Алгоритм EDF создает выполнимый график, если коэффициент использования $U \leq 1$.

Это означает, что алгоритм EDF является эффективной политикой планирования, которая может использовать 100 % мощности процессора. Алгоритм EDF может быть реализован как с периодическими, так и с аperiodическими задачами.

7.6.2.1. Аperiodический алгоритм EDF

Рассмотрим сначала аperiodический случай с заданием, приведенным в табл. 7.4. Все три задачи являются аperiodическими (однократными) и имеют разные абсолютные конечные сроки.

График планирования алгоритма EDF этого набора задач показан на рис. 7.11.

Таблица 7.4. Пример с набором задач

τ_i	Время прибытия a_i	C_i	Абсолютный крайний срок d_i
1	0	8	18
2	4	6	14
3	8	10	26

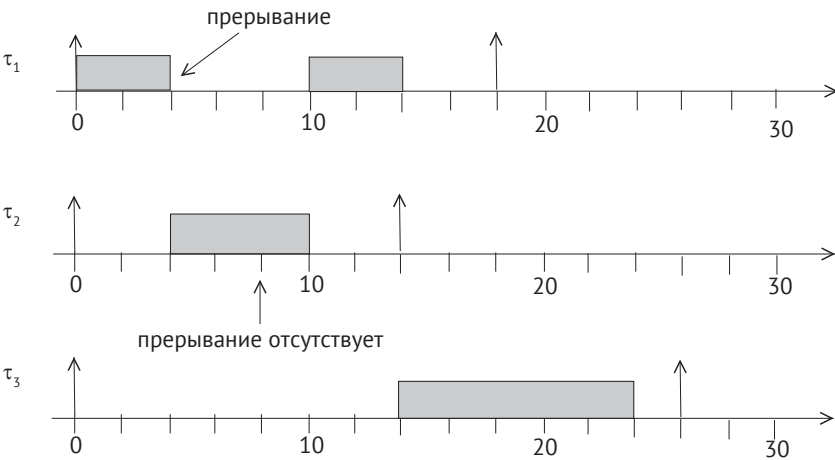


Рис. 7.11. Алгоритм EDF с тремя аperiodическими задачами

Видно, что задача τ_1 прерывается задачей τ_2 в момент времени 4, так как абсолютный срок τ_2 (14) является самым ранним сроком в этот момент времени. Позже, когда будет запущена задача τ_3 в момент времени 8, приоритет отсутствует, так как τ_2 все еще имеет ближайший срок. Когда τ_2 заканчивается при $t = 10$, τ_1 имеет текущий ранний крайний срок, и поэтому планируется эта задача. Задача τ_3 имеет самый дальний срок, и она планируется с завершением после 24 единиц времени.

Таблица 7.5. Пример набора задач

τ_i	C_i	T_i
1	3	10
2	4	13
3	8	21

7.6.2.2. Периодический алгоритм EDF

Теперь мы полагаем, что задачи являются периодическими, между ними нет отношения приоритета, не существует требования взаимного исключения между любой парой задач, крайний срок задачи равен ее периоду, а время переключения контекста ничтожно, как в алгоритме планирования RM. Время выполнения в самом худшем случае для каждой задачи является постоянным, и его значение, как в алгоритме RM, известно заранее.

Всякий раз, когда задача становится готовой к выполнению, например когда начинается ее период, приоритеты задачи могут быть изменены, чтобы назначить самый высокий приоритет задаче, которая ближе к сроку. Планировщик времени выполнения затем выбирает для исполнения задачу с самым высоким приоритетом. Можно показать, что при этом методе за счет корректировки приоритетов во время выполнения загрузка процессора может достигать 100 %. Давайте рассмотрим набор задач из табл. 7.5.

Сначала мы попытаемся запланировать этот набор, используя алгоритм RM. Коэффициент использования равен $(3/10) + (4/13) + (8/21) \approx 0,989$, что больше $U = 3(2^{(1/3)} - 1) = 0,78$. Тем не менее мы видели, что тест RM обеспечивает достаточность. Другими словами, неудача этого теста не означает, что набор задач не может быть запланирован. Планирование, полученное при применении алгоритма RM, показано на рис. 7.12, и мы можем видеть, что задача τ_3 не укладывается в срок.

Теперь мы попытаемся запланировать этот набор, используя алгоритм EDF, и соответствующий результат изображен на рис. 7.13, где показан выполнимый график. Обратите внимание, что задача τ_3 не должна ждать задачи τ_2 , поскольку она имеет более короткий срок и, следовательно, более высокий динамический приоритет.

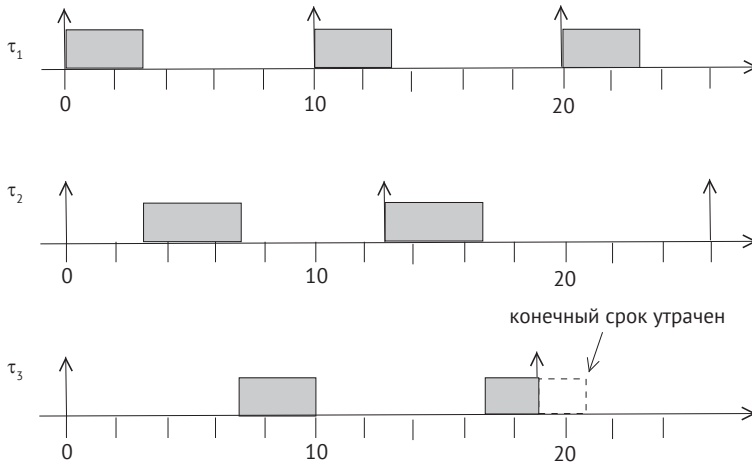


Рис. 7.12. Планирование RM трех периодических задач из табл. 7.5

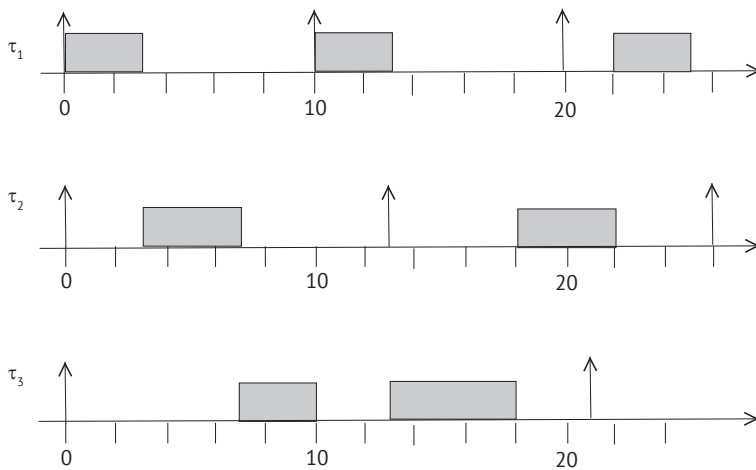


Рис. 7.13. Планирование EDF трех периодических задач из табл. 7.5

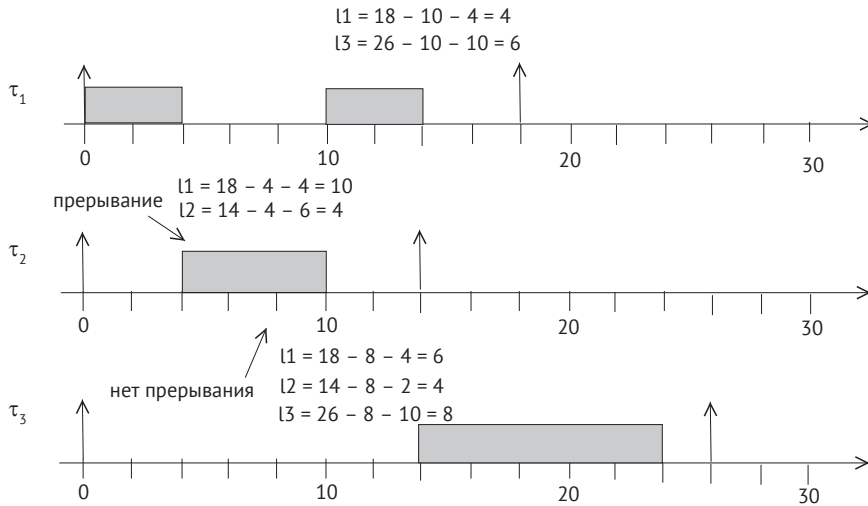


Рис. 7.14. Планирование LLF трех периодических задач из табл. 7.4

7.6.3. Алгоритм наименьшего времени незанятости

Мы определили время незанятости (резерв времени) задачи как разницу между конечным сроком выполнения и оставшимся временем его выполнения. Обратите внимание, что этот параметр является динамическим, так как оставшееся время выполнения задачи со временем меняется, и чем дольше она выполняется, тем меньше оставшееся время и, следовательно, меньше эта его неопределенность. Алгоритм первой наименьшей незанятости (*least laxity first*, LLF) оценивает значения незанятости для всех задач и планирует выполнение задачи с минимумом этого значения. Алгоритм первой наименьшей незанятости (LLF) является алгоритмом динамического планирования, который назначает задачи с динамическими приоритетами, как алгоритм EDF. Он отличается от алгоритма EDF временем вычисления задач, что влияет на решения по планированию. Давайте снова рассмотрим поставленную в табл. 7.4 задачу. Использование алгоритма LLF приводит к полученному графику на рис. 7.14 с показанными на нем значениями незанятости в критические моменты. В момент времени $t = 4$ задача τ_2 активируется и имеет более низкое значение незанятости, чем τ_1 , что приводит к вытеснению τ_1 . Там нет вытеснения τ_2 при $t = 8$, когда τ_3 становится доступной, так как τ_2 имеет наименьшую незанятость среди всех трех задач. Соответствующими задачами при $t = 10$ являются τ_1 и τ_2 , и τ_1 имеет более низкое значение незанятости, поэтому запланирована она. Мы видим, что этот график повторяет результат, полученный с помощью алгоритма EDF, однако эти два алгоритма могут создавать разные графики.

7.6.4. Анализ времени отклика

Время отклика R_i задачи τ_i – это интервал времени между временем поступления задачи τ_i и временем окончания ее выполнения. В системе на основе статического приоритета $R_i = C_i + I_i$, где I_i – задержка, вызванная задачами, которые имеют более высокие приоритеты, чем τ_i .

Первым требованием для задачи τ_i является уверенное обеспечение $R_i \leq D_i$. Давайте рассмотрим, как можно рассчитать R_i . В течение интервала времени

R_i любая задача τ_j с $P_j > P_i$ будет вызвана $\left\lceil \frac{R_i}{T_j} \right\rceil$ раз. Общая задержка, вызванная τ_j и τ_i , составит

$$\left\lceil \frac{R_i}{T_j} \right\rceil C_j.$$

Время отклика τ_i будет суммой времени его выполнения и общей задержки задачи τ_i из всех более приоритетных задач следующим образом:

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j. \quad (7.5)$$

где $hp(i)$ – набор задач с приоритетами, превышающими τ_i . Текущее соотношение для этого уравнения может быть сформировано как

$$R_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^n}{T_j} \right\rceil C_j. \quad (7.6)$$

Мы можем начать итерацию со значения R_i , которое гарантированно будет меньше или равно его окончательному значению, например $R_i^0 = C_i$, и остановить итерацию, когда R_i^{n+1} сходится к R_i^n или когда $R_i^n > D_i$.

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j. \quad (7.7)$$

Давайте рассмотрим набор задач из табл. 7.6 с задачами из табл. 7.1, упорядоченными соответственно их приоритетам.

Временем отклика для задачи τ_1 с наивысшим приоритетом является ее время вычисления, равное 3 единицам времени, поскольку никакая другая задача не будет вытеснять его, и $R_1 \leq T_1$ будет означать, что задача τ_1 уложится в срок.

Итерация для времени отклика τ_3 дает

$$R_3^1 = C_3 + \left\lceil \frac{R_3^0}{T_1} \right\rceil C_1 = 2 + \left\lceil \frac{2}{10} \right\rceil 3 = 5.$$

Таблица 7.4. Пример набора задач

τ_i	C_i	T_i	P_i
1	3	10	1
3	2	12	3
2	6	24	2

Следующая итерация дает

$$R_3^2 = 2 + \left\lceil \frac{5}{10} \right\rceil 3 = 5.$$

Поскольку изменения значения R_3 нет, $R_3 = 5 \leq T_3$, что показывает, что задача τ_3 выполняется в пределах ее крайнего срока. Нам необходимо рассмотреть задачи τ_1 и τ_3 во время отклика вычисления τ_2 следующим образом:

$$R_2^1 = C_2 + \left\lceil \frac{R_2^0}{T_3} \right\rceil C_3 + \left\lceil \frac{R_2^0}{T_1} \right\rceil C_1$$

$$6 + \left\lceil \frac{6}{12} \right\rceil 2 + \left\lceil \frac{6}{10} \right\rceil 3 = 11.$$

Вычисление с итерациями дает

$$R_2^2 = 6 + \left\lceil \frac{11}{12} \right\rceil 2 + \left\lceil \frac{11}{10} \right\rceil 3 = 14$$

$$R_2^3 = 6 + \left\lceil \frac{14}{12} \right\rceil 2 + \left\lceil \frac{14}{10} \right\rceil 3 = 16$$

$$R_2^4 = 6 + \left\lceil \frac{16}{12} \right\rceil 2 + \left\lceil \frac{16}{10} \right\rceil 3 = 16.$$

Поскольку $R_2^4 = R_2^3$, мы останавливаемся, $R_2 \leq T_2$, и мы можем заключить, что все задачи будут выполняться в свои конечные сроки, равные их периодам. Обратите внимание, что это необходимое и достаточное условие. Применим проверку RM:

$$U \leq n(2^{1/n} - 1) = 3(2^{1/3} - 1) = 0,78.$$

Поскольку $U = (C_1/T_1) + (C_2/T_2) + (C_3/T_3) = 3/10 + 2/12 + 4/24 = 0,55$, мы можем сказать, что этот набор задач выполним по срокам при планировании RM.

7.7. АПЕРИОДИЧЕСКОЕ ПЛАНИРОВАНИЕ ЗАДАЧ

В общем случае в системах реального времени не применяется смешение периодических и аperiodических задач, требующее использования механизма аperiodических задач. Аperiodические задачи обычно связаны с мягкими

сроками, а спорадические задачи, как правило, имеют жесткие сроки выполнения. Простой подход к планированию аperiodических задач включает поэтому использование двух очередей, готовых выполнять задачи, с более высоким приоритетом для периодических задач реального времени с жесткими сроками и очередь с более низкими приоритетами для аperiodических задач реального времени с мягкими сроками, как показано на рис. 7.15. Периодические задачи планируются с использованием алгоритма на основе приоритетов, таких как RM, EDF или LLF. Однако применение такой простой схемы может затрудняться возможным длительным временем отклика аperiodических задач, когда количество периодических задач велико.

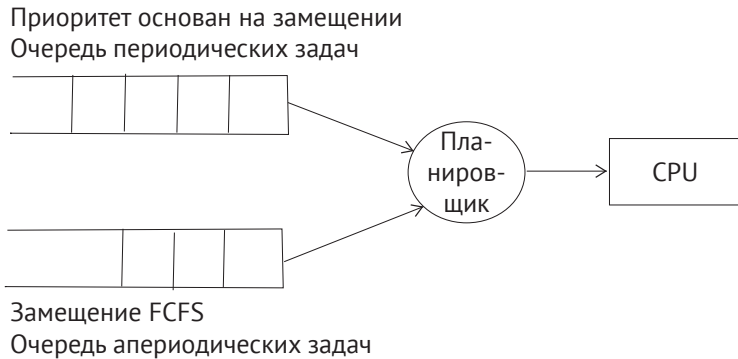


Рис. 7.15. Аperiodические задачи, планируемые с использованием двух очередей

7.7.1. Основные методы

Планирование аperiodических задач должно быть нацелено на их быстрое выполнение таким образом, чтобы сроки выполнения периодических задач также были соблюдены. В этом контексте главная цель любого аperiodического алгоритма планирования задач состоит в том, чтобы минимизировать среднее время отклика аperiodической задачи, не нарушая гарантированные крайние сроки периодических задач. Мы можем применить два основных подхода при планировании аperiodических задач в системе реального времени с периодическими задачами.

- Аperiodические задачи могут выполняться во временных интервалах, когда процессор простаивает. Это простой метод, который работает хорошо, однако аperiodические задачи могут иметь длительные задержки, вызывая значительное среднее время отклика. Рисунок 7.16 изображает планирование RM задачи $\tau_1(0, 2, 5)$ и $\tau_2(0, 5, 15)$ с аperiodической задачей $\tau_3(0, 1)$, время отклика τ_3 равно 10, хотя она имеет единичное время выполнения.

- Периодические задачи могут быть прерваны для выполнения аperiodических задач. Аperiodические задачи уменьшают время отклика, но периодические или спорадические задачи могут пропустить конечные сроки.
- Лучший метод, чем описанные выше процедуры, называемые *воровством незанятого времени (slack stealing)*, работает аналогично фоновому планированию аperiodических задач. Незанятое время (slack time) периодического времени – это интервал времени между его крайним сроком и оставшимся временем вычисления. Если $C_i(t)$ – оставшееся время вычисления задачи τ_i в момент времени t , незанятость (slack) задачи τ_i в момент времени t

$$\text{slack}_i = d_i - t - C_i(t).$$

При воровстве незанятого времени периодическое задание может быть удалено до следующей точки планирования, пропуская конечный срок, чтобы оставить время для аperiodического планирования задач. Реализация этого метода для задач рис. 7.16 показана на рис. 7.17. Отклик задачи τ_3 в этом примере с использованием воровства незанятого времени уменьшается до 1. Расчет данного времени и его перемещение требует значительных вычислений.

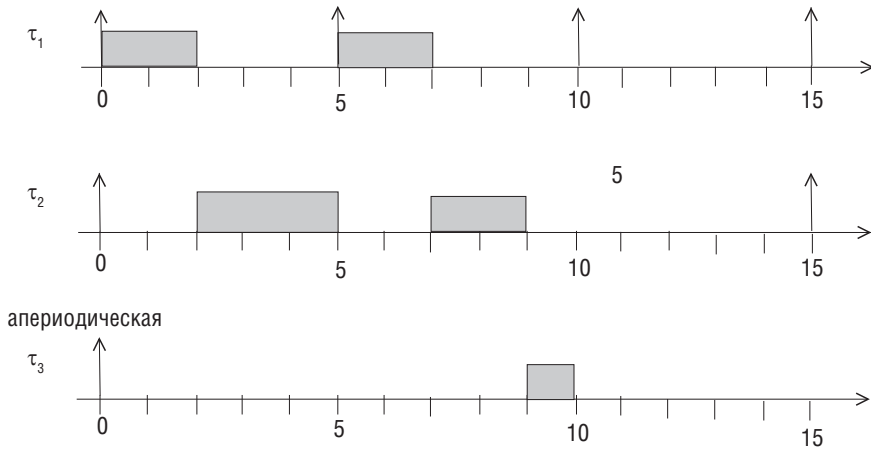


Рис. 7.16. Фоновое планирование аperiodической задачи

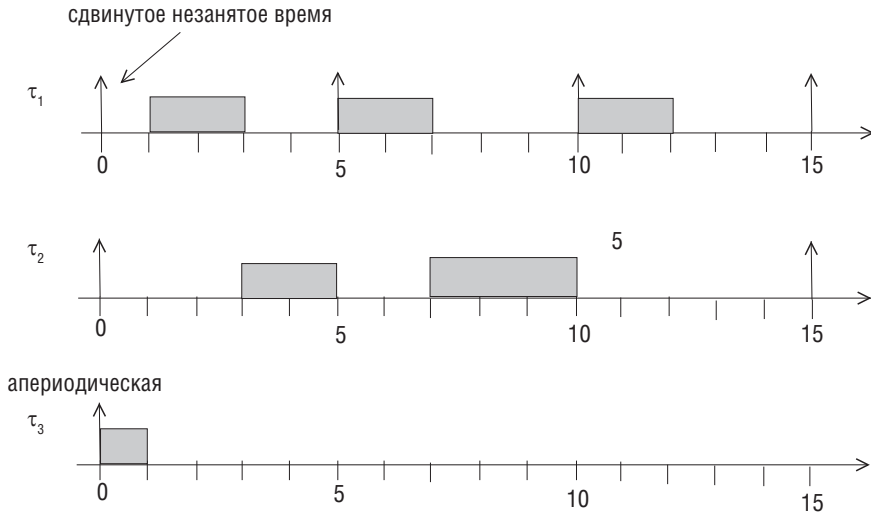


Рис. 7.17. Пример воровства незанятого времени для задачи на рис. 7.16

7.7.2. Периодические серверы

Периодический сервер – это системная задача, которая запланирована как любая другая периодическая задача с целью выполнения аperiodических задач. Он определяется как $\tau_s = (E_s, T_s)$, где T_s – период сервера и E_s – его максимальный бюджет. Бюджет может быть *израсходован* и *пополнен*, а когда он заканчивается, *истощен*. Периодический сервер может быть в одном из следующих состояний:

- простаивает, если очередь аperiodических задач пуста;
- быть в резерве, ожидании, когда аperiodическая очередь задач не пуста;
- задействован, когда он в резерве, а его бюджет не исчерпан.

7.7.2.1. Сервер опроса

Сервер опроса (Polling Server, PS) предназначен для улучшения среднего времени отклика аperiodических задач. Формируется как периодическая системная задача, как правило, с высоким приоритетом, имеет период T_s и вычислительное время C_s , выделенное ему для каждого его периода. Когда он активирован, он использует свой бюджет для любой аperiodической задачи, которая ожидает выполнения. Правила потребления бюджета PS следующие:

- когда PS выполняется, его бюджет расходуется из расчета единицы за единицу времени;
- его бюджет заканчивается, как только он перестает действовать.

Бюджет PS пополняется время от времени в начале каждого его периода, $k T_s$ для $k = 1, \dots$. Предполагается, что есть n периодических задач и PS с коэффициентом использования $U_s = T_s / C_s$ и для планирования используется алгоритм RM + PS, тогда имеем график, гарантирующий выполнение задач, если

$$U_s + \sum_{i=1}^n \frac{C_i}{T_i} \leq (n+1)(2^{\frac{1}{n+1}} - 1). \quad (7.8)$$

В левой части неравенства – сумма коэффициента использования сервера и использования n задач, в правой части неравенства – тестовое значение алгоритма RM для $(n + 1)$ задач.

7.7.2.2. Отложенный сервер

Отложенный сервер (Deferrable Server, DS) также является периодической системной задачей с бюджетом E_s и периодом T_s . В отличие от PS, он резервирует свои мощности до конца периода, разрешая выполнение аperiodических задач, которые приходят в этом периоде позже. Емкость DS полностью пополняется в начале каждого периода, как PS. В целом DS обеспечивает лучшее время отклика для аperiodических задач.

7.7.2.3. Спорадический сервер

Спорадический сервер (Sporadic Server, SS) является высокоприоритетной задачей для аperiodического планирования задач и работает аналогично DS, резервируя свой бюджет до конца своего периода, ожидая аperiodического запроса. Однако пополнение его емкости отличается. SS пополняет свою емкость только после того, как она будет использована аperiodическими задачами. Реализация SS сложнее, чем реализации PS и DS, из-за сложности расчетов моментов пополнения и управления таймером.

7.7.2.4. Серверы с динамическим приоритетом

Сервер с динамическим приоритетом имеет различный приоритет, как следует из его названия. Основными типами динамических серверов являются:

- *сервер с постоянной пропускной способностью (Constant Bandwidth Server, CBS)*: этот тип сервера имеет известное, зарезервированное для него время процессора U_{CBS} . Он имеет бюджет и правила потребления и пополнения, как и другие серверы, и планируется по принципу EDF, когда его бюджет не равен нулю. Использование этого сервера является постоянным, и отсюда его наименование. Его бюджет и конечный срок определяются таким образом, чтобы обеспечить его фиксированное использование, и он может потреблять бюджет только тогда, когда выполняется. Правила пополнения для CBS следующие:
 - ♦ вначале он имеет нулевой бюджет и нулевой конечный срок;
 - ♦ когда аperiodическая задача со временем выполнения C_A прибывает в момент времени t_A :

если $t_A < D_{CBS}$, дождитесь окончания CBS;

если $t_A \geq D_{CBS}$, установите $D_{CBS} = t_A + C_A/U_{CBS}$ и $C_{CBS} = C_A$;

- ♦ когда конечный срок CBS достигнут в момент времени t_D , если CBS находится в состоянии ожидания, установите $D_{CBS} = D_{CBS} + C_A/U_{CBS}$ и $C_{CBS} = C_A$. В противном случае подождите, так как сервер занят.

Таким образом, гарантируется, что у CBS всегда достаточно бюджета для завершения задачи в начале аperiодической очереди задач при ее пополнении;

- *сервер общей пропускной способности (Total Bandwidth Server, TBS)*: этот сервер обеспечивает лучшее время отклика для аperiодических задач, чем CBS с использованием времени, не используемого периодическими задачами. Он имеет правила пополнения, аналогичные CBS, со следующим отличием:

- ♦ когда аperiодическая задача со временем выполнения C_A поступает в момент времени t_A , установить $\max(D_{CBS}, t) + C_A/U_{CBS}$ и $C_{CBS} = C_A$;
- ♦ когда TBS заканчивает выполнение текущей аperiодической задачи τ_A , эта задача удаляется из аperiодической очереди задач, и если CBS находится в состоянии ожидания, установите $D_{CBS} = D_{CBS} + C_A/U_{CBS}$ и $C_{CBS} = C_A$. В противном случае подождите, пока сервер занят. TBS всегда готов к исполнению, когда он находится в состоянии ожидания.

7.8. ПЛАНИРОВАНИЕ СПОРАДИЧЕСКИХ ЗАДАЧ

Спорадические задачи имеют минимальный интервал между активациями, но точное время их активации заранее неизвестно. У них жесткие конечные сроки, и, следовательно, мы, возможно, не сможем их запланировать. Эти задачи проходят приемочное тестирование планировщика, и он принимает решение, выполнять их или нет. Нам необходимо убедиться, что при поступлении новой спорадической задачи ее принятие не приведет к пропаданию ранее принятой спорадической задачи. Принятые задачи будут выполнены с использованием алгоритма EDF, так как этот метод удобен в системе с динамическим приоритетом. Типичный сценарий планирования в системе реального времени будет выглядеть так, как показано на рис. 7.18.



Рис. 7.18. Планирование спорадических, периодических и аperiodических задач

Спорадическая задача определяется как $S_i(A_i, C_i, D_i)$ с A_i в качестве времени прибытия, C_i является временем вычисления, и D_i – конечный срок. Плотность спорадической задачи:

$$\Delta_i = \frac{C_i}{D_i - A_i}.$$

Общая плотность системы с n задачами:

$$\Delta = \sum_{i=1}^n \Delta_i.$$

Теорема 7.2. Система независимых приоритетных спорадических задач с использованием алгоритма EDF планируема, если общая плотность всех активных заданий в системе всегда меньше или равна 1.

Тест допустимости для спорадической задачи основан на этой теореме. Когда поступает спорадическая задача S_i , то, если определено, что общая плотность будет превышать 1 в выполняемом интервале, задача S_i отклоняется.

7.9. РЕАЛИЗАЦИЯ В DTRK

В этом разделе мы сначала покажем, как реализовать три политики планирования в DTRK: планировщик монотонного рейтинга (Rate Monotonic Scheduler), планировщик первый с наименьшей незагруженностью для периодических задач (Least Laxity First Scheduler for periodictasks) и отложенный сервер для аperiodических задач (Deferrable Server for aperiodic tasks). Обратите внимание, что планировщик является потоком POSIX, пробуждаемым семафором *после* вызова POSIX в DTRK. Мы будем проектировать его как простую быструю функцию, которая определяет, какую задачу запустить, и подает сигналы семафору POSIX этой задачи, заставляя ее начать работу.

7.9.1. Планировщик монотонного рейтинга

Мы будем полагать, что задачи являются периодическими, независимыми без ограничений предшествования, и они не разделяют память. Кроме того, тест планируемости RM выполняется в автономном режиме, так что все задачи могут уложиться в свои конечные сроки. RM-планирование DTRK предполагает, что периодические задачи находятся в очереди приоритетов *RM_Queue*, и, следовательно, нам нужна функция, которая ставит задачи в очередь при инициализации DTRK. Этот системный вызов с именем *Init_RM*, показанный ниже, вычисляет приоритеты периодических задач и вставляет их в *RM_Queue*. Он должен быть вызван в процедуре инициализации *init_system* DTRK.

```

/*****
Инициализация задач RM
*****/
void Init_RM() {
    task_ptr_t task_pt;
    for (i=0; i<N_TASKS; i++)
        if( task_tab[i].type=PERIODIC) {
            task_tab[i].priority=(int)1/task_tab[i].period;
            task_pt=&(task_tab[i]);
            insert_queue(&RM_queue,task_pt);
        }
}

```

Давайте рассмотрим подход в DTRK для периодических задач. Задача RM в DTRK задерживается на некоторый промежуток времени системным вызовом *delay_task* и ожидает своего пробуждения. *Time_ISR* при пробуждении прерыванием по таймеру проверяет *delta_queue*, и если у начала очереди не осталось времени, она выбирает задачу из начала очереди, приводит в состояние готов-

ности (ready) и вызывает планировщика. Эта операция соответствует приоритетному RM-планированию. Обратите внимание, что RM-планирование принимает динамические решения во время выполнения, используя статические приоритеты и замещение.

Код для RM-планировщика может быть сформирован на основе задержки в DTRK операции как простой процедуры, которая всегда ожидает на своем semaфоре POSIX соответственно общему планированию в DTRK. Она могла быть вызвана либо как результат прерывания таймером, поскольку необходимо активировать периодическую задачу, либо как текущая задача, завершившая обработку и ждущая следующего периода. Чтобы различать эти два случая, *RM_Scheduler* сначала проверяет состояние текущей задачи, и если она *отложена*, выбирается первая задача из готовой очереди *RM_Queue*. В противном случае она сравнивает приоритет текущей задачи с приоритетом первой задачи в начале очереди *RM_Queue* и, как показано ниже, выбирает в качестве следующей задачи для запуска задачу с более высоким приоритетом. Если текущая задача, вызвавшая планировщик, имеет более высокий приоритет, чем первая задача *RM_Queue*, она возобновляется системным вызовом планировщика активацией идентификатора задачи, хранящегося в системной таблице, как *preempted_tid*. Нам нужно хранить идентификатор замещенной задачи, поскольку задача планировщика выполняется как задача DTRK и, следовательно, как поток POSIX. Мы полагаем, что в *RM_Queue* есть хотя бы одна *готовая* задача.

```

/*****
RM-планировщик
*****/
TASK RM_Scheduler(){

    task_ptr_t task_pt;
    while(TRUE){
        sem_wait(&(System_Tab.sched_sem));
        if(task_tab[current_tid].state!=DELAYED ||
           task_tab[current_tid].state!=BLOCKED)
            if(task_tab[current_pid].priority <
               (RM_Queue.front)->priority) {
                task_pt=dequeue_task(&RM_queue);
                current_tid=task_pt->tid;
                insert_task(&RM_Queue,&task_tab[System_Tab.preempted_tid]);
            }
        else
            current_tid=System_tab.preempted_tid;
        else {
            task_pt=dequeue_task(&RM_queue);
            current_tid=task_pt->tid;
        }
        current_pt=&(task_tab[current_pid]);
        sem_post(&(task_tab[current_tid].sched_sem));
    }
}

```

7.9.2. Планировщик самого раннего первого срока

Планировщик EDF должен всегда запускать задачу с ближайшим крайним сроком. Замещение происходит, когда задача с крайним сроком ближе к той, которая активирована и выполняется. Мы будем полагать, что все задачи являются периодическими и независимыми и не разделяют ресурсы. Периодические задачи ставятся в очередь отложенных задач *delta_queue*, и сервисная подпрограмма временной обработки прерываний уменьшает значение задержки первой задачи в очереди. Если это значение равно нулю, задача изымается из очереди и готовится к обычной операции в DRTK. Планировщик EDF – это системная задача, вызываемая, когда отложенная задача из *delta_queue* активирована или текущая задача заблокирована в ожидании события. Все *готовые* задачи находятся в *EDF_Queue*, и планировщик EDF сравнивает крайний срок текущей задачи с крайним сроком разблокированной задачи в *EDF_Queue* и выбирает задачу с ближайшим крайним сроком, как показано в коде ниже. Мы полагаем, что абсолютные крайние сроки хранятся в структурах задач блока управления задачами и что в *EDF_Queue* есть как минимум одна *готовая* задача, как в алгоритме RM.

```

/*****
Планировщик EDF
*****/
TASK EDF_Scheduler(){

    task_ptr_t task_pt;
    while(TRUE) {
        sem_wait(&(System_Tab.sched_sem));
        if(task_tab[current_tid].state!=DELAYED ||
           task_tab[current_tid].state!=BLOCKED)
            if(task_tab[current_pid].abs_deadline >
               (EDF_Queue.front)->abs_deadline) {
                task_pt=dequeue_task(&EDF_Queue);
                current_tid=task_pt->tid;
                insert_task(&EDF_Queue,&task_tab[System_Tab.preempted_tid]);
            }
        else
            current_tid=System_tab.preempted_tid;
        else {
            task_pt=dequeue_task(&EDF_queue);
            current_tid=task_pt->tid;
        }
        current_pt=&(task_tab[current_pid]);
        sem_post(&(task_tab[current_tid].sched_sem));
    }
}

```

7.9.3. Планировщик первой наименее занятой задачи

Мы полагаем, что задачи являются периодическими с жесткими сроками, они не имеют никаких отношений приоритета и не разделяют ресурсы, как и ранее

для этого алгоритма. Незанятость задачи – это интервал между ее крайним сроком и временем ее завершения. Планировщик первой наименее занятой задачи (*least laxity first*, LLF), *LLF_Scheduler*, также основан на приоритете. На этот раз, однако, нам нужно проверить незанятость задач во время выполнения в точках планирования. Мы будем полагать, что начальные незанятости задач рассчитываются и сохраняются при инициализации системы следующей процедурой: вычитается относительное значение крайнего срока из наихудшего случая относительно времени выполнения (*worst-case execution time*, WCET) задачи. Эти задачи *готовы* и вставлены в очередь *LLF_Queue* в соответствии с их временем незанятости.

```

/*****
Инициализация задач LLF
*****/
void Init_LLFC() {

    task_ptr_t task_pt;
    for (i=0; i<N_TASKS; i++)
        if( task_tab[i].type=PERIODIC) {
            task_tab[i].laxity=task_tab[i].rel_deadline-task_tab[i].wcet;
            task_pt=&(task_tab[i]);
            insert_task(&LLF_queue,task_pt);
        }
}

```

Активация *LLF_Scheduler* выполняется либо по завершении выполнения задачи, либо должен быть активирован новый экземпляр периодической задачи, когда начинается его период. Если текущая задача находится в отложенном или заблокированном состоянии, планировщик просто удаляет первую задачу из очереди *LLF_Queue*, в противном случае сравниваются значения незанятости текущей задачи со значением незанятости первой задачи в *LLF_Queue* и выбирается одна с меньшей незанятостью для запуска, как в коде ниже. Обратите внимание, что нам нужно добавить поле времени выполнения для блока управления задачами, чтобы можно было рассчитать его текущую незанятость онлайн. Расчет времени выполнения задачи можно обновлять на каждом такте, добавляя значение квантов времени к значению в блоке управления задачей, которая не реализована. Мы полагаем, что в очереди *LLF_Queue* существует хотя бы одна задача.

```

/*****
Планировщик LLF
*****/
void LLF_Scheduler() {

    task_ptr_t task_pt;
    while(TRUE) {
        sem_wait(&(System_Tab.sched_sem));
        if(task_tab[current_tid].state!=DELAYED) ||
            (task_tab[current_tid].state!=BLOCKED) {

```

```

task_tab[current_tid].laxity=
task_tab[current_tid].rel_deadline -
task_tab[current_tid].executed;
if(task_tab[current_pid].laxity >
(LLF_Queue.front->laxity) {
task_pt=dequeue_task(&LLF_queue);
current_tid=task_pt->tid;
insert_task(&LLF_Queue,&task_tab[System_Tab.preempted_tid]);
}
else
current_tid=System_Tab.preempted_tid;
}
else {
task_pt=dequeue_task(&LLF_queue);
current_tid=task_pt->tid;
}
current_pt=&(task_tab[current_pid]);
sem_post(&(task_tab[current_tid].sched_sem));
}
}

```

7.9.4. Сервер опроса

Сервер опроса (PS) – это высокоприоритетная системная задача, которая запланирована как остальные задачи реального времени. Когда запланирована, она проверяет очередь аperiodических задач *AT_Queue*, и если аperiodическая задача ожидает, она удаляется из очереди и подготавливается этим сервером.

```

/*****
Сервер опроса
*****/
TASK Polling_Server()
task_ptr_t task_pt;

while(TRUE){
delay_task(current_tid);
if(AT_Queue.front!=NULL)
{ task_pt=dequeue_task(&AT_Queue);
unblock_task(task_pt->tid, YES);
}
}
}

```

7.10. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. В чем разница между абсолютным и относительным конечными сроками задачи?
2. Каково использование периодической задачи реального времени?

3. Что подразумевается под приоритетным и неприоритетным планированием задач?
4. Что делает набор задач зависимым?
5. В чем разница между табличным планированием и циклическим исполнителем?
6. Как может быть достигнуто динамическое планирование статичных приоритетных задач?
7. Каков основной принцип планирования RM?
8. Каков основной принцип планирования EDF?
9. Можно ли использовать планирование EDF для аperiodических задач?
10. В чем разница между планированием EDF и планированием LLF?
11. Что такое время отклика задачи, и как оно зависит от других задач?
12. Как работает сервер опросов?
13. В чем разница между сервером опроса и сервером с задержкой?
14. Какова основная идея спорадического сервера?
15. Как работает сервер с постоянной пропускной способностью?

7.11. ПРИМЕЧАНИЯ К ГЛАВЕ

Мы рассмотрели основные алгоритмы планирования задач в одном процессоре для независимых задач, которые не разделяют ресурсы. Как мы видели, при разрешенном и неразрешенном замещении стратегии планирования разные. В циклической модели операционной системы автономное планирование периодических задач реального времени осуществляется с использованием наименьшего общего кратного их периодов, и во время работы задачи выполняются в соответствии с заранее подготовленной таблицей. Алгоритм планирования RM обычно используется для сложных периодических задач в реальном времени, которые являются независимыми и не разделяют ресурсы. Этот метод назначает фиксированные приоритеты для задач в автономном режиме на основе их периодов. Он прост в реализации, однако имеет строгую верхнюю границу использования процессора. Алгоритм EDF работает с динамическими приоритетами, и приоритет задачи отражает близость задачи к ее конечному сроку. Этот алгоритм всегда обеспечивает выполнение задачи с наивысшим приоритетом и лучшее использование процессора. Однако алгоритм EDF требует оценки динамических приоритетов задачи, основанных на их близости к конечным срокам, и, следовательно, требует при выполнении значительных затрат времени. По этой причине алгоритм RM чаще предпочитается алгоритму EDF. Алгоритм LLF в основном работает по принципу, аналогичному алгоритму EDF, но при планировании учитывает расчет времени выполнения задач. Как алгоритм EDF, он также требует дополнительных затрат, поскольку динамические приоритеты должны быть рассчитаны во время выполнения задач. На основе нашего анализа теперь можно сформировать подробную таксономию алгоритмов планирования независимых задач в одном процессоре, как показано на рис. 7.19.

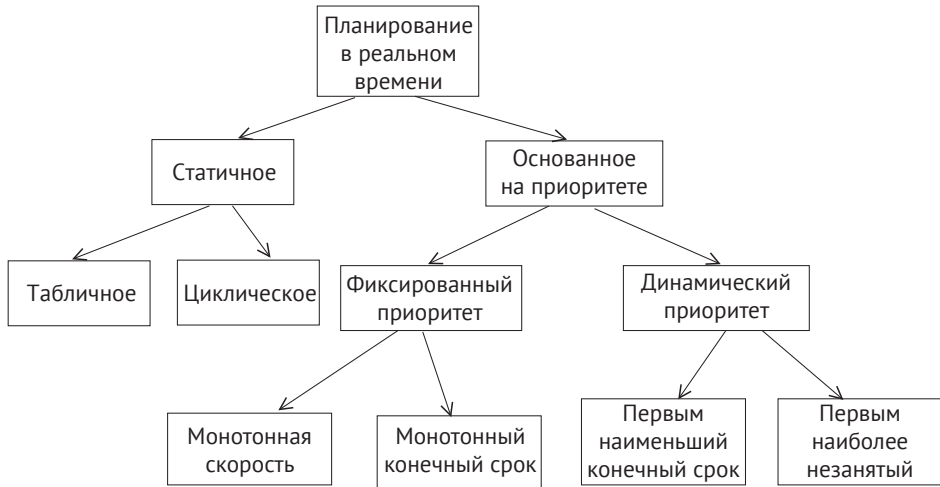


Рис. 7.19. Детальная классификация планирования однопроцессорных независимых задач

Методы для аperiодических и спорадических задач существенно отличаются. Аperiодические задачи могут быть запланированы с использованием различных типов серверов. Для систем с фиксированным приоритетом самым простым сервером с доступными затратами при predetermined планировании точек является сервер опросов. Сервер, допускающий задержку, также имеет фиксированные затраты, но допускает их периодическое пополнение. Простой подход к решению спорадических задач состоит в том, чтобы создать мнимую (*superficial*) задачу и, если спорадическая задача доступна в выделенное время, выполнить ее. В противном случае этот выделенный период времени может использоваться для периодических задач. Спорадический сервер обслуживает спорадические задачи и имеет фиксированные затраты, которые пополняются, если они были израсходованы. Системы с динамическим приоритетом имеют сервер постоянной пропускной способности и сервер с общей пропускной способностью, на которых конечные сроки не зависят от времени выполнения в первом случае и зависят во втором. На рис. 7.20 представлена классификация аperiодических серверных алгоритмов.

Наконец, мы дали пример набора алгоритмов планирования в DRTK: планировщик PM, планировщик LLF и сервер опроса с необходимой модификацией структур данных и процедур инициализации.

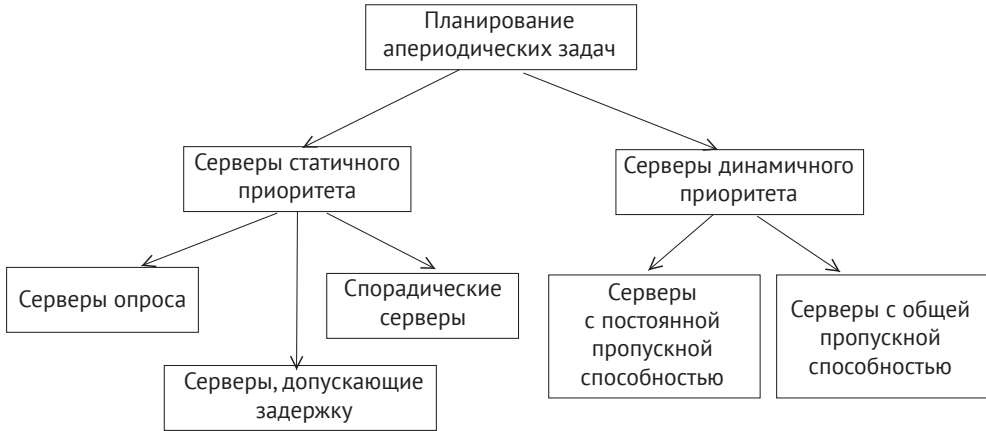


Рис. 7.20. Классификация аperiodических серверов

7.12. УПРАЖНЕНИЯ

- Дана периодическая независимая задача с набором задач $\tau_1(1,6)$, $\tau_2(3,8)$ и $\tau_3(2,12)$ ($\tau_i(C_i, T_i)$).
 - Составьте таблицу планирования для этого набора, используя планирование на основе таблиц, и нарисуйте диаграммы Ганта выполнения задач.
 - Найдите гиперпериод H и длину фрейма f , чтобы запустить для этих задач циклическую операционную систему Cyclic Executive, и нарисуйте диаграммы Ганта выполнения задач.
- Модифицируйте циклический исполнительный псевдокод Cyclic Executive алгоритма 7.2, с тем чтобы аperiodические задачи в очереди FCFS могли быть выполнены, если до следующего прерывания по таймеру есть незанятое время.
- Найдите допустимое расписание для задачи, поставленной с задачами $\tau_1(3,8)$, $\tau_2(2,12)$ и $\tau_3(4,36)$ ($\tau_i(C_i, T_i)$), используя алгоритм RM, предварительно выполнив тестовую задачу планируемости. Нарисуйте график планирования Ганта.
- Найдите допустимое расписание для набора задач с задачами $\tau_1(1,3,8)$, $\tau_2(2,3,6)$ и $\tau_3(5,4,12)$ ($\tau_i(a_i, C_i, d_i)$) с использованием алгоритма EDF, выполнив сначала тест планируемости. Нарисуйте график планируемости Ганта.
- Две задачи $\tau_1(3,6)$ и $\tau_2(3,8)$ ($\tau_i(C_i, T_i)$) планируются с использованием планирования RM. Нарисуйте график Ганта для этих задач. Найдите расписание аperiodической задачи $\tau_3(2,4,24)$ ($\tau_i(a_i, C_i, d_i)$) с использованием воровства незанятого времени в этой системе.

6. Разработайте и закодируйте аperiodический планировщик EDF, который может быть включен в код DRTK. Задачи независимы и не разделяют ресурсы. Покажите необходимые дополнения к структурам данных и коду инициализации.
7. Создайте сервер, допускающий задержку, для аperiodических задач DRTK, приняв за основу сервер опроса. Напишите код с краткими комментариями.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Buttazzo. (1993) Hard real-time computing systems: predictable scheduling algorithms and applications. Real-time systems series, 3rd edn. Springer.
2. Liu C. L. (2000) Real-time systems. Prentice Hall.
3. Liu C. L., Layland J. W. (1973) Scheduling algorithms for multiprogramming in a hard-real-time environment. J ACM 20 (1): 40–61.
4. Mullender S. (1993) Distributed systems, 2nd edn. Addison-Wesley.

Глава 8

Планирование однопроцессорной зависимой задачи

8.1. ВВЕДЕНИЕ

Мы рассмотрели однопроцессорные алгоритмы, предполагая, что задачи независимы и любое их взаимодействие отсутствует. Во многих реальных приложениях задачи обмениваются данными, и между задачами существуют отношения приоритета. Кроме того, задачи могут совместно использовать ресурсы, что ведет к взаимному исключению задач. Эти предшествующие отношения и обмен ресурсов значительно влияют на планирование решений.

В первой части данной главы мы рассмотрим основные алгоритмы для аperiodических задач с конечными сроками и с приоритетом отношений между ними, но не разделяющими ресурсы. Эти задачи удобно представить направленным ациклическим графом (directed acyclic graph, DAG). Алгоритмы соблюдения сроков выполнения для всех задач используют модифицированные формы основных алгоритмов, рассмотренных в предыдущей главе. Два представленных алгоритма – это первым самый последний крайний срок (the latest deadline first, LDF) и первым модифицированный самый ранний срок (the modified earliest deadline first, MEDF) алгоритмы.

Инверсия приоритета происходит, когда задача с более низким приоритетом захватывает ресурс и блокирует выполнение задачи с более высоким приоритетом. Во второй части главы мы пересматриваем протоколы, решая проблему инверсии приоритетов. Два проанализированных протокола – это протокол приоритета наследования (Priority Inheritance) и протокол приоритета верхней границы (потолка приоритетов) (Priority Ceiling). Наконец, мы реализуем алгоритмы LDF, MEDF и протоколы приоритетного наследования в DRTK.

8.2. ПЛАНИРОВАНИЕ ЗАВИСИМЫХ ЗАДАЧ

В нашей первой попытке мы будем предполагать в поисках решения задачи независимого планирования задач следующее: задачи не разделяют ресурсы, но между ними существуют приоритетные отношения. Эта проблема NP-hard (nondeterministic polynomial time hard) в общем случае трудна для решения, и в связи с этим мы рассмотрим два эвристических алгоритма LDF и MEDF.

8.2.1. Алгоритм первым последний конечный срок

Принцип алгоритма последнего конечного срока (LDF) заключается в том, чтобы отложить планирование задачи, которая имеет последний конечный срок, до тех пор, пока у нее нет преемников. Наличие одного или нескольких преемников задачи τ_i означает, что задержка задачи τ_i приводит к задержкам ее преемников и ее следует избегать. Следовательно, алгоритм ищет задачи, которые не имеют преемников, и выбирает среди таких задач задачу с последним сроком выполнения. Выбранная задача помещается в стек, и число преемников каждого из ее предшественников уменьшается. Продолжая подобным образом, мы помещаем все задачи на вершину стека, а затем каждая задача извлекается из стека и распределяется по расписанию, как показано в алгоритме 8.1. При работе этот алгоритм начинает выбирать одну из конечных вершин графа DAG. Обратите внимание, что задача с последним конечным сроком обрабатывается при планировании первой, но запланирована позже, чем невыбранные задачи.

Алгоритм 8.1. Алгоритм первым последний конечный срок

```

1: Вход: набор  $T = \{\tau_1, \dots, \tau_n\}$  of  $n$  аperiодических задач, набор его предшественников  $S = \{s_1, \dots, s_k\}$ 
2: Выход: Таблица планирования для задач
3:  $S$ : стек задач
4: for  $i=1$  to  $n$  do
5:   выбрать задачу  $\tau_x$ , которая не имеет последователей и имеет самый последний конечный срок
6:   вставить  $\tau_x$  в  $S$ 
7:   уменьшить число последователей каждого предшественника задачи  $\tau_x$ 
8: end for
9: for  $i=1$  to  $n$  do
10:  pop  $\tau_x$  из  $S$ 
11:  enqueue  $\tau_x$  в планируемую очередь
12: end for

```

Выбор задачи при отсутствии предшественника требует $O(n)$ операций в простом применении этого алгоритма, всего $O(n^2)$ операций. Набор зависимых задач, состоящий из семи задач $\tau_1, \dots, \tau_7$, при 0 абсолютных сроках времени прибытия показан на рис. 8.1. Помещение этих задач в стек и планирование их в обратном порядке приводят к диаграмме Гантта на рис. 8.2. Мы можем видеть, что для этого набора все задачи соответствуют их срокам без каких-либо замещений.

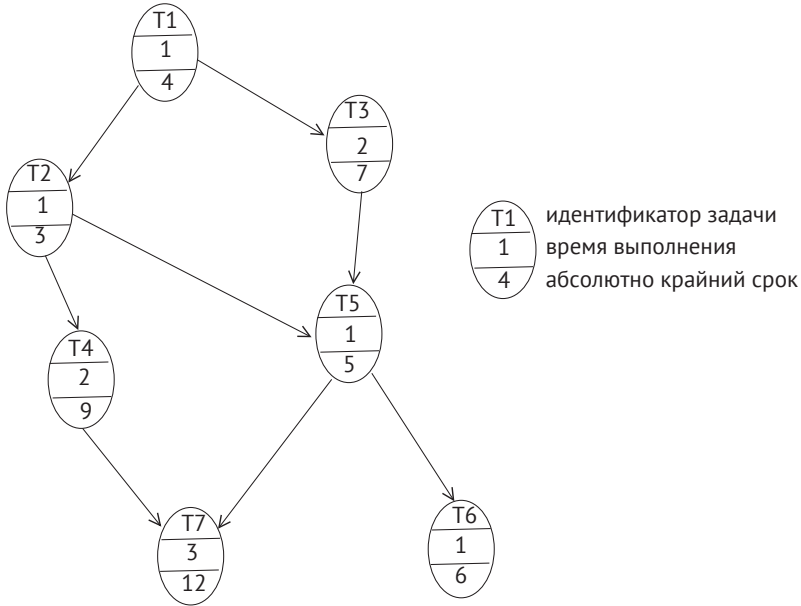


Рис. 8.1. Граф набора зависимых задач

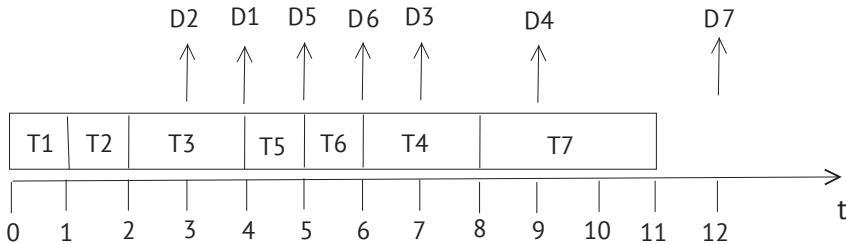


Рис. 8.2. LDF-планирование задачи рис. 8.1

8.2.2. Модифицированный алгоритм первым ранний конечный срок

Алгоритм EDF оказался оптимальным в случае одного процессора и независимых задач, и теперь мы попытаемся применить этот алгоритм для задачи планирования зависимых задач. Мы предполагаем, что задачи не разделяют ресурсы, но между ними есть приоритетные отношения. Нам нужно изменить время запуска и сроки выполнения задач в соответствии с приоритетами следующим образом, предполагая, что $\tau_i(a_i, C_i, D_i) < \tau_j(a_j, C_j, D_j)$:

- задача τ_j должна начать выполнение позже, чем время ее выполнения, и позже, чем максимальное время завершения задач ее предшественников, поэтому

$$a_i' = \max(a_i, a_i + C_i).$$

Если задача не имеет предшественников, ее эффективным временем выполнения и является это время ее выполнения, так как ей не нужно ждать, пока какая-либо другая задача начнет выполняться;

- задача τ_i должна завершиться прежде ее конечного срока, и

$$D_i' = \min(D_i, D_i' - C_i).$$

Если задача не имеет последователей, ее эффективное время конечного срока является ее конечным сроком, поскольку конечное время не влияет на другие задачи.

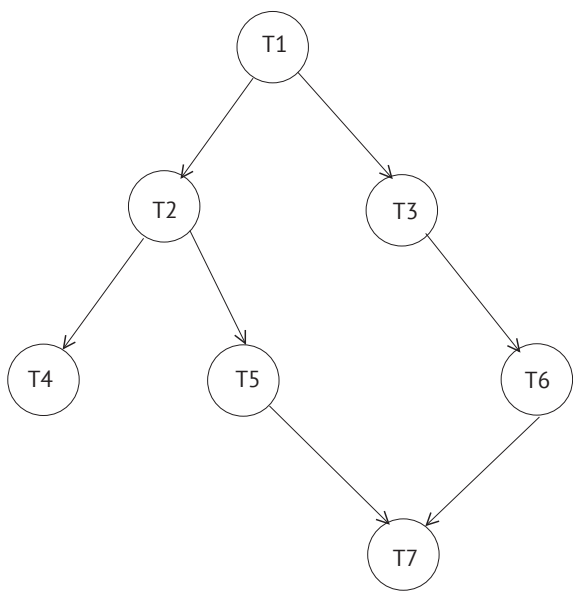


Рис. 8.3. Пример набора зависимых задач

Таблица 8.1. Пример набора задач

τ_i	C_i	D_i	a_i	D_i'
1	1	3	0	3
2	1	5	1	5
3	1	4	1	4
4	1	7	2	7
5	1	5	2	5
6	1	7	2	5
7	1	6	1	6

Теперь мы можем сформировать метод, чтобы найти расписание для зависимых задач. Сначала вычисляются эффективное время реализации и эффективные сроки выполнения задач на основе вышеуказанных правил, а затем алгоритм EDF применяется к задаче, заданной с этими новыми параметрами. Рассмотрим граф задач на рис. 8.3. Характеристики этих задач показаны в табл. 8.1: каждая задача имеет единичное время выполнения, и все они доступны в момент 0. Последние два столбца таблицы показывают измененное время прибытия и измененные сроки. Планирование MEDF этих задач изображено на рис. 8.4. Обратите внимание, что мы могли бы прервать выполнение задачи в момент времени t , если задача с более близким сроком в момент времени t станет готовой.

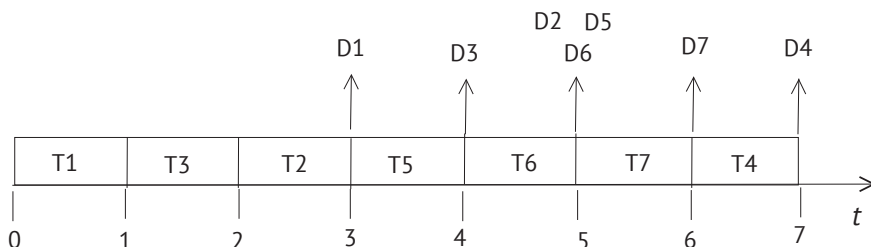


Рис. 8.4. MEDF-планирование задач из рис. 8.3

8.3. ПЛАНИРОВАНИЕ ЗАДАЧ ПРИ СОВМЕСТНОМ ИСПОЛЬЗОВАНИИ РЕСУРСОВ

В реальных приложениях задач реального времени совместно используют ресурсы: структуры данных, файлы, устройства ввода/вывода или основную память. Эти ресурсы должны быть защищены от одновременного доступа с помощью механизмов взаимного исключения, предоставляемых операционной системой. Сегмент кода, который управляет доступом задач к общему ресурсу, называется критическим разделом, и для обеспечения взаимного исключения обычно, как отмечалось в главе 4, используются семафоры. Как правило, задача, которая хочет начать выполнение своего важного раздела, устанавливает ожидание семафора, который защищает ресурс, а это может привести к его блокировке, если ресурс использует другая задача. Когда она завершает выполнение своего критического раздела, с которым обращалась к ресурсу, то посылает на семафор сигнал реализовать любые ожидающие задачи. Задачи, ожидающие один и тот же ресурс, стоят на семафоре, защищающем ресурс, в очереди.

Большинство методов планирования в реальном времени основаны на приоритете, то есть каждая задача имеет приоритет, и задача с наивысшим приоритетом планируется в любое время. Приоритет задачи может быть статическим, не изменяющимся со временем, как в планировании монотонного рейтинга приоритетов, или динамическим, который назначается примени-

тельно к конечным срокам выполнения задач, как в методе первого самого раннего срока. Рассмотрим случай с двумя задачами τ_1 и τ_2 , показанными на рис. 8.5, с более низким идентификатором задачи, означающим более высокий приоритет. Предположим, что τ_2 входит в критичный раздел доступа к ресурсу R в момент времени t_1 , а затем прерывается посредством задачи τ_1 в момент времени t_2 , поскольку τ_1 имеет более высокий приоритет. Задача τ_1 хочет получить доступ к R , и поскольку τ_2 занимает R , она блокируется операционной системой на семафоре, защищающем R в момент времени t_3 . Задача τ_1 должна ждать, пока τ_2 не закончит работу с ресурсом и не освободит его, сигнализируя семафору в момент времени t_4 , после чего τ_1 входит в свой критичный раздел и получает доступ к R . Она заканчивает выполнение критичного раздела в момент времени t_5 , и в момент t_6 задача τ_2 начинает выполнение своего некритичного кода.

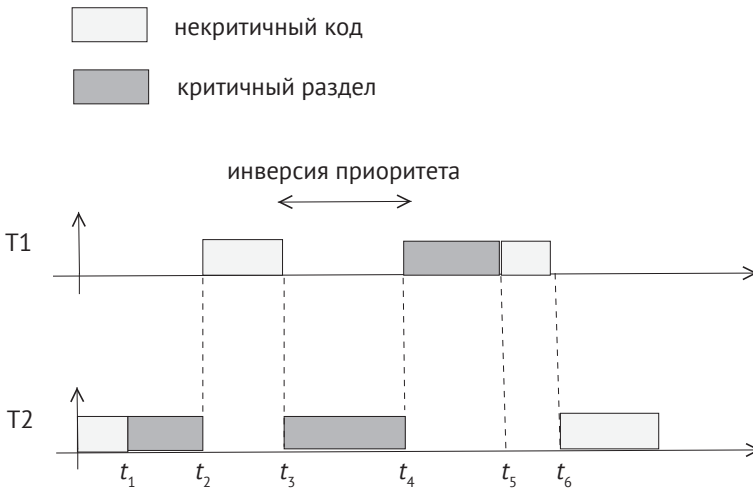


Рис. 8.5. Инверсия приоритетов для двух задач

Последовательность описанных событий привела к тому, что задача с более высоким приоритетом ожидает выполнения задачи с более низким приоритетом, что называется *инверсией приоритета* (*priority inversion*). Время ожидания τ_1 ограничено временем, которое требуется τ_2 , чтобы завершить критичный раздел, и отсюда этот тип инверсии приоритетов называется *инверсией с ограниченным приоритетом* (*bounded priority inversion*).

Более сложным случаем является случай, когда в системе есть три задачи τ_1 , τ_2 и τ_3 в порядке уменьшения приоритета, как показано на рис. 8.6.

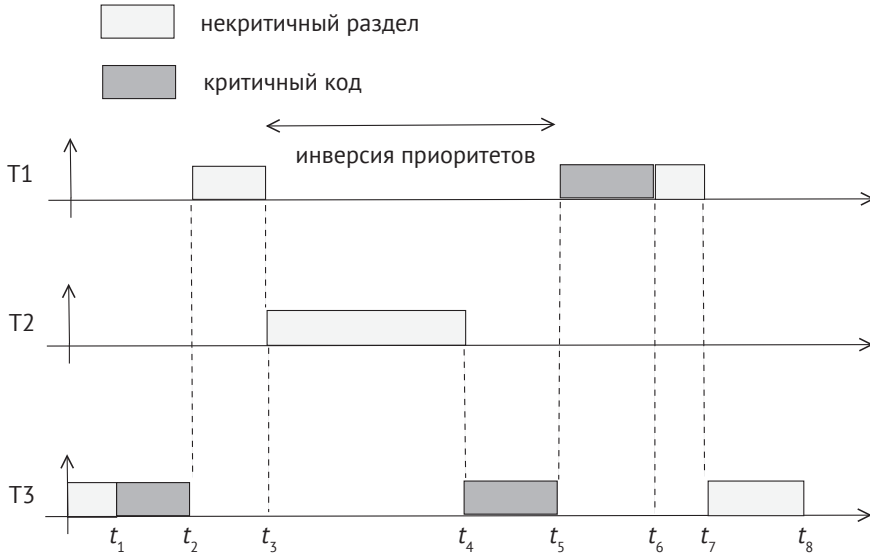


Рис. 8.6. Инверсия приоритетов для трех задач

На этом рисунке происходит следующая последовательность событий.

1. Задача τ_3 с наименьшим приоритетом выполняет ожидание семафора S и входит в критичный раздел для доступа к общему ресурсу R в момент времени t_1 .
2. Задача τ_3 вытесняется задачей τ_1 , поскольку $P(\tau_1) > P(\tau_3)$ в момент времени t_2 .
3. Задача τ_1 ожидает на S , так как τ_3 блокируется на этом семафоре в момент времени t_3 .
4. Задача со средним приоритетом τ_2 имеет более высокий приоритет, чем τ_3 , и, поскольку τ_1 заблокирована, запланирована к запуску, пока не завершится в момент времени t_4 .
5. Задача τ_3 является единственной выполняемой задачей и, следовательно, выполняется до тех пор, пока не завершит свой критичный раздел в момент времени t_5 .
6. Задача τ_1 сигнализируется теперь задачей τ_3 на семафоре S и запускается, выполняя свой критичный раздел до момента времени t_6 , а затем выполняет свой некритичный код до момента времени t_7 и завершает работу.
7. Задача τ_3 снова запланирована, поскольку она является единственной оставшейся в системе задачей и выполняется до завершения момента времени t_8 .

В отличие от предыдущего случая, задача τ_2 может выполняться в течение длительного времени, заставляя задачу τ_1 с высоким приоритетом пропустить свой срок. Эта ситуация называется *неограниченным ожиданием* (*unbounded waiting*) инверсии приоритетов. Возможное решение данной проблемы – отключить приоритет, однако, когда задача находится в критичном разделе, многим несвязанным задачам, которые не разделяют заблокированный ресурс, придется ждать, что может привести к несоблюдению сроков. Два эффективных решения этой проблемы – протокол наследования приоритетов и протокол приоритетного потолка. Сначала рассмотрим случай марсианского зонда (Mars Pathfinder).

8.3.1. Случай марсианского зонда

Марсианский зонд (Mars Pathfinder) был спроектирован и построен Лабораторией реактивного движения (Jet Propulsion Laboratory, JPL) Калифорнийского института технологий для посадки на Марсе и сбора научных данных о Марсе [1]. Зонд был оснащен операционной системой реального времени VxWorks, которая обеспечивает приоритетное планирование с фиксированным приоритетом в форме циклического планировщика. Космический корабль начал переустанавливаться вскоре после того, как совершил посадку на Марсе 4 июля 1997 года, что привело к задержке сбора данных, хотя данные не были потеряны. Зонд решал три основные периодические задачи:

- τ_1 : поток информационной шины; выступает в роли диспетчера шин, имеет высокую частоту, высокий приоритет;
- τ_2 : коммуникационный поток; имеет среднюю частоту и средний приоритет с длительным временем выполнения;
- τ_3 : поток погоды; собирает геологические данные, имеет низкую частоту и низкий приоритет.

Каждая задача проверяла, выполнялись ли другие в предыдущем цикле, и если тест не пройден, система сбрасывалась сторожевым таймером. Общая память была использована для взаимодействия задач. Данные передавались от задачи τ_3 к задаче τ_2 через задачу τ_1 , и доступ к общей памяти был защищен мьютексом (объект с функцией взаимного исключения, *mutex*). Когда зонд стал перезагружать систему, инженеры JPL стали работать над решением этой проблемы. У JPL на Земле была копия системы зонда, и проблема была воспроизведена в лаборатории в результате выполнения множества тестов. Конструкторы выяснили, что причиной проблемы являлась инверсия приоритетов. Высокий приоритет τ_1 был заблокирован, потому что τ_3 содержал мьютекс, который был необходим τ_1 , и τ_2 с длительным временем вытеснения τ_3 , что является ситуацией, показанной на рис. 8.6.

Объекты мьютекс системы VxWorks имеют необязательный флаг приоритетного наследования, и инженеры были в состоянии загрузить патч, установив этот флаг на информационной шине мьютекса. Бортовое программное обеспечение было модифицировано с земли, и параметры системы были изменены,

с тем чтобы включить приоритетное наследование. После этого обновления перезагрузки системы больше не происходило.

8.3.2. Основной протокол наследования приоритетов

Протокол для решения задачи инверсии приоритетов был предложен в [2]. Мы будем предполагать, что задачи обращаются к ресурсам через ожидание и сигнализируют о вызовах на семафорах. Главная идея этого протокола, который мы назовем основным протоколом наследования приоритета (*Basic Priority Inheritance Protocol*, BPIP), заключается в том, чтобы задача с более низким приоритетом наследовала приоритет задачи с более высоким приоритетом, удерживая ресурс. Мы также предполагаем, что задачи имеют приоритеты и планируются на основе их приоритетов, то есть задача с наивысшим приоритетом выполняется в любое время. Ниже показана возможная последовательность событий в этом протоколе.

- Задача τ_i выдает вызов ожидания на семафор S , связанный с ресурсом R , и блокируется, поскольку R удерживается задачей с более низким приоритетом τ_j .
- Когда это происходит, задача τ_i переносит свой приоритет P_i в τ_j , поэтому $P_j = P_i$.
- Когда задача τ_j завершает выполнение своего критичного раздела, она передает свой текущий приоритет задаче τ_i , следовательно, $P_i = P_j$, и она восстанавливает свой первоначальный приоритет.

Выполнение задач на рис. 8.6 теперь происходит так, как показано на рис. 8.7, с использованием протокола приоритета наследования. Мы можем видеть, что задача τ_3 наследует приоритет задачи τ_1 между моментами t_3 и t_4 и может закончить свой критичный раздел. Задача τ_1 возвращает свой приоритет в момент времени t_4 и может войти в свой критический раздел. Обратите внимание, что выполнение задачи с промежуточным приоритетом τ_2 теперь задерживается до момента t_5 и только в момент времени t_6 , когда τ_2 заканчивает выполнение своего некритичного кода, τ_3 может начать выполнение своего некритичного кода. Приоритетное наследование является переходным, то есть если задача τ_i заблокирована задачей τ_j , которая заблокирована задачей τ_k с $P_i > P_j > P_k$, то τ_k наследует приоритет от τ_i через τ_j . Анализ BPIP показывает две основные проблемы: длительные задержки и тупики, что рассматриваются ниже.

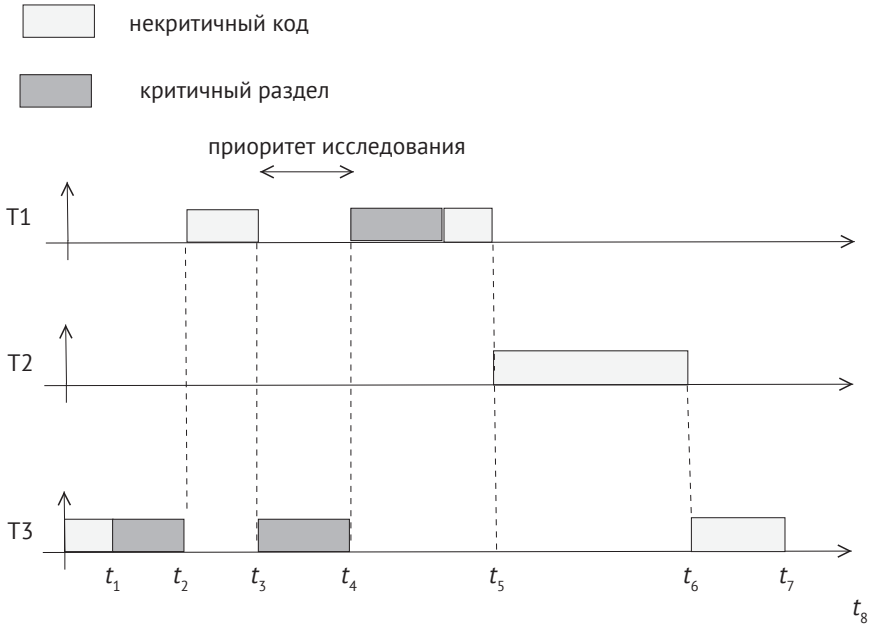


Рис. 8.7. Пример приоритета наследования

Длительные задержки

Предположим, что n задач $\tau_1, \tau_2, \dots, \tau_n$ расположены в порядке убывания приоритетов. Рассмотрим случай, в котором τ_n начинает свой критичный раздел после ожидания на семафоре S , блокируется и передает свой приоритет задаче τ_{n-1} , после чего выполняется запланированная задача τ_{n-1} , ожидающая на S . Предположим далее, что эта ситуация продолжается до задачи τ_1 , приоритет которой через все другие задачи передается задаче τ_n . Задача с самым высоким приоритетом τ_1 должна ждать выполнения n критичных разделов, которые могут быть достаточно длинными для большого значения n , чтобы заставить τ_1 пропустить свой срок. Эта ситуация называется цепной блокировкой (*chain blocking*), как показано на рис. 8.8.

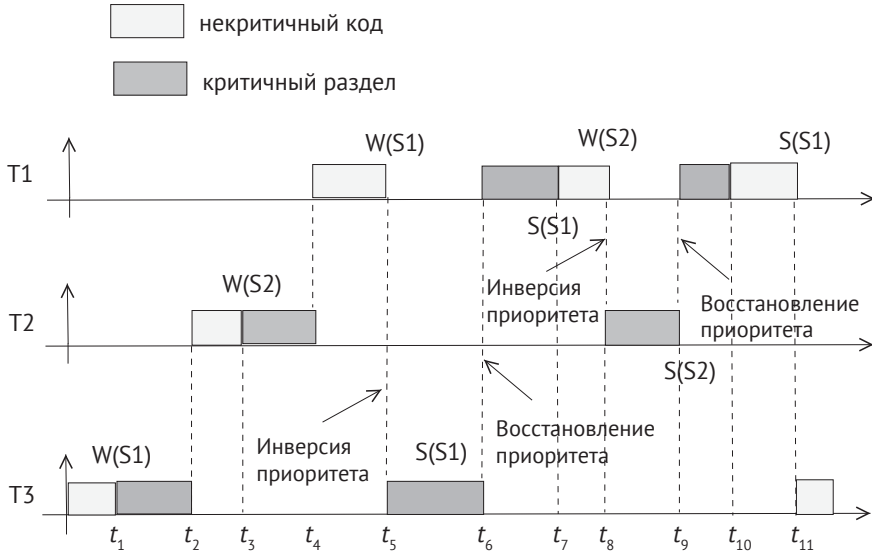


Рис. 8.8. Пример цепной блокировки

В системе есть два семафора S_1 и S_2 , и задачи τ_1 , τ_2 и τ_3 имеют приоритеты, обратно пропорциональные их индексам. Ниже приведена последовательность событий на этом рисунке:

- 1) t_1 : задача τ_3 , которая является задачей с самым низким приоритетом, выполняет *ожидание* на S_1 , блокирует его и начинает выполнять свой критичный раздел;
- 2) t_2 : задача τ_2 выгружает τ_3 , поскольку она имеет более высокий приоритет, и начинает свой некритичный раздел;
- 3) t_3 : задача τ_2 выполняет *ожидание* на S_1 , блокирует его и начинает выполнение своего критичного раздела;
- 4) t_4 : задача τ_1 выгружает τ_2 , так как она имеет более высокий приоритет, и начинает свой некритичный раздел;
- 5) t_5 : задача τ_1 вызывает *ожидание* на S_1 , и поскольку этот семафор заблокирован τ_1 , она переносит свой приоритет на τ_3 , которая продолжает выполнять свой критичный раздел;
- 6) t_5 : задача τ_3 завершает выполнение своего критичного раздела, сигнализирует S_1 и восстанавливает свой приоритет и приоритет τ_1 ;
- 7) t_6 : задача τ_1 теперь может войти в свой критичный раздел, защищенный семафором S_1 ;
- 8) t_7 : задача τ_1 выходит из своего критичного раздела, сигнализирует S_1 и начинает свой некритичный раздел;
- 9) t_8 : задача τ_1 выполняет *ожидание* семафора S_2 , и поскольку S_2 заблокирован задачей τ_2 , она передает свой приоритет задаче τ_2 , которая начинает выполнение своего критичного раздела;

- 10) t_8 : задача τ_1 выполняет ожидание на семафоре S_2 , и поскольку S_2 заблокирован задачей τ_2 , она передает ее приоритет τ_2 , которая начинает выполнение своего критичного раздела;
- 11) t_9 : задача τ_1 запланирована, поскольку выполняет ожидание на семафоре S_2 , и так как S_2 является заблокированным задачей τ_2 , он передает свой приоритет τ_2 , который начинает выполнение своего критичного раздела;
- 12) t_{10} : задача τ_1 завершает свой критичный раздел, защищенный S_2 , и сигнализирует S_2 . Затем она продолжает выполнение своего некритичного раздела;
- 13) t_{11} : задача τ_1 завершает свой некритичный раздел, и в это время задача τ_3 начинает свой некритичный раздел, а затем завершает его.

Этот пример демонстрирует, что задаче с самым высоким из трех задач приоритетом, τ_1 , пришлось ждать τ_3 на семафоре S_1 и τ_2 на семафоре S_2 . В общем, цепочка блокировок может привести к тому, что задача τ_i ожидает время блокировки B_i :

$$B_i = \sum_{k=1}^M \text{block}(k, i) C(k), \quad (8.1)$$

где M – количество критичных разделов задач с более низким приоритетом, блок (k, i) , $C(k)$ – это время выполнения критичного раздела k в наихудшем случае. Использование процессора, включающее время блокировки для планирования RM, может быть модифицировано при рассмотрении времени блокировки задач следующим образом:

$$\sum_{i=1}^n \frac{C_i}{T_i} + \frac{B_i}{T_i} = U_i + \frac{B_i}{T_i} \leq n(2^{1/n} - 1), \quad (8.2)$$

где B_i – самое большое время инверсии приоритета, которое может ждать τ_i . На самом деле $B_i = CS_j + CS_1 + \dots + CS_k$, где значения CS являются критичными разделами задач с более низким приоритетом, что может заблокировать τ_i . Время ответа задачи τ_i зависит от времени блокировки B_i следующим образом:

$$R_i = C_i + B_i + \sum_{j=1}^{i-1} \left\lceil \frac{R_j}{T_j} \right\rceil C_j \leq D_i.$$

Вероятность тупика

Рассмотрим случай с двумя задачами τ_1 и τ_2 , которые выдают ожидающие и сигнальные вызовы двум семафорам S_1 и S_2 следующим образом:

τ_1 : {ожидание (S_1), ..., ожидание (S_2), сигнал (S_2), сигнал (S_1)};

τ_2 : {ожидание (S_2), ..., ожидание (S_1), сигнал (S_1), сигнал (S_2)}.

Этот сценарий может привести к тупику, изображенному на рис. 8.9. Последовательность событий на этом рисунке следующая:

- 1) t_1 : задача τ_2 выполняет ожидание на S_2 , блокирует его и начинает выполнять свой критичный раздел;
- 2) t_2 : задача S_2 выгружает τ_2 , поскольку она имеет более высокий приоритет, и начинает свой некритичный раздел;
- 3) t_3 : задача τ_1 выполняет ожидание на S_1 , блокирует его и начинает выполнять свой критичный раздел;
- 4) t_4 : задача τ_1 вызывает ожидание семафора S_2 внутри своего критичного раздела, защищенного S_1 , и так как τ_2 держит S_2 , она передает свой приоритет τ_2 , который продолжает выполнять свой критичный раздел, защищенный S_2 ;
- 5) t_5 : задача τ_2 вызывает ожидание на семафоре S_1 внутри его критичного раздела, защищенного S_2 , и так как τ_2 держит S_1 , он не может продолжаться.

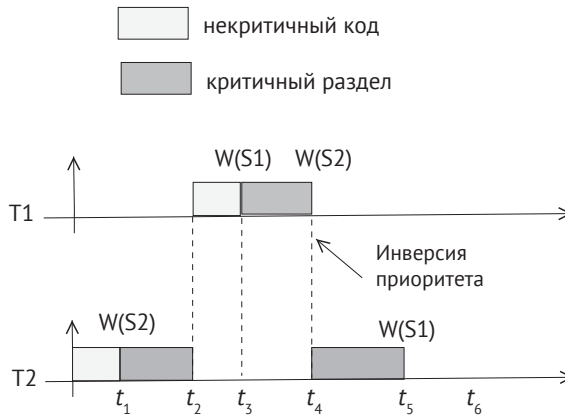


Рис. 8.9. Пример тупика, вызванного BPIP

В этой ситуации τ_1 удерживается S_2 , и для продолжения необходимо разблокировать S_1 , а τ_2 удерживается S_1 , и для продолжения необходимо разблокировать S_2 , что является тупиковым условием.

8.3.3. Протокол приоритетного потолка

Приоритетные потолочные протоколы решают все проблемы BPIP. Основная идея этих протоколов заключается в предвидении блокировки задачи. Если задача потенциально может заблокировать задачу с более высоким приоритетом, закрывая семафор, то это не разрешается. Эти протоколы могут классифицироваться как первоначальный приоритетный протокол потолка (Original Priority Ceiling Protocol, OPCR) и непосредственный приоритетный протокол потолка (Immediate Priority Ceiling Protocol, IPCP). Эти протоколы рассмотрены в следующих разделах. Протокол OPCR решает две проблемы, связанные с BPIP. Основная идея данного протокола заключается в ассоциации значения потолка для каждого семафора в системе и наибольшего приоритета задач, которые используют семафор. Формально

$$\text{ceil}(S_i) = \max \{\text{Prio}_i \mid \tau_i, \text{использующей } S_k\}.$$

Это значение является статичным и может быть вычислено в автономном режиме. Работа ОРСР основана на следующих правилах:

- потолок семафора $\text{ceil}(S)$ является приоритетом задачи с наивысшим приоритетом, которая использует семафор во время выполнения;
- задача τ_i может только блокировать (выполнять ожидание, не будучи блокированной) семафор, если $\text{Prio}(i)$ строго выше потолков всех семафоров, в настоящее время заблокированных другими задачами;
- в противном случае τ_i блокируется на S , и задача, которую в данный момент содержит S , наследует приоритет задачи τ_i .

Далее подробно иллюстрируется работа ОРСР:

- пусть S_m будет семафором, который имеет наибольшее значение потолка среди всех семафоров в системе;
- **If** $\text{Prio}(\tau_i) > \text{ceil}(S_m)$, **then** τ_i входит в свой критичный раздел;
- **Else** τ_i передает свой приоритет задаче, которая в настоящее время выполняется, как в BPIP.

Пример ОРСР с тремя задачами τ_1 , τ_2 и τ_3 в порядке убывания приоритетов, использующих два семафора S_1 и S_2 , показан на рис. 8.10. Значения потолков семафоров: $\text{ceil}(S_1) = \text{Prio}(\tau_1)$ и $\text{ceil}(S_2) = \text{Prio}(\tau_1)$.

Ниже приведена последовательность событий в этом примере:

- 1) t_1 : задача τ_3 , которая является задачей с самым низким приоритетом, выполняет ожидание на S_1 , блокирует его и начинает выполнять свой критичный раздел;
- 2) t_2 : задача τ_2 выгружает τ_3 , поскольку она имеет более высокий приоритет, и начинает свой некритичный раздел;
- 3) t_3 : задача τ_2 выполняет ожидание на S_2 и блокируется, так как $\text{Prio}_2 < \text{ceil}(S_2)$, и задача τ_3 запланирована, поскольку нет других готовых задач, и продолжает выполнение своего критичного раздела;
- 4) t_4 : задача τ_1 выгружает τ_3 и запускает свой некритичный раздел;
- 5) t_5 : задача τ_1 вызывает ожидание на S_1 , и поскольку $\text{Prio}_1 < \text{ceil}(S_2)$, она блокируется, и запускается τ_3 , которая выполняет оставшуюся часть своего критичного раздела;
- 6) t_6 : задача τ_3 заканчивает свой критичный раздел, и теперь запланирована τ_1 , которая может начать критичный раздел, связанный с семафором S_2 ;
- 7) t_7 : задача τ_1 заканчивает свой критичный раздел, сигнализирует семафору S_2 и начинает выполнение своего некритичного раздела;
- 8) t_8 : задача τ_1 выполняет ожидание семафора S_2 , и поскольку S_2 заблокирован задачей τ_2 , он передает свой приоритет τ_2 , которая начинает выполнение своего критичного раздела;
- 9) t_9 : задача τ_1 запланирована как выполняющая ожидание семафора S_2 , и поскольку S_2 заблокирована задачей τ_2 , она передает свой приоритет τ_2 , который начинает выполнение своего критичного раздела;

- 10) t_{10} : задача τ_1 завершает свой критичный раздел, защищенный S_2 , и сигнализирует S_2 . Затем она продолжает выполнение своего некритичного раздела;
- 11) t_{11} : задача τ_1 завершает свой некритический раздел, и в это время задача τ_3 начинает свой некритический раздел, а затем завершает его.

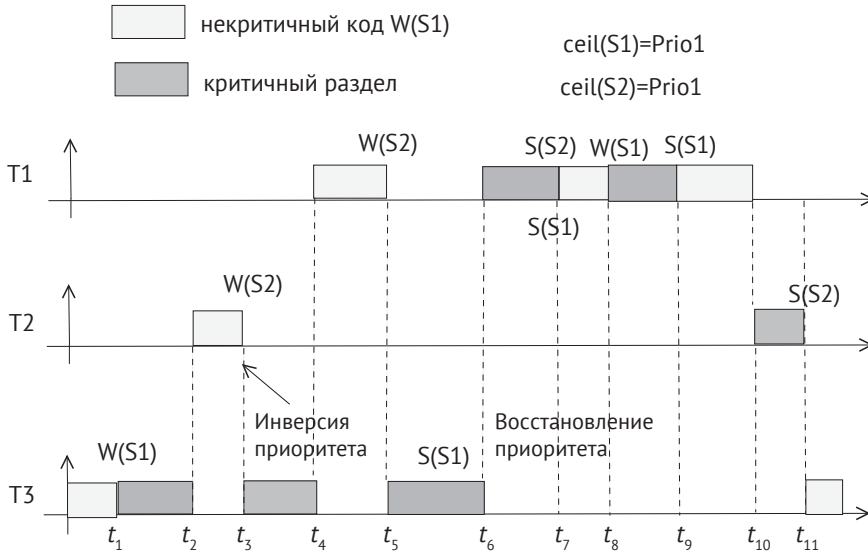


Рис. 8.10. Пример цепочной блокировки

Анализ

Приоритет ОРСР свободен от тупиков, и задача τ_i может быть заблокирована на время не более самого продолжительного критичного раздела. Использование системы по схеме РМ такое же, как по протоколу ВРІР, но время блокировки задачи τ_i равно $B_i = \max \{CS_j, CS_{j+1}, \dots, CS_k\}$. Обратите внимание, что с ВРІР B_i была суммой всех времен блокировки, которое может наступить для задачи из-за инверсии приоритета задач с низким приоритетом. В итоге ОРСР предотвращает появление тупиков в связи с вложенностью критичных разделов, и задача в этом протоколе может быть заблокирована любой другой задачей не более одного раза.

8.4. РЕАЛИЗАЦИЯ DRTK

Сначала мы реализуем алгоритм LDF, который будет использоваться с DRTK. Для реализации ВРІР и ОРСР в DRTK нам нужно модифицировать ожидание семафора и сигнальные процедуры, чтобы разрешить приоритетные передачи.

8.4.1. Зависимое планирование задач LDF

Алгоритм LDF реализуется с использованием структуры стека «последним пришел – первым вышел» (LIFO). Приведенная ниже функция вводит *очередь* задач, представляющую граф DAG со всеми задачами, имеющий в блоке управления задачи всех предшественников и последователей. Эта очередь обходится до тех пор, пока не будет найдено задание без преемников и в случае более одной задачи, а такие задачи существуют, одна из них с последним конечным сроком выбирается и помещается в стек (*stack_queue*). Временной указатель блока управления задачами используется на каждой итерации для хранения адреса задачи, которая не имеет преемников и имеет максимальное значение конечного срока. После помещения задачи в стек значения всех преемников задач-предшественников считаются уменьшенными. Этот процесс продолжается до тех пор, пока все задачи не будут помещены в стек, после чего задачи поочередно извлекаются из стека и ставятся в очередь в планировщике вывода (*sched_queue*).

```
/******
```

```
Алгоритм зависимой задачи LDF
```

```
*****/
```

```
void LDF( task_queue_t input_task_que ){
```

```
    task_queue_t sched_queue, stack_queue;
    task_ptr_t task_pt, temp_pt;
    task_control_block_t task_temp;
```

```
    task_pt=input_task_que.front;
    temp_pt=task_pt;
    n_tasks=input_task_que.n_tasks;
```

```
    for(i=1;i<=input_task_que.n_tasks;i++){
        while(task_pt->next != NULL){
            if (task_pt->n_successors==0 &&
                task_pt->deadline>temp_pt.deadline)
                temp_pt=task_pt;
            task_pt=task_pt->next;
        }
        push_stack(stack_queue,task_pt);
        for(j=1;j<=task_pt->n_predecessors;j++){
            task_id=task_pt->predecessors[i];
            temp_pt=&(task_tab[task_id]);
            temp_pt->n_successors--;
        }
    }
```

```
    for(i=1;i<=input_taskque_pt->n_tasks;i++){
        task_pt=pop_stack(stack_queue);
        enqueue(sched_queue,task_pt);
    }
    return(DONE);
}
```

8.4.2. Протокол приоритетного наследования

Для реализации BPIP задаче разрешено иметь два приоритета, *номинальный приоритет* (*nominal priority*) и *активный приоритет* (*active priority*). Номинальный приоритет – это поле, зарезервированное в блоке управления задачами как приоритет задачи. Активный приоритет является при использовании BPIP текущим приоритетом задачи. Это новое поле *active_priority* добавляется в структуру управления задачами данных. Мы модифицируем системные вызовы DRTK *ожидание* и *сигнализация*, чтобы включить BPIP-операцию. Вызов ожидания семафора в DRTK уменьшает значение семафора, и если это значение отрицательно, то есть уже другая задача получила семафор, он блокируется в очереди семафоров, и вызывается планировщик. Сначала мы изменим семафорную структуру DRTS путем добавления *держателя* поля, который показывает задачу, удерживающую в настоящее время семафор, как показано ниже. Это поле необходимо для сравнения приоритетов задачи, вызывающей ожидание, и задачи, которая находится в ее критичном разделе. Другое новое поле – это *transfer_id* для хранения идентификатора задачи, которая передает свой приоритет.

```

/*****
Структура BPIP семафора данных
*****/
typedef struct{
    int state;
    int value;
    int holder_id; // new field
    int transfer_id; // new field
    task_queue_t task_queue;
} semaphore_t; semaphore_t *semaphore_ptr_t;

semaphore_t semaphore_tab_t[N_SEM];

```

Новый системный вызов для ожидания на семафоре – *pip_wait_sema* – показан ниже. Эта процедура проверяет, содержит ли какая-либо задача семафор и имеет ли более низкий приоритет, чем вызывающий абонент, передает приоритет вызывающего абонента и осуществляет перепланировку, чтобы позволить держателю завершить выполнение своего критичного раздела. Идентификатор *mtransfer_id* обновляется в семафоре и используется в сигнале вызова для восстановления приоритетов. Планировщик может уже планировать ожидающую своей очереди задачу с нижним приоритетом, поскольку теперь ее приоритет повышен.

```

/*****
Ожидание семафора при использовании BPIP
*****/
int pip_wait_sema(ushort sem_id){

    semaphore_ptr_t sem_pt;
    task_ptr_t holder_pt;
    ushort holder,

```

```

if (sem_id < 0 || sem_id >= system_tab.N_SEM)
return(ERR_RANGE)
sem_pt=&(semaphore_tab[sem_id]);
sem_pt->value--;
if (sem_pt->value < 0) {
if (sem_pt->transfer_id==0) { // new lines start
holder=sem_pt->holder_id;
holder_pt=&task_tab[holder];
if (holder_pt->priority < current_pt->priority) {
temp=holder_pt->priority;
holder_pt->priority=current_pt->priority;
current_pt->priority=temp;
sem_pt->transfer_id=current_tid;
Schedule();
} // новые строки закончены
insert_task(sem_pt->task_queue, current_pt);
block(current_tid);
}
}
sem_pt->holder_id=current_tid;
}

```

Сигнализация семафора в DRTK осуществляется с помощью показанной ниже *pip_signal_sema*, которая увеличивает счетчик семафора, и если на семафоре есть задачи, ожидающие очереди, он восстанавливает первоначальный приоритет, заменяя его приоритетом задачи, который хранится в *transfer_id* семафора. На этот раз нам нужно где-то взять задачу из очереди семафора, так как, возможно, были другие задачи, которые установили *ожидание* на этом семафоре между операциями *ожидания* и *сигнализации*. Если поле *transfer_id* не заполнено, выполняется нормальная *сигнальная* операция.

```

/*****
Сигнал светофора
*****/
int pip_signal_sema(ushort sem_id){

    semaphore_ptr_t sem_pt;
    task_ptr_t task_pt;

    if (sem_id < 0 || sem_id >= system_tab.N_SEM)
return(ERR_RANGE)
sem_pt=&(semaphore_tab[sem_id]);
sem_pt->value++;
if (sem_pt->value <= 0) {
if(sem_pt->transfer_id!=0) { // начинаются новые строки
task_pt=&(task_tab[sem_pt->transfer_id]);
temp=current_pt->priority;
current_pt->priority=task_pt->priority;
task_pt->priority=temp;
take_task(sem_pt->task_queue, task_pt);

```

```

unlock(task_pt->id, YES); // новые строки заканчиваются
}
else {
    task_pt=dequeue_task(sem_pt->task_queue);
    unlock_task(task_pt->task_id, YES);
}
}
}
}

```

8.5. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какова основная идея алгоритма LDF, используемого для планирования зависимых задач?
2. В чем главное отличие алгоритмов EDF для независимых и зависимых задач?
3. Опишите проблему инверсии приоритетов.
4. В чем состояла проблема в проекте марсианского зонда?
5. Какова основная идея BPIP?
6. Как может возникнуть тупик в BPIP?
7. Каковы основные проблемы BPIP?
8. Какова основная идея OPCP?
9. Как решаются основные проблемы BPIP в OPCP?

8.6. ПРИМЕЧАНИЯ К ГЛАВЕ

В этой главе мы рассмотрели основные методы планирования зависимых задач. Мы рассмотрели два разных случая: аperiodические задачи реального времени с конечными сроками и отношениями приоритетов и аperiodические задачи, которые разделяют ресурсы. Мы описали алгоритмы для первого случая, который может быть использован для периодических задач, а также когда сроки рассматриваются как периоды задач. Входными данными для этих алгоритмов является граф DAG, показывающий приоритет задач. Алгоритм LDF начинается с одной из конечных задач графа DAG и помещает эту задачу в стек. Любая задача, которая не имеет преемника с последним сроком выполнения таких задач, помещается в стек до тех пор, пока все задачи не будут обработаны. Порядок выполнения – это порядок задач, извлеченных из стека. Алгоритм MEDF начинает работать в обратном направлении, выбирая задачу, у которой нет предшественников и которая имеет самый ранний срок среди тех задач, которые должны быть запланированы первыми. Это продолжается до тех пор, пока все задачи не будут поставлены в очередь в FCFS-очереди и порядок планирования не будет определяться положением задач в этой очереди. Оба алгоритма требуют временных затрат $O(n^2)$.

Затем мы проанализировали проблему инверсии приоритетов, когда задачи в реальном времени делятся ресурсами. Эта проблема фактически возникла в проекте марсианского зонда Pathfinder в 1985 году. Основными протоколами

для устранения данной проблемы являются наследование приоритетов и потолок приоритетов. Протоколы BPIP обеспечивают передачу задачи с высоким приоритетом задаче с более низким приоритетом, которая удерживает ресурс, и, следовательно, задача, использующая ресурс, может быть завершена. Этот протокол имеет несколько недостатков: высокоприоритетная задача может ждать выполнения многих критичных разделов задач с более низкими приоритетами, а это может вызвать взаимные блокировки. Для обеспечения бесперебойной работы могут использоваться протоколы приоритетного потолка, в которых задача с высоким приоритетом может ожидать не более одного критичного раздела, выполняемого задачей с более низким приоритетом. Мы показали реализацию алгоритмов LDF, MEDF, BPIP и OPCP в DRTK. Протоколы BPIP и OPCP требуют добавления новых полей в структуре семафоров, модификации системных вызовов сигналов и ожидания для семафоров. Сравнение выполнения задач без приоритетного замещения, BPIP и OPCP, приведено в табл. 8.2, где показано количество блокировок и состояний тупика.

Таблица 8.2. Сравнение протоколов

Протокол	Ограниченный PI	Блокируется в большинстве случаев	Отсутствие тупиков
Неприоритетный	✓	✓ _a	✓ _a
BPIP	✓	–	–
OPCP	✓	✓	✓

^a Если только не блокированы в критичном разделе.

8.7. УПРАЖНЕНИЯ

1. Пример аperiodической зависимой задачи с абсолютными сроками приведен в табл. 8.3. Запустите алгоритм LDF из этого набора и проверьте, все ли задачи соответствуют их конечным срокам с этим алгоритмом.

Таблица 8.3. Пример набор задач

τ_i	C_i	d_i	Предшественники
1	2	7	–
2	1	5	1
3	3	8	1
4	1	4	2
5	1	12	2, 4
6	2	10	1, 4

2. Запустите алгоритм MED для набора зависимых задач, показанного на рис. 8.1, и проверьте с помощью этого алгоритма, все ли задачи соответствуют их конечным срокам.

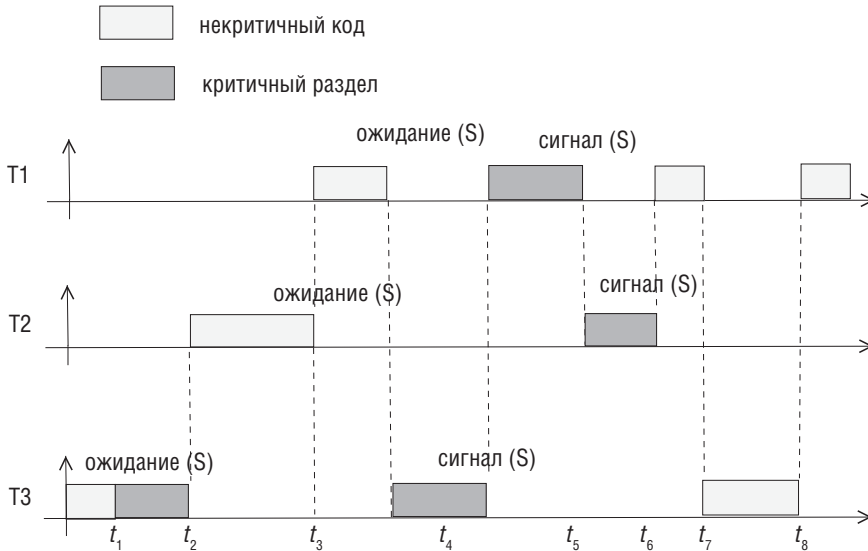


Рис. 8.11. Выполнение задачи для упражнения 3

3. Набор задач $T = \{\tau_1, \tau_2, \tau_3\}$ имеет следующие характеристики выполнения при увеличении времени $t_1, t_2, \dots, t_8$, показан на рис. 8.11 и использует ресурс, защищенный семафором S_1 . Опишите события в каждый момент времени и предложите решение, используя BPIP. Нарисуйте диаграмму решения Гантта.
4. Чтобы избежать инверсии приоритета и вызывать задачу, можно применять неприоритетный протокол с наивысшим доступным приоритетом. Следовательно, он будет выполнен без выгрузки, предотвращая инверсию приоритета. Покажите, как этот протокол можно реализовать в DRTK, сделав необходимые модификации кода.
5. Протокол немедленного приоритета наследования IPIP работает следующим образом. Любая задача, которая хочет войти в критичный раздел, выполняет ожидание на семафоре, защищая ресурс. Если этот вызов успешен, он наследует самый высокий потолок среди всех семафоров в системе, гарантируя, что она не будет прервана какой-либо другой задачей во время ее критичного раздела. Реализуйте новые системные вызовы ожидания и сигнализации в DRTK, которые проявят свойство этого протокола.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Mars Pathfinder Official Website. http://www.nasa.gov/mission_pages/mars-pathfinder/index.html.
2. Sha L., Rajkumar R., Lehoczky J. P. (1990) Priority inheritance protocols: an approach to real-time synchronization. IEEE Trans Comput 39 (9): 1175–1185.

Глава 9

Планирование многопроцессорных распределенных задач реального времени

9.1. ВВЕДЕНИЕ

До сих пор мы рассматривали планирование задач в однопроцессорной системе и проанализировали несколько часто используемых алгоритмов для периодических и аperiodических задач. Планирование задач в реальном времени на многопроцессорных и распределенных аппаратных средствах – значительно более сложная проблема, чем однопроцессорный случай. На самом деле найти оптимальный график для конкретной задачи в реальном времени в многопроцессорных или распределенных системах – трудная проблема (NP-Hard – non-deterministic polynomial time hard), и поэтому обычно ищутся подоптимальные эвристические алгоритмы.

Затраты на межзадачное взаимодействие при планировании задач в реальном времени в многопроцессорных системах могут быть проигнорированы, так как общение обычно осуществляется через общую память, и для передачи данных используются высокоскоростные параллельные шины памяти, даже когда связь включает в себя копирование сообщений из одной локальной памяти процессора в другую. В распределенных системах реального времени затраты на связь между задачами сопоставимы со временем выполнения задачи с момента передачи сообщений через сеть связи, и поэтому эти затраты не могут быть проигнорированы в таких системах. Приоритетность отношений между задачами представляет собой еще одну проблему, которую необходимо решать. Наконец, совместное использова-

ние ресурсов в многопроцессорной или распределенной системе добавляет еще один уровень сложности при планировании задачи в реальном или не в реальном времени.

Планирование задач реального времени с известными свойствами в многопроцессорных или распределенных системах выполняется в два этапа: распределение задач среди процессоров и планирование задач в процессорах. В некоторых случаях эти два этапа могут чередоваться.

Наша цель в этой главе – изучить методы планирования для многопроцессорной системы и распределенной системы реального времени. Мы примем, что затраты на межпроцессорное взаимодействие незначительны в связи с наличием скоростной межпроцессорной связи, но в распределенной системе учитывать задержки сообщений в сети связи необходимо. Мы рассмотрим независимые задачи, которые во всех случаях не разделяют ресурсы, если только не будет указано иначе.

9.2. МНОГОПРОЦЕССОРНОЕ ПЛАНИРОВАНИЕ

Целью многопроцессорного алгоритма планирования в реальном времени является назначение n набора задач $T = \{\tau_1, \dots, \tau_n\}$ m процессорам из множества $P = \{P_1, \dots, P_m\}$ такого, что срок выполнения каждой задачи соблюдается, а нагрузка равномерно распределяется между процессорами. Заметим, что в общем случае алгоритм многопроцессорного планирования не в реальном времени заключается в том, чтобы обеспечить баланс нагрузки. В [10] показано, что многопроцессорное планирование – трудная (NP-Hard) проблема.

Аппаратное обеспечение может состоять из *однородных* (*homogeneous*) процессоров, и в этом случае время выполнения задачи не зависит от процессора, на котором оно выполняется, или *неоднородных* процессоров с задачами, имеющими разное время выполнения. Для простоты анализа мы будем полагать, что процессоры идентичны. Мы можем иметь следующие варианты многопроцессорного планирования: *раздельное планирование*, *глобальное планирование* и *полураздельное планирование* (*partitioned scheduling*, *global scheduling*, *semi-partitioned scheduling*). Раздельное планирование выполняется в автономном режиме путем назначения задачи процессорам, после чего каждый процессор применяет алгоритм планирования применительно к назначенной ему задаче, как показано на рис. 9.1a. Любые однопроцессорные алгоритмы планирования, которые мы рассматривали в главе 7, могут использоваться для локального планирования при раздельном планировании.

Глобальное планирование использует другой подход, размещая в любое время n задач с самым высоким приоритетом в m процессорах. Таким образом, это динамический подход, когда любая входящая в систему задача планируется онлайн, основываясь на некоторых критериях планирования, как показано на рис. 9.1b. Существует одна готовая очередь задач, в отличие от m готовых очередей задач для m процессоров раздельного метода.

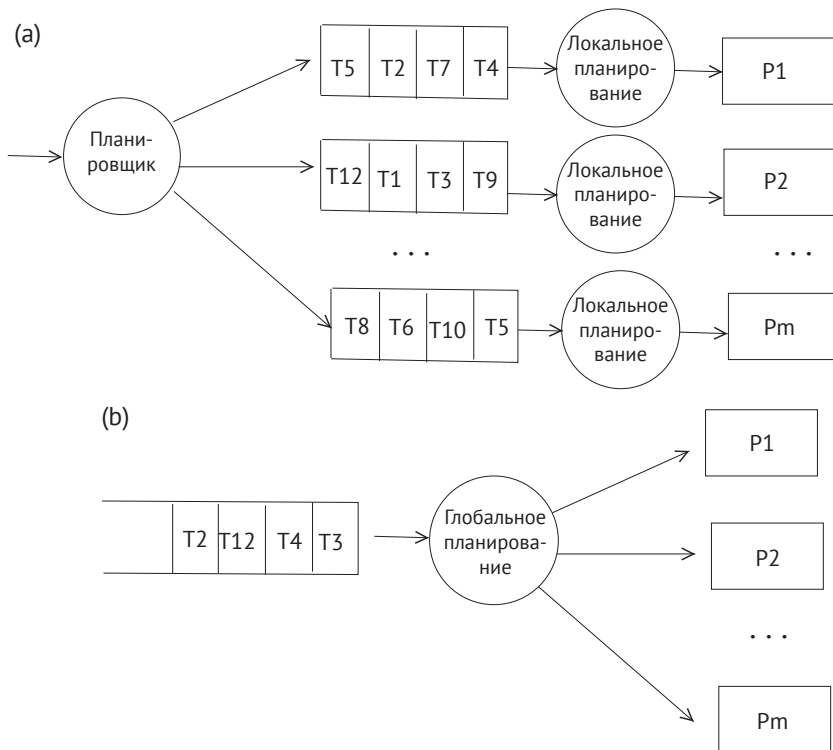


Рис. 9.1. Планирование: а – раздельное; б – глобальное

9.2.1. Раздельное планирование

В этом режиме мы будем считать, что все экземпляры задачи выполняются на одном процессоре, как это обычно делается на практике. Будем полагать, что каждый процессор имеет свою собственную очередь готовых задач, и как только задача τ_i назначена процессору P_j , все экземпляры задач τ_i будут выполняться на процессоре P_j . Миграции задач в этом подходе не допускается. Наша цель состоит в том, чтобы разделить набор задач на m групп так, чтобы каждая группа задач выполнялась на разных процессорах. В этом случае планирование задач выполняется индивидуально каждым процессором, как указано выше. В этом методе мы можем реализовать большую часть алгоритмов и протоколов управления ресурсами однопроцессорного планирования.

9.2.1.1. Распределение задач

Первым шагом разбиения при планировании является назначение задач процессорам многопроцессорной системы. За этим шагом следует реализация однопроцессорного алгоритма планирования в каждом процессоре. Критерием успеха является выполнимое планирование задач во всех процессорах. Кри-

теории остановки могут быть определены несколькими способами, например после неудачных попыток, как показано в следующих шагах такой процедуры:

1. **Input:** набор задач $T = \{\tau_1, \tau_2, \dots, \tau_n\}$
2. m процессоров $P = \{P_1, \dots, P_m\}$
3. Распределение задач по процессорам с отображением в $M: T \rightarrow P$
4. Планирование задач на каждом процессоре с T_i , отображая как P_i .
5. **If** планирование на всех процессорах выполнено
6. **Output** планирования $S = \{S_1, S_2, \dots, S_n\}$, где S_i – планирование задачи из набора $T_i \subset T$ на процессоре P_i
7. **Else if** не соответствует критериям, перейти к 3
8. **End if**
9. **Output** неудача

Нам нужно проверить, обеспечивает ли распределение задач на шаге 3 возможные графики для всех наборов задач, назначенных каждому процессору в системе. Если это не удастся, мы можем попробовать другое назначение, в худшем случае может быть увеличено количество процессоров. Мы рассмотрим три примера алгоритмов для распределения задач при распределенном планировании: алгоритм балансировки использования, алгоритм EDF с первым соответствием (first-fit) и алгоритм RM с первым соответствием.

9.2.1.2. Алгоритм балансировки использования с EDF

В качестве первой попытки задачи могут быть назначены процессорам, так чтобы использование процессоров было сбалансировано. Из набора задач выбирается неназначенная задача τ_i , и для нее назначается наименее используемый процессор, как показано в алгоритме 9.1. Мы проверяем условие планирования с задачей τ_i , назначенной процессору i , и если это не удастся, нам нужен новый процессор и, следовательно, необходимо увеличить число процессоров в строке 9 алгоритма. Использованием процессора i является U_i , использованием задачи τ_i является u_i . Обратите внимание, что мы проверяем, меньше ли единицы $U_j + u_i$, проверяя тем самым условие планируемости EDF.

Алгоритм 9.1. Алгоритм балансировки использования

- 1: **Input:** набор $T = \{\tau_1, \dots, \tau_n\}$ из n задач, набор процессоров $P = \{P_1, P_2, \dots, P_n\}$
 2. **Output:** $M: T \rightarrow P$ – назначение процессорам задач
 - 3: **for all** $\tau_i \in T$ **do**
 - 4: выбрать P_j такое, что U_j является минимумом
 - 5: **if** $(U_j + u_i) < 1$ **then**
 - 6: **назначить** τ_i процессору P_j с минимумом текущей нагрузки
 - 7: $U_j \leftarrow U_j + u_i$
 - 8: **else**
 - 9: $m \leftarrow m + 1$
 - 10: $U_m \leftarrow u_i$
 - 11: **end if**
 - 12: **end for**
-

9.2.1.3. Алгоритмы

Метод пакетной упаковки имеет целью эффективное размещение в заданное количество пакетов некоторого количества продуктов [6]. Как мы увидим, этот подход можно использовать для распределения задач на этапе их разбиения. Формально его можно определить следующим образом:

Определение 9.1 (проблема упаковки в пакеты). Дан набор из n составляющих элементов $\{1, 2, \dots, n\}$, имеющих размер $s_i \in [0, 1]$. Найти минимальное количество пакетов емкостью 1 такое, что все составляющие набора будут размещены в этих пакетах.

Решение этой проблемы является NP-hard решением [8], и для поиска субоптимальных решений обычно используются различные эвристики.

- *Первое соответствие (First-fit)*: выбран пакет с самым низким индексом в последовательности, в котором может разместиться элемент.
- *Лучшее соответствие (Best-fit)*: выбран пакет с наименьшей вместимостью, который может вместить элемент из последовательности.
- *Худшее соответствие (Worst-fit)*: выбран пакет с максимальной вместимостью, который может вместить элемент из последовательности.

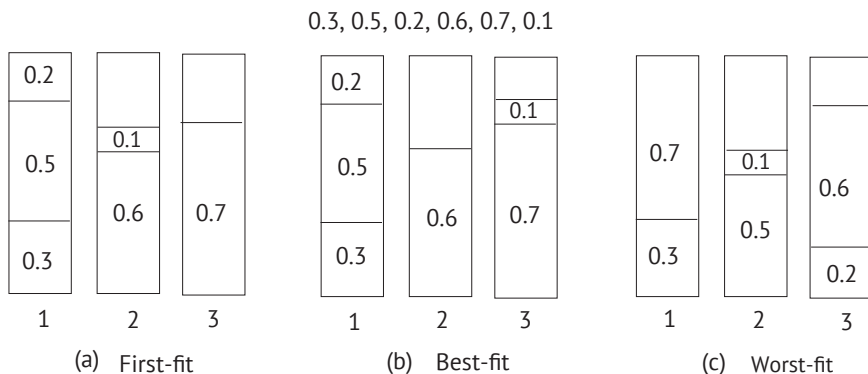


Рис. 9.2. Пример алгоритма балансировки использования

Работа этих алгоритмов проиллюстрирована на рис. 9.2 с тремя пакетами единичной емкости каждая из шести составляющих элементов, которые представлены в порядке 0,3, 0,5, 0,2, 0,6, 0,7 и 0,1. Для многопроцессорного планирования могут использоваться алгоритмы пакетной упаковки [12]. В этом случае процессоры – это пакеты, а элементы – задачи со временем выполнения. Алгоритм 9.2 отображает псевдокод для алгоритма *следующее соответствие (Next-fit)* в предположении, что n – это число задач, а m – это количество процессоров.

Алгоритм 9.2. Алгоритм следующего соответствия (Next-fit)

```

1: Input: набор  $T = \{\tau_1, \dots, \tau_n\}$  из  $n$  задач, набор процессоров  $P = \{P_1, P_2, \dots, P_m\}$ 
2: Output:  $M: T \rightarrow P$  – назначение процессорам задач
3: for all  $\tau_i \in T$  do
4:    $i \leftarrow 1$ ;  $j \leftarrow 1$ 
5:   while  $i < n \wedge j < m$  do
6:     if  $\tau_i$  может быть размещен в  $P_j$  then
7:       назначить  $\tau_i$  процессору  $j$ 
8:        $i \leftarrow i + 1$ 
9:     else
10:       $k \leftarrow k + 1$ 
11:    end if
12:  end while
13: if  $i < n$  then
14:   return НЕ ОСУЩЕСТВИМО
15: end if

```

Алгоритм упаковки пакета назначения задач *First-Fit* (FF) имеет похожую структуру, но запускается для первого процессора, чтобы соответствовать текущей задаче, как показано в алгоритме 9.3, где мы полагаем, что номер процессора на этот раз постоянен. Мы ищем процессоры, начиная с первого, чтобы соответствовать текущей задаче, и увеличиваем количество процессоров, пока не достигнем числа m .

Алгоритм 9.3. Алгоритм первого соответствия (First-fit)

```

1: Вход: набор  $T = \{\tau_1, \dots, \tau_n\}$  из  $n$  задач, набор процессоров  $P = \{P_1, P_2, \dots, P_m\}$ 
2: Выход:  $M: T \rightarrow P$  – назначение процессорам задач
3: for  $i = 1$  to  $n$  do
4:    $j \leftarrow 1$ 
5:   while  $j < m \wedge \tau_i$  задача не может быть назначена процессору  $P_k$  do
6:      $j \leftarrow j + 1$ 
7:   end while
8:   if  $j < m$  then
9:     назначить  $\tau_i$  to  $P_j$ 
10:     $i \leftarrow i + 1$ 
11:   else
12:    return НЕ ОСУЩЕСТВИМО
13:   end if
14: end for

```

Алгоритмы упаковки пакетов предполагают, что временные затраты на связь незначительны, поскольку данные размещены в глобальной памяти или передаются по высокоскоростным параллельным шинам.

Рассмотрим множество задач $T = \{\tau_1, \dots, \tau_n\}$ для запуска в многопроцессорной системе. Сначала нам нужно определить количество необходимых процессоров. Это можно примерно определить с учетом общего использования U_T задач и проверки используемого алгоритма. Например, поскольку мы знаем, что ал-

горитм EDF позволяет коэффициент использования 1, нам нужно, по крайней мере, $[U_T]$ процессоров.

9.2.1.4. Алгоритм EDF с упаковкой первое соответствие

Для распределения задач между процессорами может использоваться алгоритм EDF с эвристикой первого приближения. Алгоритм EDF в одном процессоре может обеспечить использование единства, и нам нужно проверить это свойство при назначении задач, как показано в алгоритме 9.4. Нашей целью в этом случае является размещение задач на доступных процессорах, начиная с самого низкого индекса процессора, и проверка использования процессора перед каждым назначением. Мы также определяем количество требуемых процессоров (j).

Теорема 9.1 ([11]). *Набор задач T планируется алгоритмом EDF-FF на m процессорах, если*

$$U(T) \leq \frac{m+1}{2}. \quad (9.1)$$

Если все задачи имеют коэффициент использования C/T меньший, чем значение α , то в худшем случае использование алгоритма EDF-FF [11]:

$$U(m, b) = \frac{\beta m + 1}{\beta + 1}, \quad (9.2)$$

Алгоритм 9.4. EDF-FF

```

1: Вход: набор  $T = \{\tau_1, \dots, \tau_n\}$  из  $n$  задач, набор процессоров  $P = \{P_1, P_2, \dots, P_n\}$ 
2: Выход:  $M: T \rightarrow P$  > назначение процессорам задач
3:  $i \leftarrow 1$ ;  $j \leftarrow 1$ 
4: while  $i \leq n$  do
5:   if  $U_j + u_i < 1$  then >проверить критерий EDF
6:      $U_j \leftarrow U_j + u_i$  > назначить текущую задачу процессору
7:   else
8:      $j \leftarrow j + 1$  > увеличить число процессоров
9:    $u_i \leftarrow u_i$ 
10:  end if
11:   $i \leftarrow i + 1$ 
12: end while
13: return (  $j$  )
```

где $\beta = [1/\alpha]$. Обратите внимание, что когда $\alpha = 1$, то $\beta = 1$, и уравнение (9.2) превращается в уравнение (9.1).

9.2.1.5. Монотонный алгоритм первое соответствие (RM-FF)

Этот алгоритм, предложенный Даллом и Лю (Dhall and Liu) [7], основан на монотонном методе для однопроцессорной системы. Предполагается, что задачи являются периодическими с конечными сроками, приравненными к их периодам, и они независимы. Следующая теорема определяет использование в такой системе.

Теорема 9.2. Если запланирован набор из m задач, соответствующих монотонному алгоритму планирования, то минимально достижимый коэффициент использования $m(2^{1/m} - 1)$.

Предложенный авторами алгоритм состоит из следующих этапов: задачи сначала сортируются соответственно их периодам, а затем присваиваются процессорам. Затем задачи, назначенные процессору, планируются с использованием алгоритма RM для задач $\tau_1, \dots, \tau_n$, а задача τ_i , которая назначена процессору P_j , пытается планироваться на P_j , а если такой процессор отсутствует, τ_i назначается новый процессор. Обратите внимание, что этот алгоритм похож по структуре на алгоритм 9.1 с тестовым условием, адаптированным к алгоритму. Авторы показали, что алгоритм RMFF в худшем случае использует около $2.33U$ процессоров, где U является нагрузкой на рассматриваемую задачу. Возможная реализация этого алгоритма показана в алгоритме 9.5.

Анализ

Гарантированная граница использования U_{RMFF} с m процессорами при планировании RMFF обеспечивается по Оу и Бейкеру (Oh и Baker) [13] следующим образом:

$$m(2^{1/5} - 1) \leq U_{RMFF}.$$

Алгоритм 9.5. RM-FF

```

1: Вход: набор  $T = \{\tau_1, \dots, \tau_n\}$  из  $n$  задач, набор процессоров  $P = \{P_1, P_2, \dots, P_n\} \dots >$ 
    $m$  заранее неизвестно
2: Выход:  $F: T \rightarrow F >$  назначение процессорам задач
3: for all  $\tau_i \in T$  do
4:   Выбрать самую низкую из прежде использованных  $j$  такую задачу  $\tau_i$ , которую может
   обеспечить процессор  $P_j$ , основываясь на тесте использования RM
5:   назначить  $\tau_i$  процессору  $P_j$ 
6:   if это невозможно then
7:     add добавить к набору процессоров новый процессор
8:   end if
9: end for

```

Это означает, что на многопроцессорной платформе при применении данного алгоритма возможен набор задач с максимальным использованием общей емкости процессоров примерно 41 %.

9.2.2. Глобальное планирование

Как говорилось ранее, этот тип планирования характеризуется единственной готовой очередью задач, которая используется для назначения задач узлам графа. Кроме того, задачам может разрешаться миграция от одного узла к другому. Мы выделим три случая миграции, позволяющих *миграцию на уровне задачи, или, иначе, перенос экземпляра задачи*. Это означает, что экземпляр задачи

может работать на любом процессоре, но запущенная задача не может мигрировать на другой процессор. Миграция на уровне задач позволяет вместе с тем запускать задачу на любом процессоре в любое время.

В глобальном планировании существует единственная глобальная готовая очередь задач, и готовая задача может быть назначена процессору с наименьшей текущей нагрузкой. Могут быть классифицированы два типа глобальных алгоритмов планирования: может быть адаптирован однопроцессорный алгоритм планирования, такой как RM или EDF, или может быть разработан новый алгоритм. Мы рассмотрим оба подхода. Два общих случая использования существующего алгоритма могут различаться следующим образом:

- *глобальный EDF (Global EDF)*: планировщик всегда выбирает m задач из очереди с кратчайшими конечными сроками, планируя их на m процессоров;
- *глобальный RM (Global RM)*: планировщик всегда выбирает m задач из очереди с самым высоким приоритетом, основанным на их RM-критерии, планируя их на m процессоров.

Основной проблемой данного метода является невозможность использования существующего однопроцессорного алгоритма планирования. Однако с этими алгоритмами может быть достигнуто эффективное использование вычислительной мощности.

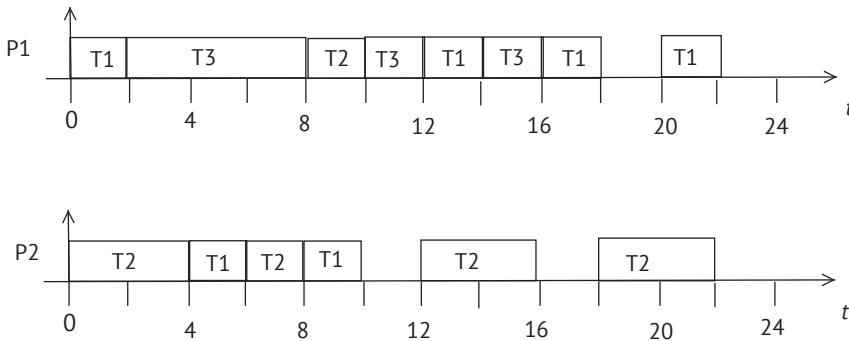


Рис. 9.3. Пример глобального RM

9.2.2.1. Глобальный монотонный алгоритм

Рассмотрим набор задач $T = \{\tau_1(2, 4), \tau_2(4, 6), \tau_3(8, 24)\}$, который необходимо запланировать на двух процессорах P_1 и P_2 . Диаграмма Ганта возможного графика изображена на рис. 9.3 присвоением приоритетов задачам в соответствии с их периодами. В любой планируемой точке мы выбираем для запуска задачу с наивысшим приоритетом, и это может привести к выгрузке задачи с более низким приоритетом. Например, в момент времени $t = 8$ задача τ_2 готова к запуску, и задача τ_3 выгружается. Обратите внимание, что при этом планировании мы разрешаем миграцию задач.

9.2.2.2. Аномалии

При использовании алгоритмов глобального RM или глобального EDF в многопроцессорной системе могут присутствовать следующие типы аномалий:

- периодические наборы задач с использованием, близким к 1, не могут быть запланированы с глобальным RM или глобальным EDF. Этот факт известен как эффект Дхалла (Dhall Effect) [7]. Рассмотрим случай с $n = m + 1$, $\forall \tau_i, P_i = 1$ и $C_i = 2\epsilon$, $\forall 1 \leq i \leq m$. $P_{m+1} = 1 + \epsilon$, $C_{m+1} = 1$, $u_{m+1} = 1/(1 + \epsilon)$. График этих задач с использованием глобальной RM изображен на рис. 9.4. Можно видеть, что $\tau_m + 1$ не уложилась в срок, хотя ее использование близко к единице;
- увеличение продолжительности периода задач с фиксированным приоритетом может привести для некоторых задач к несоблюдению сроков [1].

9.2.2.3. Пропорциональный алгоритм справедливого планирования

Алгоритм пропорциональной справедливости (P-Fair), разработанный для периодических задач с жестким реальным временем, является одним из первых оптимальных алгоритмов многопроцессорного планирования в реальном времени [3]. Основные предположения в этом алгоритме: есть m идентичных процессоров, задачи могут быть прерваны и могут мигрировать без затрат. Задача делится на подзадачи фиксированного размера, и период задачи делится на возможно перекрывающиеся *окна*. Алгоритмы P-справедливого (P-fair) планирования назначают приоритеты подзадачам и пытаются планировать каждую подзадачу в соответствующем окне. Таким образом, каждая работа соответствует своему конечному сроку. Задача P-fair планирования преобразуется

в проблему интегрального сетевого потока, и было показано, что $\sum_{i=1}^n \frac{C_i}{P_i} \leq m$, где m – число идентичных процессоров, является необходимым и достаточным условием для выполнимого P-справедливого планирования [3].

Алгоритмы PF [4] и PD [5] представили две версии алгоритма P-fair, из которых алгоритм PD более эффективен. Основным недостатком алгоритмов P-fair являются затраты, связанные с частыми выгрузками и миграцией подзадач. Они обеспечивают выполнимый график для периодических задач. Алгоритм PD2 [1] упрощает определение приоритета алгоритмов P-fair.

9.3. РАСПРЕДЕЛЕННОЕ ПЛАНИРОВАНИЕ

Распределенное планирование задач в системе реального времени обычно имеет следующие цели:

- нам необходимо обеспечить априори выполнимое планирование задач с известными характеристиками. Эти задачи обычно являются жесткими периодическими задачами реального времени с известным временем вычисления;

- когда аperiodическая задача или sporadicкая задача с крайним сроком поступает онлайн, нам необходимо выполнить предварительные тесты. Задачи, прошедшие тестирование, в системе разрешены;
- мы стараемся поддерживать балансировку нагрузки в системе, перенося задачи из загруженных узлов в менее загруженные узлы.

Мы видели, как разделение может обеспечить выполнимые расписания для задач с известными характеристиками. В этом разделе мы рассмотрим подходы к планированию задач, которые поступают асинхронно во время выполнения программы путем балансировки нагрузки в системе в это время. Сначала опишем два простых метода для балансировки нагрузки в общей системе и затем кратко рассмотрим два алгоритма для планирования задач в реальном времени с использованием балансировки нагрузки.

9.3.1. Балансировка нагрузки

Динамическое планирование задач сосредоточено вокруг концепции *миграции задач*, которая в основном состоит в передаче задач вместе с их текущей средой из сильно загруженного узла на другой узел распределенной системы. Миграция задач довольно сложна из-за твердой необходимости передачи сред, таких как локальная и глобальная память и т. д. Еще один момент, который необходимо учитывать, заключается в осуществлении передачи при выполнении задачи, например когда вытесняется ожидание ресурса, или разрешении миграции только для задач, которые еще не начались. Мы примем последнее, иллюстрируя два простых протокола динамической балансировки нагрузки, описанных в следующих разделах. Далее предположим, что критичная задача τ_i имеет количество копий τ_{ij} для $i = 1$ до k , и, следовательно, нам не нужно передавать код задачи; а просто нужно инициировать копию задачи в узле с низкой нагрузкой. Еще одним вопросом, вызывающим озабоченность, является определение нагрузки в узле распределенной системы реального времени. Общепринятой стратегией является определение количества готовых задач в готовых очередях планировщика. Там могут быть веса, связанные с очередями, и сами задачи. Мы примем, что жесткие сроки периодических задач реального времени гарантируются локальными политиками планирования в реальном времени, и динамическое планирование включает в себя миграцию аperiodических задач с мягкими сроками или sporadicкие задачи.

9.3.1.1. Центральная балансировка нагрузки

Метод центральной балансировки нагрузки основан на центральном узле графа, который мы назовем супервизором. Он периодически контролирует нагрузки на каждом узле. Это делается путем отправки зонда сообщения каждому узлу и получения локальных загрузок в начале каждого периода. Если существует пара узлов с низкой и высокой нагрузками, он дает команду узлу с высокой нагрузкой об отправке нагрузки, а узлу с низкой для приема нагрузки – как показано в алгоритме 9.6.

С другой стороны, обычный узел имеет системную задачу, которая всегда ждет получения запроса на опрос от супервизора и отправляет статус своей текущей загрузки супервизору, когда такой запрос получен. Исходя из своей нагрузки, он может затем ждать передачи ему нагрузки или может отправить свою загрузку в соответствии с указаниями супервизора. Это проиллюстрировано в алгоритме 9.7.

Такой подход облегчает нагрузку на обычные узлы, но, как и в любом протоколе, который опирается на центральный объект, возникает две проблемы, с которыми можно столкнуться: супервизор становится узким местом, когда количество узлов в системе велико, и вторая проблема – супервизор является единственной точкой отказа, и в случае его отказа необходимо будет выбирать нового супервизора с помощью алгоритма выбора лидера.

Алгоритм 9.6. *Супервизор центральной балансировки нагрузки*

```

1: Вход: набор вычислительных элементов  $P = \{P_1, \dots, P_n\}$ 
2: Выход:  $M: T \rightarrow P$  – назначение задач процессору  $\theta$ 
3: while справедливо do
4: по истечении времени таймера
5: передать экземпляр всем вычислительным элементам
6: получить загрузку от каждого вычислительного элемента
7: если есть  $v$  пар верхних и нижних узлов do
8: отправить  $load\_send(v)$  в верхний узел
9: отправить  $load\_recv(u)$  в нижний узел
10: end while
11: установить счетчик
12: end while

```

Алгоритм 9.7. *Обычный узел*

```

1: while справедливо do
2: определить  $my\_load$ 
3: получить элемент
4: отправить  $my\_load$  супервизору
5: если  $my\_load$  является верхним then
6: ожидать приема  $load\_send(v)$  от супервизора
7: отправить нагрузку узлу  $v$ 
8: else if  $my\_load$  является нижним then
9: ожидать получения  $load\_send(u)$  от супервизора
10: получить нагрузку от узла  $u$ 
11: end if
12: end while

```

9.3.1.2. Распределенная балансировка нагрузки

Подход в данном случае заключается в выполнении балансировки нагрузки без использования центрального компонента из-за недостатков его использования. Каждый узел теперь должен знать о состоянии нагрузки всех других

узлов. Это может быть осуществлено каждым узлом периодической передачей на все остальные узлы своей текущей нагрузки и ожиданием получения величины их загрузки. В подходе инициации приемника, если узел находит, что он имеет низкую нагрузку и обнаруживает узел с высокой нагрузкой, он просто запрашивает передачу от верхнего узла. Верхний узел может получать несколько таких запросов, а тогда выбирает узел с самой низкой нагрузкой, передает ему свою нагрузку и отклоняет все другие запрашивающие узлы, как показано в алгоритме 9.8.

Алгоритм 9.8. Узел динамической балансировки нагрузки

```

1: Вход: набор вычислительных элементов  $P = \{P_1, \dots, P_n\}$ 
2: while справедливо do
3:   по истечении времени таймера
4:   передать экземпляр всем вычислительным элементам
5:   получить загрузку от каждого вычислительного элемента
6:   если  $my\_load$  является нижним и  $\exists P_u \in P$  такой, что  $load(P_u) = \text{нижний}$  then
7:     отправить  $req\_load$  узлу  $u$ 
8:     ожидать получения загрузки  $load$  или отказа от узла  $u$ 
9:   else if  $my\_load$  является верхним then
10:    ожидать истечения времени получения  $req\_load$  от нижнего узла
11:    если есть по крайней мере один  $req\_load$ , получить then
12:      выбрать узел  $v$  с самой легкой нагрузкой
13:      отправить нагрузку узлу  $v$ 
14:      отправить отказ всем другим узлам, которые прислали  $req\_load$ 
15:    end if
16:  end if
17: end while

```

9.3.2. Метод целенаправленной адресации и назначения ставок

Схема целенаправленной адресации и назначения ставок (focused addressing and bidding, FAB) – это онлайн-процедура для планирования задачи в DRTS [15]. Набор задач в такой системе состоит из не критичных и критичных задач реального времени. Первоначальное планирование критических задач гарантирует, что их конечные сроки соблюдаются, и для этих задач отводится достаточно времени. Планирование не критичных задач зависит от состояния системы. Для любой задачи, поступающей на узел DRTS, сначала делается попытка запланировать ее на этом узле, и если это невозможно, ищется целевой узел, который может запланировать прибывающую задачу. Эта схема состоит из целенаправленных алгоритмов адресации и назначения ставок.

Каждый узел в системе ведет таблицу состояния, отображающую список критичных задач, которые ему были назначены ранее алгоритмом статического планирования и другими не критичными задачами и которые он, возможно, принял. У каждого узла есть таблица загрузки, содержащая избыточные мощности всех других узлов в системе. Время делится на окна фиксированной продолжительности, и в конце окна каждый узел передает долю мощности

компьютера, которую он оценивает как свободную, в следующее окно. Каждый узел получает эту трансляцию и периодически обновляет свою таблицу загрузки. Таблица загрузки может быть устаревшей из-за того, что система является распределенной.

Для каждой новой задачи τ_i с конечным сроком, поступающей в узел P_j , делается попытка запланировать ее на этом узле. Это возможно, только если избыточная мощность, доступная на этом узле, больше, чем время между конечным сроком и временем прибытия задачи. Если это невозможно из-за перегрузки узла P_j , P_j выбирает возможный узел P_s , называемый *фокусным процессором*, и передает задачу τ_i в P_s . Но, как было сказано, информация о наличии избытка, может оказаться устаревшей, и P_j инициирует процедуру под названием *торги (bidding)*, во взаимодействии с фокусным узлом, с тем чтобы увеличить вероятность планирования τ_i . По этому алгоритму претендент P_j выполняет следующие действия:

- 1) выбирает k узлов с достаточным избытком;
- 2) отправляет сообщение с запросом цены (request-for-bid, RFB), которое содержит ожидаемое время выполнения, требования к ресурсам и конечный срок τ_i для всех выбранных узлов;
- 3) отправляет задачу на узел, который предлагает лучшую ставку;
- 4) если полученные предложения не являются удовлетворительными, отклоняет задачу.

Выбранный узел P_k выполняет следующие действия:

- 1) рассчитывает ставку, которая является вероятностью того, что задание гарантировано;
- 2) отправляет заявку участнику, если она превышает минимальную требуемую ставку.

9.3.3. Алгоритм Buddy

Для сокращения времени, затрачиваемого на сбор информации о состоянии, проверку состояния и торги, может быть использован алгоритм Buddy (buddy, приятель). Алгоритм Buddy работает аналогично методу FAB путем передачи задач от сильно загруженных узлов менее загруженным. Узлы в системе классифицируются как *недогруженные (Underloaded)*, *нормально загруженные (Normal-Fully Loaded)* и *перегруженные (Overloaded)*. Состояние загрузки узла определяется номером задачи в подготовленной планировщиком очереди. Узел имеет ряд связанных с ним узлов, называемых его приятелями (buddy). Когда узел меняет свое состояние на недогруженное или выходит из этого состояния, он сообщает об этом своим приятелям. Формирование набора приятелей требует тщательного рассмотрения; большой набор означает много сообщений и, следовательно, значительные затраты на связь, а малый набор означает, что поиск приемника нагрузки может стать невозможным. Выбор порогов также влияет на производительность. Следует учитывать размер набора приятелей, пропускную способность сети и топологию.

9.3.4. Планирование сообщений

Для обеспечения сквозного планирования задач в конечные сроки нужно учитывать задержки сообщений по сети. Передача сообщений в режиме реального времени обычно требует приоритета сообщений, как мы рассмотрели в главе 3. Как и в случае задач в реальном времени, связь в реальном времени может быть вызвана временем или событием. ТТП является примером первого, в то время как CAN выступает примером второго. Управляемые временем и управляемые событиями системы требуют различных подходов для совместного планирования задач и сообщений в реальном времени. В общем, анализ совместного планирования задач и сообщений зависит от используемого протокола связи, такого как TR (кольцевая локальная вычислительная сеть с маркерным доступом, Token Ring) или TDMA (множественный доступ с временным разделением каналов, time-division multiple access).

Анализ общего времени отклика (holistic response time analysis, HRTA) – это метод анализа планируемости для расчета верхних границ при совместном планировании задач и сообщений в распределенной системе реального времени [9]. Рассмотрим случай, когда узел датчика вводит некоторые данные и отправляет их по сети к вычислительному узлу, который обрабатывает эти данные и отправляет по сети команду на исполнительный механизм. Общее время отклика – это временной интервал между восприятием данных и активацией механизма.

Метод HRTA итеративно запускает алгоритмы анализа узлов и сетей. Первый шаг выполняется путем расчета времени отклика всех задач и сообщений в сети и предполагает, что все временные искажения (дрожания, jitter) равны нулю. Затем на втором этапе эти искажения сообщения инициализируются искажением отправляемой задачи, вычисленной на первом этапе, и каждая задача получаемого сообщения наследует искажение, равное времени отклика получаемого сообщения. Третий шаг выполняется путем вычисления времени ответа всех задач и сообщений. Эти вычисленные значения сравниваются со значениями, полученными на первом этапе, и эта процедура продолжается до получения равных значений в шагах 1 и 2. Псевдокод алгоритма HRTA показан в алгоритме 9.9, где J обозначает значение временного искажения, а R – время отклика.

Анализ HRTA впервые был предложен в [9] для управляемых событиями задач, связанных через сеть TDMA, и позже был расширен с учетом динамических смещений задач. В общем, сообщения могут быть статичными с известным временем и продолжительностью активации или динамическими с асинхронной активацией. Случай управляемой временем системы со статичными сообщениями для построения расписания управляемых временем задач и статичных сообщений, а также обеспечение анализа планируемости для управляемых событиями задач и динамических сообщений в этой системе рассмотрен в [14].

Алгоритм 9.9. Анализ общего времени отклика

```

1: Вход: набор задач  $T = \{\tau_1, \dots, \tau_n\}$ 
   набор сообщений  $M = \{m_1, \dots, m_k\}$ 
3:  $R \leftarrow 0$  › установить общее время отклика равным 0
4: while true do
5:   for all  $\tau_i \in T \wedge m_i \in M$  do
6:      $J_{ni} \leftarrow R_{senderi}$ 
7:      $J_{receiveri} \leftarrow R_{ni}$ 
8:   compute время отклика всех сообщений
9:   compute время отклика всех задач
10:  if  $R_i \neq R_{i-1}$  then
11:     $R_{i-1} = R_i$ 
12:  else break;
13:  end if
14:  end for
15: end while

```

9.4. РЕАЛИЗАЦИЯ DRTK

Мы предоставляем задачу центральной балансировки нагрузки и задачу распределенной балансировки нагрузки для реализации DRTK. Проанализируем область данных транспортного уровня для загрузки задач балансировки, чтобы сохранять значение нагрузки, отправителя и получателя значения нагрузки, как в следующем ниже типовом сообщении. Этот тип сообщения будет реализован как *объединенная часть части данных data_unit_t*.

```

/*****
Структура кластера сообщений
*****/
typedef struct {
    ushort load_sender;
    ushort load_receiver;
    ushort load;
} load_msg_t

```

9.4.1. Центральные задачи балансировки нагрузки

Есть два типа задач для этой реализации; центральная задача и обычная задача. Центральная задача спит и периодически просыпается, чтобы передать фрейм с типом LOAD_CHECK для получения состояния загрузки всех узлов в системе. Когда все другие узлы отправят свое состояние, центральная задача может решить, какой узел может получать нагрузку от какой другой, и отправляет сообщения для передач этим узлам. Процедура `find_load` просто подсчитывает количество задач в готовой очереди, чтобы оценить нагрузку на узел как высокую или низкую. Фактические процедуры передачи задач у нас отсутствуют и ниже не показаны. Мы также предполагаем безошибочной работу, то есть все узлы работают правильно.


```

/*****
Центральный балансер нагрузки
*****/
/* central_load.c */

#define HIGH_LOAD 120
#define LOW_LOAD 20
#define LOAD_CHECK 1
#define SEND_LOAD 2
#define RECV_LOAD 3
#define LOAD_STATUS 4
#define REQ_LOAD 3
#define LOAD_SENDING 4

TASK Central_Load() {

data_unit_ptr_t received_msg_pts[System_Tab.N_NODES],
data_pt, data_pt1, data_pt2;
ushort load, sender1, sender2, flag=0;
while(TRUE) {
    if(current_tid==System_Tab.Central_Load_id) { // сервер
        delay_task(current_id, System_Tab.DELAY_TIME);
        data_pt=get_data_unit(System_Tab.Net_Pool);
        data_pt->MAC_header.sender_id=System_Tab.this_node;
        data_pt->MAC_header.type=BROADCAST;
        data_pt->TL_header.type=LOAD_CHECK;
        data_pt->TL_header.sender_id=System_Tab.Central_Load_id;
        send_mailbox_notwait(System_Tab.DL_Out_mbox,data_pt);
        for (i=0;i<System_Tab.N_NODES-1;i++){
            data_pt=recv_mbox_wait(&task_tab[current_tid].mailbox_id);
            received_msg_pts[i]=data_pt;
        }
        for (i=0;i<System_Tab.N_NODES-1;i++)
            for (j=0;j<System_Tab.N_NODES-1,j++) {
                if(received_msg_pts[i].TL_header.type==LOW_LOAD
                && received_msg_pts[j].TL_header.type==HIGH_LOAD) {
                    low=i;
                    high=j;
                    flag=1;
                }
                else if(received_msg_pts[i].TL_header.type==HIGH_LOAD
                && received_msg_pts[j].TL_header.type==LOW_LOAD) {
                    low=j;
                    high=i;
                    flag=1;
                }
            }
        if (flag==1) {
            data_pt1=received_msg_pts[low];
            data_pt2=received_msg_pts[high];
            sender1=data_pt1->MAC_header.sender_id;
            sender2=data_pt2->MAC_header.sender_id;

```

```

data_pt2->MAC_header.type=UNICAST;
data_pt2->MAC_header.sender_id=System_Tab.this_node;
data_pt2->MAC_header.receiver_id=sender2;
data_pt2->TL_header.receiver_id=data_pt2->TL_header.sender_id;
data_pt2->TL_header.sender_id=System_Tab.Central_Load_id;
data_pt2->TL_header.type=SEND_LOAD;
data_pt2->data.load_msg.receiver_id=sender1;
send_mailbox_notwait(System_Tab.DL_Out_mbox,data_pt2);
data_pt1->MAC_header.type=UNICAST;
data_pt1->MAC_header.sender_id=System_Tab.this_node;
data_pt1->MAC_header.receiver_id=sender1;
data_pt1->TL_header.receiver_id=data_pt1->TL_header.sender_id;
data_pt1->TL_header.sender_id=System_Tab.Dist_Load_id;
data_pt1->TL_header.type=RECV_LOAD;
data_pt1->data.load_msg.sender_id=sender2;
send_mailbox_notwait(System_Tab.DL_Out_mbox,data_pt1);
}
flag=0;
}}
else { // ordinary nodes
data_pt=recv_mbox_wait(&task_tab[current_tid].mailbox_id);
if(data_pt->TL_header.type==LOAD_CHEC
load=find_load();
data_pt->MAC_header.type=UNICAST;
data_pt->MAC_header.sender_id=System_Tab.this_node;
data_pt->MAC_header.receiver_id=data_pt->MAC_header.sender_id;
data_pt->TL_header.receiver_id=data_pt->TL_header.sender_id;
data_pt->TL_header.sender_id=System_Tab.Central_Load_id;
data_pt->TL_header.type=LOAD_STATUS;
data_pt->data.load_msg.load=load;
send_mailbox_notwait(System_Tab.DL_out_mbox,data_pt);
if(load==LOW_LOAD || load==HIGH_LOAD) {
data_pt=recv_mailbox_wait(&task_tab[current_tid].mailbox_id);
if (load==LOW_LOAD && data_pt.TL_header.type==RECV_LOAD)
// ожидание получения нагрузки
data_pt=recv_mailbox_wait(&task_tab[current_tid].mailbox_id);
else if (load==HIGH_LOAD && data_pt.TL_header.type==SEND_LOAD)
// передача нагрузки определенному узлу
}
}
}
}
}
}

```

9.4.2. Задача балансировки распределенной нагрузки

В этой реализации у нас нет центральных узлов, и все узлы функционируют в равной степени. Каждый узел периодически пробуждается, а затем передает свою нагрузку всем остальным узлам. В реализованном подходе инициации приемником *нижний* узел запрашивает передачу от *верхнего* узла.

```

/*****
Балансировка распределенной нагрузки
*****/
/* distributed_load.c */

data_unit_ptr_t data_pt, received_msg_pts[System_Tab.N_NODES],
ushort load;

TASK Dist_Load() {
data_unit_ptr_t data_pt;

while(TRUE){
delay_task(current_id, System_Tab.DELAY_TIME);
data_pt=get_data_unit(Sys_Tab.Net_Pool);
data_pt->MAC_header.type=BROADCAST;
data_pt->MAC_header.sender_id=System_Tab.this_node;
data_pt->TL_header.sender_id=System_Tab.Dist_Load_id;
data_pt->TL_header.type=LOAD_CHECK;
send_mailbox_notwait(System_Tab.DL_Out_mbox,data_pt);
for (i=0;i<System_Tab.N_NODES-1;i++){
data_pt=recv_mbox_wait(&task_tab[current_tid].mailbox_id);
received_msg_pts[i]=data_pt;
}
load=find_load();
if(load==LOW_LOAD)
for (i=0;i<System_Tab.N_NODES-1;i++){
data_pt=received_msg_pts[i];
if(data_pt->TL_header.type==HIGH_LOAD){
data_pt->MAC_header.type=UNICAST;
data_pt->MAC_header.sender_id=System_Tab.this_node;
data_pt->MAC_header.receiver_id=data_pt->MAC_header.sender_id;
data_pt->TL_header.receiver_id=data_pt->TL_header.sender_id;
data_pt->TL_header.sender_id=System_Tab.Dist_Load_id;
data_pt->TL_header.type=REQ_LOAD;
send_mailbox_notwait(System_Tab.DL_Out_mbox,data_pt);
data_pt=recv_mbox_wait(task_tab[current_tid].mailbox_id);
if (data_pt->TL_header_type==LOAD_SENDING)
// получение задачи и ее активация
break;
}
else if(data_pt->TL_header.type==HIGH_LOAD){
data_pt=recv_mbox_wait_tout(&task_tab[current_tid].mailbox_id);
if (data_pt->TL_header_type==REQ_LOAD) {
data_pt->MAC_header.type=UNICAST;
data_pt->MAC_header.sender_id=System_Tab.this_node;
data_pt->MAC_header.receiver_id=data_pt->MAC_header.sender_id;
data_pt->TL_header.receiver_id=data_pt->TL_header.sender_id;
data_pt->TL_header.sender_id=System_Tab.Dist_Load_id;
data_pt->TL_header.type=LOAD_SENDING;
// послать задачу
break;
}
}
}
}
}

```

9.5. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы основные методы многопроцессорного планирования?
2. Каковы две фазы раздельного многопроцессорного планирования?
3. Сравните раздельное и глобальное многопроцессорное планирование с точки зрения сложности алгоритмов и справедливого распределения нагрузки.
4. Как работает алгоритм пакетной упаковки, и каковы основные алгоритмы пакетной упаковки?
5. Какова основная идея алгоритма балансировки использования?
6. Как работает EDF с First-Fit?
7. Как работает RM с First-Fit?
8. Что такое эффект Dhall?
9. Каков основной принцип алгоритма планирования P-Fair?
10. Как работает алгоритм фокусированной адресации и назначения ставок?

9.6. ПРИМЕЧАНИЯ К ГЛАВЕ

Два основных метода планирования задач в многопроцессорной системе – это разбиение на разделы и схемы глобального планирования. В методе разбиения на первом этапе нам нужно назначить наборы задач для процессоров, и на втором этапе каждому процессору задачи планируются независимо от готовой очереди. Хорошей стороной этого метода является то, что на втором этапе могут использоваться рассмотренные нами основные однопроцессорные алгоритмы, такие как RM и EDF. Вместе с тем проблема присваивания является NP-сложной задачей, требующей использования эвристических субоптимальных решений. Задачи не могут мигрировать, и это может привести к ситуациям, когда возможное расписание может быть найдено, только если возможна миграция.

Глобальное планирование использует единую готовую очередь, и функция планировщика – это обеспечение выполнения в любое время t задач наивысшего приоритета на m процессорах. Глобальное планирование с помощью таких алгоритмов, как RM и EDF, может привести к низкой загрузке процессора [7]. Предложенный новый класс алгоритмов глобального планирования – алгоритм P-fair с его вариациями обеспечивает оптимальное решение проблем многопроцессорного планирования. Обзор алгоритма многопроцессорного планирования в реальном времени дан в [16], а подробное исследование смежных методов приведено в [2].

Распределенное планирование в реальном времени относится к системе реального времени, состоящей из взаимодействующих вычислительных узлов. Сетевая задержка является важным параметром при рассмотрении планирования в этом режиме. Алгоритмы планирования для таких систем обычно имеют две части: статичное планирование периодических задач в реальном времени и динамическое планирование аperiodических и спорадических задач. Ба-

лансировка нагрузки и соблюдение сроков выполнения задач являются здесь основными целями. Фокусный алгоритм адресации и назначения ставок обеспечивает начальное планирование сложных задач в реальном времени и процедуры для любых входящих задач, поступающих в динамике, а алгоритм Buddy уменьшает трафик сообщений в этом алгоритме. Есть перспектива дальнейших исследований в части разделения, глобального планирования и гибридных методов. Нами также не рассматривались зависимость ресурсов и обмен ими, а это добавляет еще один уровень сложности, который следует учитывать.

9.7. УПРАЖНЕНИЯ

1. Имеется три пакета одинаковой вместимости и шесть элементов в порядке 0,2, 0,6, 0,5, 0,1, 0,3 и 0,4. Покажите, как эти элементы можно поместить в пакеты, используя методы First-Fit, Next-Fit, Best-Fit и Worst-Fit.
2. Модифицируйте код Next-Fit алгоритма 9.2, предполагая, что имеется неограниченное количество процессоров.
3. Дан набор задач τ_1 (2, 4), τ_2 (4, 5) и τ_3 (3, 6) (τ_i (C_i , d_i)). Сначала проверьте возможность планирования этого набора задач в двух процессорах. Если это возможно, разработайте планирование EDF-FF, составьте расписание и нарисуйте график Гантта, демонстрирующий это расписание.
4. При заданном наборе задач τ_1 (6, 12), τ_2 (4, 6) и τ_3 (3, 8) (τ_i (C_i , T_i)) разработать планирование RM-FF, при условии что все необходимые процессоры доступны, и нарисуйте график Гантта, демонстрирующий это расписание.
5. Модифицируйте алгоритм 9.4, чтобы получить EDF с Next-Fit.
6. Модифицируйте алгоритм 9.5, чтобы получить RM с Next-Fit.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Anderson A., Srinivasan A. (2000) Early-release fair scheduling. In: Proceedings of the Euromicro conference on real-time systems, pp. 35–43.
2. Baruah S., Bertogna M., Buttazzo G. (2015) Multiprocessor scheduling for real-time systems (Embedded systems), 2015th edn. Springer embedded systems series.
3. Baruah S. K., et. al. (1996) Proportionate progress: a notion of fairness in resource allocation. *Algorithmica* 15 (6): 600–625.
4. Baruah S., Cohen N., Plaxton C. G., Varvel D. (1996) Proportionate progress: a notion of fairness in resource allocation. *Algorithmica* 15: 600–625.
5. Baruah S., Gehrke J., Plaxton C. G. (1995) Fast scheduling of periodic tasks on multiple resources. In: Proceedings of the international parallel processing symposium, pp. 280–288.
6. Coffman E. G., Galambos G., Martello S., Vigo D. (1998) Bin packing approximation algorithms: combinatorial analysis. In: Du D. Z., Pardalos P. M. (eds). Kluwer Academic Publishers.

7. *Dhall S. K., Liu C. L.* (1978) On a real-time scheduling problem. *Oper Res* 26 (1): 127–140.
8. *Garey M., Johnson D.* (1979) Computers and intractability. A guide to the theory of NP-completeness. W. H. Freeman & Co., New York.
9. *Tindell K., Clark J.* (1994) Holistic schedulability analysis for distributed hard real-time systems. *Microprocess Microprogram* 40: 117–134.
10. *Leung J. Y. T., Whitehead J.* (1982) On the complexity of fixed-priority scheduling of periodic real-time tasks. *Perform Eval* 2: 237–250.
11. *Lopez J. M., Diaz J. L., Garcia F. D.* (2000) Worst-case utilization bound for EDF scheduling on real-time multiprocessor systems. In: *Proceedings of 12th Euromicro conference on real-time systems (EUROMICRO RTS 2000)*, pp. 25–33.
12. *Moriyama I., Ibaraki T., Hasegawa T.* (1983) Bin packing and multiprocessor scheduling problems with side constraints on job types. *Disc Appl Math* 6: 173–191.
13. *Oh D. I., Baker T. P.* (1998) Utilization bounds for N-processor rate monotone scheduling with static processor assignment. *Real Time Syst Int J Time Crit Comput* 15: 183–192.
14. *Pop T., Eles P., Peng Z.* (2003) Schedulability analysis for distributed heterogeneous time/event triggered real-time systems. In: *Proceedings of 15th Euro-micro conference on real-time systems*, pp. 257–266.
15. *Stankovic J. A., Ramamritham K., Cheng S.* (1985) Evaluation of a flexible task scheduling algorithm for distributed hard real-time systems. *IEEE Trans Comput C* 34 (12): 1130–1143.
16. *Zapata O. U. P., Alvarez P. M.* (2005) EDF and RM multiprocessor scheduling algorithms: survey and performance evaluation. *CINVESTAV-IPN, Seccion de Computacion Av, IPN*, p. 2508.

ЧАСТЬ IV



ПРИКЛАДНОЕ ПРОЕКТИРОВАНИЕ

Глава 10

Разработка программного обеспечения систем реального времени

10.1. ВВЕДЕНИЕ

Разработка программного обеспечения определяется стандартом IEEE как применение систематического, упорядоченного, поддающегося количественному определению подхода к разработке, эксплуатации и обслуживанию программного обеспечения. Методы разработки программного обеспечения общей компьютерной системы хорошо установлены, однако по ряду причин разработка программного обеспечения для систем реального времени и систем со встроенными элементами отличается от систем, работающих не в реальном времени. Прежде всего потому, что с самого начала должна рассматриваться платформа аппаратного обеспечения, так как некоторые из необходимых функций будут выполняться в общей системе аппаратно, особенно когда речь идет о системе со встроенными элементами. В результате использование подхода сверху вниз в системе реального времени может оказаться непрактичным, поскольку совместное проектирование аппаратного и программного обеспечения обычно требуется на начальном этапе. Во-вторых, система реального времени должна реагировать на внешние события в определенный период времени, что означает, что анализ времени является важным компонентом процесса проектирования в такой системе и также должен быть выполнен на начальном этапе проектирования. Как мы увидим, какие-то подходы традиционного проектирования все же применимы в системе реального времени. Разработка системного программного обеспечения распределенных систем реального времени добавляет еще один уровень сложности в проектировании ввиду необходимости учитывать синхронизацию передач по сети в задаче связи, а также необходимости назначения задач узлам системы. В заключение надо сказать, что не существует

устоявшихся методов проектирования программного обеспечения систем реального времени ни для однопроцессорных, ни для распределенных систем, однако при использовании эффективных методов моделирования, включающих временной анализ, некоторый формализм все же может быть достигнут.

Мы начнем эту главу с основных и общих концепций разработки программного обеспечения, которые могут быть реализованы в системе реального времени. Затем, вслед за временным анализом, мы опишем процесс спецификации требований. Рассматривается процедурное и объектно-ориентированное проектирование применительно к системам реального времени. Разбираются также спецификация и детальное проектирование систем реального времени с использованием конечных автоматов, временных автоматов и сетей Петри.

10.2. ЖИЗНЕННЫЙ ЦИКЛ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Жизненный цикл разработки программного обеспечения (software development life cycle, SDLC) – это базовая структура, описывающая процессы, которые необходимо выполнить на каждом этапе, обычно в форме диаграмм. Это обеспечивает описание и порядок действий, которые будут выполнены при разработке проекта программного обеспечения. Последовательные процессы, которые не перекрываются, являются общими в SDLC, хотя дублирование в некоторых процессах неизбежно. Следующие этапы описаны кратко и являются общими для большинства структур SDLC [8]:

- *сбор и анализ требований*: цели, которые необходимо достичь, сервис, который должен быть обеспечен системой, и ограничения, определяемые посредством взаимодействия с пользователем/клиентом;
- *проект*: общая архитектура системы разрабатывается на основе спецификации системных требований (system requirements specification, SRS) – документа, предыдущего этапа. На этом этапе проектирования обычно создается документация проекта данных, проекта архитектуры, проекта интерфейса и процедуры проектирования;
- *реализация*: программное обеспечение, которое будет реализовываться, делится на модули, и каждая команда выполняет кодирование назначенного ей модуля. Программисты разрабатывают программное обеспечение на основе SRS и спецификации проектной документации;
- *интеграция и тестирование*: отдельные программные модули интегрируются и тестируются для проверки работоспособности в соответствии с требованиями, установленными на первом этапе;
- *развертывание и обслуживание*: система установлена и запущена в эксплуатацию. Во время технического обслуживания система модифицируется в соответствии с новыми требованиями, и исправляются любые возможные ошибки.

Некоторые часто используемые модели SDLC – это модель водопада (Waterfall Model), спиральная модель (Spiral Model) и V-модель (V-Model), которые описываются далее.

10.2.1. Пошаговая модель Waterfall

Модель Waterfall (водопад) – это простой и старый программный дизайн высокого уровня, техника которого содержит шесть этапов. Течение проекта напоминает водопад: каждая ступень завершается только после завершения предыдущей ступени, как показано на рис. 10.1. Обратите внимание, что обратная связь от каждого этапа означает, что предыдущий этап может быть изменен на основе дизайна более поздней ступени. Оригинальная модель Waterfall не имеет обратной связи, при которой каждая фаза проектирования должна быть завершена до начала следующей. Этот простой метод может быть использован для малых и средних проектов.

Модель Waterfall проста для понимания и реализации и основана на документах. Вместе с тем она предполагает точное определение требований в начале проекта, что может оказаться нереальным для многих проектов. Программное обеспечение реализуется в этой модели на поздних стадиях проекта, что ведет к позднему обнаружению ошибок.

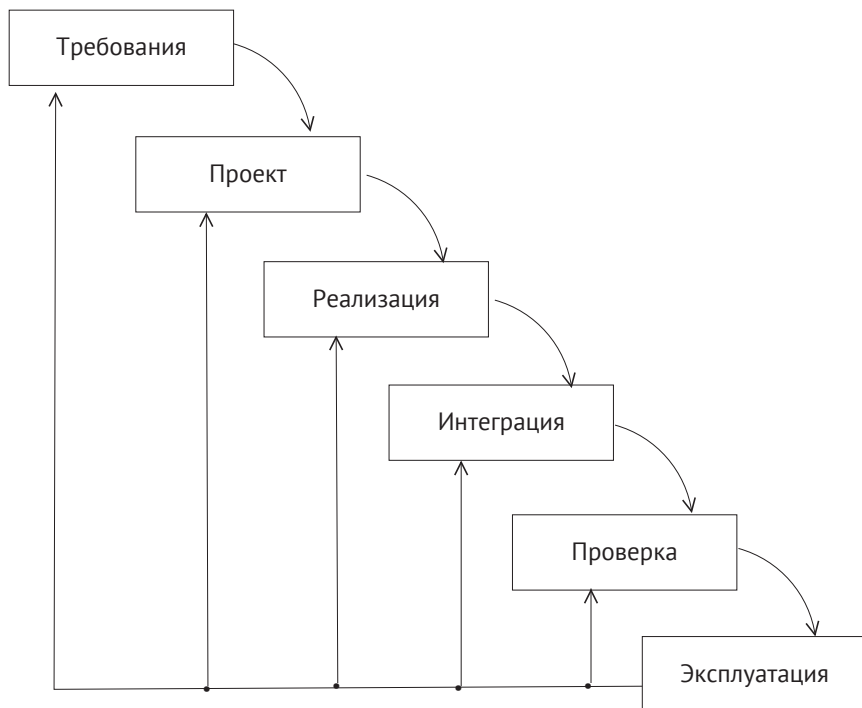


Рис. 10.1. Пошаговая модель Waterfall

10.2.2. V-модель

V-модель обеспечивает графическое представление жизненного цикла разработки программного обеспечения, содержащее этапы разработки проекта в графической форме буквы V. Левая часть, как показано на рис. 10.2, содержит анализ требований, проектирование высокого уровня и подробные этапы проектирования, как и в модели Waterfall. Внизу графика – реализация, содержащая коды, а справа, как показано ниже, – этапы модульного тестирования, тестирования интеграции, тестирования системы и приемочного тестирования.

- *Модульное тестирование*: каждый модуль тестируется самостоятельно в соответствии с конкретными тестовыми примерами.
- *Интеграционное тестирование*: модули интегрируются и тестируются с использованием тестовых примеров.
- *Системное тестирование*: тестируется полное приложение в соответствии с функциональными и нефункциональными требованиями.
- *Пользовательское тестирование*: эти тесты проводятся в пользовательской среде для проверки соответствия всем требованиям пользователя.

V-модель может использоваться для разработки небольших проектов программного обеспечения. Каждый этап обеспечивает конкретные результаты, и проблемы с дизайном могут быть обнаружены на ранней стадии. Модель, возможно, не очень удобна для больших и сложных проектов с недостаточно четкими требованиями.

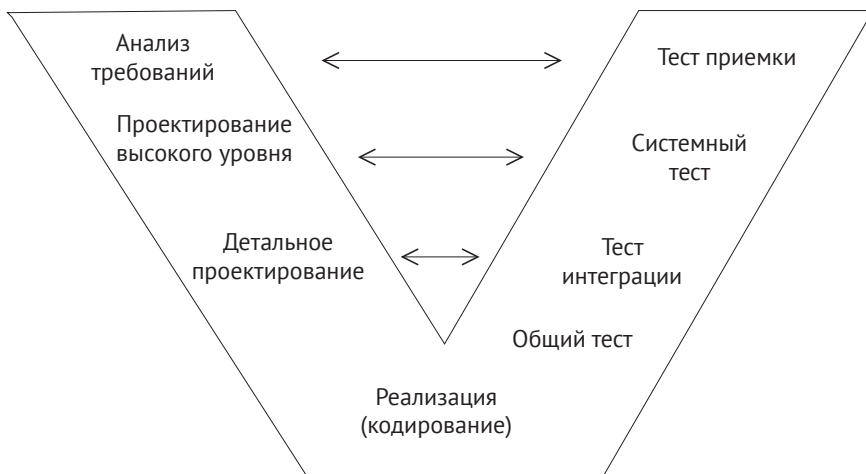


Рис. 10.2. V-модель

10.2.3. Спиральная модель Spiral Model

По сравнению с другими моделями разработки программного обеспечения спиральная модель значительно более абстрактна. Она подчеркивает значение

анализа рисков и включает четыре этапа: анализ проблемы, анализ, реализация и планирование рисков. Развитие проходит через эти этапы итерациями, называемыми спиралями. Совершив итерацию спирального цикла, эти четыре этапа реализуются, как показано на рис. 10.3.

Первой фазой является определение целей и требований, за которым следует анализ рисков, где определяются риски и ищутся методы их уменьшения. Текущее состояние разработки реализуется на третьем этапе, а планирование делается на заключительном этапе. Например, один полный цикл может быть посвящен спецификации требований в классическом смысле с проектированием высокого уровня на втором.

Этот метод обычно используется в проектах со средней и высокой степенью риска, а также когда у клиента нет уверенности относительно требований. Также требования к системе могут быть сложными, и может потребоваться их более точная оценка. Пользователь системы может оценить поведение системы на более ранней стадии, чем при использовании модели Waterfall. Однако управление этапами в этой модели является сложным, и сроки завершения проекта определить затруднительно.



Рис. 10.3. Спиральная модель

10.3. РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ СИСТЕМ РЕАЛЬНОГО ВРЕМЕНИ

Разработка программного обеспечения для систем реального времени со встроенными элементами отличается от его разработки для систем общего применения в нескольких отношениях. Прежде всего в системах реального времени первостепенное значение имеют безопасность и надежность, и это

влияет на выбор метода проектирования в целом. Соблюдение сроков – следующая требующая решения проблема. Разработка программного обеспечения системы реального времени со встроенными элементами должна включать следующее [9]:

- *выбор платформы*: выбор операционной системы реального времени и аппаратного обеспечения оказывает существенное влияние на производительность системы и поэтому должно осуществляться на самых ранних этапах проектирования;
- *анализ ввода/отклика и синхронизации*: должны быть указаны входы и ответы на эти входы с требуемым временем ответа. Затем могут быть определены конечные сроки выполнения задач. Этот анализ может быть частью спецификации требований;
- *задачи*: ввод с датчиков и вывод на исполнительные механизмы, а также задачи, которые запускают алгоритмы, осуществляющие вычисления между вводом и выводом, выполняются как параллельные задачи;
- *планирование задач*: должно быть обеспечено соблюдение конечных сроков выполнения задач.

Имея в виду перечисленные проблемы, для разработки программного обеспечения систем реального времени могут применяться все вышеперечисленные модели Waterfall, Spiral Model и V-модель.

10.4. АНАЛИЗ ТРЕБОВАНИЙ И СПЕЦИФИКАЦИЯ

Цель этого этапа разработки программного обеспечения – уточнить требования к системе посредством обсуждения с пользователем и клиентом и документировать их. *Пользовательские требования* определяют услуги, предоставляемые системой, а *системные требования* являются спецификациями выполняемых функций [8]. Этот этап разработки программного обеспечения состоит из двух этапов: *анализа требований* и *спецификации требований*. Данные, необходимые для анализа требований, поступают от пользователя/клиента, а анализ делается посредством их обсуждения. В требованиях пользователя/клиента могут быть неоднозначности и противоречия, и они должны быть разрешены на этапе анализа.

Когда анализ закончен, требования формализуются как спецификация требований к программному обеспечению (software requirement specification, SRS), иногда называемая документом анализа требований (requirement analysis document, RAD), который содержит функциональные требования, нефункциональные требования и цели системы. Эта задача выполняется *системным аналитиком*, который понимает и определяет точные требования клиента/системы. Он будет собирать данные, удалять все несоответствия и аномалии, такие как противоречивые запросы, организовывать и формировать документ SRS. Этот документ служит договором между пользователем и разработчиком

системы, а также выступает как справочный документ и определяет дизайн системы на высоком уровне. Разработанный продукт принимается, когда он соответствует всем требованиям документа SRS. Этот документ определяет, что должно выполняться системой, не затрагивая вопроса, как достичь этих требований при проектировании. Документ SRS должен быть написан с использованием терминологии, понятной пользователю, но с указанием точных требований. Типичный документ SRS состоит из следующих разделов:

- 1) *введение*: обычно в эту часть включены обзор системы, ее области применения, ссылки на существующие системы, цель проекта и критерии успеха проекта;
- 2) *функциональные требования*: в этой части описывается функциональность высокого уровня системы;
- 3) *нефункциональные требования*: в эту часть включаются требования пользователя, такие как надежность и производительность, которые не связаны напрямую с функциональностью системы;
- 4) *гlossарий*: содержит словарь терминов, которые должны быть понятны пользователю.

10.5. ВРЕМЕННОЙ АНАЛИЗ

Временной анализ системы реального времени должен учитывать выполнимость периодических, аperiodических и спорадических задач. Аperiodическое и спорадическое выполнение задач нельзя предсказать заранее, однако можно выполнить их вероятностный анализ. При анализе сроков выполнения требований периодических задач необходимо рассмотреть следующие вопросы:

- *характеристики задачи*: к ним относятся срок выполнения задачи, время и период выполнения. Относительно более простым временной анализ системы реального времени становится, когда задачи являются независимыми, периодическими и не разделяют ресурсы;
- *зависимости задач*: должен быть проанализирован порядок выполнения независимых задач. Если задача τ_i предшествует другой задаче τ_j , то задача τ_i должна завершиться до того, как может начаться задача τ_j . Отношения между задачами можно отобразить в графе задач, и мы видели в главе 8, что алгоритмы планирования задач должны быть изменены, когда задачи зависимы;
- *совместное использование ресурсов*: когда задачи используют ресурсы совместно, они могут быть заблокированы, и это повлияет на их время отклика. Также может возникнуть проблема инверсии приоритета, что приводит к тому, что задача с более низким приоритетом блокирует выполнение задачи с более высоким приоритетом. В главе 8 мы рассмотрели, как протоколы приоритетного наследования могут предотвратить эту ситуацию.

Таким образом, временной анализ систем реального времени, которые имеют зависимые задачи и совместно используют ресурсы, сложен, однако этот анализ необходим, чтобы иметь возможность предсказать поведение системы, выполнить приемочные тесты для спорадических задач и найти подходящие временные интервалы для аperiодических и спорадических задач, не нарушая конечных сроков выполнения периодических задач.

10.6. СТРУКТУРНОЕ ПРОЕКТИРОВАНИЕ С ДИАГРАММАМИ ПОТОКОВ ДАННЫХ

Диаграмма потока данных (data flow diagram, DFD) программного обеспечения системы отображает поток данных через различные модули системы. DFD состоит из компонентов, изображенных на рис. 10.4, где *процесс*, показанный кружком, является компонентом, который выполняет системную функцию, а компонент памяти данных – это место для хранения данных совместного использования процессами. Обычной практикой является маркировка памяти данных именами данных, поступающих в память и выходящих из памяти. *Внешние объекты*, или *терминаторы*, показаны прямоугольниками, а *потоки данных* между различными компонентами изображаются направленными дугами. Поток данных между терминаторами и системой определяет интерфейс системы с внешним миром.

DFD-метод представления системы используется главным образом в связи с его простотой и возможностью обеспечить много уровней иерархии. Очень простая диаграмма, изображающая систему лишь в виде круга и внешних объектов, может быть расширена, и системный цикл представлен в виде другой DFD, состоящей из различных процессов и потоков данных между ними. Первая диаграмма на уровне 0 называется *контекстной диаграммой*, или *диаграммой уровня 0*. Поток данных, показанный дугой, имеет метку данных, которые он передает в DFD. Память данных также имеет имена для определения хранящихся в ней типов данных. Словарь данных имеет список всех меток, используемых в DFD, а также объявление любых возможных составных элементов данных в терминах компонентов, которые они содержат.

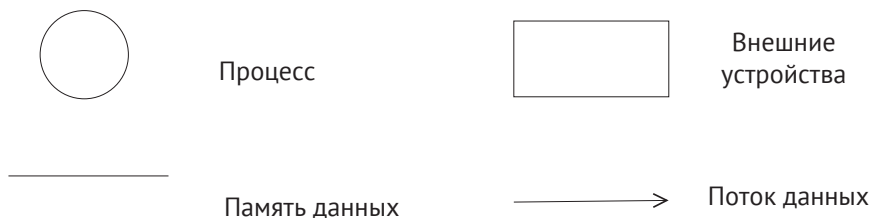


Рис. 10.4. Компоненты DFD

Контекстная диаграмма типичной системы реального времени изображена на рис. 10.5. Внешняя система состоит из датчиков, исполнительных механизмов, интерфейса оператора и дисплея. Датчики и интерфейс оператора обеспечивают входы в систему в терминах данных и команд/данных соответственно, а выход системы направлен к исполнительным механизмам и дисплею.

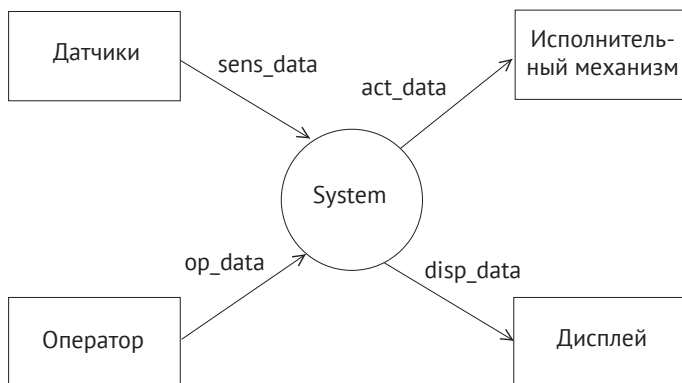


Рис. 10.5. Диаграмма DFD типичной системы реального времени

Давайте рассмотрим простой пример, где системой контролируется температура окружающей среды. Основные функциональные требования будут следующими:

- 1) периодически вводятся данные датчика, и проверяется, находится ли он в указанном диапазоне. Если нет, отображается тревога и статус;
- 2) если оператор хочет выполнить какое-либо действие, активируется требуемая процедура.

Возможная реализация этой системы с использованием диаграммы DFD может быть отображена путем уточнения контекстной диаграммы, показанной на рис. 10.5, диаграммой уровня 1, показанной на рис. 10.6. Круг, обозначенный как система в контекстной диаграмме, теперь отображается большой пунктирной окружностью.

Внутри его четыре задания: *Sense_In* для ввода данных датчика, *Op_In* для ввода команд оператора, *Act_Out* для вывода выходных данных исполнительного механизма и задача управления *Control*, которая является основным контроллером. Два хранилища памяти данных это *all_data* для хранения всех входящих данных и *control_data* для задачи *Act_Out*. Обратите внимание, что маркировка данных контекстной диаграммы должна быть сохранена. Места хранения памяти данных могут быть реализованы с помощью очередей или почтовых ящиков, и задачи могут быть смоделированы FSM для реализации потоками POSIX с использованием практического метода разработки программного обеспечения, который будет описан в разделе 10.10.

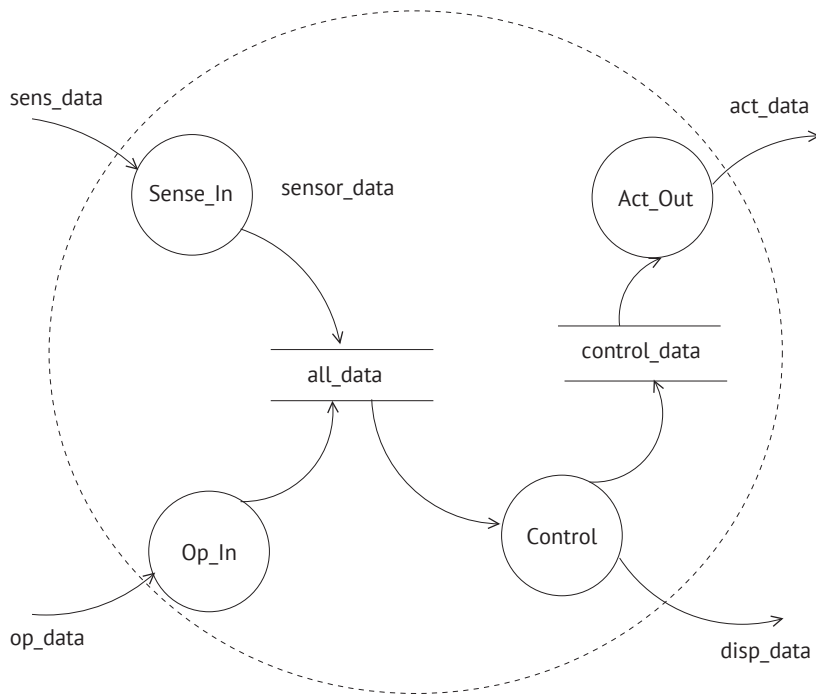


Рис. 10.6. Диаграмма DFD типичной системы реального времени

Диаграммы DFD практичны и просты, однако недостатком этого метода является отсутствие какой-либо структуры управления. Кроме того, нет никаких формальных методов для последовательного формирования более тонких диаграмм путем разложения функции на ее подфункции и перехода к окончательной диаграмме для начала детального проектирования. Эта проблема решается подходящим образом (ad hoc) для конкретного случая на основе опыта аналитика.

10.7. ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОЕКТИРОВАНИЕ

Объектно-ориентированное проектирование (object-oriented design, OOD) разработано для устранения некоторых проблем, которые встречаются в процедурном проектировании, таких как большие затраты на разработку и неадекватное управление неисправностями. В этом методе проблема разлагается на сущности, называемые *объектами* и *функциями*, определенными на этих объектах. *Класс* определяет содержание различных отличительных признаков, таких как процедуры и данные логических единиц. Система программного обеспечения может быть определена как структура взаимодействующих объектов. OOD опирается на *абстракцию*, которая фокусируется на том, что делает объект, а не на том, как это делается. Например, *резать объект ножницами*

и есть объект вилкой – это аналогии OOD в повседневной жизни. Функции, вызываемые методами, специфичны для объектов. Класс представляет собой набор объектов аналогичных типов и является общим определением объекта, а объект является элементом класса. Определение класса характеризует атрибуты объекта и методы, которые могут быть применимы к объекту. Объединение атрибутов и методов в определении класса называется *инкапсуляция*, она обеспечивает ограниченный доступ к данным и методам объекта. Техника *наследования* в объектно-ориентированном проектировании обеспечивает иерархическое получение переменных и методов из суперкласса. *Производный класс* разделяет атрибуты с классом, которым он произведен. Новый класс может быть сформирован путем добавления дополнительных атрибутов существующему классу, что обеспечивает возможность повторного использования и является еще одной благоприятной особенностью OOD. Одно и то же имя функции может использоваться для реализации различных операций с использованием разных данных в *полиморфизме*. Таким образом, основные преимущества использования OOD заключаются в следующем:

- безопасные программы могут быть разработаны с использованием *сокрытия данных (data hiding)*;
- наследование и модульность обеспечивают возможность многократного использования и простоту программирования;
- программный проект может быть легко разделен на несколько групп;
- можно эффективно управлять большим и сложным программным обеспечением.

10.8. МЕТОДЫ РЕАЛИЗАЦИИ В РЕАЛЬНОМ ВРЕМЕНИ

Детальное проектирование программного обеспечения в реальном времени может быть выполнено с использованием нескольких надежных методов. Основным требованием этих методов является то, что они должны охватывать все возможные сценарии, возможно, обеспечивать некоторый способ включения в них времени и легкость перевода методологии в реальный код. Мы рассмотрим три таких метода, которые обычно используются в детальном проектировании программного обеспечения в реальном времени: конечные автоматы, сети Петри и унифицированный язык моделирования.

10.8.1. Еще о конечных автоматах

Мы кратко рассмотрели модель конечного автомата (FSM) применительно к системам реального времени в главе 3. Автомат FSM состоит из нескольких состояний и переходов между этими состояниями и представляется ориентированным графом (диграф, digraph). FSM могут быть недетерминированными или детерминированными. Детерминированный FSM имеет ровно один переход для любого входа, а недетерминированный может иметь один или несколько переходов или не иметь их для данного входа.

В этом тексте мы рассмотрим только детерминированные автоматы. Детерминированный автомат FSM представляется шестью кортежами: I – набор конечных входов, O – набор конечных выходов, S – набор конечных состояний, $S_0 \subset S$ – начальное состояние, δ – следующая функция состояния: $I \times S \rightarrow S$, λ – функция выхода.

Диграф, называемый *диаграммой конечных состояний* (*finite-state diagram*, FSD), служит наглядным пособием работы автомата. Состояния, показанные кружками, являются взаимоисключающими, и система в любое время может находиться только в одном из состояний. Появление ввода на входе или изменение значения некоторых параметров часто является причиной события, которое запускает переход между двумя состояниями автомата. Изменение состояния может привести к определенному выводу, который указан в метке дуги перехода. Например, такая метка, как x/y на дуге между состояниями A и B , означает, что когда вводится x , вырабатывается выход y и переход в состояние B , как показано на рис. 10.7. Один и тот же вход x может вызвать другой выход z , оставаясь в том же состоянии, как показано на этом рисунке.

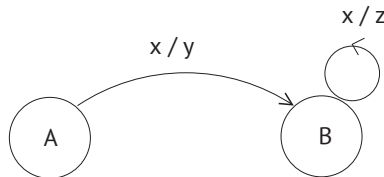


Рис. 10.7. Пример диаграммы FSM

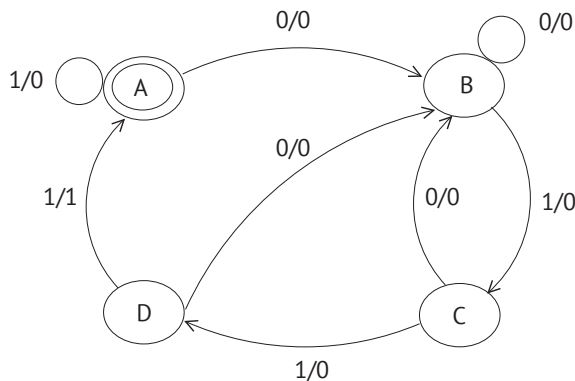


Рис. 10.8. FSM для распознавания образов

Два основных типа автоматов FSM – это автомат Мура (Moore FSM) и автомат Мили (Mealy FSM). В первом выход зависит только от состояния автомата, а во втором выход зависит как от входа, так и от состояния автомата.

Другими словами, автомат Мили создает выход, реагируя лишь на переходы, а не на состояния. Автомат FSM может использоваться для *распознавания образов*, обнаруживая появление предопределенного образа во входных данных. FSM, который обнаруживает двоичный вход 0111 и выводит 1, если такой вход обнаружен, показан на рис. 10.8. Автомат имеет четыре состояния, помеченных как *A*, *B*, *C* и *D* с *A* в качестве начального состояния. Входной набор *I* равен {0, 1}, а выходной набор *O* также равен {0, 1}. Для каждого состояния нам нужно определить следующее состояние и выход для любого входного значения. Обратите внимание, что неполучение третьей 1 возвращает автомат в состояние *B*, а не в исходное состояние *A*, так как мы уже есть первый 0 образа.

Конечные состояния для этого примера будут иметь состояния для всех возможных входов следующих состояний и выходов, как показано в табл. 10.1.

Таблица 10.1. Таблица FSM для согласования образов

Входы	0		1	
	Следующее состояние	Выход	Следующее состояние	Выход
<i>A</i>	<i>B</i>	0	<i>A</i>	0
<i>B</i>	<i>B</i>	0	<i>C</i>	0
<i>C</i>	<i>B</i>	0	<i>D</i>	0
<i>D</i>	<i>B</i>	0	<i>A</i>	1

10.8.1.1. Параллельные иерархические конечные автоматы

Основной проблемой детального проектирования на основе FSM являются сложность определения и реализация FSM, когда число состояний велико. Количество переходов равно $O(n^2)$, если n – число состояний, что вызывает значительные трудности в визуализации и реализации автоматов. На практике многие состояния FSM имеют тенденцию быть аналогичными, и способ повторного использования этих состояний имеет преимущества.

Диаграммы состояний, предложенные Харелем (Harel) [4], обеспечивают вложение состояний для решения этих проблем FSM. В этой модели, также известной как иерархические автоматы, состояние автомата S_1 можно разбить на несколько подсостояний, например S_{11} , ..., S_{1k} . Состояние S_1 называется *суперсостоянием*, и любое событие, которое не может быть обработано, – *подсостоянием* и доставляется к суперсостоянию на следующем уровне вложенности. Подсостояния наследуют свойства суперсостояний и определяют отличия от них, обеспечивая тем самым повторное использование аналогичных состояний. Иерархический FSM изображен на рис. 10.9, где состояние *B* является суперсостоянием, а состояния B_{11} и B_{12} – подсостояниями.

Параллельный иерархический автомат FSM (*concurrent hierarchical FSM*, CHF-SM) имеет несколько FSM, работающих параллельно, каждый из которых может быть иерархическим FSM. Модель CHF-SM может быть использована для моделирования распределенной системы реального времени.

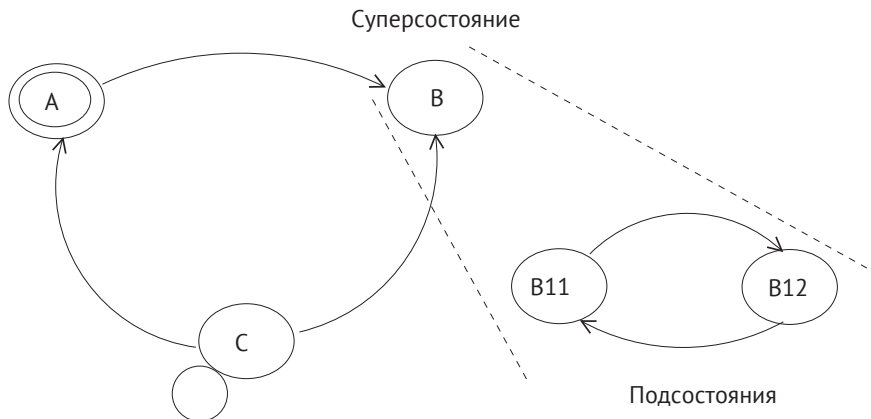


Рис. 10.9. Иерархический конечный автомат

10.8.2. Временные автоматы

Временной автомат (timed automaton, ТА) является автоматом конечного состояния с добавленными часами реального времени [1], другими словами, ТА содержит набор часов, которые увеличивают FSM. В дугах перехода диаграммы FSM теперь содержатся значения часов, которые могут сравниваться с некоторыми другими значениями, чтобы включить переход. Все часы увеличивают время в одном темпе, чтобы представлять глобальный прогресс ТА. Значение часов может быть протестировано или сброшено, но часам нельзя присвоить значение. На рис. 10.10 показан ТА с двумя часами x и y . Чтобы выполнить переход в состояние B , выполнив действие a и сбросив x , значение часов должно быть больше или равно 5. Точно так же возврат к состоянию A включается, когда часы показывают $y = 3$, выполняя действие b и сбрасывая y .

Модель сети временных автоматов (network of timed automata, NTA) состоит из набора параллельных ТА с синхронизацией между ними [2]. Используя этот подход, можно анализировать распределенную систему в режиме реального времени. Каждый узел такой системы является ТА, который связывается и синхронизируется с другими ТА по сети.

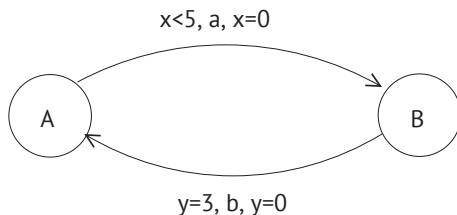


Рис. 10.10. Пример временного автомата

10.8.3. Сети Петри

Сети Петри (Petri nets, PN) предоставляют математический инструмент для моделирования совместно действующих и распределенных систем [6]. Простая сеть Петри – это кортеж из пяти элементов $PN = (P, T, F, W, M_0)$, где

- $P = \{p_1, p_2, \dots, p_n\}$ – конечное множество мест, представленных кружками;
- $T = \{t_1, t_2, \dots, t_m\}$ – конечное множество переходов, представленных полосоками с $P \cup T \neq \emptyset$ и $P \cap T = \emptyset$;
- $F \subseteq (P \times T) \cup (T \times P)$ – конечное множество дуг;
- $W: F \rightarrow \mathbb{N}^+$ – весовая функция.

Поток токенов среди мест показывает динамическое поведение PN. Компоненты PN изображены на рис. 10.11, где дуги направлены между местоположением и переходом и переходом и местоположением. Дуга имеет ассоциированный с ней вес и пропущенный вес, означающий одинаковость. Состояние PN определяется распределением мест токенов.

PN работает в соответствии с правилами перестрелки (firing), которые формулируются следующим образом.

1. *Возможность перехода*: переход возможен, если каждое местоположение ввода помечено токенами, имеющими, по крайней мере, вес перехода. Такой переход может или не может включаться.
2. *Запуск (fire) перехода*: когда переход запущен, токены, равные его весу, удаляются из каждого из его мест входа и добавляются к его местам выхода. Запуск имеет *неделимый характер (atomic)* – происходит включение одновременно только одного перехода.

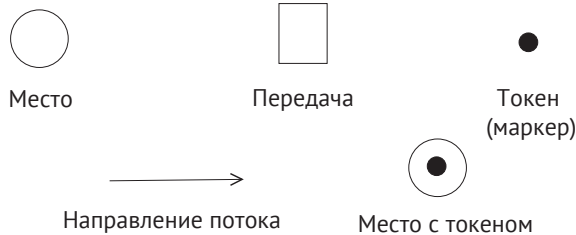


Рис. 10.11. Компоненты сети Петри

Переход без какого-либо ввода, называемый *переходом источника (source transition)*, может срабатывать в любое время и выдает токен каждый раз при срабатывании, как показано на рис. 10.12а. Переход без вывода, называемый *приемником (sink)*, может срабатывать при включении и потреблять входной токен, когда переход запущен, как показано на рис. 10.12б. Сети Петри недетерминированы; два перехода с одним вводом, имеющим токен, конкурируют за обладание одним и тем же токеном и следующим переходом, который будет запущен произвольно. Эта ситуация иллюстрируется на рис. 10.12с, где может быть запущен или переход t_1 , или переход t_2 .

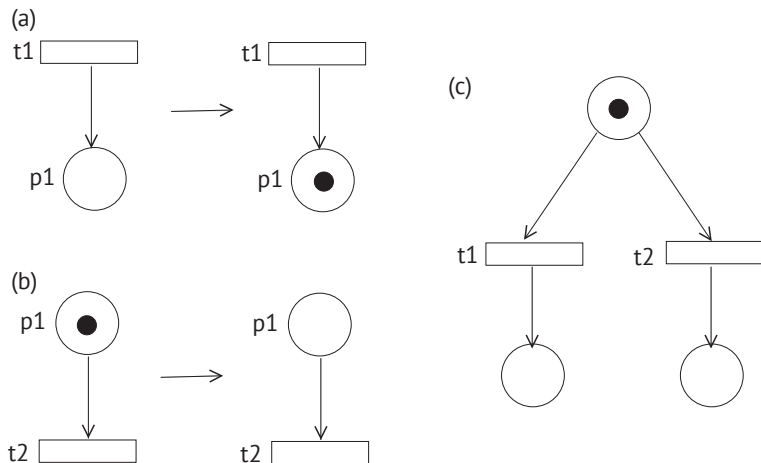


Рис. 10.12. Переходы в сети Петри

Присвоение токенов местам PN называется *маркировкой*. Моделирование программного обеспечения системы посредством PN обеспечивается токенами, представляющими ресурсы, местами, представляющими состояния или условия, и переходами, представляющими события или преобразования. PN с тремя местами p_1 , p_2 и p_3 и один переход t_1 показаны на рис. 10.13а. Переход t_1 включен, так как входящие в него дуги имеют места ввода, по крайней мере с весами дуг. Когда срабатывает переход, мы имеем ситуацию рис. 10.13б, где два токена размещены в месте вывода p_3 перехода t_1 .

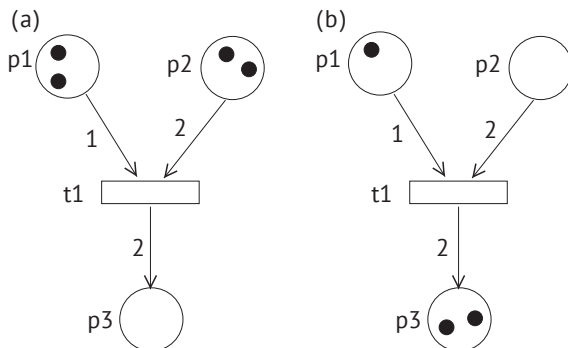


Рис. 10.13. Переходы в сети Петри

Сеть Петри со всеми дугами, имеющими единичные веса, называется обычной сетью Петри, и она отображается без весов на дугах. Обычная PN без циклов с начальной маркировкой одного токена представляет собой обычный конечный автомат. PN может находиться в одном из следующих состояний:

- *текущее состояние*: текущая маркировка, называемая также конфигурацией токенов на местах. Например, текущее состояние PN, как показано на рис. 10.13а для (p_1, p_2, p_3) , есть $(3, 2, 0)$;
- *достижимое состояние*: состояние, которое может быть достигнуто из текущего состояния путем запуска последовательности разрешенных переходов;
- *состояние запрета (тупика)*: состояние, в котором переход не разрешен.

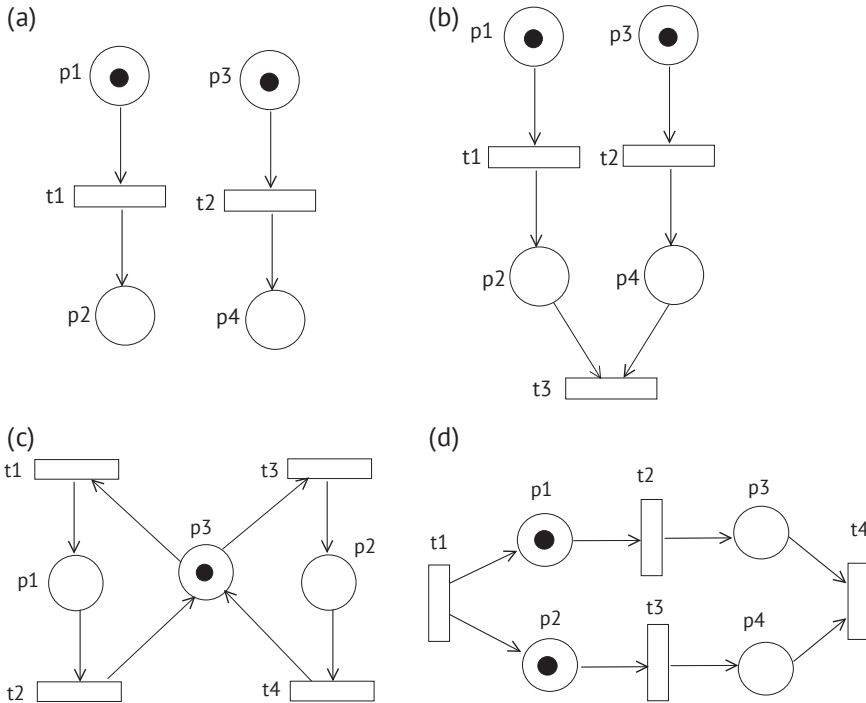


Рис. 10.14. Различные режимы работы в сети Петри

Ясно, что необходимо гарантировать абсолютное отсутствие тупиковых состояний. Сети PN могут быть использованы для моделирования взаимодействующих и распределенных систем, как показано на рис. 10.14а, где присутствует два последовательных независимых процесса. Когда для завершения работы системы есть необходимость в такой синхронизации, при которой должны завершиться оба процесса, используется переход t_3 , который активируется, когда оба процесса завершаются, как показано на рис. 10.14b. Конфигурация сети PN, показанная на рис. 10.14с, может использоваться для взаимного исключения, поскольку включен или переход t_1 , или переход t_3 , но не оба. Последний рисунок показывает, как в сети PN может быть достигнута параллельная обработка между переходами t_1 и t_4 .

С использованием сети PN может быть реализован протокол связи. Рисунок 10.15 иллюстрирует реализацию с помощью PN базового протокола «Остановись и жди» (Stop-and-Wait), в котором отправитель отправляет сообщение и ожидает подтверждения перед отправкой следующего сообщения. Получатель ожидает сообщения и отправляет сообщение подтверждения отправителю, когда он получает сообщение правильно.

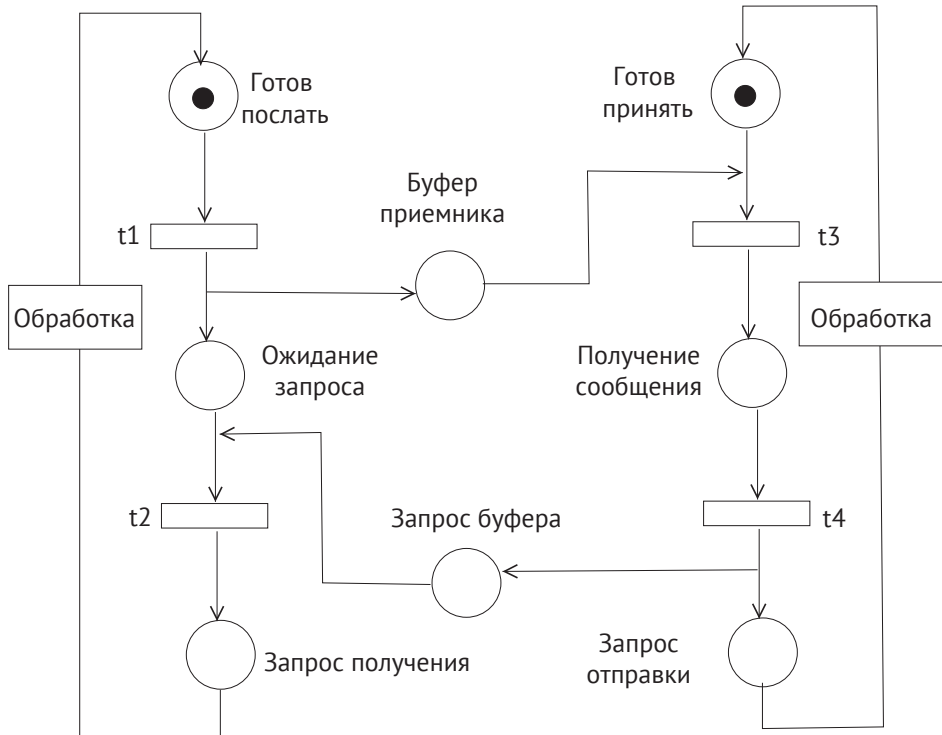


Рис. 10.15. Выполнение сетью PN протокола «Остановись и жди»

10.8.3.1. Временные сети Петри

Сети PN высокого уровня могут быть сформированы путем добавления к ним цвета, времени и иерархии. *Временная сеть Петри (Timed Petri Nets, TPN)* имеет $t_{\min}$ и $t_{\max}$, связанные с каждым переходом, определяющим самый ранний и последний переход, когда переход возможен [5, 10]. Возможный переход может запуститься, если значение его часов находится в интервале $[t_{\min}, t_{\max}]$. Сеть TPN с тремя местами и тремя переходами с интервалами изображена на рис. 10.16.

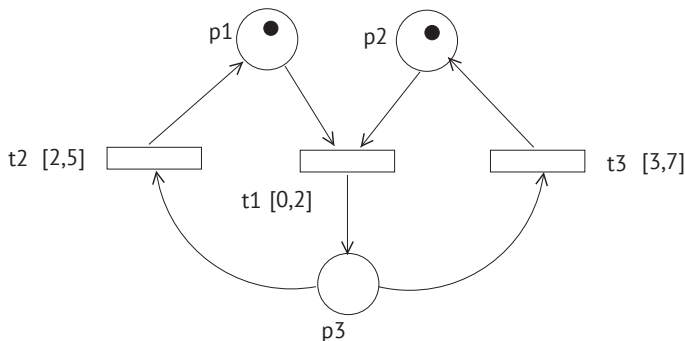


Рис. 10.16. Пример сети TRN

10.9. УНИФИЦИРОВАННЫЙ ЯЗЫК МОДЕЛИРОВАНИЯ В РЕАЛЬНОМ ВРЕМЕНИ

Унифицированный язык моделирования в реальном времени (*Unified Modeling Language, UML*) – это графический язык, который помогает анализу, дизайну, разработке и реализации программного обеспечения [3]. Обычно при разработке больших программных систем используется объектно-ориентированное проектирование (OOD). Системы реального времени требуют своевременного ответа, профилактики неисправностей, обеспечения безопасности и надежности, а также качества сервиса. UML – это язык, учитывающий наличие этих проблем.

10.9.1. UML-диаграммы

UML воплощает различные типы диаграмм для определения программных систем. UML-диаграммы можно классифицировать как структурные и поведенческие диаграммы. Структурная диаграмма показывает, какие типы диаграмм должна содержать система. Мы перечислим только основные типы диаграмм UML. Они приведены ниже.

- *Диаграмма классов*: показывает отношения между классами, используемыми в системе. Операции и атрибуты класса могут быть частными, общедоступными или защищенными в соответствии с принципами OOD. Диаграмма классов показывается в виде прямоугольника с тремя полями, разделенными горизонтальными линиями: наименование класса, атрибуты класса и методы класса.
- *Диаграмма состояний*: показывает конечный автомат для моделирования динамического поведения объекта.
- *Диаграмма объектов*: объект является экземпляром класса. Диаграмма объектов показывает взаимодействие между объектами.
- *Диаграмма компонентов*: показывает зависимости между программными компонентами, такими как библиотеки программного обеспечения.

- *Диаграмма составной структуры*: отображает внутреннюю структуру классификатора (класс, компонент или вариант использования) и взаимодействие с другими компонентами системы.
- *Диаграмма развертывания*: показывает архитектуру системы, отображающую, как система будет физически установлена в аппаратурной среде.
- *Диаграмма пакетов*: показывает, как компоненты организованы в пакеты, которые являются логическими группами программных компонентов.
- *Диаграмма взаимодействия объектов*: отображает взаимодействия между объектами системы.

Основные поведенческие диаграммы отображают активность системы и показаны ниже.

- *Диаграмма действий*: показывает внешнее окружение системы, ее связи и взаимодействия с внешним миром. Это помогает отображать функциональные требования к системе с точки зрения пользователя. Она состоит из *сценариев использования*, *актеров*, и *отношений*, таких как зависимость и связь между ними, как показано на рис. 10.17. Актер представляет любую сущность вне системы, такую как человек, группа людей или машина. Возможная схема использования торгового автомата изображена на рис. 10.18.
- *Диаграмма активности*: используется для демонстрации динамического поведения системы как процедурного потока между объектами. Состояния в диаграмме активности являются функциями. Они могут быть использованы для моделирования одновременного выполнения задач.
- *Схема конечного автомата*: отображает поток управления в системе, как в модели FSM.
- *Диаграмма связей/совместной работы*: этот тип диаграмм отображает взаимодействие объектов системы.
- *Диаграмма последовательности*: отображает взаимодействие между объектами с акцентом на порядок сообщений. Диаграмма последовательности показывает поток для конкретного варианта использования путем отображения вызовов между различными объектами. Имеет вертикальную ось, показывающую сообщение и последовательность вызовов во времени, и горизонтальную ось, состоящую из общающихся объектов.

Использование объектов UML в жизненном цикле программного обеспечения можно описать следующим образом:

- *требования*: представлены диаграммами действий;
- *статическая структура*: представлена диаграммами классов;
- *поведение объекта*: представлено конечным автоматом, который показывает время жизни объекта;
- *взаимодействие с объектами*: представлены диаграммами активности, последовательности и совместной работы;
- *физическая реализация*: представлена программными модулями и их отображением на физические узлы системы.

Состояние в UML представляет собой продолжительность жизни объекта, в течение которого объект удовлетворяет некоторому условию, выполняет какое-либо действие или ожидает события. Состояние обычно ассоциируется с предикатом, который имеет значение *true*, когда состояние активно. Предикат может быть определен в терминах значений некоторых атрибутов класса или ссылки на другой объект. Состояния в UML изображаются в виде прямоугольников.



Рис. 10.17. Компоненты диаграммы действий

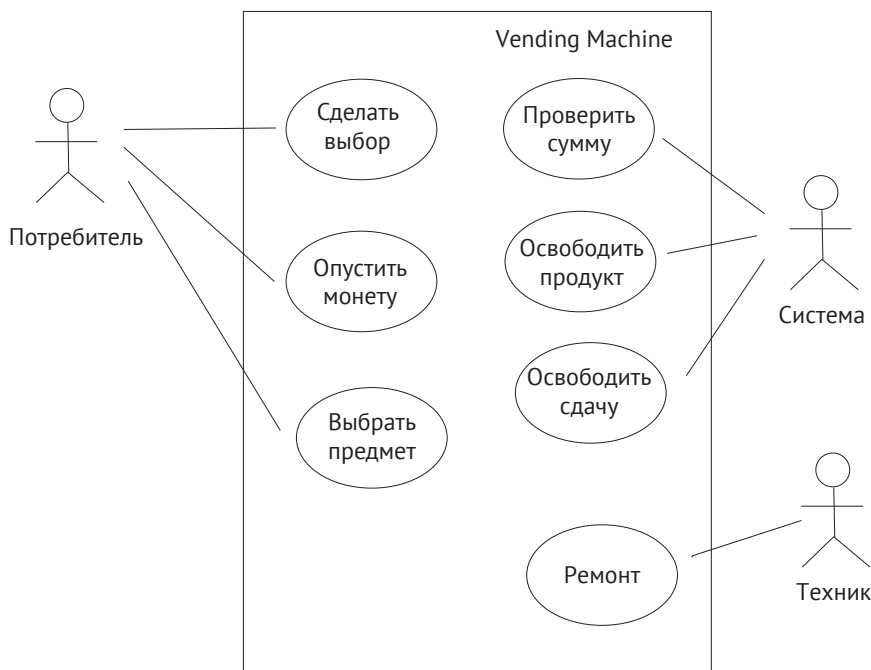


Рис. 10.18. Диаграмма действий торгового автомата

10.9.2. Функции реального времени

Диаграммы последовательности являются важной для реализации программного обеспечения в реальном времени особенностью UML. Они отобража-

ют взаимодействие между объектами, используя сообщения, и такой подход удобен для управляемых событиями систем реального времени. Временные метки могут быть использованы с диаграммами последовательности для определения временных ограничений. Диаграммы активности, которые обеспечивают спецификации параллельных задач, хорошо масштабируются для сложных систем – это еще одна особенность UML, которую можно использовать для проектирования в реальном времени. Конечные автоматы являются подходящими компонентами систем реального времени, управляемых событиями. UML обеспечивает систему таймерами, которые периодически отправляют временные события, и активный объект, который стимулируется в режиме состояний исключительно периодическим таймером, может представлять периодическую задачу системы реального времени.

UML – это отраслевой стандарт, используемый для анализа и проектирования объектно-ориентированных систем. Он использует сценарий использования, класс, конечный автомат, взаимодействие, последовательность, активность, компонент и схемы развертывания. Применение UML для проектирования систем в режиме реального времени имеет ряд преимуществ, включая инкапсуляцию, наследование и полиморфизм. Однако UML-стандарты обработки в реальном времени не являются законченными, планирование адекватно не поддерживается, и модели реального времени, использующие при разработке UML, могут оказаться сложными.

10.10. МЕТОД ПРАКТИЧЕСКОГО ПРОЕКТИРОВАНИЯ И РЕАЛИЗАЦИИ

Практичный современный метод проектирования и реализации для малых и средних систем реального времени может быть построен следующим образом.

1. *Требования:* перечислите требования к системе в соответствии с представлениями пользователя и создайте документ с описанием системных требований. Этот документ должен указать все сроки выполнения сложных задач в реальном времени.
2. *Операционная система:* выберите операционную систему реального времени (RTOS), которая подходит для приложения. RTOS с возможностями потоков POSIX будет полезна.
3. *Проект верхнего уровня 1:* проектирование системы с использованием метода диаграммы потоков данных (DFD) начиная с контекстной диаграммы на уровне 0. Эта диаграмма должна показать все внешние сущности и поток данных между этими сущностями и системой.
4. *Проект верхнего уровня 2:* рекурсивно детализируйте контекстную диаграмму до уровня 1 DFD и затем до уровня 2 и т. д., сохраняя маркировку данных. Продолжайте до тех пор, пока не появится схема, примерно соответствующая функции, которая должна быть реализована, с потоками входных и выходных данных.

5. *Анализ времени*: выполните анализ времени, и если должна быть разработана система DRTS, разработайте распределенное планирование, чтобы сроки выполнения задач были соблюдены.
6. *Детальный дизайн*: спроектируйте FSM для каждой функциональной схемы с состояниями и переходами между состояниями.
7. *Кодирование 1*: реализуйте каждый FSM как поток POSIX на языке C.
8. *Кодирование 2*: используйте любые существующие функции библиотеки POSIX для синхронизации потоков и связи между FMS.
9. *Кодирование 3*: используйте возможности RTOS для других функций, таких как управление памятью и обработка прерываний.

Эта процедура была успешно реализована в различных реальных приложениях. Мы увидим конкретный пример с использованием этого подхода и DRTK, как RTOS в главе 13. Укрупненные этапы данного метода изображены на рис. 10.19.

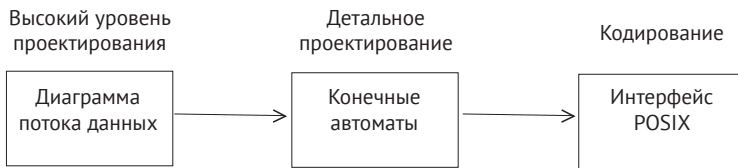


Рис. 10.19. Основные этапы практического метода проектирования

10.11. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы различия между разработкой программного обеспечения общей системы и системой реального времени?
2. В чем проблема классической модели Waterfall, и как ее можно преодолеть?
3. Какова основная характеристика V-модели?
4. Каковы результаты анализа требований разработки программного обеспечения?
5. Каковы основные типы программных проектов, где можно использовать модель Spiral Model?
6. Каковы основные положения, которые следует учитывать при временном анализе системы реального времени?
7. Каков основной принцип работы DFD?
8. Каковы основные преимущества использования объектно-ориентированной парадигмы в дизайне при реализации систем реального времени?
9. Каковы основные проблемы применения FSM, и как эти проблемы решаются в CHFSM?
10. В чем разница между автоматом FSM и автоматом ТА, и что такое автоматы NTA и сетевые синхронизированные автоматы?

11. Что такое местоположения и переходы в сети Петри, и какова функция токена?
12. Каковы основные диаграммы UML?
13. Какова функция диаграммы действий (сценариев) UML?

10.12. ПРИМЕЧАНИЯ К ГЛАВЕ

Мы описали основные концепции разработки программного обеспечения, относящиеся к системам реального времени. Хотя общие методы разработки программного обеспечения хорошо известны, какие-либо традиционные формальные методы разработки программного обеспечения систем реального времени, будь то однопроцессорные или распределенные системы, отсутствуют. Эти системы во многих случаях имеют встроенные элементы, что требует совместного проектирования аппаратного и программного обеспечения на ранней стадии процесса проектирования. Кроме того, во многих проектах обычно используется подход «снизу вверх».

Основными этапами разработки программного обеспечения являются анализ требований, проектирование, реализация, тестирование и обслуживание. Различные модели используются для разработки жизненного цикла программного обеспечения, основными из которых являются модель Waterfall, V-модель и модель Waterfall – это классическая модель, в которой результат заверченного этапа используется в качестве входа в следующий этап. Такой подход на практике проблематичен, поскольку для реализации данного этапа может потребоваться возврат к предыдущему этапу и его модернизация. Поэтому вводится усложненная модель Waterfall, обеспечивающая обратную связь между этапами. V-модель представляет процессы проектирования с левой ветви и процессы тестирования на правой ветви V-образного графа с реализацией внизу. Spiral Model состоит из циклов, каждый из которых проходит фазы общего анализа, анализа рисков, реализации и планирования. Этот метод может быть использован в сложных системах, в которых требования вначале могут быть недостаточно ясными.

Анализ требований обычно является первым этапом разработки программного обеспечения. Потребности клиента формируются в документе спецификации системных требований (system requirement specification, SRS), которая является основой для проектирования высокого уровня, а также контрактом между проектировщиком и заказчиком/пользователем. Методы проектирования высокого уровня могут быть классифицированы как процедурный и объектно-ориентированный. Диаграммы потока данных, структурные блок-схемы потоков являются широко используемыми инструментами для процедурного проектирования высокого уровня в инженерных проектах, а объектно-ориентированный унифицированный язык моделирования UML может использоваться для высокоуровневого и детального проектирования программного обеспечения.

Методы детального проектирования в системах реального времени обычно используют конечные автоматы FSM, синхронизированные временные автоматы TA и сети Петри. FSM состоит из состояний и переходов между этими состояниями. Временные автоматы – это в основном автоматы FSM, дополненные синхронизированными переходами. Сеть Петри состоит из мест и переходов, а временные сети Петри обеспечивают синхронизированные переходы между местоположениями. Все эти методы могут быть использованы для спецификации и выполнения детального проектирования систем реального времени. Распределенная система реального времени вносит еще один уровень сложности проектирования: анализ времени должен быть обработан в терминах сквозного соблюдения сроков и анализа сетевых задержек. UML – это объектно-ориентированная платформа для проектирования, анализа и реализации программного обеспечения системы. Он может быть использован для систем реального времени, обеспечивая ряд таких преимуществ, как инкапсуляция данных, наследование и полиморфизм, однако для целей проектирования прогнозируемых систем, планирования и функций моделирования в реальном времени метод UML пока нельзя считать адекватным и зрелым.

Наконец, мы показали проверенный, простой и практичный метод проектирования системы реального времени. Этап разработки требований спецификации выполняется не в режиме реального времени, проектирование системы на высоком уровне выполняется с помощью диаграмм DFD, а диаграммы уровней формируются рекурсивно, до тех пор, пока схема не становится соответствующей функции. Эта функция затем реализуется посредством FSM (или синхронизированного автомата), который может быть закодирован потоком POSIX. Таким образом, могут быть доступны все библиотечные функции, относящиеся к потокам POSIX, и, кроме того, используемая RTOS может обеспечить любую другую необходимую функцию, такую как обработка прерываний и управление часами. Программная инженерия является устоявшейся дисциплиной для системы не в реальном времени, и детальный анализ концепций разработки ее программного обеспечения может быть найден в [9] и [7]. Формализованные методы разработки программного обеспечения для систем реального времени все еще находятся на ранней стадии развития и требуют дальнейшей разработки.

10.13. ПРОГРАММИРОВАНИЕ ПРОЕКТОВ

1. Должно быть реализовано программное обеспечение для лифта, который перемещается между двумя этажами. Привлечь диаграммы DFD, начиная с контекстной диаграммы. Определить задачи и реализовать их, используя потоки POSIX.
2. В ДНК человека четыре типа нуклеотидов – это аденин (A), тимин (T), гуанин (G) и цитозин (C). В геноме человека нужно искать нуклеотид шаблона ACCGTA. Нарисуйте схему FSM и таблицу переходов состояний для обнаружения этого шаблона и напишите код на языке Си для реализации FSM.

3. Сравните модели FSM, Timed Automata и Petri Net в распределенной системе реального времени с точки зрения представления времени и реализации распределенной обработки.
4. Нарисуйте диаграмму прецедентов UML для терминала торговой точки (point of sale, POS) с клиентом, продавцом и POS-машиной.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Alur R., David L., Dill D. L. (1994) A theory of timed automata. Theor Comput Sci 126: 183–235.
2. Balaguer S. (2012) Concurrency in real-time distributed systems. PhD thesis, Laboratoire Specification et Verification.
3. Booch G., Rumbaugh J. E., Jacobson I. (1998) UML user guide. Addison Wesley.
4. Harel D. (1987) Statecharts: a visual formalism for complex systems. Sci Comput Program 8: 231–274.
5. Merlin P. M. (1974) A study of the recoverability of computing systems. PhD thesis, Department of Information and Computer Science, University of California, Irvine, CA.
6. Petri C. A. (1962) Kommunikation mit Automaten. PhD thesis, University of Bonn.
7. Pressman R. S. (2014) Software engineering: a practitioner's approach, 8th edn. McGraw-Hill Education.
8. Sommerville I. (2011) Software engineering, 9th edn. Addison Wesley.
9. Sommerville I. (2011) Software engineering, 9th edn. Addison Wesley (Chap. 20).
10. Zuberek W. M. (1991) Timed Petri Nets-definitions, properties, and applications. Microelectron Reliab 31 (4): 627–644.

Глава 11

Языки программирования в реальном времени

11.1. ВВЕДЕНИЕ

После выполнения высокоуровневых и детальных проектов следующим этапом разработки системы реального времени является кодирование, которое обычно выполняется на языке программирования в режиме реального времени. Такой язык характеризуется своими возможностями по временному управлению, синхронизации задач и связи, а также обработке исключений и планированию. Еще одно важное свойство при поиске такого языка – он должен позволять распределенную обработку. На первый взгляд, подобные услуги предоставляются на уровне операционной системы в режиме реального времени, рассмотренной в главе 4. Однако целесообразно расширить возможности уровня программирования, поскольку решение некоторых вопросов при разработке программного обеспечения в режиме реального времени предоставляется программисту и требует соответствующего опыта программирования в реальном времени. Кроме того, код будет переносимым на различные операционные системы, и его будет легче поддерживать, когда язык используется в реальном времени. С другой стороны, модель операционной системы может радикально отличаться от модели, используемой языком программирования, что сделает его применение затруднительным [2]. Язык ассемблера наиболее близок аппаратному оборудованию, и во многих случаях, чтобы получить прямой доступ к аппаратному обеспечению при программировании в реальном времени, программист должен писать ряд патчей на ассемблере.

Есть только несколько языков программирования высокого уровня в режиме реального времени. Языками, выбранным нами для обзора, являются C/Real-time POSIX, Ada и Java. Мы начнем эту главу с краткого описания языка C/Real-time POSIX, за которым следуют обзоры Ada и Java. Наше внимание при анализе этих языков обращено к описанным выше свойствам, которые делают их пригодными для применения в реальном времени.

11.2. ТРЕБОВАНИЯ

Как отмечалось, требования к языку программирования в реальном времени обязывают наличие дополнительных возможностей по сравнению с языком не в реальном времени. Общие требования к программированию в реальном времени, как и к языкам не в реальном времени, могут быть перечислены следующим образом:

- *управление модулями*: в крупных проектах необходима независимая разработка программного обеспечения. По этой причине удобное разделение программного обеспечения на модули является основным требованием для любого языка программирования, как в реальном времени, так и не в реальном времени;
- *инкапсуляция данных*: данные должны быть защищены от ошибочного использования. Объективно-ориентированная парадигма обеспечивает удобную инкапсуляцию данных.

Следующие свойства более важны в языке реального времени, чем в режиме нереального времени:

- *поддержка параллелизма*: как правило, система реального времени связана с асинхронными событиями во внешнем мире. Эти события требуют параллельной обработки, которая может быть реализована операционной системой или языком программирования, поддерживающим многозадачность. Кроме того, время выполнения задачи и анализ взаимосвязей могут выполняться в многозадачной системе для выполнения анализа планируемости, как мы видели в главе 4. Синхронизация между задачами и обмен данными, а также совместное использование ресурсов являются основными проблемами, которые должны решаться в параллельной системе. Языки программирования Ada и Java обеспечивают поддержку параллелизма, тогда как язык программирования Си с интерфейсом POSIX может использоваться для параллельных приложений;
- *поддержка ввода/вывода*: язык программирования должен обеспечивать механизмы для доступа к оборудованию ввода/вывода с использованием регистров и манипуляций на уровне оборудования;
- *управление временем*: время – самый ценный ресурс в системе реального времени, и язык программирования в реальном времени должен иметь средства для управления таймерами и удобного кодирования периодических и аperiodических задач;
- *поддержка планирования*: системы реального времени в большинстве случаев используют планирование на основе приоритетов; следовательно, язык программирования реального времени должен иметь средства для назначения задач приоритетов, а также некоторую поддержку планирования на основе приоритетов;
- *поддержка обработки исключений*. Обработка исключений необходима в любой системе программирования, как в режиме реального времени, так и в режиме нереального времени. Однако последствия отказа в системе

реального времени могут быть более серьезными, чем в системе нереального времени, и потому желательной особенностью языка программирования в реальном времени является эффективная обработка исключений.

Обратите внимание, что большинство этих свойств являются необходимыми в операционной системе реального времени. На основе описанных выше свойств мы дадим краткий обзор языков программирования C/Real-time POSIX, Ada и Java.

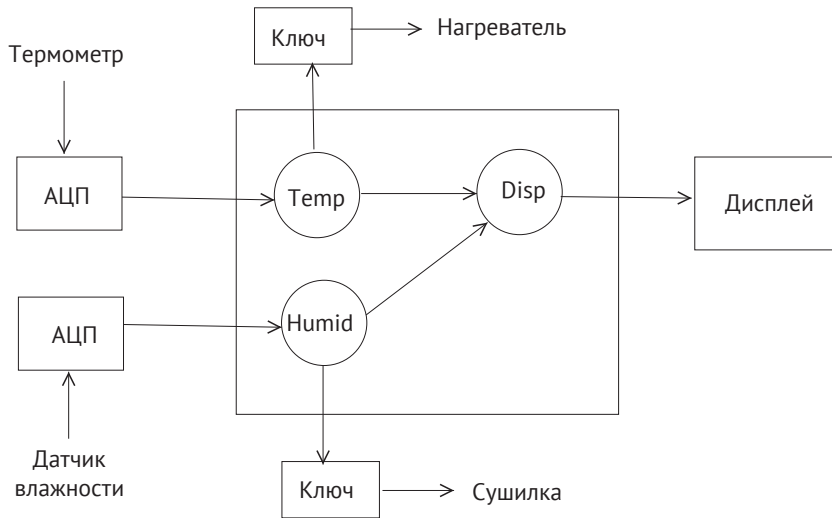


Рис. 11.1. Система управления процессом в реальном времени

11.3. ПРИЛОЖЕНИЕ В РЕАЛЬНОМ ВРЕМЕНИ

Мы опишем приложение в реальном времени, которое будем использовать для демонстрации того, как язык программирования может реализовывать необходимые функции. Приложение предназначено для мониторинга температуры и влажности в системе управления процессом. Есть два датчика для определения температуры и влажности окружающей среды. Оба входа поступают через аналого-цифровой преобразователь в интерфейс ввода компьютера реального времени, который выполняет три задачи: *контроль температуры*, *контроль влажности* и *отображение мониторинга*, как показано на рис. 11.1. Ключи предназначены для включения или выключения нагревателя либо сушилки.

11.4. ОПЕРАЦИОННАЯ СИСТЕМА C/POSIX В РЕАЛЬНОМ ВРЕМЕНИ

Язык программирования Си – это язык программирования общего назначения, который обычно используется для низкоуровневого программирования, написания кода операционной системы и встроенных системных приложений.

У него относительно небольшое количество ключевых слов по сравнению с другими языками программирования. Мы коснемся свойств языка C/POSIX, связанных с необходимыми атрибутами языка программирования реального времени.

11.4.1. Инкапсуляция данных и управление модулями

Модули в языке Си имеют структуру данных и обычно объявляются в файле заголовка с расширением «.h», а код в файле с расширением «.c» – так же, как мы это делали в различных модулях образца ядра операционной системы DRTK. Отдельная компиляция может быть более предпочтительной, когда программное обеспечение, которое должно быть реализовано, является большим и должно быть разделено между командами разработчиков программного обеспечения. Мы реализуем простой пример стека, который содержит целочисленные значения для описания инкапсуляции данных и отдельную компиляцию в Си. Файл заголовка *stack.h*, показанный ниже, определяет константы, связанные со стеком и структурой данных стека.

```

/*****
Структура стека данных
*****/
//файл stack.h

# define STACK_SIZE 1024
# define STACK_FULL -1
# define STACK_EMPTY -2
# define DONE 1

typedef struct {
    int state;
    int data[STACK_SIZE];
    ushort index;
}stack_t;

typedef stack_t *stack_ptr_t;

```

Две операции на стеке типа данных – это функции *push* и *pop*, находящиеся в файле *stack.c*, который включает файл заголовка *stack.h*.

```

/*****
Функции стека
*****/
// файл stack.c
#include <stack.h>
int push(stack_ptr_t stack_pt, int item){
    if (stack_pt->index == 0)
        return(STACK_FULL);
    stack_pt->data[stack_pt->index]=item;
    stack_pt->index++;
    return(DONE);
}

```

```
int pop(stack_ptr_t stack_pt){
    if (stack_pt->index == STACK_SIZE)
        return(STACK_EMPTY);
    stack_pt->index++;
    return(stack_pt->data[stack_pt->index]);
}
```

В файле *stack_test.c* мы имеем тестовую программу, которая вводит в стек 10 целых чисел от 1 до 10 и затем одну за другой извлекает из стека и печатает каждую составляющую данных.

```
/******
Основная программа
******/
// файл stack_test.c
#include <stdio.h>
#include "stack.h"
void main(){
    int i, data;
    stack_t stack_ex={0,{0},STACK_SIZE-1};
    stack_ptr_t spt=&stack_ex;
    for(i=1; i<=10; i++)
        if(push(spt,i) <0)
            exit(0);
    for(i=1; i<=10; i++)
        { if((data=pop(spt)) <0)
            exit(0);
          printf(" data retrieved: %d \\n",data);
        }
}
```

Мы можем отдельно компилировать исходные файлы языка C, чтобы тестировать наличие в нем ошибок. Финальным шагом является компиляция файлов в единственный исполняемый файл *stack_test*, показанный ниже.

```
gcc -c stack.c
gcc -c stack_test.c
gcc -o stack_test stack_test.o stack.o
```

11.4.2. Управление потоком POSIX

Мы уже привели несколько примеров с использованием потоков POSIX, поэтому будем перечислять только основные функции управления потоками интерфейса POSIX.

```
int pthread_attr_init(pthread_attr_t *attr);

int pthread_create(pthread_t *thread, const pthread_attr_t *att,
void *(*start_routine)(void *), void *arg);
```



```
int pthread_join(pthread_t thread, void **value_ptr);
int pthread_exit(void *value_ptr);
pthread_t pthread_self(void);
```

Функция *pthread_create* обычно вызывается из основного потока, возможно, с некоторыми входными параметрами. Основной поток ожидает потока, созданного вызовом *pthread_join*. Функция *pthread_exit* используется, когда поток завершает выполнение, и он также может передать адрес параметра потоку, ожидающему присоединения к нему, используя эту функцию. Последний вызов применяется, когда потоку необходимо узнать свой идентификатор. Заметим, что мы можем передать целое число потоку, которое может быть обозначено как его идентификатор во время его создания, отличное от идентификатора, который назначен потоку операционной системой. Назначение пользователем идентификатора потоку полезно, когда выполняется параллельная обработка нескольких потоков, и каждый поток работает с определенной частью данных на основе его идентификатора. В следующем фрагменте кода показано, как назначать идентификаторы и объединять потоки. Основной поток создает 10 потоков и передает индекс цикла в качестве его идентификатора для созданного потока. Результаты вычислений потока собираются в массиве *results*.

```
#include <pthread.h>
#define N_THREADS 10
pthread_t tid[N_THREADS];
int results[N_THREADS];

void Thread((void *)me)
{
    // произвести ряд вычислений
    // получить результат
    pthread_exit(&result);
}

void main() {
    int i;
    for ( i=1; i<=N_THREADS; i++)
        pthread_create(&tid[i], NULL, T, (void *)i);
    for ( i=1; i<=N_THREADS; i++)
        pthread_join(&results[i]);
}
```

11.4.2.1. Управление временем

Стандартным способом управления временем в языке C является использование данных *timespec*, определяемых следующим образом:

```
struct timespec {
    time_t tv_sec; // number of seconds
    long tv_nsec; // number of nanoseconds
}
```

Установка и получение времени осуществляются следующими функциями в интерфейсе POSIX:

```
int clock_gettime(clockid_t clock_id, struct timespec *tp);
int clock_settime(clockid_t clock_id, const struct timespec *tp);
int clock_getres(clockid_t clock_id, struct timespec *res);
```

где *clock_t* представляет тип используемых часов реального времени. Задержка задачи в RT-POSIX достигается посредством системных вызовов *sleep* или *nanosleep*.

```
unsigned sleep(unsigned seconds);
int nanosleep(const struct timespec *ts, struct timespec *tw)
```

где *ts* определяет интервал поддерживаемого потока. Когда спящий поток прерывается, его оставшееся время записывается в структуру, указываемую *tw* до тех пор, пока *tw* не становится NULL. Периодический поток может быть применен путем использования системного вызова *nanosleep*, как показано ниже, где период определен равным 80+ вычисления в миллисекундах.

```
void *thread(void *arg) {
    struct timespec period;
    period.tv_sec = 0;
    period.tv_nsec = 80 * 1000000; // 80 msec
    while(1) {
        // сделать ряд вычислений
        nanosleep(&period, 0);
    }
}
```

11.4.2.2. Синхронизация потоков и связь

Интерфейс C/POSIX предоставляет два метода управления параллелизмом: использование UNIX, управление процессами с межпроцессным взаимодействием и методами синхронизации и с помощью потоков POSIX. Мы подробно рассмотрели обе эти концепции в главе 4. В общем, в POSIX есть четыре метода синхронизации потоков интерфейса:

- сигналы;
- взаимное исключение;
- условная синхронизация;
- семафоры.

Мы кратко рассмотрим эти методы в следующих разделах с акцентом на их свойства в реальном времени.

11.4.2.3. Сигналы

Процессы UNIX для синхронизации используют сигналы. Процесс, который хочет получить сигнал, использует вызов *signal* следующим образом:

```
signal(int signum, sighandler_t handler);
```

где *signum* – номер сигнала, который нужно перехватить, а *handler* – адрес функции, которая вызывается при получении сигнала. Поле обработчика может быть установлено в SIG_IGN для игнорирования сигнала или SIG_DFL для действия по умолчанию. Отправка сигнала в указанный процесс выполняется функцией *kill*, как показано ниже:

```
int kill(pid_t pid, int signum);
```

где *pid* – идентификатор процесса получателя, а *signum* – номер сигнала. Следующий вызов может использоваться потоком для отправки сигнала другому потоку, являющемуся таким же процессом.

```
int pthread_kill(pthread_t thread, int signum);
```

Системный вызов *pause()* заставляет вызывающий процесс или поток ожидать сигнала. Вызовы *sigwaitinfo*, *sigtimedwait* и *sigwait* могут использоваться для ожидания, пока один из сигналов в указанном наборе принимается. В следующем примере показано, как ловить сигнал SIGALRM, генерируемый, когда время истекает.

```
#include <signal.h>

void my_handler(int sig) {
    signal(SIGALRM, my_handler);
    .... // сделать обслуживание прерывания
}

main(void) {
    signal(SIGALRM, my_handler);
    while(true) {
        alarm(20);
        .... // выполнить некоторую работу
    }
}
```

11.4.2.4. Взаимное исключение

Потоки могут иметь критичные разделы для манипулирования глобально доступными данными и доступом к этим критичным разделам и должны быть взаимноисключающими. Библиотека POSIX обеспечивает структурой данных *pthread_mutex_t*, которая может быть установлена при входе в критичный раздел и сброшена на выходе. Переменная мьютекса *m* должна быть объявлена первой, как показано ниже:

```
pthread_mutex_t m;
```

Структура данных мьютекса и функции, связанные с мьютексами, следующие:

```
int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_timedlock(pthread_mutex_t *mutex,
    const struct timespec *abstime);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Первая функция инициализирует созданный мьютекс, а две другие функции *pthread_mutex_lock* и *pthread_mutex_unlock* используются для блокировки и разблокировки мьютекса соответственно. Обратите внимание, что вызовы этих двух функций блокируют и разблокируют структуру данных, но не другие потоки. Основное правило заключается в том, что всякий раз, когда два или более потоков обращаются к одним и тем же глобальным данным, их доступ должен быть инкапсулирован путем блокировки и разблокировки переменной мьютекса, чтобы предотвратить состояние гонки. Вызов *pthread_mutex_trylock* пытается заблокировать мьютекс и выдает ошибку, если мьютекс уже заблокирован. Вызов *pthread_mutex_timedlock* также пытается заблокировать мьютекс и выдает ошибку, если мьютекс не может быть заблокирован в указанный интервал времени. Эти две подпрограммы могут использоваться задачами в реальном времени для предотвращения непредсказуемой блокировки.

11.4.2.5. Условная синхронизация

Условная переменная типа *pthread_cond_t* используется для синхронизации между потоками. Поток может ожидать условия при вызове *pthread_cond_wait*, чтобы быть разбуженным другим потоком с помощью вызова *pthread_cond_signal*, который сигнализирует об этом. Основные функции условий следующие:

```
int pthread_cond_init(pthread_mutex_t *mutex, NULL);
int pthread_cond_wait(pthread_mutex_t *mutex);
int pthread_cond_timed_wait(pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_mutex_t *mutex);
int pthread_cond_destroy(pthread_mutex_t *mutex);
```

Ожидание условия по времени удобно, когда поток в режиме реального времени может ждать конкретного условия только в течение ограниченного времени. Вызов *pthread_cond_destroy* используется для удаления условия из системы. Обратите внимание, что условная синхронизация выполняется с помощью переменных мьютекса.

11.4.2.6. Семафоры

Семафор в интерфейсе POSIX – это структура данных, которая может быть использована для взаимного исключения и синхронизации между потоками и между задачами. Мы рассмотрели POSIX реализации семафоров в главе 4, и потому будем перечислять здесь только основные семафорные операции.

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
```

```
int sem_getvalue(sem_t *sem, int *valpt);
int sem_destroy(sem_t *sem);
```

Семафор инициализируется системным вызовом *sem_init* с начальным значением, и переменная *pshared* указывает, будет ли она использоваться другими процессами, которые активны. Системный вызов *sem_wait* блокирует вызывающий, если ресурс недоступен, и вызов *sem_post* увеличивает значение семафора и разблокирует ожидание потока в очереди семафора. Последняя процедура просто удаляет семафор из системы.

11.4.3. Обработка исключений и низкоуровневое программирование

В интерфейсе C/POSIX нет явной обработки исключений, однако простой способ получения устойчивости к отказу может быть достигнут путем использования процедуры обработки исключений, когда вызов функции возвращает ошибку. Программирование на ассемблере интерфейса на языке Си представляется утилитой *pragma asm*, как показано в сегменте кода ниже. Выполняется инкапсуляция в блок команд на ассемблере, после чего команды C начинают выполняться.

```
extern void test();

void main( void ) {
    ....
    pragma asm
        JMP address
    pragma endasm
}
```

11.4.4. Реализация управления процессом C/POSIX в реальном времени

Теперь мы реализуем управление процессом, показанным на рис. 11.1, с использованием C/POSIX. Три задачи будут представлены потоками POSIX: задачи «Температура» (*Temp*) и «Влажность» (*Humid*), запускаемые по времени периодические задачи и задача «Дисплей» (*Disp*), которая управляется событиями. Поток *Temp* показан ниже; каждые 200 миллисекунд периодически возникает задача ввода и проверки данных температуры относительно нижнего предельного значения. Если температура ниже этого предела, задача *Temp* заставляет сработать ключ нагревателя.

```
/******
Задача «Температура»
******/
#define LOW_T_LIMIT 12
#define INTERVAL_T 0.2
```

```

typedef struct {
    int type;
    int alarm_cond;
    float value;
}data_t;
typedef struct{
    data_t temp_data;
    data_t humid_data;
}disp_data_t

disp_data_t disp_data;

void Temp(void *) {

    while(true) {
        sleep(INTERVAL);
        temp_value=Read_T();

        sem_wait(temp_sem);
        if (temp_value<LOW_T_LIMIT) {
            Turn_H_Switch();
            disp_data.temp_data.alarm_cond=ON;
        }
        disp_data.temp_data.value=temp_value;
        sem_post(disp_sem);
    }
}

```

Задача «Влажность» выполняется подобно задаче «Температура», как показано в коде ниже, с более низким предельным значением и величиной периода.

```

/*****
Задача «Влажность»
*****/
#define LOW_LIMIT 12
#define INTERVAL_H 0.1

void *Humid(void *) {

    while(true) {
        sleep(INTERVAL);
        humid_value=Read_H();
        sem_wait(temp_sem);
        if (humid_value<LOW_H_LIMIT) {
            Turn_D_Switch();
            disp_data.humid_data.alarm_cond=ON;
        }
        disp_data.humid_data.value=humid_value;
        sem_post(disp_sem);
    }
}

```

Задача «Дисплей» всегда ждет на своем семафоре и, когда активирована, находит источник данных, проверяет условия тревоги и затем отображает данные, как показано в коде ниже.

```

/*****
Задача «Дисплей»
*****/
#define LOW_LIMIT 12
#define INTERVAL_H 0.1

void *Disp(void *) {

    while(true) {
        sem_wait(disp_sem);
        if (disp_data.type=TEMP){
            printf("Temperature: f", disp_data.value);
            if(disp_data.alarm_cond==ON)
                Start_T_Alarm();
            sem_post(temp_sem);
        }
        else {
            printf("Humidity: f", disp_data.value);
            if(disp_data.alarm_cond==ON)
                Start_H_Alarm();
            sem_post(humid_sem)
        }
    }
}

```

Основная программа будет инициализировать структуры данных, включая семафоры, и активировать эти потоки, которые мы не показываем.

11.5. Ада

Язык Ада был разработан по просьбе Министерства обороны США, которое хотело иметь язык программирования для критически важных систем. Это широко используемый язык с поддержкой параллелизма и реального времени. Он был опубликован в 1983 году как стандарт ANSI/MIL и в 1987 году как стандарт ISO [1]. Вторая крупная ревизия Ада привела к языку Ада 95, который мы кратко рассмотрим. Ада имеет блочную структуру и состоит из одного или нескольких следующих модулей:

- **подпрограмма:** она похожа на процедуру или функцию в общем языке программирования;
- **упаковка:** используется для инкапсуляции и модульного проектирования;
- **задача:** задача – это базовая единица параллелизма, представляющая задачу операционной системы;
- **защищенный блок:** в основном используется для совместных данных с добавленной синхронизацией.

Блок в языке Ада имеет следующую структуру [2]:

```
declare
- определения объектов, подпрограмм, типов и т. п.
begin
- последовательность объявлений
exception
- исключение обработки
end;
```

Давайте посмотрим, как мы можем реализовать функцию, которая находит сумму элементов вектора целых чисел в языке Ада. Функция *Summation* возвращает переменную *Sum*. Заметим, что *V'Range* – это все, что нам нужно сделать, чтобы запустить индекс цикла *I*, проходящего через все значения вектора *V*.

```
function Summation(V: Vector) return Integer is
  Sum : Integer := 0;
begin
  for I in V'Range loop
    Sum := Sum + V(I);
  end loop;
  return Sum;
end Summation;
```

Ада имеет основные характеристики объектно-ориентированного языка, и модуль может объявить некоторые из его переменных и функций *частными*, чтобы они не были доступны извне. Кроме того, объектно-ориентированные средства программирования, такие как наследование, конструкторы, деструкторы и полиморфизм, в языке Ада 95 доступны.

11.5.1. Параллелизм

Базовая единица параллелизма в языке Ада называется *задачей*, которая объявляется явно и использует ключевое слово *задача*. Задачи синхронизируются и общаются как операционная система задач, использующая общие переменные, защищенные блоки или метод, называемый *rendezvous*. Задача имеет спецификацию, которая содержит при создании свое имя, возможные входные параметры, видимую часть и частную часть. Тело задачи содержит ее исполняемый код, как показано в примере, показанном ниже:

```
procedure Example1 is
  task type A_Type;
  task B;
  A, C : A_Type;
  task body A_Type is
    - локальные описания для задач A и C
  begin
    - последовательность инструкций для задач A и C
```



```

end A_Type;
task body B is
- локальное описание для задачи B
begin
- последовательность инструкций для задачи B
end B;
begin
- задачи A, C и B начинают выполнение с первой инструкции этой процедуры
end

```

В этом программном сегменте мы определяем тип задачи *A_type*, который можно использовать для классификации задачи. Описание тела задачи содержит локальные описания и фактический код для задачи. Когда процедура начинается, все задачи начинают выполняться одновременно.

11.5.1.1. Управление временем

Тип данных под названием *Time* представляет в языке Ada95 реальное время с разрешением 1 миллисекунда. Значение текущего реального времени можно прочитать с помощью функции *Clock*. Следующий пример демонстрирует, как измерить время, которое занимает выполнение цикла.

```

task body Example2 is
  First, Second, Interval : Time;
begin
  First := Clock; - запись времени перед началом цикла
  loop
  ...
end loop;
  Second := Clock; - запись времени, когда цикл окончен
  Interval := Second - First;
end Example2;

```

Нельзя не сказать, что мы могли бы использовать инструкцию «Interval:=Clock-First» вместо двух операторов после окончания цикла. Для такого эффективного управления временем язык Ада обеспечивает использование пакета *Calendar*.

11.5.1.2. Periodic Tasks

Периодическая задача в языке Ада может быть определена использованием структуры «задержка до тех пор, пока» (*delay until*), как показано в примере ниже. Размер интервала может быть установлен для указания периода. Обратите внимание, что *Interval* должен быть установлен для задачи как $T - C$, например если задача выполняется 20 миллисекунд и ее следует активировать каждые 100 миллисекунд, нам нужно установить значение *Interval* 80 миллисекунд.

```

task T1 is
  Interval : constant Duration := 0.08;
  Next_Time : Time;

```

```

begin
Next_Time := Clock + Interval;
loop
Action; - процедура выполняет полезную работу
delay until Next_Time;
Next_Time := Next_Time + Interval;
end loop;
end T;

task T1 is
#pragma Priority(12)
end T;

```

11.5.1.3. Приоритеты задач

В языке Ada предоставляется поддержка статических и динамических приоритетов задач. Статичный приоритет задачи задается с помощью прагмы *Priority*, которая содержится в спецификации задачи, как показано ниже.

```

task T1 is
#pragma Priority(12)
end T;

```

В языке Ada 95 для систем реального времени дополнительно поддерживаются динамические приоритеты.

```

package Ada.Dynamic_Priorities is
procedure Set_Priority(...);
function Get_Priority(...) return Any_Priority;
end Ada.Dynamic_Priorities;

```

Используя этот пакет, приоритет задачи можно прочитать и изменить во время ее выполнения.

11.5.1.4. Синхронизация задач и связь

Основным методом синхронизации и связи в языке Ада является использование пары *entry/accept* (вход/прием) вызовов. Две задачи, *вызывающая сторона* и *сервер*, синхронизируются и обмениваются данными этими вызовами. Вызывающая сторона вызывает *entry* в сервере, и сервер выполняет инструкцию *accept*, чтобы получить вызов. Если *accept* была выполнена до вызова *entry* вызывающей стороной, сервер блокируется в точке *accept*. В противном случае, если выполнение *entry* осуществлялось до выполнения сервером *accept*, вызывающая сторона блокируется в ее точке выполнения *entry*. Вызывающая сторона остается заблокированной во время обработки *accept* сервером. Таким способом достигается полная и надежная синхронная связь. Вызывающая сторона должна знать сервер и его входы, но сервер не должен не знать вызывающие стороны. Только одна вызывающая сторона может встретиться с сервером, а все попытки связать всех других абонентов заблокированы. В следующем примере показан сервер,

который принимает целое число от клиента, возводит его в квадрат и печатает выходные данные. Например, клиент должен сделать вызов *Square.Calculate* (3), чтобы найти квадрат 3. Выход достигается путем вызова *Put*.

```
task Server is
  entry Square(x in Integer);
end Server;

task body Server is
  a : Integer;
begin
  accept Square(x : in Integer, a : out integer) do
    a := x * x;
    Put(a);
  end Square;
end Server;
```

Когда серверу нужно ждать два или более разных типов вызовов *accept*, можно использовать оператор *select*. Следующий пример иллюстрирует использование *select*, где задача сервера может принимать entries *Add* (добавить) или *Subtract* (вычесть) в любом порядке и блокировать, пока принятый вызов не выполнен.

```
task Calculate is
  entry Add(x,y in Integer, z: out Integer);
  entry Subtract(x,y in Integer, z: out Integer););
end Calculate;

task body Calculate is
  a : Integer;
begin
  loop
    select
      accept Add(x, y: in Integer, a: out Integer) do
        a:= x + y;
        Put(a);
      end Add;
    or
      accept Subtract(x, y:in Integer, a:out Integer)do
        a:= x - y;
        Put(a);
      end Subtract;
    end select;
  end loop;
end Calculate
```

11.5.2. Обработка исключений

Исключения в языке Ada – это ошибки, возникающие при выполнении программы. *Обработка исключений* – это процесс перехвата этих ошибок во время выполнения и выполнение некоторых действий по исправлению или отображению состояния ошибки. В языке Ada оператор блока имеет декларативная часть, по-

следовательность операторов и раздел обработки исключений. Предопределенные типы исключений в Ada определены в пакете *Standard* следующим образом:

- *ошибка ограничения (Constraint_Error)*: возникает, когда нарушается ограничение диапазона;
- *численная ошибка (Numeric_Error)*: возникает, когда числовая операция не может быть выполнена из-за таких условий, как переполнение или деление на ноль;
- *программная ошибка (Program_Error)*: возникает, когда функция завершается, не встретив инструкции ответа;
- *ошибка памяти (Storage_Error)*: возникает, когда пространство памяти исчерпано из-за динамического создания объекта или исчерпания стекового пространства;
- *ошибка в задаче (Tasking_Error)*: возникает во время связей задачи, например при попытке синхронизироваться с помощью бездействующей задачи, используя *rendezvous* (рандеву).

Исключения могут быть обработаны разделом *исключение* в конце блока, используя ключевое слово *when*, чтобы указать тип исключения. Давайте проиллюстрируем эти понятия с помощью рекурсивной функции, которая вычисляет факториал целого числа на входе. Если входное значение слишком велико, чтобы его можно было вычислить, возникает исключение.

```
function Factorial(n:integer) return integer is
begin
  if n=1 then return 1;
  else return n * Factorial(n-1);
  end if;
exception
  when NUMERIC_ERROR
    Put_Line("Вход слишком большой");
end
```

Мы можем иметь более одной обработки исключений в блоке, который, например, используется для открытия и выполнения какой-либо операции с файлом, как показано ниже. Обработка исключений выполняется аналогично команде *case*, содержащей один обработчик исключений.

```
begin
  -- operations on the file
exception
  when File_Not_Found
    Put_Line("File does not exist");
  when End_Of_File
    Close(file);
  when others
    Put_Line("Error");
end
```

Пользователь может активировать исключение, используя **повышение** вызова, а затем указав тип исключения в любом месте программы, как показано в примере ниже.

```
raise CONSTRAINT_ERROR
```

Пользователи могут определить свои собственные исключения, вставив их как часть описания:

```
my_exception: exception;
```

Исключения могут быть подавлены с помощью прагмы `suppress`. Например, синтаксис для подавления `STORIGE_ERROR`:

```
pragma suppress (storage_check);
```

11.5.3. Реализация управления процессами на языке ADA

Реализация программного обеспечения реального времени для системы управления процессами на рис. 11.1 включает в себя три задачи для языка Ada, как и в реализации языка C/POSIX. Взаимное исключение между задачами *Temp* и *Humid* может быть обеспечено *защищенным объектом Display*. Любая процедура в этом объекте будет выполняться только одной задачей, и вызывающая задача будет заблокирована, если какая-либо процедура выполняется другой задачей.

```
protected type Display is
  procedure Temp_Disp(data: in Temp_Data);
  procedure Humid_Disp(data: in Humid_Data);
end Display;
protected body Display is
begin
  procedure Temp_Disp(data : in Temp_Data) is
  begin
    Printline("Temperature: ", data);
  end Temp_Disp;
  procedure Humid_Disp(data : in Humid_Data) is
  begin
    Printline("Humidity: ", data);
  end Humid_Disp;
end Display;
```

Теперь две задачи можно кодировать следующим образом: каждая задача после прочтения величины от своего датчика проверяет, является ли эта величина ниже или выше пороговых значений, и вызывает свою процедуру в объекте *Display*.

```
procedure Process_Control is
  task Temp;
  task Humid;
```

```

task body Temp is
temp_Uthreshold: constant := (some_value);
temp_Lthreshold: constant := (some_value);
begin
  loop
    Read_Temp(temp_value);
    if temp_value > temp_Uthreshold or
    temp_value < Ltemp_threshold then
      Set_Temp(Temp_Switch, temp_value);
    end if
    Display.Temp_Dis(temp_value);
  end loop
end Temp;
task body Humid is
begin
  Humid_Hthreshold: constant := (some_value);
  Humid_Lthreshold: constant := (some_value);
  loop
    Read_Humid(humid_value);
    if humid_value > Humid_Uthreshold or
    temp_value < Humid_Lthreshold then
      Set_Humid(Humid_Switch, humid_value);
    end if
    Display.Humid_Dis(humid_value);
  end loop
end Humid;

begin
  null; -задачи Temp и Humid запускаются одновременно
end Process_Control;

```

11.6. ЯЗЫК ПРОГРАММИРОВАНИЯ JAVA

Язык Java является независимым от платформы объектно-ориентированным языком с синтаксисом, аналогичным языку C/C++. Одна из основных целей языка Java – быть переносимым, и по этой причине исходный код языка Java компилируется в промежуточный код, называемый *байт-кодом Java (Java bytecode)*, который может выполняться на любой виртуальной машине Java (JVM). Он демонстрирует все основные характеристики объектно-ориентированного программирования, такие как классы, наследование и полиморфизм. Давайте реализуем предыдущую функцию суммирования элементов вектора в Java. Сначала определим класс *Vectors*, а затем метод в этом классе под названием *Summation*, который вычисляет сумму входов входного вектора.

```

class Vectors
{ public int Summation(Vector V )
  { int sum = 0;
    int i;
    for (i=0; i<V.length; i++)

```

```

    sum = sum + V[i];
    return sum;
}
}

```

Класс в Java содержит данные и методы и может принадлежать *пакету* (*package*). Методы и объектные переменные в Java могут быть *открытыми* (*public*), находящимися вне класса, защищенными (*protected*), которые видны только внутри пакета или в классе, или *частными* (*private*), которые могут не быть доступными вне класса.

11.6.1. Поток Java

Поток в Java является основной процедурой исполнения так же, как поток POSIX или задача Ada. Основной поток виртуальной машины (JVM) начинается с вызова метода *main*. Класс *java.lang.Thread* может быть расширен до класса *thr*, чтобы расширить создаваемые потоки. Существует также стандартный интерфейс, который можно использовать для одновременного выполнения, как показано ниже.

```

public interface Runnable
public abstract void run();

```

В качестве примера создания первого потока в следующем примере мы используем класс *Thread*, где он расширен до класса *thr*, и экземпляр *T1* этого класса создан системным вызовом *new* в главном потоке. Созданный поток просто пишет на экране «Hello». Код, специфичный для потока, может быть размещен внутри метода *run* в этой реализации.

```

class thr extends Thread {
    public void run() {
        System.out.println("Hello");
    }
    public static void main(String args[]) {
        thr T1= new thr();
        T1.start();
    }
}

```

Второй тип реализации включает использование интерфейса *Runnable* и начинается с заголовка «**class thr implements Runnable**», чтобы указать метод *run*. Динамическое создание потоков возможно, и основная программа завершается, когда все пользовательские потоки завершились. Потоки Java могут быть или *пользовательскими потоками*, или серверными присоединенными потоками, называемыми демонами (*daemon threads*). Обычно используемые потоками Java методы следующие:

- `void run ()`: точка входа для потока;
- `void start ()`: начать поток, активировав метод *run*;

- `boolean isAlive ()`: определяет, работает ли еще поток;
- `sleep (long ms)`: приостановить поток на указанный период времени;
- `void join ()`: дождитесь окончания потока.

11.6.2. Синхронизация потоков

Критичный раздел в Java защищен *объектами блокировки*. Два метода, определенных на заблокированном объекте, – это *блокировка* и *разблокировка*, как показано в следующем сегменте кода:

```
import java.util.concurrent.locks.*;
// не критичный раздел
lock my_lock;
while (true) {
    my_lock.lock(); // ввести критичный раздел
    try {
        // критичный раздел
    }
    finally { // это необходимо для уверенной реализации блокировки, даже когда
        // присутствует исключение
        my_lock.unlock(); // возврат нормального режима
    }
}
```

Открытый класс *семафоров* в Java имеет следующие реализуемые методы:

- `count ()`: возвращает текущее значение семафора;
- `up ()`: увеличивает счетчик семафоров. Этот вызов аналогичен вызову `signal_sema` в DRTK;
- `down ()`: ожидает положительного значения счетчика, а затем уменьшает его. Этот вызов – аналог вызова `wait_sema` в DRTK.

Семафоры могут быть использованы для синхронизации между потоками, а также для взаимного исключения в качестве замков. Предполагая, что семафорный объект `sem` создан, критичный раздел может быть затем защищен вызовами `sem.down` и `sem.up` при входе и выходе соответственно.

11.6.2.1. Управление временем и расписание

Язык Java предоставляет открытый класс *Clock* и *getTime* и другие методы этого класса для управления временем. Поток в Java может быть прерван другим потоком с помощью функции *interrupt ()*, которая повышает *InterruptedException*. Спящий или ожидающий поток может перехватить это прерывание, однако работающий поток не будет перехватывать прерывание. Функция *interrupted()*, которая возвращает логическое значение *true* при возникновении прерывания, может быть проверена потоком, чтобы перехватить прерывание, как это происходит в следующем далее фрагменте кода. Периодический поток спит в течение 200 миллисекунд перед следующей активацией. Однако он может получать прерывания и обслуживать их.


```
class thr extends Thread {  
    public void run() {  
        try {  
            for(;;) {  
                while(!interrupted) {  
                    // выполнить некоторую работу  
                    Thread.sleep(200);  
                }  
            }  
        } catch (InterruptedException e) {  
            // выполнить обслуживание прерывания  
        }  
    }  
}
```

11.6.3. Обработка исключений

Обработка исключений в Java осуществляется с помощью блока *try-block*, который обеспечивает защиту блока.

```
try  
// код, который может вызвать исключение  
// ...  
  
catch (исключение типа x)  
// обработчик исключений для x  
finally (exception type y)  
// код выполняется во всех случаях
```

Возможны следующие типы исключений в Java:

- *NullPointerException* генерируется, когда нулевой указатель передается в метод остановки;
- *IllegalThreadStateException* генерируется, когда был вызван метод *start* и поток уже начался;
- *SecurityException* генерируется, когда был вызван метод *stop* или *destroy* потока, для которого вызывающая сторона не имеет разрешений на запрашиваемую операцию;
- *InterruptedException* генерируется, когда поток ожидает, или спит, или прерван либо до активности, либо во время этой активности.

11.7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каковы основные требования к любому языку программирования реального времени?
2. Как язык программирования Си обеспечивает параллелизм?
3. Какова основная функция в UNIX для задержки задачи? Какая функция задерживает задачу в интерфейсе POSIX?

4. Каков основной системный вызов в языке Ада для выполнения периодической задачи?
5. Какой основной метод синхронизации задач в языке Ада?
6. Каковы основные типы исключений в языке Ада?
7. Что является основной характеристикой языка Java, которая отличает ее от многих других языков программирования?
8. Каковы два основных метода создания потоков в языке Java?
9. Как в языке Java достигается взаимное исключение при доступе к общей переменной?
10. Каковы основные методы класса семафоров в языке Java?
11. Как обнаруживается исключение в языке Java?
12. Каковы основные типы исключений в языке Java?

11.8. ПРИМЕЧАНИЯ К ГЛАВЕ

Основными требованиями к языку программирования реального времени являются эффективное управление временем, поддержка параллелизма, средства для исключения и обработки прерываний и поддержка планирования в реальном времени. Мы рассмотрели три основных языка программирования в реальном времени с учетом этих проблем: C/POSIX, Ada и Java. Язык Си все еще широко используется для реализации программного обеспечения в реальном времени, хотя сам по себе является языком нереального времени. Тем не менее расширение POSIX реального времени обеспечивает удобное управление временем, многозадачность, используя потоки, синхронизацию временных задач и примитивы связи. Язык Ада разработан для критически важных систем и имеет все указанные выше средства программирования в режиме реального времени. Задача может быть определена как отдельный исполняемый модуль, и различные методы предоставлены для межзадачной синхронизации и связи. Ада имеет средства для обработки и планирования исключений. Язык программирования Java может использоваться для реализации программного обеспечения в системах реального времени, поскольку обеспечивает параллелизм потоков и имеет средства для обработки прерываний и исключений.

Основное решение заключается в том, использовать ли в первую очередь операционную систему в реальном времени или язык программирования в реальном времени для реализации программного обеспечения в реальном времени. Первое означает, что мы можем использовать общий язык программирования с интерфейсом операционной системы реального времени, например C/C++, с легкостью переносимости и, возможно, с более богатой библиотекой системных вызовов, чем язык программирования в реальном времени. С другой стороны, применение языка программирования в реальном времени может быть сложным, если модели, используемые языком, и операционная система радикально отличаются. Мы сделали очень краткое введение относительно характеристик трех упомянутых языков в реальном времени: язык C/

POSIX в режиме реального времени описан в [2], подробное изучение языка Ада можно найти в [3], а использование языка Java для приложений реального времени приведено в [4].

11.9. УПРАЖНЕНИЯ ПО ПРОГРАММИРОВАНИЮ

1. Напишите программу на языке C/POSIX с тремя потоками $T1$, $T2$ и $T3$, где входом $T1$ является целое число, поступающее от пользователя, записываемое в общую память. Поток $T2$ читает это целое число, проверяет его на соответствие нижнему и верхнему пределам, умножает его на константу и помещает его в другое место общей памяти, так что $T3$ вводит его значение и отображает его. Все критичные разделы должны быть защищены.
2. Напишите программу на языке Ада с задачей сервера, который является калькулятором выполнения четырех основных арифметических операций, и задачей вызывающей стороны ввода от пользователя в виде двух чисел и нужной операции. Вызывающая сторона получает вход в принимающей части сервера.
3. Напишите программу на языке Ada, которая объявляет массив X из 10 целых чисел и инициализирует каждую запись, так что $X(I) = 2 \cdot I$. Выполняемая программа вводит индекс от пользователя и отображает ввод X в этом индексе. Обеспечьте часть программы обработки исключений, чтобы повысить `CONSTRAINT_ERROR`, и сообщение об ошибке, отображаемой, когда пользователь вводит неверный индекс.
4. Java-модуль обеспечивает периодические задачи в реальном времени $T1$, $T2$ и $T3$. Задача $T1$ читает данные из файла, а затем отправляет прерывание на задачу $T2$, которая записывает эти данные в другой файл и выполняет семафор, ожидающий задачу $T3$. Задача $T3$ читает содержимое файла и отображает его. Написать эту программу на Java с коротким комментарием.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. Ben-Ari M. (2005) Ada for software engineers. Weizmann Institute of Science.
2. Burns A., Wellings A. (2001) Real time systems and programming languages: Ada 95, real-time Java and real-time C/POSIX, 3rd edn. Addison-Wesley.
3. Burns A., Wellings A. (2007) Concurrent and real-time programming in Ada, 3rd edn. Cambridge University Press.
4. Real-time specification for Java 2.0 (RTSJ 2.0). <https://www.aicas.com/cms/en/rtsj>.

Глава 12

Отказоустойчивость

12.1. ВВЕДЕНИЕ

Отказы в компьютерной системе являются следствием неисправности компонентов, связанных с аппаратным или программным обеспечением. Дефекты компонентов могут возникнуть из-за производственного брака или из-за старения – износа при длительном использовании или воздействии окружающей среды, таких как применение компонента вне диапазона допустимой температуры или диапазона вибрации и т. д. По различным причинам могут возникнуть программные ошибки, такие как ошибки программного обеспечения, ошибочный дизайн и т. д. Ошибки могут стать причиной выхода из строя части или всей системы и привести к трагическим последствиям, которые случались в прошлом, – авиакатастрофам, падениям ракет и прочим. Например, неисправность в компоненте схемы отображения высоты над уровнем моря самолета вызовет ошибку в чтении дисплея, что может ввести пилота в заблуждение и привести к катастрофе.

Неисправность должна быть обнаружена, с тем чтобы, если это возможно, предпринять действия по устранению вызвавших ее причин. Отказоустойчивость – это способность системы продолжать корректно функционировать при наличии неисправностей. В системе реального времени отказоустойчивость является императивом, поскольку сбой в системе может привести к гибели людей или потере имущества.

Мы начнем эту главу с основных понятий и терминологии, относящихся к отказоустойчивости в целом. Затем рассмотрим основные примеры аппаратного и программного сбоя, обзор методов устранения неисправностей и описание отказоустойчивого планирования в системах реального времени. Использование группирования рабочих задач является фундаментальным методом достижения отказоустойчивости в распределенных системах реального времени, и мы рассмотрим и обсудим методы для упорядоченной и надежной доставки сообщений в целевую группу.

12.2. Понятия и терминология

Неисправности в компьютерной системе возникают по многим причинам, основными из которых являются такие, как неадекватность спецификации, ошибки проектирования программного обеспечения, сбои аппаратных компонентов, сетевые ошибки, возникающие в основном из-за помех. Активный отказ является причиной ошибки, вызывающей неисправное состояние системы, которое может привести к отказу. Ошибка может распространяться, вызывая ряд ошибок. Нам нужно определить ряд терминов, связанных с отказоустойчивостью [8, 10]:

- *уровень обслуживания*: это способность системы предоставлять своим пользователям необходимый уровень обслуживания [10]. Основными характеристиками являются надежность, доступность и безопасность;
- *надежность, ненадежность*: надежность $R(t)$ – это вероятность того, что система функционирует согласно ее спецификациям в интервале $[0, t]$, принимая, что она функционирует правильно в момент 0. Ненадежность $F(t)$ – вероятность того, что система выйдет из строя в любой момент времени в интервале $[0, t]$. Отсюда следует, что $R(t) = 1 - F(t)$;
- *доступность*: доступность $A(t)$ – это вероятность того, что система работает правильно в соответствии с ее техническими характеристиками в момент времени t . Обратите внимание, что доступность оценивается для конкретного момента t , тогда как надежность оценивается для интервала $[0, t]$;
- *безопасность*: безопасность $S(t)$ – это вероятность того, что система не выйдет из строя в интервале $[0, t]$ и будет работать правильно, не причиняя вреда людям, имуществу или окружающей обстановке;
- *ремонтпригодность* $M(\Delta t)$: это вероятность того, что система с отказом будет восстановлена в течение указанного интервала времени Δt ;
- *система критической безопасности*: это система, выход из строя которой может привести к гибели людей, повреждению имущества и/или окружающей среды. Распределенные системы реального времени и некоторые системы со встроенными элементами обычно попадают в эту категорию;
- *эксплуатационная безопасность*: это способность системы защитить себя от потенциального повреждения;
- *отказ, ошибка и неисправность*. Разработанная система имеет характеристики, которые определяют ее поведение. Когда поведение системы отклоняется от ее спецификации, состояние системы является неисправностью, ошибкой, возникшей в результате отказа. Рисунок 12.1 отображает эти отношения;
- *задержка ошибки*: это время между возникновением ошибки и полученным в результате отказом.

Предотвращение отказов – это набор методов для предотвращения возникновения отказов. Существует два основных подхода к обработке отказов –

маскировка отказов и *реконфигурация отказов*. Маскировка отказов – это процесс предотвращения возникновения ошибок в системе. Реконфигурация отказов включает в себя удаление неисправных компонентов из системы и приведение ее в рабочее состояние и состоит в общем из следующих последовательных шагов:

- 1) *обнаружение отказа*: отказ должен быть обнаружен перед началом любой процедуры восстановления;
- 2) *местоположение неисправности*: это поиск места отказа;
- 3) *ограничение отказа*: отказ должен быть изолирован, чтобы предотвратить его распространение в системе;
- 4) *устранение отказа*: это процесс восстановления системы в рабочее состояние после возникновения отказа.



Рис. 12.1. Отношения между отказом, ошибкой и неисправностью

12.3. КЛАССИФИКАЦИЯ НЕИСПРАВНОСТЕЙ

Отказы могут возникать по разным причинам, например ошибки в спецификации или ошибки из-за внешних помех, таких как шум или радиация. Причины возникновения и продолжительность отказа могут быть классифицированы следующим образом:

- *постоянные отказы*: постоянные отказы обычно вызываются неисправным компонентом, и восстановление после такой неисправности в целом возможно только путем замены или ремонта этого компонента;
- *временные ошибки*: временная ошибка возникает только в течение определенного промежутка времени, и система, как правило, продолжает после этого функционировать правильно. Эти ошибки через некоторое время исчезают. К данному типу отказов относятся многие неисправности сети связи;
- *прерывистые отказы*: прерывистая неисправность – это повторяющийся отказ, который трудно обнаружить. Возникает ошибка, а затем система работает правильно. Эта ситуация встречается довольно часто. К таким отказам может привести плохое соединение в электронной цепи.

Неисправность можно классифицировать с другой точки зрения, исходя из ее значения и времени, когда она произошла [1, 6]:

- *отказоустойчивая система*: система продолжает работать правильно при наличии временных отказов;
- *отказы с задержкой*: система выдает правильные значения, но они появляются с временной задержкой;

- *частичная деградация (амортизация отказов)*: система при возникновении ошибки продолжает выполнять свои основные функции с потерей некоторых сервисов при возникновении отказа;
- *молчаливый отказ*: система, которая работает правильно и во временной области, и в области значений, пока не происходит пропуск процесса или линии связи, после чего происходят отказы во всех сервисах.

12.4. РЕЗЕРВИРОВАНИЕ

Резервирование – это дублирование критически важных аппаратных и/или программных компонентов системы для повышения надежности за счет использования дубликата, если активный компонент перестает работать. Основными типами резервирования являются аппаратное резервирование, избыточность информации, резервирование рабочего времени и резервирование программного обеспечения.

12.4.1. Аппаратное резервирование

Аппаратное резервирование в компьютерной системе может быть реализовано в виде *пассивного резервирования*, *активного резервирования* или *гибридного резервирования* [5]. В пассивной аппаратной избыточности используется маскировка ошибок. Как правило, n независимых идентичных аппаратных модулей выполняют одни и те же функции, результаты которых для принятия решения о правильном выходе определяются посредством голосования. Система М-из-N состоит из N компонентов, и для правильной работы этой системы требуется как минимум М компонентов, работающих правильно. Система с тройным модульным резервированием (triple modular redundancy, TMR) представляет собой систему 2-из-3 с $M = 2$ и $N = 3$, которой для правильной работы нужны два правильно работающих компонента системы. Система TMR реализуется тремя компонентами, выполняющими одно и то же действие, а результат их работы оценивается. Схема голосования в TMR может быть реализована с помощью логических элементов И-ИЛИ и называется *однобитная мажоритарная схема голосования*. Она показана на рис. 12.2. Есть три модуля А, В и С, которые производят три однобитных выхода а, b и с соответственно. Давайте предположим, что неисправное состояние создает *ложный* вывод, а правильное состояние производит *истинный* вывод. Обратите внимание, что вывод логического элемента ИЛИ является истинным, даже если произойдет отказ одного из компонентов. Мажоритарная n -битная схема голосования состоит из n таких однобитных схем мажоритарного голосования, параллельно участвующих в голосовании. Полная задержка в такой схеме голосования – задержка в двух логических элементах И и элементе ИЛИ.

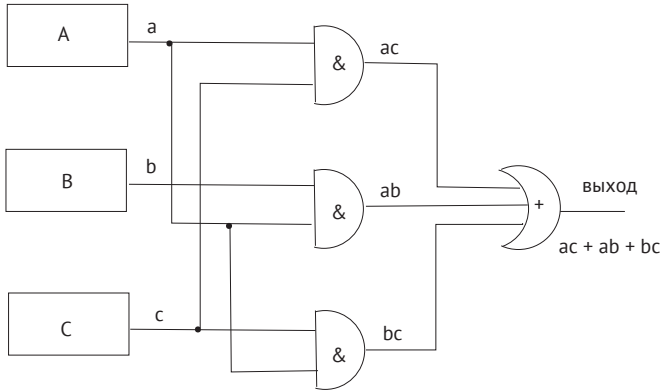


Рис. 12.2. Реализация схемы TMR с тремя элементами

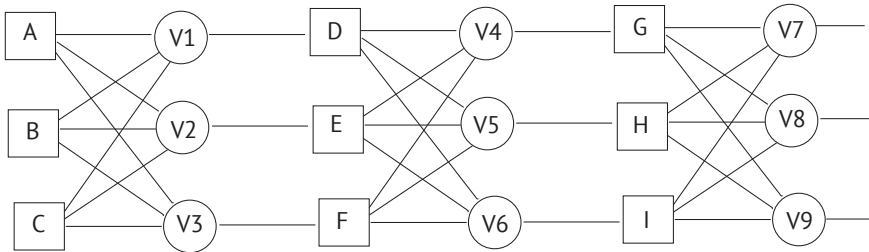


Рис. 12.3. Общая реализация схемы TMR. Адаптировано из [13]

Голосование TMR может быть выполнено в программном обеспечении с помощью простой программы, которая сравнивает выходы модулей, используя три сравнения и выводя тот выход, который является тем же, что и выходы, по крайней мере двух модулей. Аппаратная схема дороже, чем программный вариант, из-за необходимости дополнительных аппаратных схем, но он быстрее. В более общем случае TMR с тремя ступенями реализуется, как показано на рис. 12.3, без необходимости в логических переменных. Кроме того, предусматривается, что избиратели могут быть неисправными, отсюда три ступени.

N-модульное резервирование – это обобщение TMR с N модулями. При $N > 2k$ может быть обнаружено до k отказов. *Сторожевой таймер* периодически сбрасывается системными/прикладными задачами, и неудача в этом свидетельствует об отказе. В системах реального времени обычно используются сторожевые таймеры. Активное аппаратное резервирование достигается путем обнаружения отказа с последующими процедурами его локализации, ограничения распространения неисправности и его устранения. Гибридное аппаратное резервирование сочетает пассивные и активные подходы к резервированию, которые предусматривают как предотвращение отказов, так и принятие мер активного резервирования [5].

12.4.2. Избыточность информации

Данные, перенесенные из одной ячейки памяти в другую, или чаще из одного узла в сети в другой, могут содержать ошибки. Информационная избыточность добавляет больше информации к данным для выявления ошибок.

12.4.2.1. Кодирование

Эта операция на уровне слов данных известна как *кодирование*, в котором к данным добавляются контрольные биты. D -битное слово кодируется в s -битное кодовое слово, и получатель/пользователь данных декодирует кодовое слово для извлечения исходных данных [8]. Схема кодирования может быть разработана таким образом, чтобы ошибки, вносимые в кодовое слово, выводили кодовые слова из установленного диапазона, и поэтому ошибки могли бы быть обнаружены. Кодирование может быть разработано так, чтобы сделать возможным исправление кодового слова из ошибочного слова в слово с *откорректированной ошибкой*. *Кодовое расстояние по Хэммингу (Hamming distance)* двух двоичных слов x и y – это количество битов, которое их различает. Например, даны два двоичных слова $x = 0100\ 1000$ и $y = 0101\ 1010$, расстояние Хэмминга x и y , $H_d(x, y)$, составляет 2.

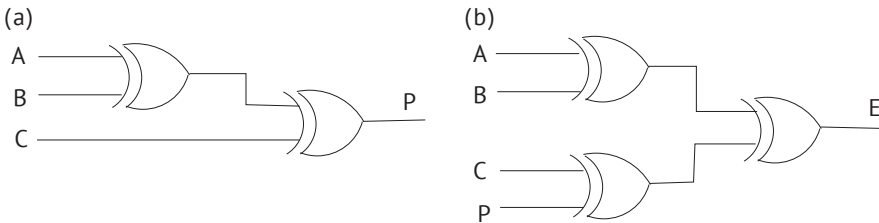


Рис. 12.4. Реализация контроля четности и проверка с помощью вентилях XOR

Таблица 12.1. Пример матрицы кодового слова вертикальной и горизонтальной четности

c_4	c_3	c_2	c_1	c_0
1	0	1	0	0
0	1	1	1	1
1	0	0	1	0
1	1	1	1	0
1	0	1	1	1

Коды четности

К двоичному слову можно добавить дополнительный бит, чтобы полученное слово имело четное (*even parity*) или нечетное (*odd parity*) число единиц. Например, учитывая двоичное семибитное слово 0110 101, нам нужно добавить к последнему биту 0 для четности слова или 1 для нечетности. Получатель кодового слова может затем проверить четность и обнаружить однобитную ошибку. Простота этого метода позволяет использовать его для защиты дан-

ных в памяти. Генерация паритета и его обнаружение обычно выполняются на оборудовании с использованием логических элементов XOR, как показано на рис. 12.4, где генерируется бит четности для трехбитных данных с использованием четности на рис. 12.4а и четырехбитное кодовое слово проверяется на четность, чтобы вывести бит ошибки (Е), на рис. 12.4б.

Для выявления более чем однобитных ошибок может быть применена горизонтальная и вертикальная генерация четности тестирования. Этот метод добавляет горизонтальные биты четности, как и раньше, и дополнительно добавляет биты вертикальной четности, чтобы сформировать новое кодовое слово внизу. Этим способом может быть исправлена ошибка в единственном бите блока данных. Такой ошибочный бит в кодовом слове будет иметь ошибку четности как для строки, так и для столбца, которым оно принадлежит. Пример горизонтальной и вертикальной четности показан в табл. 12.1, где четыре четырехбитных слова данных (c_4 по c_1) с добавленным четным битом четности (c_0) добавляются по вертикали в пятое слово со всеми добавленными битами четности, выделенными жирным шрифтом.

Резервирование с циклической проверкой

Циклическая проверка (Cyclic Redundancy Check, CRC) – эффективный метод обнаружения ошибок, используемый обычно на канальном уровне семиуровневой иерархической модели стандарта OSI (Open System Interconnection,). CRC-узел вычисляет короткий двоичный код, называемый *контрольным значением*, и добавляет его к данным, подлежащим отправке/записи, чтобы сформировать кодовое слово. Это делается с помощью следующих шагов, примененных к n битам двоичного кода данных $D(n) = \{d_{n-1}, \dots, d_0\}$.

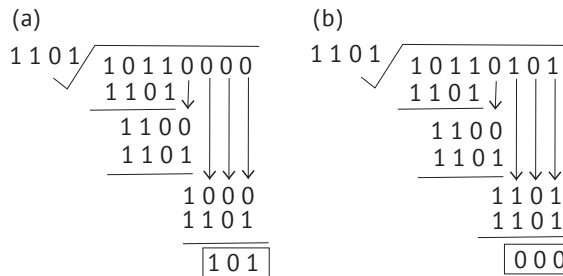


Рис. 12.5. Пример циклической проверки

1. Выберите полиномиальное слово $P(k)$ размера k .
2. Добавьте $k - 1$ нулей в конец $D(n)$, чтобы получить $D(n + k - 1)$.
3. Разделите $D(n + k - 1)$ на $P(k)$, используя арифметику по модулю 2. Это как бинарное суммирование, за исключением того, что биты переноса отбрасываются.
4. Добавьте остаток $R(k - 1)$, полученный к концу $D(n)$, чтобы получить $D^*(n + k * 1)$, и отправьте/напишите его.

Приемник/считыватель делает следующие шаги.

1. Разделите полученное/прочитанное $D^*(n + k - 1)$ с помощью $P(k)$ с использованием арифметики по модулю 2.
2. Если остаток равен нулю, полученное сообщение не содержит ошибок, отправьте $D^*(n)$ для использования.
3. В противном случае сообщите об ошибке.

Пример отправляющей и принимающей сторон показан на рис. 12.5, где $D(5) = 10110$ и $P(4) = 1101$. Остаток, полученный из деления по mod-2 в (а), равен 101, который добавляется к $D(5)$, чтобы получить 10110101, который передается/записывается. Затем получатель/считыватель делит эти двоичные данные на один и тот же полином и получает ноль в (б), что означает, что в этом случае ошибок не было.

Можно показать, что при использовании данного метода остаток на приемнике всегда будет равен нулю, если нет ошибок. Выбор $P(k)$ важен, так как он должен максимизировать возможности обнаружения ошибок и минимизировать вероятность конфликтов. Наиболее часто используемые длины полинома CRC составляют 9, 17, 33 и 65 бит.

12.4.3. Резервирование времени

Резервирование времени достигается за счет выделения дополнительного времени для проверки неисправностей, например повторное выполнение программного модуля и сравнение результатов обеспечат обнаружение кратковременных отказов, так как эти отказы обычно происходят всего несколько раз за время жизни системы. На рис. 12.6 показана логическая схема резервирования по времени, где одни и те же вычисления повторяются с одним и тем же вводом x в трех временных точках; t_1 , t_2 и t_3 и выходы a , b и c сравниваются. Если один из первых двух выходов отличается от c , произошла временная ошибка. Обратите внимание, что если c отличается от обоих a и b , нам нужно выполнить еще один тест в другой момент времени, чтобы решить, отказ в момент времени t_3 является постоянным или временным. Этот тип резервирования не требует дополнительного оборудования, но может привести к большим вычислительным затратам.

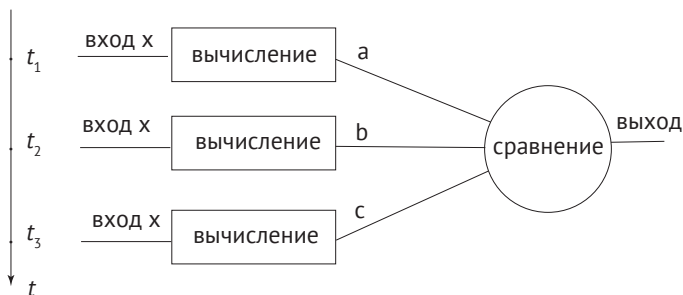


Рис. 12.6. Резервирование времени

12.4.4. Резервирование программного обеспечения

Программная ошибка обычно является результатом неправильного проектирования, которое часто возникает из-за конкретной входной последовательности или условия. Методы резервирования программного обеспечения основаны на использовании хранения дополнительного программного обеспечения для запуска его при возникновении отказов. Эти методы могут быть в общем случае классифицированы как *метод с одной версией* (однократное резервирование) и *метод со многими версиями* (многократное резервирование).

12.4.4.1. Методы с одной версией

Эти методы используют дополнительные программные модули одной версии программного модуля для устранения отказов. Существует два метода устранения отказов: *обработка исключений* (*exception handling*) и *контрольные точки* (*checkpointing*).

Обработка исключений

Исключениями являются программные прерывания из-за непредвиденных условий, таких как деление на ноль, повреждение памяти, а также эффекты ошибочного проектирования. Необходимо правильное применение исключений, так как они могут привести к неисправности системы. Методы обработки исключений можно в общем случае классифицировать как *запрограммированную обработку исключений* (*programmed exception handling*) и *обработку исключений по умолчанию* (*default exception handling*). Тип исключений, который может появиться, известен из возможных предыдущих действий, предпринимаемых программистом. Обработчик исключений постарается скрыть отказ, и если это невозможно, систему можно перевести в предыдущее безопасное состояние путем восстановления контрольной точки. Обработчики исключений должны по умолчанию иметь дело со всеми непредсказуемыми исключениями, используя особенности конструкций языка программирования. Как упоминалось в главе 11, обработка исключений присутствует во многих языках программирования, таких как ADA и Java. Обработка исключений по умолчанию обычно выполняется программистом с использованием существующей конструкции языка программирования.

Обработка исключений важна в системах реального времени, так как отказ в этих системах может иметь катастрофические результаты. Исключение в такой системе приведет к вычислительным затратам, которые могут привести к несоблюдению сроков выполнения сложных задач в реальном времени. Следовательно, код обработки исключений в системе реального времени должен быть очень эффективным.

Контрольная точка

Метод контрольных точек основан на восстановлении состояния системы до предыдущего безошибочного состояния, когда происходит отказ. Текущее со-

стояние системы сохраняется в энергонезависимой памяти периодически или перед выполнением критичного кода. При этом сохраняется состояние только модифицированной системы в *инкрементной контрольной точке* (*incremental checkpointing*), что обычно диктуется необходимостью экономии времени и пространства при сохранении состояния системы. Обнаружение отказа является частью программного обеспечения для контрольных точек, которое обычно инициируется таким условием, как обнаружение завершающей задачи, которая должна работать непрерывно. Восстановление системы осуществляется путем загрузки из последнего записанного состояния с последующим продолжением нормальной работы. В случае отказа в контрольной точке состояние системы полностью восстанавливается в последней контрольной точке, а затем осуществляются инкрементные изменения.

Этот метод сможет обнаружить отказы переходных процессов, но постоянные неисправности продолжат сказываться на результатах дальнейших восстановительных процессов. Проявившаяся ошибка может быть сохранена в состоянии системы, и восстановление состояния системы ошибку не устранил. В таких случаях может быть выполнена перезагрузка системы или сообщение о неисправности. Также в системе может использоваться в таком случае другой, возможно, менее эффективный программный модуль с несколько худшими характеристиками.

12.4.4.2. Многократное резервирование

Методы с многократным резервированием используют несколько версий компонента программного обеспечения для резервирования при восстановлении после отказа. В этой технике используются два основных подхода – *блоки восстановления* и *N-версии программирования*.

Блоки восстановления

В этом методе программное обеспечение разделено на блоки, и каждый блок имеет ряд функционально эквивалентных блоков, работающих параллельно с первичным блоком. Входная точка в блок – это *точка автоматического восстановления*, а на выходе формируется точка *приемочного теста*. После выполнения основного блока выполняется приемочный тест для проверки того, находится ли система в приемлемом состоянии. Если этот тест не пройден, выполнение отменяется до точки автоматического восстановления в начале блоков, и для исполнения выбирается другой блок. Сбой вновь приводит к выбору другого, параллельного блока, и если все блоки выходят из строя, сообщается об отказе, и процесс восстановления осуществляется на более высоком уровне абстракции.

Программирование N версий

Резервирование программного обеспечения может быть осуществлено с помощью программирования *N* версий. Основная идея этой техники состоит в том, чтобы протестировать разные версии программы и проверить, одинаковы ли выходы, чтобы найти неисправный компонент [3]. Этот метод анало-

гичен методу NMR в аппаратном обеспечении. Функциональные эквиваленты программы называются *версиями*. В этом методе параллельно выполняется N различных версий программы, а выходы сравниваются посредством *драйвера процесса*. Когда начальные условия совпадают, входы одинаковы для всех версий, и алгоритм принятия решения надежен, все выходы из версий должны быть одинаковыми. Версии обычно разрабатываются различными командами разработчиков, которые не взаимодействуют во время разработки. При этом поощряется применение различных алгоритмов, языков программирования и сред разработки, что приводит к разработке существенно отличающихся конструкций [12]. Последовательная разработка версий также возможна, но для приложений реального времени не подходит.

Программирование N -самоконтроля

Метод программирования N -самоконтроля использует как программирование N версий, так и блоки восстановления. Существует N версий программного модуля и *приемочные испытания* (*acceptance test*, АТ) на выходе каждого модуля. Версия с самым высоким рейтингом, которая проходит приемочный тест, принимается за выход мультиплексора N -в-1, как показано на рис. 12.7 [5]. Программирование N -самоконтроля сравнивает результаты из каждой пары версий, и мультиплексор выбирает вход, имеющий самый высокий стабильный рейтинг.

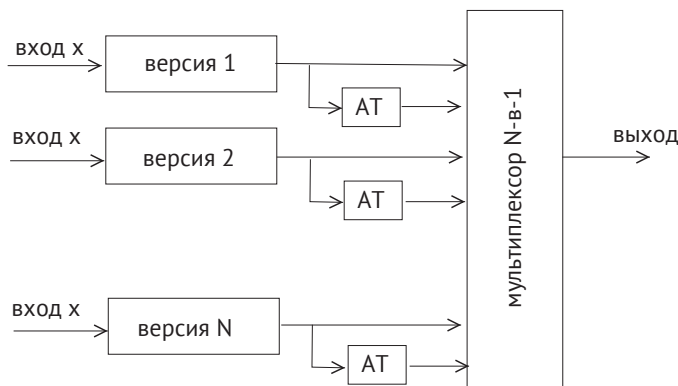


Рис. 12.7. Программирование N -самоконтроля

12.5. ОТКАЗОУСТОЙЧИВЫЕ СИСТЕМЫ РЕАЛЬНОГО ВРЕМЕНИ

Отказоустойчивость в системе реального времени является обязательной, поскольку отказ может привести к катастрофическим последствиям. Все описанные методы резервирования, аппаратные, информационные, программные и временные применимы в таких системах. В реальности в критичных, с точки зрения безопасности системах, таких как самолеты, часто используется аппаратное резервирование. Основное требование к системе реального времени – уложиться в сроки выполнения задачи. Для этого, помимо общих

отказоустойчивых методов, необходимы отказоустойчивые методы планирования при наличии неисправностей. Мы рассмотрели в главе 7 статические алгоритмы планирования, работающие в автономном режиме, и алгоритмы динамического планирования, которые принимают решение о следующей задаче планирования во время работы. Анализ отказоустойчивого планирования в реальном времени описан далее.

12.5.1. Статичное планирование

Отказоустойчивые статические методы планирования обычно резервируют время простоя при планировании в автономном режиме так, что когда в системе появляется неисправность, это время доступно и может быть использовано онлайн. В отказоустойчивом планировании существует три основных подхода: основное/аварийное планирование, маскирование и перепланировка онлайн, как описано в [7].

При основном/аварийном планировании существует основное расписание и предварительно рассчитанные расписания на случай непредвиденных обстоятельств. В чрезвычайных ситуациях, когда появляется неисправность, такое расписание активируется, с тем чтобы задачи соответствовали своим срокам. Метод маскировки использует n версий задачи, всегда выполняющихся на наборе вычислительных элементов, и является в основном методом резервирования программного обеспечения. Перепланировка онлайн осуществляется в два этапа. Сначала вычисляется статичное расписание в автономном режиме. Отказ при выполнении задачи генерирует аperiодический запрос, по которому некоторые из периодических задач переупорядочиваются без нарушения их конечных сроков, и затем в оставшееся свободное время активируется соответствующая поддержка для устранения неисправности.

12.5.2. Динамическое планирование

Мы рассмотрим расширения для оценки алгоритма монотонного рейтинга (rate monotonic, RM), чтобы сделать его более отказоустойчивым. Алгоритм RM предполагает независимые периодические задачи и назначает приоритеты, основанные на продолжительности их периодов. Если задан коэффициент

использования $U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^n - 1)$, где C_i – время вычисления, а T_i – период задачи

τ_i , то алгоритм RM обеспечивает выполнимое расписание.

Алгоритм RM расширен для учета отказоустойчивости в [7, 11], где задача имеет несколько версий, каждая из которых назначена другому процессору. Для распределения версий задач используется алгоритм пакетной упаковки первоочередной эвристики, как показано в алгоритме 12.1. У нас есть задание $T = \{\tau_1, \dots, \tau_n\}$, и каждая задача τ_j имеет r версий; $\tau_j^1, \tau_j^2, \dots, \tau_j^r$. При каждой итерации алгоритма проводится проверка, с тем чтобы определить, какая из версий τ_j^i может быть запланирована процессору P_k , как RM вместе с уже на-

значенными задачами. Если этот тест не пройден, число процессоров увеличивается (строка 7). Если это число превышает число доступных процессоров, то оно восстанавливается до максимального возможного числа. Для каждой новой рассматриваемой задачи для номера процессора устанавливается значение 1 (строка 13).

Алгоритм 12.1. *FT_RMSched*

```

1: Вход: установка  $n$  периодических задач  $T = \{\tau_1, \dots, \tau_n\}$ 
2: Output: Расписание  $T$ 
3: for all  $\tau_i \in T$  do
4:   while  $\exists$  неназначенных задач  $\tau_j^i$  do
5:     if  $\exists$  RM расписание для  $\tau_j^i \cup \{\text{задачи уже назначены } P_k\}$  и  $\exists \tau_j^w$  для любого  $w$  назначен  $P_k$  then
6:       назначить  $\tau_j^i$  процессору  $P_k$ 
7:     else  $k \leftarrow k + 1$ 
8:     end if
9:   if  $k > n$  then
10:     $n \leftarrow k$ 
11:   end if
12: end while
13:  $k \leftarrow 1$ 
14: end for

```

12.6. ОТКАЗОУСТОЙЧИВОСТЬ В РАСПРЕДЕЛЕННЫХ СИСТЕМАХ РЕАЛЬНОГО ВРЕМЕНИ

Аппаратный метод резервирования дублирующими компонентами и голосования часто используется в распределенных системах реального времени и в системах со встроенными элементами, таких, например, как самолет. Программное обеспечение резервирования с целевыми группами также обычно применяется в этих системах. Для начала мы определим режимы отказов в DRTS и опишем, как группы задач могут обеспечить отказоустойчивость.

12.6.1. Классификация отказов

Как упоминалось, отказы являются результатом ошибок. Отказы задач в распределенной системе могут классифицироваться следующим образом [4, 13]:

- *полный отказ*: задача перестает функционировать навсегда;
- *ошибка пропуска*: задача не может ответить на отправку и получение сообщения из-за пропуска сообщения;
- *сбой синхронизации*: сервер не отвечает в течение требуемого интервала времени;
- *ошибка ответа*: ответ сервера неправильный. Из-за ошибочного значения значение ответа неверно, и при переходе состояния сервер выдает неправильный поток управления;

- *византийский сбой*: византийская задача (задача со сложными, многоуровневыми связями) может быть вовлечена в любое произвольное действие – оно может дублировать сообщения, посылать нереальные сообщения и т. д.

Детектор неисправности – это оракул, который предоставляет информацию о состоянии задач в системе. Детектор неисправности может состоять из распределенных модулей, каждый из которых связан в системе с некоторой задачей [2].

12.6.2. Пересмотр состава целевых групп

В главе 6 мы рассмотрели группы задач как общий модуль промежуточного программного обеспечения в системе, как в нереальном режиме времени, так и в режиме реального времени. В этот модуль включены системные вызовы для создания группы, чтобы присоединиться к группе или выйти из нее, а также отправлять и получать многоадресные сообщения с многоадресным сообщением, отправленным всем членам группы. Целевая группа может быть плоской с равенством всех членов или иерархической с лидером (или координатором). В последнем случае запрос на некоторые услуги из группы передается координатору группы, который управляет требуемой функцией в группе. Группа называется *закрытой*, если участники группы могут отправлять сообщения только в группе. Отправлять сообщения вне группы можно в *открытой* группе. *Перекрывающаяся группа* позволяет своим членам быть членами других групп, тогда как *неперекрывающаяся группа* имеет всех членов, принадлежащих только ей. Членство в группе может управляться *групповым сервером*, который отслеживает созданные группы и членство в группе.

12.6.2.1. Надежная многоадресная связь

Многоадресная связь предполагает отправку сообщения *всем* членам группы. Мы будем различать *получение* и *доставку* сообщения. Получение сообщения подразумевает, что сообщение прибыло на узел, но оно еще не обработано, тогда как его доставка означает, что оно достигло верхнего промежуточного программного обеспечения или уровня приложения.

Многоадресная связь может быть получена с использованием различных подходов. В самом простом случае многоадресное сообщение может быть передано всем узлам сети, а затем фильтрация на каждом узле обеспечивает доставку только членам локальной группы. Протокол Ethernet поддерживает широко-вещательную доставку сообщений. Многоадресная связь может быть получена несколькими одноадресными рассылками, когда отправитель знает обо всех членах группы, как показано на рис. 12.8а, за счет большого количества подтверждений сообщения от каждого получателя. Этот тип многоадресной рассылки называется *основной многоадресной связью (B-Multicast)* и основан на надежной операции «один к одному».

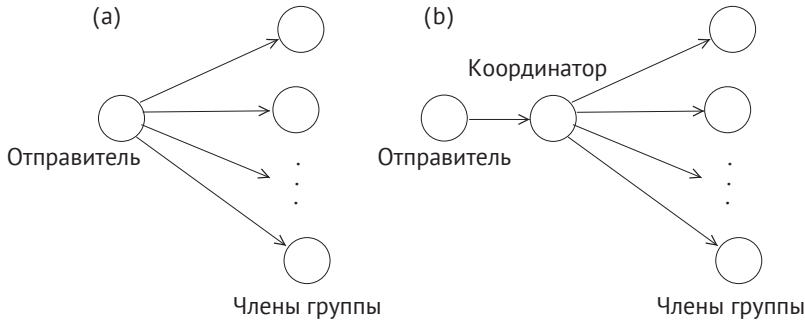


Рис. 12.8. Режим многоадресной связи

Иерархическая реализация обычно выполняется путем отправки сообщения координатору группы (серверу), который распределяет сообщение всем членам группы, как показано на рис. 12.8b. Мы выбрали этот режим работы при реализации DRTK целевых групп. Надежный многоадресный алгоритм может быть реализован с использованием алгоритмов B-Multicast, как в алгоритме 12.2, где показаны отправка сообщения m группе G и получение многоадресного сообщения с помощью задачи τ . Отправитель отправляет сообщение себе и всем членам группы, используя основной алгоритм многоадресной рассылки. После получения сообщения, если отправитель сообщения отличается от идентификатора получателя, он осуществляет многоадресную передачу в группу с использованием алгоритма B-Multicast, а затем доставляет сообщение промежуточному программному обеспечению/приложению. Надежный многоадресный протокол не выполняет упорядочение сообщений, и упорядоченный протокол доставки может быть реализован поверх надежного протокола доставки.

Алгоритм 12.2. *R-многоадресное сообщение*

```

1: received  $\leftarrow \emptyset$ 
2:
3: procedure Send_R- Multicast( $m, G$ )
4:   B-Multicast( $m, G$ )
5: end procedure
6:
7: procedure Receive_R- Multicast( $m, G$ )
8:   if  $m = \text{received}$  then
9:     received  $\leftarrow \text{received} \cup \{m\}$ 
10:    if  $m.\text{sender\_} = \text{my\_id}$  then
11:      B-Multicast( $m, G$ )
12:    end if
13:    R-Deliver( $m$ )
14:  end if
15: end procedure
  
```

Порядок сообщений

Упорядочение сообщений в группе задач необходимо для согласованного состояния задач. Порядок сообщений может быть обеспечен с использованием следующих методов:

- «*первым пришел – первым вышел*» (FIFO): это подразумевает, что протокол обеспечивает доставку сообщений в том же порядке, в котором они были отправлены. Другими словами, если многоадресная задача m_1 расположена до многоадресной задачи m_2 , то корректное получение задачи m_2 осуществляется, только если ранее получена задача m_1 . Этот протокол может быть реализован путем назначения сообщениям порядковых номеров. Отправитель вставляет порядковые номера для многоадресной рассылки сообщений, связанных с группой, и получатель располагает полученные сообщения в указанном порядке, как это показано в алгоритме 12.3. Когда сообщение с порядковым номером k больше ожидаемого порядкового номера s , он помещается в отложенную очередь и доставляется только тогда, когда все промежуточные сообщения с порядковыми номерами в интервале $(k - s)$ доставлены;
- *случайный порядок*: случайность задач в распределенной системе была специфицирована Лампортом (Lamport) и означает следующее [9]. Пусть отношение приоритета ($\rightarrow$) между событиями e_i и e_j означает, что e_i предшествует e_j .
 1. События e_i и e_j – это два события в одном узле распределенной системы.
 2. Событие e_i – отправка сообщения m , а e_j – прием сообщения m .
 3. Существует такое событие e_k , что $e_i \rightarrow e_k$ и $e_k \rightarrow e_j$.

Случайно упорядоченная многоадресная рассылка может быть получена использованием векторной концепции временной синхронизации, которую мы рассматривали в главе 6. Это показано в алгоритме 12.4, где p_j является идентификатором задачи, полученное сообщение имеет p_j в качестве отправителя, а вектором часов p_j является $V_i^G[N]$ для группы G . Нам нужно обеспечить два условия перед доставкой сообщения его получателю: любое сообщение, отправленное p_j до текущего сообщения, было уже доставлено, и любое сообщение, доставленное от p_j , также было доставлено. Эти две проверки в строке 10 алгоритма гарантируют упорядоченную доставку, и сообщение доставляется только при соблюдении данных условий;

- *общий порядок*: этот тип порядка гарантирует всем правильным задачам получение многоадресных сообщений в одном и том же порядке. Простым способом иметь общий порядок многоадресной рассылки является наличие токена, который циркулирует в сети в некотором предопределенном порядке. Любая задача, которая имеет токен, может осуществлять многоадресную рассылку с использованием многоадресной рассылки FIFO.

Алгоритм 12.3. *Порядок FIFO*

```

1:  $seq\_no_G \leftarrow 0$ 
2:
3: procedure Send_FIFO- Multicast( $m, G$ )
4:  $m.seq\_no_G \leftarrow seq\_no_G$ 
5:  $B\text{-Multicast}(m, G)$ 
6:  $seq\_no_G \leftarrow seq\_no_G + 1$ 
7: end procedure
8:
9: procedure Receive_FIFO- Multicast( $m, G$ )
10: if  $m.seq\_no_G = seq\_no_G$  then
11:  $FIFO\text{-Deliver}(m)$ 
12: else if  $m.seq\_no_G > seq\_no_G$  then
13: insert сообщение в отложенную очередь
14: когда все промежуточные сообщения доставлены, FIFO-Deliver(m)
15: end if
16: end procedure

```

12.7. РЕАЛИЗАЦИЯ DRTK

Мы реализуем случайную упорядоченную многоадресную рассылку, используя векторные часы с менеджером группы на каждом узле, применяемом для отправки и получения многоадресных сообщений. Многоадресное сообщение содержит векторные значения часов в поле данных сообщения, как показано на рис. 12.9. Нам нужно сформировать объединенную структуру в поле данных сообщения, чтобы иметь возможность использовать эту область для различных функций промежуточного программного обеспечения.

Алгоритм 12.4. *Случайный порядок*

```

1:  $V_i^G[N] \leftarrow 0$ 
2:
3: procedure Send_Casual- Multicast( $m, G$ )
4:  $m.V_i^G[j] \leftarrow V_i^G[j] + 1$ 
5:  $B\text{-Multicast}(m, G)$ 
6: end procedure
7:
8: procedure Receive_Casual- Multicast( $m, G$ )
9: вставить m в отложенную очередь сообщений
10: ждать, пока  $m.V_i^G[j] = V_i^G[j] + 1$  и  $m.V_j^G[k] \leq V_i^G[k], k \neq j$ 
11:  $Casual\text{-Deliver}(m)$ 
12:  $V_i^G[j] = m.V_i^G[j] + 1$ 
13: end procedure

```

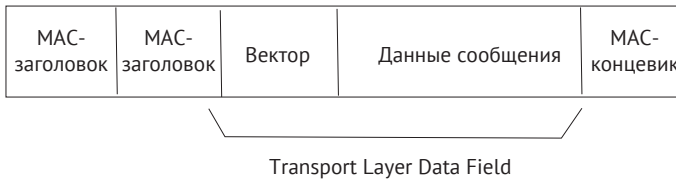


Рис. 12.9. Формат многоадресного сообщения

Мы реализуем этот протокол, предполагая, что многоадресные сообщения хранятся в групповом почтовом ящике, а не доставляются индивидуально каждому члену группы. Обратите внимание, что почтовый ящик менеджера группы и почтовый ящик группы – это два разных места хранения, и все сообщения группы теперь должны проходить через почтовый ящик менеджера группы. Задача ввода канального уровня (*DL_In*), описанная в главе 6, просто проверяет тип поля в сообщении, и если это многоадресное сообщение, помещает сообщение в почтовый ящик менеджера группы с именем *менеджер группы причинного (случайного) порядка* (*causal order group manager, COGM*). Эта задача постоянно ожидает в своем почтовом ящике, и когда он получает многоадресное сообщение, реализует правила векторных часов, чтобы доставить их получателям через групповой почтовый ящик. Любое сообщение, находящееся не в очереди, должно быть отложено и помещено в очередь. Блок управления структурой группы данных DRTK должен быть теперь изменен. Мы имеем массив векторных данных и очередь блока данных, которая хранится в блоке управления группой, как показано ниже. Любое задержанное сообщение после применения правила причинного порядка помещается в очередь данных.

```

/*****
Структура группы данных
*****/
/* group.h */
#define ERR_GR_NONE -2
#define N_MEMBERS 30

typedef struct group { ushort id;
    ushort state;
    ushort mailbox_id;
    vector_t vector;
    data_unit_queue data_que;
    ushort n_members;
    ushort local_members[N_MEMBERS];
}group_t;
typedef group_t* group_ptr_t;

```

Нам также необходима функция *compare_vector*, чтобы сравнить содержание двух векторов, используемых правилом причинного порядка, как показано в коде ниже.

```

/*****
Структура причинного порядка данных и функции
*****/
/* casual_order.c */

#define WAIT_MSG -1
#define DELIVER 1

int compare_vec(ushort* local_pt, ushort* rem_pt){
    int i;
    for (i=0;i<N_GROUP_MEM;i++)
        f (*local_pt++>*rem_pt++)
    return(WAIT_MSG);
    return(DELIVER);
}

```

Задача COGM может быть реализована следующим образом.

1. Получите многоадресное сообщение из канала передачи данных.
2. Проверьте содержание векторных часов во входящем сообщении m . Предположим, j является идентификатором задачи отправителя, а i – этим узлом; если $m.V_i^G[j] = V_i^G[j] + 1$ и $m.V_j^G[k] \leq V_i^G[k]$, $\forall k \neq j$, доставьте сообщение. Это выполняется путем хранения сообщения в почтовом ящике указанной группы. В противном случае сообщение помещается в таблицу векторов в блоке управления группой (лидера) с соответствующим вектором.
3. Когда сообщение доставлено, нам нужно проверить записи таблицы векторов в блоке управления группой. Любое сообщение, которое подчиняется правилу причинного порядка, также должно быть доставлено.

Вышеуказанные шаги применяются в задаче COGM, показанной ниже, где в очередь ставятся сообщения в векторной таблице, когда сообщение должно быть отложено. Это обновляет вектор в блоке управления группой, а затем ищет блок очереди данных полей вектора данных, чтобы проверить, нужно ли теперь доставлять какое-либо сообщение с задержкой. Если такое сообщение найдено, оно берется из очереди и доставляется в почтовый ящик группы.

Детали функции `take_data_unit` не показаны.

```

/*****
Менеджер причинного порядка группы
*****/
TASK COGM() {

    data_unit_ptr_t recvd_pt, data_pt;
    group_ptr_t group_pt;
    ushort group_id, sender_id, mbox_id2;
    ushort mbox_id1=&(tcb_tab[current_pid])->mailbox_id;

    while(TRUE) {
        recvd_pt=recv_mailbox_wait(mbox_id1);
    }
}

```

```

group_id=(recvd_pt->TL_header).receiver_id;
group_pt=&(group_tab[group_id]);
mbox_id2=group_tab[group_id].mailbox_id;
sender_id=data_pt->MAC_header.sender_id;
if (recvd_pt->data.vector[sender_id]==
group_pt->vector[sender_id]+1) &&
(compare_vec(data_pt->data.vector,group_pt->vector)) {
send_mbox_notwait(mbox_id2,data_pt);
group_pt->vector[sender_id]++;
data_pt=(group_pt->data_que).front;
while (data_pt->next!=NULL)
if (compare_vec(recvd_pt->data.vector,
data_pt->data.vector)) {
data_pt=take_data_unit(group_pt->data_que);
group_pt->vector[data_pt->sender_id]++;
send_mbox(mbox_id2,data_pt);
}
}
else
enqueue_data_unit(group_pt->data_que, recvd_pt);
}
}

```

Отправка многоадресного сообщения аналогична процедуре *send_msg_notwait* DRTK, за исключением того, что нам нужно увеличить количество сообщений для получателя и сохранить его в сообщении перед отправкой следующим образом, предполагая, что *data_pt* является адресом сообщения. Остальной код, который совпадает с *send_msg_notwait*, не показан.

```

vector_tab[current_tid].V[tid]=vector_tab[current_tid].V[tid]+1;
data_pt->data.V[tid]= vector_tab[current_tid].V[tid];

```

12.8. Контрольные вопросы

1. Какова связь между отказом, ошибкой и неисправностью?
2. Что такое надежная система?
3. Какова связь между надежностью, ненадежностью и неисправностью?
4. Как классифицируются отказы?
5. Каковы основные типы методов резервирования?
6. Каковы основные типы резервирования программного обеспечения?
7. Что такое контрольная точка?
8. Как работает блочный метод восстановления программного резервирования?
9. В чем разница между N-версией программирования и N-самоконтролем программирования?
10. Какие основные методы используются в статичном планировании отказоустойчивости в реальном времени?

11. Как модифицируется расписание RM для обеспечения безотказной работы в распределенной системе реального времени?
12. Каковы основные типы неисправностей в распределенной (в реальном времени) системе?
13. Как целевая группа используется для обеспечения отказоустойчивости?
14. Что такое надежная многоадресная рассылка?
15. Какова связь между векторными часами и причинным порядком?

12.9. ПРИМЕЧАНИЯ К ГЛАВЕ

В этой главе мы рассмотрели основные отказоустойчивые методы в системах не в реальном времени и в реальном времени. Резервирование в виде дублирования аппаратного или программного обеспечения является фундаментальной техникой для обеспечения отказоустойчивости. Основными типами резервирования являются аппаратные, информационные, временные и программные избыточности. Аппаратное резервирование обычно реализуется путем дублирования аппаратных компонентов и запуска программного обеспечения во всех компонентах, а затем голосования, чтобы получить правильный результат. Информационное резервирование достигается в основном за счет кодирования с использованием дополнительных данных для проверки ошибок. Время резервирования обеспечивается путем повторения вычислений в разные моменты времени, а затем сравнением результатов. Для резервирования программного обеспечения может быть использована техника одной версии или нескольких версий. Дополнительные программные модули, такие как исключение обработки и контрольные точки, добавляются в единый программный модуль для обнаружения отказов, и с их помощью осуществляется восстановление предыдущего и отказавшего блоков при программировании N-версии. В последнем используются N-самоконтролируемые методы программирования. Методы резервирования программного обеспечения Multiple-version обычно используют N версий программного обеспечения, которые запускаются, а затем сравниваются полученные результаты.

Все эти методы резервирования могут быть реализованы и реализуются в системах реального времени. Кроме того, к основному требованию, предъявляемому к системе реального времени, должна быть добавлена возможность их обеспечения при наличии неисправностей. Мы представили обзор некоторых исследований по этой теме, основанных на статичном и динамическом планировании. Подробный обзор некоторых тем, которые мы обсуждали в этой главе, может быть найден в [8], а отказоустойчивые методы проектирования рассмотрены в [5].

12.10. УПРАЖНЕНИЯ

1. Создайте горизонтальные и вертикальные нечетные биты четности, исходя из четырех четырехбитных данных в табл. 12.2.

2. Разработайте четырехбитное контрольное слово, которое будет добавлено к двоичным данным 1000 0110 11 с использованием метода обнаружения ошибок CRC и использованием полиномиального слова 11011.
3. Сравните надежные протоколы многоадресной рассылки FIFO, казуальные и атомарные.
4. Реализуйте протокол многоадресной рассылки FIFO, используя структуру DRTK. Напишите Си-код с краткими комментариями.
5. Предложите протокол на основе токенов для общего количества упорядоченных многоадресных рассылок. Начните проектирование с создания протокола FSM. Напишите Си-код, который будет являться интерфейсом для DRTK, с небольшими комментариями.

Таблица 12.2. Пример четырехбитных данных для упражнения 1

c_4	c_3	c_2	c_1	c_0
0	1	1	0	
1	0	1	1	
1	0	1	0	
1	1	1	1	

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. *Burns A., Wellings A.* (2009) Real-time systems and programming languages, 4th edn, Chap. 2. Addison-Wesley.
2. *Chandra T. D., Toueg S.* (1996) Unreliable failure detectors for reliable distributed systems. *J ACM* 43 (2): 225–267.
3. *Chen L., Avizienis A.* (1978) N-version programming: a fault tolerance approach to reliability of software operation. In: Digest of 8th annual international symposium on fault tolerant computing, pp. 3–9.
4. *Defago X., Schiper A., Urban P.* (2004) Totally ordered broadcast and multicast algorithms: taxonomy and survey. *J ACM Comput Surv* 36 (4): 372–421.
5. *Dubrova E.* (2013) Fault tolerant design, Chap. 4. Springer, New York.
6. *Lee P. A., Anderson T.* (1990) Fault tolerance: principles and practice, 2nd edn. Springer.
7. *Kandasamy N., Hayes J. P., Murray B. T.* (2000) Task scheduling algorithms for fault tolerance in real-time embedded systems. In: Avresky DR (ed) Dependable network computing. The Springer international series in engineering and computer science, vol. 538. Springer, Boston, MA.
8. *Koren I., Krishna C. M.* (2007) Fault tolerant systems. Morgan-Kaufman.
9. *Lamport L.* (1978) Time, clocks, and the ordering of events in a distributed system. *Commun ACM* 21 (7): 558–565.
10. *Laprie J. C.* (1985) Dependable computing and fault tolerance: concepts and terminology. In: The 15th international symposium on fault-tolerant computing, pp. 2–11 References 297.

11. *Oh Y., Son S.* (1994) Scheduling hard real-time tasks with tolerance of multiple processor failures. *Microprocess Microprogr* 40: 193–206.
12. *Punnekkat S.* (1997) Schedulability analysis for fault tolerant real-time systems. PhD thesis, Department of Computer Science, University of York.
13. *Tanenbaum A. S., Van Steen M.* (2006) Distributed systems, principles and paradigms, Chap. 8. Prentice Hall.

Глава 13

Тематический пример: мониторинг окружающей среды по беспроводной сенсорной сети

13.1. ВВЕДЕНИЕ

Беспроводная сенсорная сеть (wireless sensor network, WSN) состоит из небольших вычислительных узлов, каждый из которых оснащен средствами беспроводной связи. Сеть WSN обычно используется в опасных средах для мониторинга и спасательных операций. Мы покажем пример реализации тематического исследования: мониторинг сложного здания с помощью сети беспроводных датчиков с использованием примера ядра DRTK распределенной операционной системы реального времени. Сеть WSN будет использоваться для мониторинга здания и защиты от пожара, наводнений и злоумышленников. Этот небольшой проект поможет лучше понять многие концепции, изложенные в данной книге. Разработанная система будет распределенной системой реального времени со встроенными элементами, и мы будем реализовывать все этапы проектирования, описанные в книге, начиная с высокоуровневого дизайна, включая коды языка C. Мы начинаем с требований спецификации (system requirements specification, SRS) и сформируем этот документ в обычном порядке. Следующим шагом является выполнение временного анализа, а затем создается проект высокого уровня с учетом совместного аппаратно-программного проектирования. Инструмент, используемый для высокого уровня дизайна, – это диаграммы потоков данных (data flow diagrams, DFD), и мы начинаем эту процедуру с формирования контекстной схемы системы. Детальный дизайн системы включает в себя проектирование Си-функций, которые должны быть реализованы задачами DRTK.

Мы предоставляем также альтернативную реализацию этого проекта без использования DRTK. В этом случае шаги, которые необходимо выполнить, описаны как практический метод дизайна в главе 10, и функции реализуются потоками POSIX со всеми доступными методами синхронизации потоков POSIX. Для связи потоков мы используем современный метод связи между потоками, описанный в главе 4.

13.2. ВОПРОСЫ ПРОЕКТИРОВАНИЯ

Мы будем использовать этот пример для реализации всех этапов дизайна распределенной системы реального времени, которые определяются следующим образом.

- *Требования к спецификации*: официально сформулировать требования пользователя/клиента. В конце этого этапа мы подготовим документ SRS.
- *Анализ времени*: прежде чем выполнить дизайн высокого уровня, мы должны проверить, как выполняются необходимые временные требования.
- *Проектирование высокого уровня*: мы будем использовать блок-схемы данных для постепенного усовершенствования системы функции до уровня задач.
- *Детальный дизайн*: мы реализуем необходимые функции, используя задачи реального времени, которые будут представлены задачами DRTK.
- *Кодирование*: все коды будут выполнены на языке программирования C.

Мы опишем два способа выполнения проекта, используя DRTK в качестве ядра операционной системы и интерфейс POSIX.

13.3. ТРЕБОВАНИЯ К СПЕЦИФИКАЦИИ

Основным требованием является мониторинг опасных условий большого, сложного здания с помощью сети WSN. В документе SRS указывается следующее.

1. **Введение**: контролируемое здание имеет 25 этажей с 20–30 комнатами на каждом, подвалом и автостоянкой. Здание должно быть защищено от огня, наводнения и утечки воды. Мониторинг температуры, дыма и влажности должен выполняться непрерывно, и любой злоумышленник должен быть обнаружен.
2. **Функциональные требования**:
 - а. Температура каждой комнаты, каждого этажа и других мест в здании должна контролироваться. В каждой комнате, на каждом этаже и других различных местах здания установлены светодиодные дисплеи (LED) для оценки температуры. Когда температура данного места становится больше или меньше указанных предельных значений, должны быть активированы звуковые и дисплейные тревоги. Управляющий компьютер должен обнаружить место, где значения находятся вне пределов.

- b. Защита от огня должна быть обеспечена путем обнаружения дыма в каждой комнате, на каждом этаже и в различных других помещениях здания. В каждой комнате, на каждом этаже и в других местах здания будет звуковая и световая сигнализация о пожаре. В случае пожара в коридорах будут активированы указатели выхода, будут включены звуковые и световые сигналы. Управляющий компьютер должен определить этаж возникновения пожара.
 - c. Влажность здания должна контролироваться для принятия мер в случае наводнения или других протечек воды. Для оценки влажности в каждой комнате, на каждом этаже и в других местах здания будет звуковая и световая сигнализация, должна быть установлена сигнализация о высокой влажности, и менеджер должен быть проинформирован о таком месте.
 - d. Когда здание закрыто, должен быть обнаружен любой злоумышленник и установлено его местонахождение. Должна быть установлена сигнализация о нарушении, и компьютер менеджера должен быть проинформирован о местонахождении злоумышленника.
 - e. Все значения этих параметров в здании должны отображаться в управляющем компьютере в сжатой форме, такой как средняя температура и значения влажности на каждом этаже.
3. Нефункциональные требования.
- a. Система должна быть отказоустойчивой. Неспособность обнаружить дым, тепло или влажность в каком-то одном месте не должна вызывать полного отказа системы.
 - b. Система должна гарантированно работать в течение 5 лет.

Этот документ является контрактом между пользователем/заказчиком и разработчиком системы. Кроме того, они будут обеспечивать сопровождение на протяжении всего процесса проектирования.

13.4. Временной анализ и функциональные характеристики

Внимательное рассмотрение требований показывает, что временные ограничения в проектируемой системе не жесткие. Нам нужно периодически вводить в систему медленно меняющиеся значения температуры и влажности, а обнаружение злоумышленника должно произойти в любое время. Система имеет характеристики как по времени, так и по событию. Использование узлов сети WSN, каждый из которых взаимодействует с применением многократной передачи сообщений, удобно для дизайнера, так как нет необходимости в кабельной связи, а обработка, выполняемая каждым сенсорным узлом, не является затруднительной.

Теперь тот момент, когда мы должны рассмотреть совместное проектирование аппаратного и программного обеспечения. Использование сети WSN, которая применяет многократный обмен данными, влияет на структуру программного обеспечения системы. Остовное дерево имеет корень в узле-приемнике данных сети WSN, в данном случае – управляющем компьютере, и обычно используется для многошаговой связи между приемником и обычными узлами в сети WSN. Поэтому нам изначально нужно построить остовное (связующее) дерево сети WSN. Необходимая сокращенная форма отображения средствами менеджера означает, что в промежуточных узлах должна выполняться некоторая форма предварительной обработки до того, как данные датчика доставляются в приемник. Это требование можно удовлетворить, сгруппировав узлы датчиков, которые физически расположены близко друг к другу *в кластерах с управлением* операциями с данными и трафиком связи в кластерах. Обратите внимание, что рассмотрение специального аппаратного обеспечения изначально существенно повлияло на дизайн программного обеспечения высокого уровня.

13.5. СВЯЗУЮЩЕЕ ДЕРЕВО И КЛАСТЕРИЗАЦИЯ

Связующее (остовное) дерево T графа $G = (V, E)$ является ациклическим подмножеством G с тем же множеством вершин, т. е. $T = (V, E')$, где $E' \subseteq E$. Другими словами, каждая вершина является частью связующего дерева, но только подмножество (или равный набор) ребер принадлежит T . Формирование связующего дерева сети WSN – удобный способ связи в таких сетях, поскольку отправители и получатели сообщений могут быть определены выборочно. Передача от узла-приемника может быть достигнута просто путем отправки сообщения детям каждого родительского узла, кроме листовых узлов. После построения узлы сети WSN становятся частью связующего дерева и разделены на несколько кластеров.

Наличие связующего дерева и кластерной структуры может быть получено с помощью алгоритма, описанного в [1]. Типы сообщений в этом алгоритме: PROBE, ACK и NACK, и каждый узел в сети становится узлом, управляющим кластером (clusterhead, CH), промежуточным узлом (intermediate node, INODE) или листом (LEAF) связующего дерева в конце алгоритма. Узел приемника сети WSN запускает алгоритм, передавая сообщение PROBE все соседям в пределах его дальности передачи. Любой узел, который получает сообщение впервые, помечает отправителя как своего родителя, отправляя сообщение ACK отправителю, и отклоняет любые другие сообщения PROBE, полученные после этого, сообщением NACK. Эта процедура обеспечивает формирование связующего дерева. Однако нам нужен еще один механизм формирования кластеров. Он достигается за счет резервирования поля *hop_count* в сообщениях PROBE. Любой узел, который получает сообщение PROBE впервые, также проверяет наличие *hop_count* в сообщении, и если это значение равно указанному значению глубины кластера (*clust_depth*), он присваивает себе состояние CH и сбрасывает

hop_count перед передачей сообщения PROBE соседям. Любой другой узел, который получает *hop_count* меньше указанного значения, меняет свое состояние для INODE и увеличивает *hop_count* в сообщении перед его передачей соседям. Автомат FSM этого алгоритма изображен на рис. 13.1.

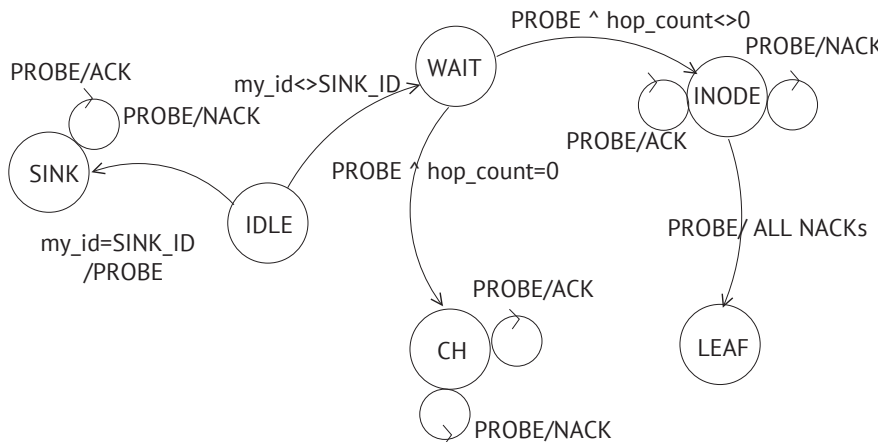


Рис. 13.1. Конечный автомат FSM задачи Cluster

Таблица 13.1. Таблица FSM для задачи кластер (cluster)

Входы		PROBE	ACK	NACK
Состояния	Не занят (IDLE)	NA	NA	NA
	Ожидание (WAIT)	act_10	NA	NA
	Промежуточное (INODE)	act_20	act_21	act_22
	Лист (LEAF)	act_30	NA	NA

Начиная с состояния IDLE проверка идентификатора узла классифицирует узел как приемник (sink) или любой другой узел. Обратите внимание, что это состояние является внутренним без ожидания какого-либо ввода. Узел в состоянии INODE может быть переведен в состояние LEAF, если все его соседи имеют ответы сообщениями NACK (принят ошибочно, negative acknowledge) на сообщения PROBE, что означает, что он может не иметь каких-либо детей, и, следовательно, это узел LEAF. Таблица автомата FSM, которая поможет нам в реализации алгоритма, приведена в табл. 13.1. При реализации кода мы будем полагать, что каждый узел имеет уникальный идентификатор. Далее будем полагать, что каждый узел сети WSN оснащен ядром операционной системы DRTK, позволяющим использовать все системные вызовы DRTK, которые мы разработали наряду с сетевыми методами связи и управления задачами подпрограммы. Один из способов реализации FSM показан в коде Си ниже. У нас есть заголовочный файл «tree_clust.h», который определяет типы сообщений и состояния показанных узлов, которые также показаны ниже.

```
// tree_clust.h
#define PROBE 1
#define ACK 2
#define NACK 3
#define CH 1
#define INODE 2
#define LEAF 3
#define N_STATE 4
#define N_INPUT 3
```

Мы можем написать код для алгоритма, называемого *Tree_Cluster*, основанного на таблице активности автомата FSM. Заметьте, что первый вход поля данных сообщения является сохраненным для переменной *hop_count*.

```
/*
*****
Процедура кластеризации
*****
*/
/* tree_clust.c */ #include "tree_clust.h"

fsm_table_t sender_FSM[N_STATE][N_INPUT];
mailbox_ptr_t mbox_pt;
data_unit_ptr_t data_pt;
ushort my_id, my_parent;
int my_state;
void act_10() {
    my_parent=data_pt->MAC_header.sender_id;
    my_neighbors[neigh_index++]=my_parent;
    if (data_pt->data[0]==N_HOPS+1) {
        current_state=CH;
        data_pt->data[0]=0;
    }
    else current_state=INODE;
    data_pt->MAC_header.sender_id=my_id;
    data_pt->MAC_header.type=ACK;
    data_pt->data->[0]++;
    send_mailbox_notwait(&(task_tab[System_Tab.DL_Out_id])
->mailbox_id,data_pt);
}

void act_20() {
    data_pt->sender=my_id;
    data_pt->MAC_header.type=NACK;
    send_mailbox_notwait(&(task_tab[System_Tab.DL_Out_id])
->mailbox_id,data_pt);
}
void act_21() {
    my_neighbors[neigh_index++]=data_pt->sender;
    return_data_unit(System_Tab.userpool1,data_pt);
}
void act_22(){
    n_rejects++;
```



```

if(n_rejects==N_NEIGHBORS)
my_state=LEAF;
return_data_unit(System_Tab.userpool1,data_pt);
TASK Tree_Cluster() {
clust_FSM[0][0]=NULL; clust_FSM[0][1]=NULL;
clust_FSM[0][2]=NULL; clust_FSM[1][0]=act_10;
clust_FSM[1][1]=NULL; clust_FSM[1][2]=NULL;
clust_FSM[2][0]=act_20; clust_FSM[2][1]=act_21;
clust_FSM[2][2]=act_22; clust_FSM[3][0]=act_20;
clust_FSM[3][1]=NULL; clust_FSM[3][2]=NULL;
mboxpt=&(tcb_tab[current_pid].rec_mailbox);
current_state=IDLE;
while(TRUE) {
data_pt=recv_mailbox_wait((task_tab[current_tid]).mailbox_id);
(*sender_FSM[current_state][data_pt->type])();
}
}

```

Сеть WSN, разделенная на пять кластеров $C_1, \dots, C_6$ с $n_hops=2$ в связующем (остовном) дереве, показана на рис. 13.2. Обратите внимание, что периодический вызов алгоритма приемником позволяет этому алгоритму создавать кластеры в связующем дереве в специальной (ad hoc) мобильной сети.

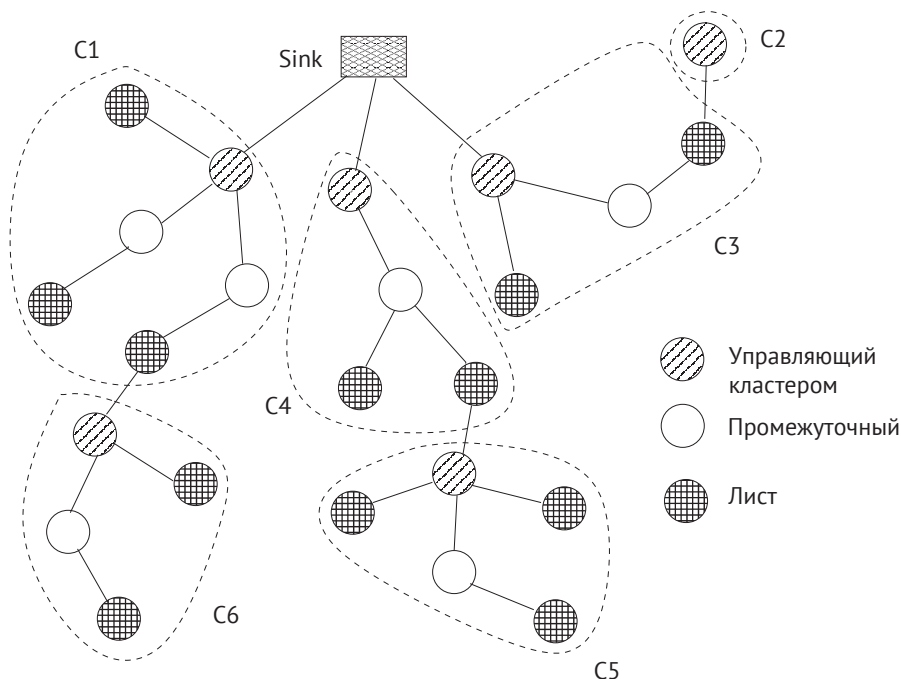


Рис. 13.2. Сеть WSN с кластерами в связующем дереве

13.6. ВОПРОСЫ ПРОЕКТИРОВАНИЯ

У нас будет высокоуровневый и детальный дизайн системы мониторинга сети WSN, основанный на типе узлов. Узел может быть одного из четырех типов: *листовой узел*, *промежуточный узел*, *управляющий кластером узел (голова кластера)* и *приемник*. Нам нужно в следующих разделах для каждого из них на основе требований спроектировать диаграммы потоков данных (dataflow diagram, DFD). Типы сообщений обхода краев связующего дерева будут следующих типов:

- *сообщение узла*: передается с конечного узла кластера на его родительский узел. Оно содержит три значения датчика и условия тревоги на этих датчиках. Этот тип сообщения также используется для локальной информации от датчиков;
- *промежуточное сообщение*: передается с промежуточного узла на другой промежуточный узел или голове кластера. Это сообщение содержит значения датчика детей промежуточного узла и состояния тревоги на этих датчиках. Обратите внимание, что этот узел не обрабатывает нелокальные значения датчика; он только объединяет эти значения;
- *сообщение головы кластера*: это сообщение может передаваться между узлами любого типа, чтобы быть переданным приемнику. Оно несет средние значения датчика в кластере и возможные условия тревоги.

Мы будем формировать эти структуры сообщений при разработке программного обеспечения узла. Область данных блока данных должна быть сформирована как объединение этих структур. Мы имеем следующие предположения, которые могут быть реализованы путем изменения алгоритма *Tree_Cluster*:

- каждый кластер в сети имеет уникальный идентификатор;
- каждый узел в сети имеет уникальный идентификатор, сформированный путем объединения локального идентификатора с идентификатором кластера;
- каждый узел в сети знает идентификатор своего кластера;
- каждый узел в сети знает номер и идентификаторы своих детей из алгоритма *Tree_Cluster*.

Нам нужна базовая операция конвергенции, часто используемая в структурированной сети WSN, представленной связующим деревом. Эта операция выполняется путем сбора сообщений от детей, а затем передачей их родителю до тех пор, пока данные не будут собраны на узле приемника. Структура сообщения в сети изображена на рис. 13.3.

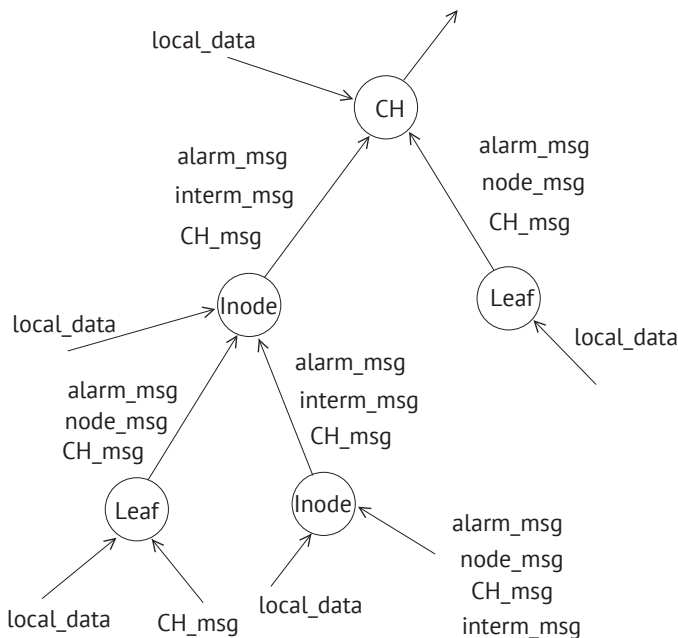


Рис. 13.3. Иерархия сообщений в сети

Обратите внимание, что каждый узел в этой иерархии обрабатывает данные локального датчика.

- *Конечный узел (лист)*: этот узел получает данные своего локального датчика и отправляет значения датчика в *node_msg* его родителю. Если возникает состояние тревоги, *alarm_msg* тотчас же отправляется к родителю. Входящее сообщение также отправляется управляющему кластером узлу (*CH_msg*) от нижних кластеров к его родителю.
- *Промежуточный узел*: сообщения промежуточному узлу (INODE) представляют собой *alarm_msg*, *node_msg* из листа, *interm_msg* из любого потомка, который не является листом, и любого входящего *CH_msg* из кластеров нижнего уровня.
- *Управляющий кластером узел (голова кластера)*: сообщения на этот узел похожи на входные сообщения в промежуточный узел.

13.7. ЛИСТОВОЙ УЗЕЛ

Листовой узел – это лист кластера, который может не быть листом связующего дерева. Следовательно, он может получать сообщения от управляющих узлов кластеров нижнего уровня. Однако из-за структуры связующего дерева он может не получать сообщения от промежуточного узла. Сначала мы опишем дизайн высокого уровня этого узла.

13.7.1. Дизайн высокого уровня

Конечный, листовой узел в основном периодически собирает данные датчика температуры и влажности и отправляет эти данные своему родителю. После получения прерывания, вызванного появлением злоумышленника, он также отправляет эти данные родителю и устанавливает кнопку тревоги, выполнив следующие шаги:

- 1) периодически собирает данные датчика и отправляет их родителю;
- 2) если данные выходят за пределы диапазона, включает данные сигнализации;
- 3) при обнаружении злоумышленника немедленно отправляет эти данные родителю и включает охранную сигнализацию;
- 4) после получения сообщения узла, управляющего кластером от узла, управляющего кластером более низкого уровня, отправляет его родителю.

Следовательно, основными входными данными для конечного узла являются входные данные датчиков и сообщение узла, управляющего кластером, как показано на контекстной диаграмме рис. 13.4. Выходами являются промежуточные сообщения и индикация тревоги.

Диаграмма уровня 1 на основе контекстной диаграммы может быть сформирована так, как показано на рис. 13.5. Данные от датчиков периодически получают задачи *Temp* (Температура) и *Humid* (Влажность), которые сначала проверяют, находятся ли эти данные в допустимом диапазоне, и помещают данные в память данных *alarm_data* для пробуждения задачи *Alarm* (Тревога), отображают их, если они выходят из пределов диапазона. В любом случае данные хранятся в памяти *sensor_data* для отправки родителю. Любой нарушитель в контролируемой области обнаруживается задачей *Accel* (Катализатор, Датчик ускорения), которая хранит данные как в памяти, активирующей тревогу, так и в памяти отчета перед родителем. Задача *Leaf_In* постоянно ожидает сообщений из сети и отправляет все входящие сообщения узла, управляющего кластером, своему родителю с помощью задачи *Leaf_Out*.

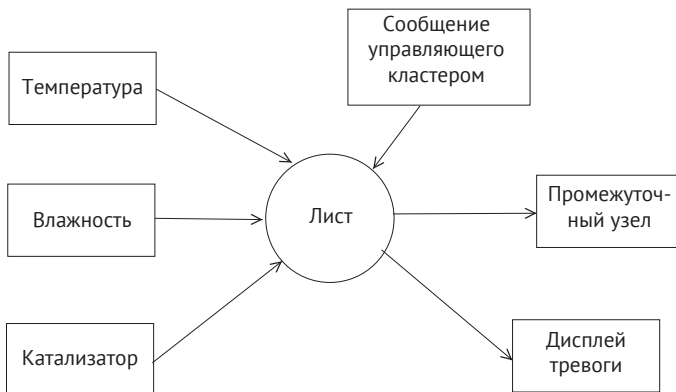


Рис. 13.4. Контекстная диаграмма узла Лист

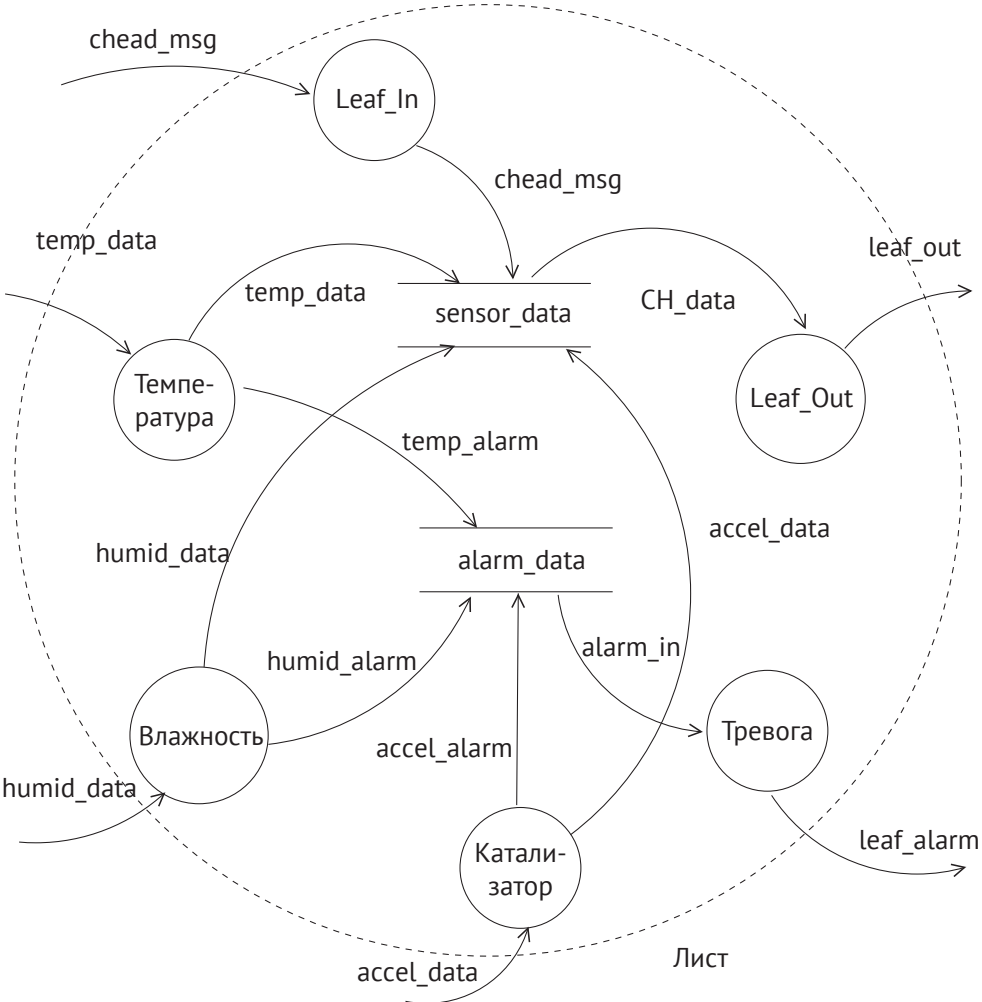


Рис. 13.5. Диаграмма DFD узла Лист

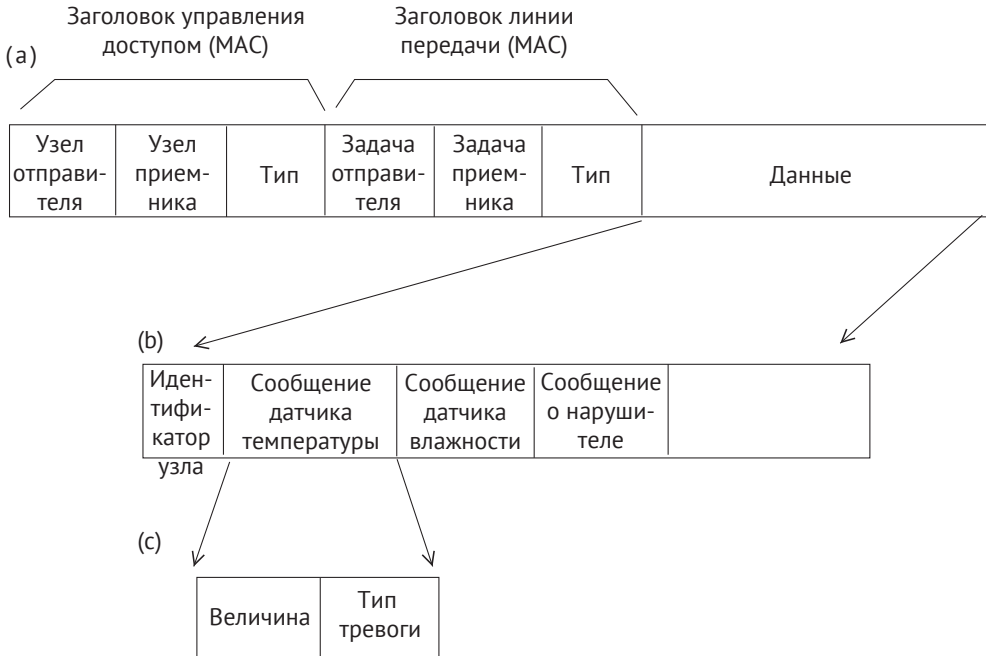


Рис. 13.6. Сообщения: а – общий формат; b – узла; c – датчика

13.7.2. Детальная разработка и реализация

Первое, что следует отметить, – это то, что кружки на диаграмме уровня 1 могут быть удобно представлены процедурами. Мы интуитивно приходим к такому выводу, рассматривая, что должна сделать каждая задача. Например, задача *Temp* должна задержаться на некоторый период времени, а затем прочитать значение температуры, проверить ее диапазон и сохранить данные в памяти данных. Общий формат сообщения, которое передается между задачами и проходит по сети, указан на рис. 13.6а, а анализ области данных для сообщений узла показан на рис. 13.6b. Сообщения датчика, показанные на рис. 13.6с, состоят из значения датчика и состояния тревоги в качестве сообщений данных самого низкого уровня. Заголовок MAC показывает отправляющий и принимающий узлы и тип на этом уровне, такой как одноадресная передача, многоадресная передача или широковещательная передача сообщений, или управляющее сообщение, такое как подтверждение (ACK) либо отрицательное подтверждение (NACK). Заголовок транспортного уровня используется для связи между задачами, поскольку основной функцией этого уровня является обеспечение связи между пользовательскими/прикладными задачами.

Структура сообщения, которая заполняется задачами ввода от датчика, имеет следующий вид структуры со значением датчика и условием тревоги.

```

/*****
Структура сообщения датчика
*****/
typedef struct {
    double value;
    int alarm_t;
} sensor_msg_t, alarm_msg_t, *sensor_msg_ptr_t;

```

Сообщение узла состоит из идентификатора узла и данных от трех датчиков.

```

/*****
Сообщение данных узлами
*****/
typedef struct {
    ushort node_id;
    sensor_msg_t temp;
    sensor_msg_t humid;
    sensor_msg_t intrude;
} node_msg_t;
Файл "local.h" содержит все постоянные:

```

```

//local.h
#define TEMP_LOW 5
#define TEMP_HIGH 2
#define HUMID_LOW 12
#define HUMID_HIGH 120
#define ERR_TEMP_LOW -1
#define ERR_TEMP_HIGH -2
#define ERR_HUMID_LOW -3
#define ERR_HUMID_HIGH -4
#define ERR_INTRUDER -5
#define N_CLUS_NODE 20
#define N_CHILD_MAX 10
#define SENSOR_MSG 1
#define NODE_MSG 2
#define INTERM_MSG 3
#define CHEAD_MSG 4
#define TEMP_MSG 1
#define HUMID_MSG 2
#define INTRUDER_MSG 3

```

Обычно мы использовали бы модель FSM для представления кружков на диаграмме уровня 1, но на этом уровне не так много внешних взаимодействий функций, и, следовательно, мы можем продолжить работу без FSM. Мы полагаем, что DRTK присутствует в каждом узле сети WSN, что позволяет нам использовать почтовые ящики в качестве мест хранения данных, и эти устройства можно объединить с датчиками. Первая задача, которую мы будем коди-

ровать, – это *Temp*. Она спит и просыпается, чтобы считать данные датчика с блока управления температурного устройства. После проверки допустимых пределов она отправляет данные в соответствующие почтовые ящики, как показано ниже. Мы будем полагать, что идентификаторы задач и идентификаторы устройств объявлены в таблице системы. Перед отправкой сообщения в задачу *Leaf_Out* задача ввода от датчика должна заполнить сообщение датчика данными датчика, как показано в коде ниже.

```

/*****
Задача Temp
*****/
TASK Temp() {
    data_unit_ptr_t data_pt1, data_pt2;
    int dev_id, mbox_id1, mbox_id2, mbox_id3;
    ushort dev_id=System_tab.temp_devid;
    ushort mbox_id1=task_tab[current_tid].mailbox_id;
    ushort mbox_id2=task_tab[System_Tab.Leaf_Out_id].mailbox_id;
    ushort mbox_id3=task_tab[System_Tab.Alarm_id].mailbox_id;

    while(TRUE) {
        delay_task(current_tid, System_Tab.LOCAL_DELAY);
        data_pt1=get_data_unit(System_Tab.userpool1);
        read_device(dev_id,&(data_pt1->data)->value,sizeof(double));
        data_pt1->TL_header.type=TEMP_MSG;
        if (data_pt1->data.value < TEMP_LOW ||
            data_pt1->data.value > TEMP_HIGH) {
            data_pt2=get_data_unit(System_Tab.userpool1);
            data_pt1->TL_header.type=ALARM_MSG;
            if (data_pt1->data.value < TEMP_LOW) {
                data_pt1->data.sensor_msg.alarm_t=ERR_TEMP_LOW;
                data_pt2->data.alarm_msg.alarm_t=ERR_TEMP_LOW;}
            else if (data_pt1->data.value > TEMP_HIGH) {
                data_pt1->data.sensor_msg.alarm_t=ERR_TEMP_HIGH;
                data_pt2->data.alarm_msg.alarm_t=ERR_TEMP_HIGH;
                send_mailbox_notwait(mbox_id3,data_pt2); }
            }
            send_mailbox_notwait(mbox_id2,data_pt1);
        }
    }
}

```

Задача *Humid* записывает влажность подобным же образом, за исключением того, что тестирование влажности обеспечивает уровни, показанные в коде ниже.

```

/*****
Задача Humid
*****/
TASK Humid() {
    data_unit_ptr_t data_pt1, data_pt2;
    int dev_id, mbox_id1, mbox_id2, mbox_id3;

```



```

dev_id=System_tab.humid_dev_id;
ushort mbox_id1=task_tab[current_tid].mailbox_id;
ushort mbox_id2=task_tab[System_Tab.Leaf_Out_id].mailbox_id;
ushort mbox_id3=task_tab[System_Tab.Alarm_id].mailbox_id;
while(TRUE) {
    delay_task(current_tid, System_Tab.LOCAL_DELAY);
    data_pt1=get_data_unit(System_Tab.userpool1);
    read_device(dev_id,&(data_pt1->data),sizeof(double));
    data_pt1->TL_header.type=HUMID_MSG;
    if (data_pt1->data.value < HUMID_LOW ||
        data_pt1->data.value > HUMID_HIGH) {
        data_pt2=get_data_unit(System_Tab.userpool1);
        data_pt1->TL_header.type=ALARM_MSG;
        if (data_pt1->data.value < HUMID_LOW) {
            data_pt1->data.sensor_msg.alarm_t=ERR_HUMID_LOW;
            data_pt2->data.alarm_msg.alarm_t=ERR_HUMID_LOW; }
        else if (data_pt1->data.value > HUMID_HIGH) {
            data_pt1->data.sensor_msg.alarm_t=ERR_HUMID_HIGH;
            data_pt2->data.alarm_msg.alarm_t=ERR_HUMID_HIGH; }
        send_mailbox_notwait(mbox_id3,data_pt2);
    }
    send_mailbox_notwait(mbox_id2,data_pt1);
}
}

```

Задача *Accel_Int*, запускаемая событием и пробуждающаяся при прерывании от датчика ускорения (катализатора). Эта задача ожидает пробуждения и посылает при пробуждении сообщение тревоги задаче тревоги.

```

/*****
Обработчик прерывания от датчика ускорения
*****/
TASK Accel_Int() {
    data_unit_ptr_t data_pt;

    ushort mbox_id1=task_tab[current_tid].mailbox_id;
    ushort mbox_id2=task_tab[System_Tab.Leaf_Out_id].mailbox_id;
    ushort mbox_id3=task_tab[System_Tab.Alarm_id].mailbox_id;

    while(TRUE) {
        data_pt=recv_mailbox_wait(mbox_id1);
        data_pt->TL_header.type=ALARM_MSG;
        data_pt->data.alarm_t=ERR_INTRUDER;
        send_mailbox_notwait(mbox_id2,data_pt);
        send_mailbox_notwait(mbox_id3,data_pt);
    }
}

```

Задача *Leaf_Alarm* – просто задача ожидания любых сообщений тревоги в своем почтовом ящике, расшифровки их и активирования соответствующих

отображений. Мы будем полагать, что есть один драйвер для активации всех тревог, который проверяет тип поля в функции *write_dev*.

```

/*****
Задача Alarm
*****/
TASK Leaf_Alarm(ushort dev_id) {

    data_unit_ptr_t data_pt;
    int err_type;
    ushort mbox_id=task_tab[current_tid].mailbox_id;
    ushort dev_id=System_tab.alarm_dev_id;

    while(TRUE) {
        data_pt=recv_mailbox_wait(mbox_id);
        err_type=data_pt->data.sensor_msg.alarm_t;
        write_dev(dev_id,&err_type,sizeof(int));
    }
}

```

Задача *Leaf_In* ожидает сообщений из сети в своем почтовом ящике и передает их задаче *Leaf_Out*, как показано в коде ниже. Мы предполагаем существование задачи передачи данных, которая получает сообщения с помощью сетевых драйверов и помещает их в почтовом ящике задачи *Leaf_In*. Для конкретной сети функция этой задачи может быть встроена в задачу передачи данных.

```

/*****
Задача Leaf_In
*****/
TASK Leaf_In() {

    data_unit_ptr_t data_pt;
    ushort mbox_id=task_tab[current_tid].mailbox_id;
    ushort dev_id=System_Tab.alarm_dcbid;
    while(TRUE) {
        data_pt=recv_mailbox_wait(mbox_id);
        if (data_pt->TL_header.type==CHEAD_MSG) {
            mbox_id=task_tab[System_Tab.Leaf_Out_id].mailbox_id;
            send_mailbox_notwait(mbox_id,data_pt);
        }
    }
}

```

Последняя задача, которая будет реализована, – это задача *Leaf_Out*, ожидающая периодических сообщений датчика, собирающая их в одно сообщение узла и передающая их своему родителю. Она вызывает функцию *check_local*, чтобы найти тип сообщения и провести необходимую «уборку», как показано ниже. Эта функция также передает любое входящее сообщение СН и аварий-

ное сообщение без немедленного изменения их содержимого, как показано в коде ниже.

```
data_unit_ptr_t data_pt, data_pt2;
ushort my_id, parent_id, sensor_count=0;
ushort mbox_id1=&(task_tab[current_tid])->mailbox_id;
ushort mbox_id2=&(task_tab[System_Tab.DL_Out_id])->mailbox_id;

int check_local(ushort type, data_ptr_t data_pt) {

    switch(type) {
    case TEMP_MSG:
        data_pt->data.node_msg.temp_val=recvd_pt
        ->data.sensor_msg.value;
        sensor_count++; break;
    case HUMID_MSG:
        data_pt->data.node_msg.humid_val=recvd_pt
        ->data.sensor_msg.value;
        sensor_count++; break;
    case ALARM_MSG:
    case CHEAD_MSG:
        data_pt2->MAC_header.sender_id=System_Tab.this_node;
        data_pt2->MAC_header.receiver_id=my_parent;
        send_mailbox_notwait(mbox_id2,data_pt2);
        data_pt2=get_data_unit(System_Tab.userpool1);
        break;
    default: return(NOT_FOUND);
    }
    return (DONE);
}
```

Задача *Leaf_Out*, код которой показан ниже, вызывает функцию *check_local*, и когда в сообщениях *sensor_msg* получены два значения датчика, она помещает их в один узел *node_msg* для передачи родителю.

```
/******
Задача Leaf Out
******/
TASK Leaf_Out() {

    data_pt=get_data_unit(System_Tab.userpool1);
    data_pt2=get_data_unit(System_Tab.userpool1);
    while(TRUE) {
        delay_task(current_tid, System_Tab.LOCAL_DELAY);
        recvd_pt=recv_mailbox_wait(mbox_id1);
        check_local(recvd_pt->TL_header.type, data_pt);
        if (sensor_count==2) {
            data_pt->TL_header.type=NODE_MSG;
            data_pt->data.node_msg.node_id=System_Tab.this_node;
            data_pt->MAC_header.sender_id=System_Tab.this_node;
```

```

data_pt->MAC_header.receiver_id=my_parent;
send_mailbox_notwait(mbox_id2,data_pt);
data_pt=get_data_unit(System_Tab.userpool1);
sensor_count=0;
data_pt=get_data_unit(System_Tab.userpool1);
}
}
}

```

13.8. ПРОМЕЖУТОЧНЫЙ УЗЕЛ

Промежуточным узлом является любой узел, кроме листьев и головы СН. Главная функция этого типа узлов заключается в сборе всех данных от его дочерних элементов, сформированных на основном дереве, объединении этих данных с локальными данными и отправке всех этих данных его родителю. Возможные сообщения для этого типа узла: сообщения листа, промежуточного узла и СН.

13.8.1. Дизайн верхнего уровня

Контекстная диаграмма промежуточного узла изображена на рис. 13.7 со всеми показанными внешними объектами. Этот тип узла собирает собственные данные среды, объединяет эти данные в одном сообщении и отправляет его на другой промежуточный узел или управляющий узел кластера.

Уровень 1 диаграммы DFD промежуточного узла изображен на рис. 13.8. Мониторинг окружения этого узла будет таким же, как и у листа, поэтому мы не будем повторять функции задач *Temp*, *Humid* и *Accel* и структуры данных. У нас есть задача *IMerge*, которая в основном объединяет все полученные значения температуры и влажности всех детей, записывает условия тревоги в полученных значениях и при необходимости активирует тревоги, помещая сообщения о тревоге (*comp_al*) в память данных *comp_alarm*.

Мы начинаем с определения содержимого сообщения, которое передается между промежуточным узлом и другим промежуточным/управляющим узлом кластера. Это сообщение содержит локальные данные всех дочерних узлов промежуточного узла, как показано на рис. 13.9.

Код для данной структуры может быть сформирован следующим образом:

```

/*****
Структура промежуточных сообщений
*****/
typedef struct {
    ushort inode_id;
    ushort n_msg;
    node_msg_t node_msg[N_CHILD_MAX];
} interm_msg_t, *interm_msg_ptr_t;

```



Рис. 13.7. Контекстная диаграмма промежуточного узла

13.8.2. Детальное проектирование и реализация

Нам нужна внешняя задача *Inter_In*, которая собирает данные от всех дочерних элементов узла и объединяет эти данные с локальными данными для отправки восходящим потоком другому промежуточному узлу или управляющему узлу кластера. Эта задача также может получить сообщение управляющего узла кластера, которое пересылается родителю. Мы будем считать, что все узлы синхронизированы по времени и задержки задач близки друг к другу, так что данные от детей поступают при выполнении этой задачи за небольшой промежуток времени, и что все узлы функционируют правильно, без отказов.

```

/*****
Задача Inter_In
*****/
TASK Inter_In() {

    data_unit_ptr_t data_pt;
    ushort mbox_id1=task_tab[current_tid].mailbox_id;
    ushort mbox_id2;

    while(TRUE) {
        delay_task(current_tid,System_Tab.LOCAL_DELAY);
        data_pt=recv_mailbox_wait(mbox_id1);
        switch(data_pt->TL_header.type) {
            case LEAF_MSG :
            case INTER_MSG :
                mbox_id2=&(task_tab[System_Tab.Imerge_id])->mailbox_id;
                break;

```

```

case CHEAD_MSG :
case ALARM_MSG :
mbox_id2=&(task_tab[System_Tab.Inter_Out_Id])->mailbox_id;
}
send_mailbox_notwait(mbox_id2,data_pt);
}
}

```

Задача объединения промежуточного узла (*Imerge*) вводит все данные, относящиеся к узлу из задачи *Inter_In*, и объединяет их в одно сообщение. Когда все дочерние узлы промежуточного узла отправили свои данные, одно сообщение может быть отправлено к выходной задаче *Inter_Out* для передачи родителю. Обратите внимание, что входами в эту задачу являются данные локального узла и любые промежуточные дочерние данные, которые необходимо объединить в промежуточное сообщение. Переменная *child_count* используется для подсчета числа полученных сообщений, и когда это значение равно количеству детей промежуточного узла, сообщение может быть передано родителю.

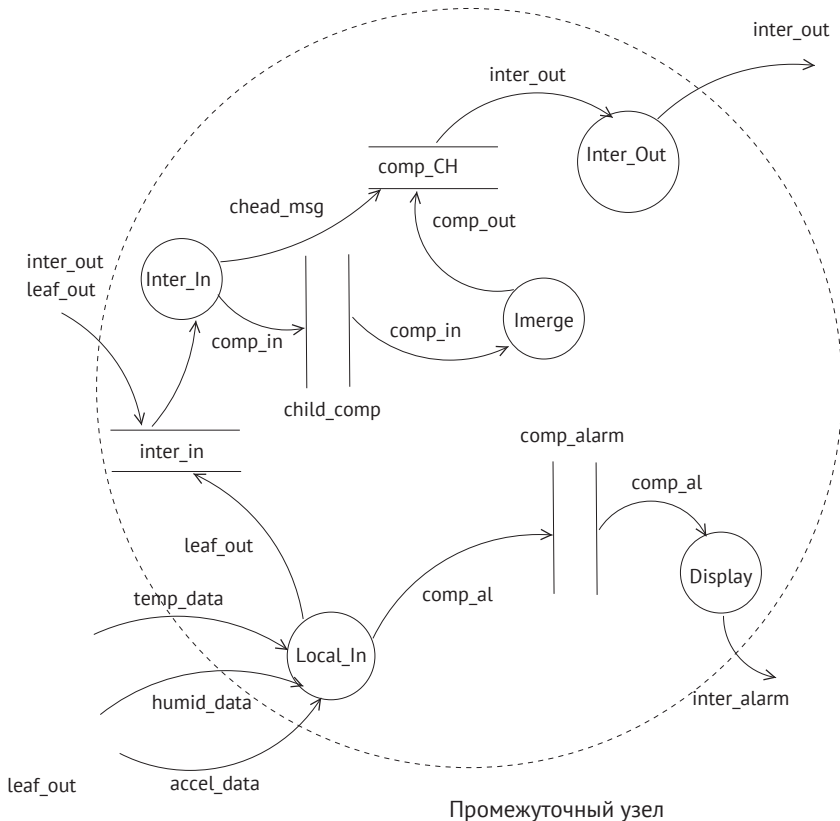


Рис. 13.8. Диаграмма уровня 1 промежуточного узла

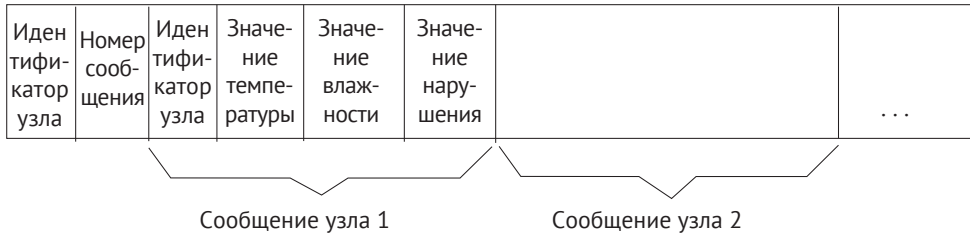


Рис. 13.9. Структура промежуточного сообщения

Мы полагаем, что значения локального датчика объединяются в сообщение узла и доставляются в эту задачу задачей *Local_In*, которая не отображена. Эта задача может получать сообщения локального датчика и должна поместить их в *node_msg*, или она может получать сообщения тревоги или СН, которые должны быть переданы немедленно. Для этой цели она использует функцию *check_local* модуля листа, и если тип сообщения в этой функции не распознается, выполняется проверка других типов сообщений в функции *check_interm*, показанной ниже.

```

/*****
Задача IMerge
*****/
data_unit_ptr_t data_pt, recvd_pt;
node_msg_ptr_t nddata_pt;
inter_msg_ptr_t intdata_pt;
ushort mbox_id1=&(task_tab[current_tid])->mailbox_id;
ushort mbox_id2=&(task_tab[System_Tab.DL_Out_id])->mailbox_id;
ushort child_count=0, msg_count=0,i;
int check_local(int);

int check_interm(ushort type, data_ptr_t data_pt) {

    if(check_local(type, data_pt)==NOT_FOUND || sensor_count==0)
    switch(type) {
    case NODE_MSG :
        memcpy(nddata_pt,
        &(recvd_pt->data.node_msg), sizeof(node_msg_t));
        msg_count++; nddata_pt++; child_count++;
        break;
    case INTERM_MSG:
        memcpy(intdata_pt,
        &(recvd_pt->data.interm_msg), sizeof(interm_msg_t));
        msg_count=msg_count+recvd_pt->data.interm_msg.n_msg;
        nddata_pt=nddata_pt+msg_count;
        child_count++;
    }
    return (DONE);
}

```

Задача *Imerge* вызывает эту функцию, чтобы найти тип сообщения и сделать необходимые передачи данных в зависимости от типа. Когда она получает сообщения от всех своих детей, конвергентные данные могут быть отправлены в виде одного *промежуточного* сообщения родителю.

```
TASK Imerge() {

    data_pt=get_data_unit(System_Tab.userpool1);
    nddata_pt=&(data_pt->data.node_msg);
    intdata_pt=&(data_pt->data.interm_msg);
    while(TRUE) {
        delay_task(current_tid,System_Tab.LOCAL_DELAY);
        recvd_pt=recv_mailbox_wait(mbox_id1);
        check_interm(recvd_pt->TL_header.type,data_pt);
        if (sensor_count==2) {
            memcpy(nddata_pt,
                recvd_pt->data.node_msg, sizeof(node_msg_t));
            msg_count++; nddata_pt++; child_count++;
            sensor_count=0;
        }
        if(child_count==&(System_Tab.n_children+1)) {
            data_pt->data.interm_msg.inode_id=System_Tab.this_node;
            data_pt->TL_header.type=INTERM_MSG;
            data_pt->TL_header.sender_id=System_Tab.this_node;
            data_pt->data.interm_msg.n_msg=msg_count;
            data_pt->MAC_header.sender_id=System_Tab.this_node;
            data_pt->MAC_header.receiver_id=my_parent;
            send_mailbox_notwait(mbox_id2,data_pt);
            data_pt=get_data_unit(System_Tab.userpool1);
            nddata_pt=&(data_pt->data.node_msg);
            intdata_pt=&(data_pt->data.interm_msg);
            child_count=msg_count=0;
            data_pt=get_data_unit(System_Tab.userpool1);
        }
    }
}
```

Код для задачи *Alarm* опущен, так как он будет очень похож на код задачи тревоги листа. Выходная задача *Inter_Out* просто получает сообщение в своем почтовом ящике и помещает сообщение в почтовый ящик задачи канального уровня передачи данных. Обратите внимание, что мы можем справиться с задачей *Imerge*, выполняя эту операцию так же, как объединение, однако наличие задач, предназначенных операциям ввода и вывода, удобно в общем виде, чтобы при необходимости изменить и улучшить функции ввода/вывода.


```

/*****
Задача Inter_Out
*****/
TASK Inter_Out() {

    data_unit_ptr_t data_pt;
    ushort mbox_id=task_tab[current_tid].mailbox_id, mbox_id2;
    mbox_id2=task_tab[System_Tab.DL_Out_Id].mailbox_id
    while(TRUE) {
        data_pt=recv_mailbox_wait(mbox_id);
        data_pt->MAC_header.sender_id=System_Tab.this_node;
        data_pt->MAC_header.receiver_id=my_parent;
        send_mailbox_notwait(mbox_id,data_pt);
    }
}

```

13.9. УЗЕЛ УПРАВЛЕНИЯ КЛАСТЕРОМ

Узел управления кластером выполняет локальное зондирование, как и все другие узлы, кроме приемника. Он также выполняет *агрегацию данных* путем вычисления средних значений температуры и значения влажности и формирует структуры сообщений, показанных на рис. 13.10. Это сообщение узла управления кластером имеет поле кластера, содержащее его идентификатор, средние значения температуры и влажности для кластера, а также идентификаторы узлов и аварийные состояния в кластере, если они существуют. Состояние тревоги для каждого узла в кластере должно быть доставлено на узел приемника, и, следовательно, у нас есть такие поля для указания этого условия.

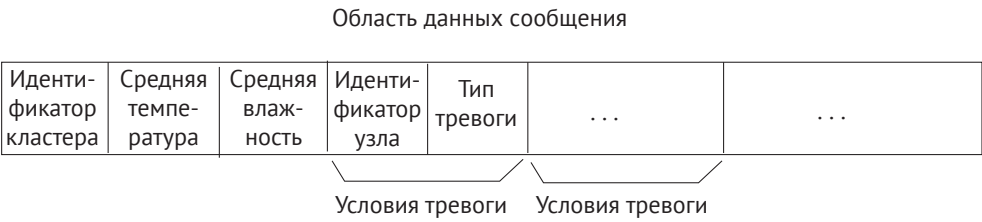


Рис. 13.10. Структура сообщения данных узлом управления кластером



Рис. 13.11. Контекстная диаграмма узла управления кластером

13.9.1. Дизайн высокого уровня

Диаграмма DFD уровня 0 группы кластеров изображена на рис. 13.11. У нас здесь может быть лист, промежуточный узел и тип узла кластера, управляющего сообщениями, приходящими на этот узел. Его выход может быть направлен листовому узлу, кластеру или в приемник.

Диаграмма DFD уровня 1 на рис. 13.12 отображает структуру задач этого узла, которая аналогична диаграмме DFD уровня 1 промежуточного узла. Основное отличие в том, что задача *IComp*, которая в основном производит вычисление средних значений всех полученных значений температуры и влажности всех детей, выделяет условия тревоги в полученных значениях и активирует, если это необходимо, аварийные сигналы, помещая аварийные сообщения (*comp_alarm*) в память данных *comp_alarm*.

13.9.1. Дизайн верхнего уровня

Локальный мониторинг этого узла будет таким же, как и конечного и промежуточного узлов, поэтому мы не будем повторять функции задач *Temp*, *Humid* и *Accel* и структуры данных. Структура сообщения, определенная ниже, может быть включена в структуру данных блока, определенную в главе 5, для применения в этом приложении.

```

/*****

```

```

Структура кластера сообщений

```

```

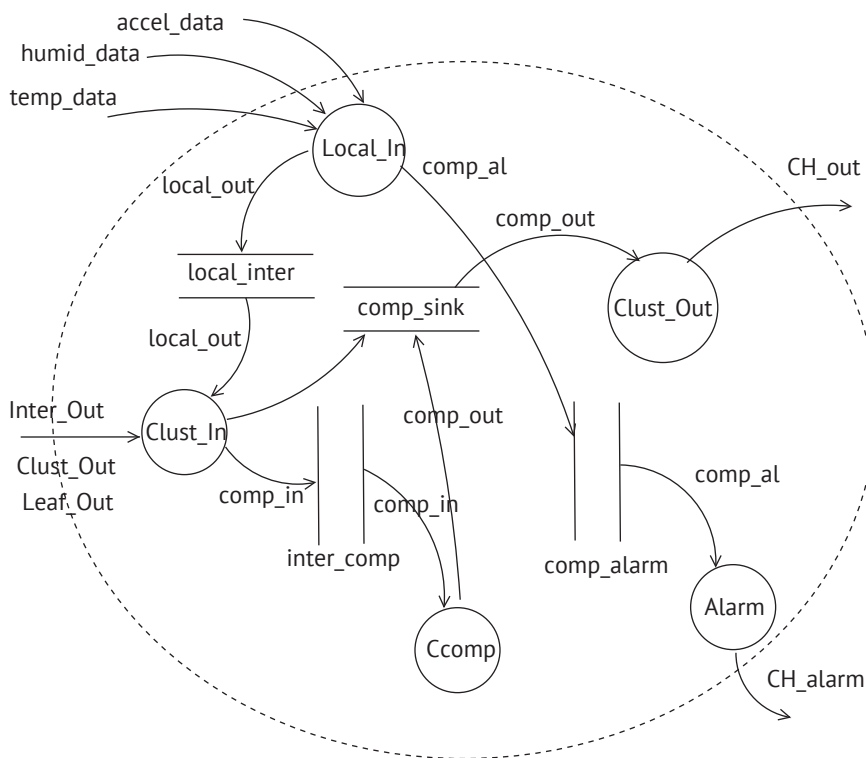
*****/

```

```

typedef struct {
    ushort node_id;
    ushort type;
} alarm_type;
typedef struct {
    ushort cluster_id;
    double ctemp_ave;
    double chumid_ave;
    alarm_type alarm_cond[N_CLUS_NODE];
} cluster_msg_t;

```



Узел управления кластером

Рис. 13.12. Диаграмма узла управления кластером уровня 1

Внешняя задача под названием *Clust_In* собирает данные от всех дочерних узлов, которые могут быть данными локальных узлов кластера или другими данными в форме сообщений кластера. Она работает как маршрутизатор, переключая любые сообщения кластера на выход задачи *Clust_Out* и межклас-

терные сообщения данных для задачи *Ccomp*, которые будут обрабатываться как в промежуточном узле. Мы опускаем код для этой задачи, так как он очень похож на *Inter_In* – код задачи промежуточного узла. Задача *Ccomp* работает иначе, чем ее аналог задачи *Imerge*, как показано ниже. Она вводит все данные и принимает среднее полученных значений, формирует компактное сообщение, содержащее эти значения вместе с условиями тревоги, и помещает это сообщение в почтовый ящик задачи *Clust_Out*, как показано ниже. Обратите внимание, что проверка типа сообщения аналогична проверке сообщения в промежуточном узле, чтобы мы могли использовать здесь функцию *check_interm*. Тип сообщения будет определяться этой функцией либо локально, либо путем вызова функции *check_local*.

```

/*****
Задача Ccomp
*****/
data_unit_ptr_t data_pt, data_pt2, recvd_pt;
node_msg_ptr_t nddata_pt;
inter_msg_ptr_t intdata_pt;
clust_msgs_ptr_t clusdata_pt;
ushort mbox_id1=task_tab[current_tid].mailbox_id;
ushort mbox_id2=task_tab[System_Tab.DL_Out_id].mailbox_id;
double temp_tot=0.0, humid_tot=0.0;
ushort child_count=0, msg_count=0,i,n;

TASK Ccomp() {

    data_pt=get_data_unit(System_Tab.userpool1);
    nddata_pt=&(data_pt->data.node_msg);
    intdata_pt=&(data_pt->data.interm_msg);
    clusdata_pt=&(data_pt->data.clust_msg);
    while(TRUE) {
        delay_task(current_tid,System_Tab.LOCAL_DELAY);
        recvd_pt=recv_mailbox_wait(mbox_id1);
        check_interm(recvd_pt->TL_header.type, data_pt);
        if(child_count==task_tab[current_tid].n_children+1) {
            data_pt2=get_data_unit(System_Tab.userpool1);
            nddata_pt=&(data_pt2->data.node_msg);
            for(i=0;i<msg_count;i++) {
                temp_tot=temp_tot+nddata_pt->temp.value;
                humid_tot=humid_tot+nddata_pt->humid.value;
                nddata_pt++;
            }
            data_pt2->data.cluster_msg.temp_ave=(double)temp_tot/msg_count;
            data_pt2->data.cluster_msg.humid_ave=(double)humid_tot/msg_count;
            data_pt2->data.cluster_msg.cluster_id=System_Tab.this_node;
            data_pt2->TL_header.type=CHEAD_MSG;
            data_pt2->TL_header.sender_id=System_Tab.this_node;
            data_pt2->MAC_header.sender_id=System_Tab.this_node;
            data_pt2->MAC_header.receiver_id=my_parent;

```

```

send_mailbox_notwait(mbox_id2,data_pt2);
data_pt=get_data_unit(System_Tab.userpool1);
nndata_pt=&(data_pt->data.node_msg);
intdata_pt=&(data_pt->data.interm_msg);
child_count=msg_count=0;
data_pt2=get_data_unit(System_Tab.userpool1);
}
}
}

```

13.10. ПРИЕМНИК

Приемник – это узел с расширенными вычислительными возможностями. Он принимает информацию о кластере от кластерных голов и отображает эту информацию в своем мониторе с подсветкой условий тревоги. Он также предоставляет короткий отчет удаленной станции через шлюз.

13.10.1. Дизайн высокого уровня

Основываясь на требованиях узла приемника, можно нарисовать контекстную диаграмму, как на рис. 13.13.

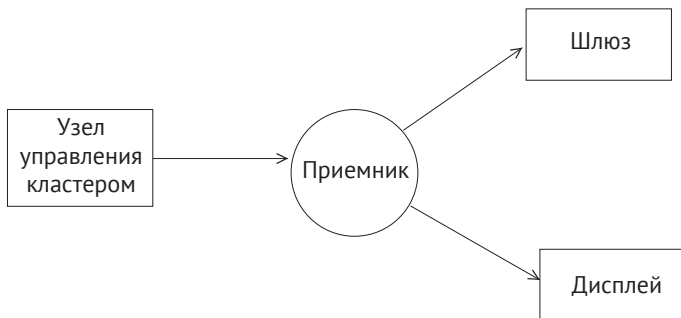


Рис. 13.13. Контекстная диаграмма приемника

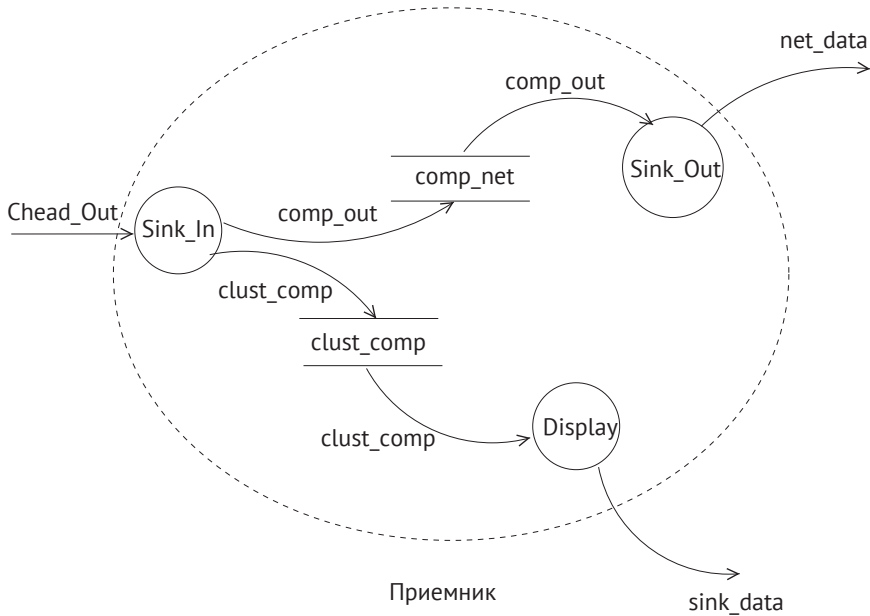


Рис. 13.14. Диаграмма DFD приемника уровня 1

Диаграмма DFD уровня 1 приемника изображена на рис. 13.14. Обратите внимание, что приемник не делает никакой локальной обработки, поэтому мы не имеем этой функциональности на данном узле. Условия тревоги узлов уже определены, поэтому нет и проверки тревоги. Наконец, мы полагаем, что данные кластера по мере их получения отображаются в режиме онлайн; следовательно, приемник не должен ждать сбора данных от своих дочерних элементов. Можно видеть, что разделение функций между различными типами узлов привело к простому дизайну приемника. Детальный дизайн приемника аналогичен дизайну и кодировке промежуточных узлов и узлов управления кластером и оставлен как проект программирования (см. упражнение 3).

13.11. ТЕСТИРОВАНИЕ

Для моделирования будет использоваться образец ядра DRTK, а входы от датчиков будут реализованы путем чтения значений из входных файлов. Нам нужно изменить структуру блока данных, определенную в главе 5. Заголовок транспортного уровня и заголовки уровня MAC остаются прежними, но область данных должна быть объединением введенных новых типов сообщений. Кодированными являются только созданные задачи.

```

/*****
Структура блока новых данных
*****/
typedef struct *data_ptr_t{
    TL_header_t TL_header;
    MAC_header_t MAC_header;
    int type;
    int data[N_DATA];
    union data {
        sensor_msg_t sensor_msg;
        node_msg_t node_msg;
        interm_msg_t interm_msg;
        cluster_msg_t cluster_msg;
    }
    ushort MAC_trailer;
    data_ptr_t next;
}data_unit_t;

/*****
Основная программа каждого узла, спецификация узла управления кластером
*****/
#include <pthread.h>
#include "drtk.h"
#include "drtk.c"

#define LEAF 0
#define INODE 1
#define CH 2
#define SINK 3

void main() {
    ushort my_id, my_parent;

    // выберите, что надо компилировать

#ifdef SINK // make sensor devices, sink doesn't have them
    System_Tab.temp_devid=make_dev();
    System_Tab.humid_devid=make_dev();
    System_Tab.alarm_devid=make_dev();
    System_Tab.Temp_id=make_task(Temp,SYSTEM,1,NO);
    System_Tab.Humid_id=make_task(Humid,SYSTEM,1,NO);
    System_Tab.Accel_id=make_task(Accel_Int,SYSTEM,1,NO);
    System_Tab.Alarm_id=make_task(Leaf_Alarm,SYSTEM,1,NO);
    System_Tab.Leaf_In_id=make_task(Leaf_In,SYSTEM,1,NO);
    System_Tab.Leaf_Out_id=make_task(Leaf_Out,SYSTEM,1,NO);
#endif

#ifdef INODE
    System_Tab.Inter_In_id=make_task(Inter_In,SYSTEM,1,NO);
    System_Tab.Imerge_id=make_task(Imerge,SYSTEM,1,NO);
    System_Tab.Inter_Out_id=make_task(Inter_Out,SYSTEM,1,NO);

```

```

#endif
#ifdef CH
System_Tab.Ccomp_id=make_task(Ccomp,SYSTEM,1,NO);
#endif
. . .
Schedule();
}

```

13.12. АЛЬТЕРНАТИВНАЯ РЕАЛИЗАЦИЯ С ПОТОКАМИ POSIX

Теперь мы рассмотрим альтернативный метод детальной реализации с POSIX-потоками без использования функций DRTK. В этом случае каждая задача реализуется с помощью потока POSIX и для связи задач может быть использован метод связи между потоками, описанный в главе 4. Мы просто опишем изменения, которые будут внесены в существующий код, описанный до этого момента:

- задачи реализуются POSIX-вызовом *pthread_create* вместо системного вызова DRTK *make_task*;
- процедуры связи задач DRTK заменены простыми процедурами *write_fifo* и *read_fifo*, описанными в главе 4;
- задержка задач может быть достигнута с помощью функций *sleep* и *usleep* вместо функции DRTK *delay_task*;
- планирование потоков POSIX выполняется системой.

Этот подход может показаться более простым, чем реализация DRTK; однако нам обычно необходимо иметь интерфейс POSIX, подобный используемому в системе UNIX, что потребует значительного объема памяти.

13.13. ПРИМЕЧАНИЯ К ГЛАВЕ

Мы описали в этой главе в виде тематического исследования проектирование и реализацию распределенной системы реального времени. В соответствии со спецификацией требований и их анализом для системного оборудования было принято решение использовать сеть WSN, учитывая стоимость, функциональность и простоту развертывания. Грубый временной анализ показал, что строгие требования отсутствуют. Рассмотрение совместного аппаратно-программного проектирования привело к решению использования сети WSN и, соответственно, реализации остоного дерева и алгоритма кластеризации, которые были подробно описаны.

Используемый алгоритм классифицирует узлы WSN как листовые, промежуточные, или узлы управления кластерами. Затем мы выполнили дизайн высокого уровня, детальный дизайн и кодирование для всех этих типов узлов и компьютера-приемника с использованием диаграмм DFD. В реальном времени ядром операционной системы является ядро DRTK, которое обеспечивает основные примитивы синхронизации между задачами и связи, а также

функции для создания объектов, таких как задачи, блоки управления устройством и управление сетью. Тестирование включает компиляцию модулей для каждого узла и с последующим выполнением. В DRTK должны быть сделаны некоторые изменения, которые не были показаны подробно. Они включают добавление нескольких полей в системную таблицу, а для реального приложения потребуется интеграция сетевых драйверов узлов сети WSN. Тем не менее мы полагаем, что этот способ применения этапов проектирования дает представление о реализации распределенной системы реального времени.

13.14. УПРАЖНЕНИЯ ПО ПРОГРАММИРОВАНИЮ

1. Показать, как можно добиться отказоустойчивости при сборе данных от детей в случае промежуточных узлов, узлов кластера и приемника, когда один или несколько дочерних узлов не предоставит свои данные.
2. Модифицируйте код для задачи *Icompr* промежуточного узла, чтобы эта задача напрямую отправляла данные в восходящую задачу по сети.
3. Выполните детальное проектирование программного обеспечения приемника, сформировав все необходимые задачи.
4. *Проект для команды*: реализовать систему мониторинга сети WSN с использованием потоков POSIX и межпоточный коммуникационный модуль.

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. *Erciyes K. (2018) Guide to graph algorithms: sequential, parallel and distributed. Springer Nature.*

Приложение А

Соглашение о псевдокоде

А.1. ВВЕДЕНИЕ

В этом разделе представлены соглашения о псевдокоде для написания алгоритма. Используемые нами соглашения следуют современным правилам программирования и аналогичны использованным в [1, 2]. Каждый алгоритм имеет имя, указанное в заголовке, и каждая строка алгоритма пронумерована, чтобы обеспечить цитирование. Первая часть алгоритма обычно начинается с его входов. Блоки в алгоритмах показаны отступами. Принятые соглашения о псевдокоде описываются как структуры данных, структуры управления и структура распределенного алгоритма.

А.2. СТРУКТУРЫ ДАННЫХ

Выражения строятся с использованием констант, переменных и операторов, как в любом функциональном языке программирования, и дают определенную величину. *Утверждения* состоят из выражений и являются основной единицей выражений. Все операторы представлены в форме пронумерованных строк. Объявление переменной выполняется так же, как в языках типа Pascal и Си, где ее тип предшествует ее метке с возможной инициализацией следующим образом:

set of int *neighbors* $\leftarrow \{\emptyset\}$

Здесь мы объявляем множество, называемое *соседями* вершины графа, каждый элемент которой является целым числом. Этот набор инициализируется значением $\{\emptyset\}$ (пусто). Другие, обычно используемые типы переменных в алгоритмах логические для логических переменных и *типы сообщений* для возможных типов сообщений. Для присваиваний мы используем оператор $\leftarrow$, который показывает, что значение справа присвоено переменной слева. Например, оператор

$a \leftarrow a + 1$

увеличивает значение целочисленной переменной *a*. Два или более утверждения в строке, разделенных точкой с запятой, и комментарии показаны символом в конце строки следующим образом:

$1 : a \leftarrow 1; c \leftarrow a + 2; \triangleright c \text{ теперь } 3$

Общие соглашения по алгоритмам приведены в табл. А.1.

Таблица А.2 суммирует арифметические и логические операторы, используемые в тексте, с их значениями.

Вместо массивов для представления коллекции похожих переменных часто используются наборы. Включение элемента u в набор S можно сделать следующим образом:

$$S \leftarrow S \cup \{u\},$$

а удаление элемента v из S выполняется так:

$$S \leftarrow S \setminus \{v\}.$$

В табл. А.3 показаны используемые в тексте операции над множествами с их значениями.

Таблица А1. Комментарии к общим алгоритмам

Условные знаки	Значение
$x \leftarrow y$	Присваивание
$=$	Равенство
$\neq$	Неравенство
$true, false$	Логические истина, ложь
$null$	Несуществующее
$\triangleright$	Комментарий

Таблица А2. Арифметические и логические операторы

Условные знаки	Значение
$\neg$	Логическое отрицание
$\wedge$	Логическое и
$\vee$	Логическое или
$\oplus$	Логическое исключение – или
x/y	Деление x на y
$x \cdot y$ или xy	Умножение

Таблица А3. Операции над множествами

Условные знаки	Значение
$ S $	Мощность множества S
$\emptyset$	Пустое множество
$u \in S$	u из множества S
$S \cup R$	Объединение множеств S и R
$S \cap R$	Пересечение множеств S и R

Условные знаки	Значение
$S \setminus R$	Деление множеств S и R
$S \subset R$	S правильное подмножество R
$\max/\min S$	Максимум/минимум величин элементов множества S
$\max/\min\{\dots\} S$	Максимум/минимум величин из набора величин множества S

А3. УПРАВЛЕНИЕ СТРУКТУРАМИ

В последовательной операции операторы выполняются последовательно. Ветвление к другому утверждению может быть сделано путем выбора, описанного ниже.

Выбор

Выбор осуществляется с помощью условных операторов, которые реализуются с использованием *if-then-else* обычным способом, а отступы используются для указания блоков, как показано в приведенном ниже примере кода:

Алгоритм А.1. Структура *if-then-else*

```

1: if условие then › первая проверка
2: утверждение1
3: if условие2 then › второе (вложенное) если
4: утверждение2
5: end if › окончание второго если
6: else if условие3 then › иначе, если первое если
7: утверждение3
8: else
9: утверждение4
10: end if › окончание первого если

```

Для выбора из нескольких ветвей используется конструкция *case-of*. Выражение в этой конструкции должно возвращать значение, которое проверяется по количеству постоянных значений, и соответствующая ветвь берется следующим образом:

```

1. case выражение of
2. постоянная1 : утверждение1
3. ⋮
4. постояннаяn : утверждениеn
5. end case

```

Повторение

Основными циклами в соответствии с обычным синтаксисом языка высокого уровня являются конструкции *while* и *loop*. Цикл *for-do* используется, когда количество итераций можно оценить перед входом в цикл следующим образом:

```

1. for  $i \leftarrow 1$  to  $n$  do
2. ⋮
3. end for

```

Второй формой этой конструкции является цикл *for all*, который произвольно выбирает элемент из указанного набора и выполняет итерацию, пока все члены набора не будут обработаны, как показано ниже, где дан набор S с тремя элементами и пустым набором R и каждый элемент S копируется в R итеративно.

```

1.  $S \leftarrow \{3, 1, 5\}; R \leftarrow \emptyset$ 
2. for all  $u \in S$  do
3.    $R \leftarrow R \cup \{u\}$ 
4. end for

```

Для неопределенных случаев, когда цикл вообще не может быть введен, может использоваться конструкция *while-do*, в которой оценивается логическое выражение, и если это значение истинно, цикл вводится следующим образом:

```

1. while булево выражение do
2.   утверждение
3. end for

```

А.4. СТРУКТУРА РАСПРЕДЕЛЕННОГО АЛГОРИТМА

Структуры распределенных алгоритмов имеют существенные отличия от структуры последовательных алгоритмов, поскольку их модель выполнения определяется типом сообщений, которые они получают от своих соседей. По этой причине общий распределенный алгоритм псевдокода обычно включает структуру, аналогичную шаблону алгоритма, показанному в алгоритме А.2.

В этой структуре алгоритма может быть n типов сообщений, и тип действия зависит от типа полученного сообщения. Для этого примера цикл *while-do* выполняется до тех пор, пока значение флага логической переменной *flag* не станет равным *true*. Как правило, сообщение, полученное этим узлом (i), в какой-то момент запускает действие, которое меняет значение переменной *flag* в *true*, которое затем приводит к завершению цикла. В другой, часто используемой структуре распределенного алгоритма цикл *while-do* выполняется всегда, и одно или несколько действий должны обеспечить выход *exit* из этого бесконечного цикла *while*, как показано в алгоритме А.3.

Алгоритм А.2. Структура 1 распределенного алгоритма

```

1: int  $i, j$  ›  $i$  является узлом;  $j$  является отправителем текущего сообщения
2: while  $\neg \text{flag}$  do › все узлы выполняют один и тот же код
3:   receive  $\text{msg}(j)$ 
4:   case  $\text{msg}(j).type$  of
5:      $type\_1$  :  $Action\_1$ 
6:     . ... : ...
7:      $type\_n$  :  $Action\_n$ 
8:   if условие then
9:      $flag \leftarrow true$ 
10:   end if
11: end while

```

Алгоритм А.3. Структура 2 распределенного алгоритма

```
1: while всегда do
2: receive msg(j)
3: case msg(j).type of
4: type_1 : Action_1 : if условие1 then exit
5: ... : ...
6: type_x : Action_1 : if условие x then exit
7: type_n : Action_n
8: end while
```

Неопределенная структура этого типа цикла делает его пригодным для распределенных алгоритмов, где тип сообщения, как правило, не может быть определен заранее.

Приложение В

Функции нижнего ядра

В1. СИСТЕМНЫЕ ВЫЗОВЫ ОЧЕРЕДИ БЛОКОВ ДАННЫХ

```
/* data_unit_que.c*/
/*****
Проверка очереди блоков данных
*****/
int check_data_que(data_que_ptr_t dataque_pt) \{

    if (dataque_pt->front == NULL)
        return (EMPTY);
    return(FULL);
}

/*****
Включение блока данных в очередь
*****/
int enqueue_data_unit(data_que_ptr_t dataque_pt, data_unit_ptr_t
data_pt) \{

    data_unit_ptr_t temp_pt;
    data_pt->next=NULL;
    if (dataque_pt->front != NULL) \{
        temp_pt=dataque_pt->rear;
        temp_pt->next=data_pt;
        dataque_pt->rear=data_pt;
    }
    else
        dataque_pt->front=dataque_pt->rear=data_pt;
    return(DONE);
}
```

```

/*****
Исключение блока данных из очереди
*****/
data_unit_ptr_t dequeue_data_unit(data_que_ptr_t dataque_pt) {

    data_unit_ptr_t data_pt;
    if (dataque_pt->front!=NULL) {
        data_pt=dataque_pt->front;
        dataque_pt->front=data_pt->next;
    }
    return(data_pt);
}
return(ERR_NOT_AV);
}

```

В.2. СИСТЕМНЫЕ ВЫЗОВЫ ОЧЕРЕДИ ЗАДАЧ

```

/* task_que.c */
/*****
Проверка очереди задач
*****/
int check_task_que(task_que_ptr_t taskque_pt) {

    if (taskque_pt->front == NULL)
        return (EMPTY);
    return(FULL);
}

/*****
Включение задачи в очередь задач
*****/
int enqueue_task(task_que_ptr_t taskque_pt, task_ptr_t task_pt){

    task_ptr_t temp_pt;
    task_pt->next=NULL;
    if (taskque_pt->front != NULL) {
        temp_pt=taskque_pt->rear;
        temp_pt->next=task_pt;
        taskque_pt->rear=task_pt;
    }
    else
        taskque_pt->front=taskque_pt->rear=task_pt;
    return(DONE);
}

```



```

/*****
Исключение задачи из очереди задач
*****/
task_ptr_t dequeue_task(task_que_ptr_t taskque_pt) {

    task_ptr_t task_pt;
    if (taskque_pt->front!=NULL) {
        task_pt=taskque_pt->front;
        taskque_pt->front=task_pt->next;
        return(task_pt);
    }
    return(ERR_NOT_AV);
}

/*****
Включение задачи в очередь задач в соответствии с приоритетом
*****/
int insert_task(task_que_ptr_t taskque_pt, task_ptr_t task_pt) {

    task_ptr_t task_pt, temp_pt, previous_pt;

    if(task_pt->priority < taskque_pt->front->priority) {
        temp_pt=taskque_pt.front;
        taskque_pt.front=task_pt;
        task_pt->next=temp_pt;
    }
    else {
        previous_pt=taskque_pt.front->next;
        while(task_pt-> priority >= previous_pt.priority) {
            previous_pt=temp_pt;
            temp_pt=temp_pt->next;
        }
        previous_pt.next=task_pt;
        task_pt->next=temp_pt;
    }
    return(DONE);
}

/*****
Включение задачи в дельта-очередь
*****/
int insert_delta_queue(ushort task_id, ushort n_ticks) {
    task_ptr_t task_pt, temp_pt, previous_pt, next_pt;
    task_queue_ptr_t taskque_pt;
    ushort total_delay=0;
    if (task_id < 0 || task_id >= System_Tab.N_TASK)
        return(ERR_RANGE);
    task_pt=&(task_tab[task_id]);
    task_pt->delay_time=n_ticks;

```

```

taskque_pt=&delta_que;
if (task_pt->delay_time < taskque_pt->front->delay_time) {
temp_pt=taskque_pt.front;
taskque_pt.front=task_pt;
task_pt->next=temp_pt;
temp_pt->delay_time=temp_pt->delay_time-task_pt
->delay_time;
}
else {
previous_pt=taskque_pt.front;
total_delay=taskque_pt->front->delay_time;
while(task_pt->delay_time > total_delay) {
previous_pt=next_pt;
next_pt=previous_pt->next;
if (next_pt==NULL) {
next_pt->next=task_pt;
task_pt->next=NULL;
task_pt->delay=task_pt->delay-total_delay;
}
total_delay=total_delay+next_pt->delay_time;
}
previous_pt.next=task_pt;
task_pt->next=next_pt;
task_pt->delay=task_pt->delay-total_delay-next_pt->delay;
next_pt->delay=next_pt->delay-task_pt->delay;
}
return(DONE);
}

```

СПРАВОЧНЫЕ МАТЕРИАЛЫ

1. *Cormen T. H., Leiserson C. E., Rivest R. L., Stein C.* (2001) Introduction to algorithms. MIT Press.
2. *Smed J., Hakonen H.* (2006) Algorithms and networking for computer games. Wiley. ISBN: 0-470-01812-7.

Предметный указатель

Ada 302

параллелизм 303

исключения 306

задача 305

управление временем 304

Упаковка 244

Контрольная точка 323

C/Real-timePOSIX 293

Драйверы устройств 111

Распределенное реальное время 63

связь 72

запуск по событию 64

конечные автоматы 65

промежуточное программное
обеспечение 68

модели 63

операционная система 68

планирование 69

запуск по времени 64

Распределенное планирование 240

Динамическая балансировка
нагрузки 71

Система со встроенными
элементами 23

Распределенные, управляемые событиями
задачи 64

Отказоустойчивость

Доступность 316

контрольная точка 323

классификация 316

уровень обслуживания 316

ошибка 316

неисправность 316

отказ 316

системы реального времени 325

классификация отказов 327

многоадресная связь 328

планирование 326

группы задач 328

блоки восстановления 324

резервирование 318

аппаратура 318

информация 319

N версий блоков 323

программное обеспечение 323

время 321

надежность 316

Конечный автомат 65

Аппаратура 31

вход/выход 50

ввод/вывод с прерыванием 55

память 46

кеш 48

прямой доступ к памяти (DMA) 56

процессор 31

блок управления 37

многоцикличность 38

конвейер 40

риски 42

один цикл 33

Ввод/вывод с прерыванием 55

Java 309

исключения 312

поток 310

синхронизация 311

управление временем 311

Ядро

- коммуникации 136
- структуры данных 119
- ввод/вывод 130
- прерывания 126
- управление памятью 139
- планировщик 123
- синхронизация 132
- блок управления задачами 120
- управление задачами 121
- состояние задачи 121

Марсианский зонд 226

Управление памятью 108

- реального времени 109

Отображаемый в памяти
ввод/вывод 53

**Промежуточное программное
обеспечение 69**

- распределенное, реального
времени 45

Многоядерные процессоры 58

Мультипроцессоры 59

**Многопроцессорное
планирование 241****Сетевые коммуникации**

- CAN 77
- передача данных 76
- IEEE 802 стандарт
реального времени 80
- модель OSI 73
- реального времени 72
- сеть реального времени Ethernet 80
- TTP 79

N версий блоков 328

Операционная система

- распределенная, реального
времени 68
- потoki 103

Сеть Петри 279

Временные сети Петри 272

POSIX/исключения 300

POSIX/коммуникация потоков 295

POSIX/потоки 295

POSIX/синхронизация потоков 297

POSIX/управление временем 296

Приоритетный потолок 231

Приоритетное наследование 227

Реальное время

Коммуникации 72

драйверы устройств 111

управление вводом/выводом 110

управление памятью 109

операционная система 92

связь 75

FreeRTOS 113

управление сообщениями 110

RTLinux 113

межзадачные коммуникации 98

синхронизация задачи 97

VxWorks 113

Разработка программного
обеспечения 265

**Операционная система реального
времени 265**

уровень канала передачи данных 151

распределенная 114

транспортный уровень 150

Программирование в реальном времени

Ada 302

C/Real-timePOSIX 293

Java 309

Системы реального времени 21

архитектура 23

характеристики 25

классификация 25

- пример [27](#)
- Аппаратура [31](#)
 - исключения [57](#)
 - вход/выход [50](#)
 - Ввод/вывод с прерыванием [55](#)
 - отображение в памяти [53](#)
 - память [46](#)
 - DMA [56](#)
 - микроконтроллеры [46](#)
 - многоцикличность [38](#)
 - конвейер [40](#)
 - один цикл [33](#)
 - таймеры [57](#)
- Блоки восстановления [324](#)
- Резервирование** [318](#)
 - аппаратура [318](#)
 - информация [320](#)
 - программное обеспечение [323](#)
 - время [322](#)
- надежная связь [328](#)
- Планирование** [151](#)
 - Апериодическое [198](#)
 - Сервер [206](#)
 - потактовое планирование [191](#)
 - циклическое выполнение [195](#)
 - монотонное с конечными сроками [195](#)
- Зависимые задачи [189](#)
 - первым последний конечный срок [220](#)
 - марсианский зонд [226](#)
 - модифицированный первым ранний конечный срок [220](#)
 - приоритетный потолок [231](#)
 - наследование
 - приоритетов [227](#)
 - совместные ресурсы [223](#)
 - распределенное [69](#)
 - buddy-алгоритм [253](#)
 - DRTK [255](#)
 - Целенаправленная адресация [252](#)
 - Балансировка нагрузки [250](#)
 - Pfair [249](#)
 - реализация DRTK [210](#)
 - динамическое [188](#)
 - EDF [197](#)
 - независимое [189](#)
 - LLF [201](#)
 - Многопроцессорное [241](#)
 - EDF-FF [246](#)
 - глобальное [247](#)
 - аномальное [249](#)
 - распределенное [249](#)
 - пакетная упаковка [244](#)
 - RM-FF [246](#)
 - политики [186](#)
 - приоритетное [186](#)
 - основанное на приоритете [195](#)
 - монотонное [195](#)
 - время отклика [202](#)
 - спорадическое [208](#)
 - статичное [188](#)
 - на основе таблиц [191](#)
- Разработка программного обеспечения** [265](#)
 - реального времени [269](#)
 - диаграмма потоков данных [272](#)
 - FSM [236](#)
 - объектно-ориентированное [274](#)
 - сети Петри [279](#)
 - временные автоматы [275](#)
 - временной анализ [271](#)
 - UML [283](#)
 - анализ требований [270](#)

спиральная модель [268](#)

V-модель [268](#)

модель водопада [267](#)

Таймеры [57](#)

Запускаемые по времени

Распределенные [64](#)

UML [283](#)

Диаграммы [283](#)

UNIX

Вилка [95](#)

очередь задач [101](#)

магистраль [95](#)

Книги издательства «ДМК Пресс» можно заказать
в торгово-издательском холдинге «Планета Альянс» наложенным платежом,
выслав открытку или письмо по почтовому адресу:
115487, г. Москва, 2-й Нагатинский пр-д, д. 6А.
При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.
Желательно также указать свой телефон и электронный адрес.
Эти книги вы можете заказать и в интернет-магазине: **www.a-planet.ru**.
Оптовые закупки: тел. **(499) 782-38-89**.
Электронный адрес: **books@aliants-kniga.ru**.

Кайхан Эрджиес

Распределенные системы реального времени

Главный редактор *Мовчан Д. А.*

dmkpress@gmail.com

Перевод *Яроцкий В. А.*

Корректор *Синяева Г. И.*

Верстка *Луценко С. В.*

Дизайн обложки *Мовчан А. Г.*

Формат 70×100 1/16.

Гарнитура «PT Serif». Печать цифровая.

Усл. печ. л. 31,04. Тираж 200 экз.

Веб-сайт издательства: www.dmkpress.com

Отпечатано в ООО «Печатное дело»
142300, Московская обл., Чехов, ул. Полиграфистов, 1